合肥大學
HEFEI UNIVERSITY



Comparing Optimization Algorithms

Thomas Weise (汤卫思)
tweise@hfuu.edu.cn · https://thomasweise.github.io

School of Artificial Intelligence and Big Data
Hefei University
Hefei, Anhui, China

人工智能与大数据学院
合肥大学
中国安徽省合肥市

Version: 2026-11-04

Outline

1. Introduction
2. Views on Performance and Time
3. Statistical Measures
4. Statistical Comparisons
5. Testing is Not Enough
6. Other Stuff
7. Summary
8. Advertisement



Introduction



Introduction



- There are many optimization algorithms.

Introduction



- There are many optimization algorithms.
- For solving an optimization problem, we want to use the algorithm most suitable for it.

Introduction

- There are many optimization algorithms.
- For solving an optimization problem, we want to use the algorithm most suitable for it.
- What does this mean?

Introduction



- There are many optimization algorithms.
- For solving an optimization problem, we want to use the algorithm most suitable for it.
- What does this mean?
- And how do we find this algorithm?

Introduction



- There are many optimization algorithms.
- For solving an optimization problem, we want to use the algorithm most suitable for it.
- What does this mean?
- And how do we find this algorithm?
- Hopefully this lesson will help answering these questions.

Introduction



- There are many optimization algorithms.
- For solving an optimization problem, we want to use the algorithm most suitable for it.
- What does this mean?
- And how do we find this algorithm?
- Hopefully this lesson will help answering these questions.
- As a complement to this lesson, I suggest the report *"Benchmarking in Optimization: Best Practice and Open Issues"*[11] on arXiv.

Exact vs. Heuristic Algorithms

- In optimization, there exist exact and heuristic algorithms.

Exact vs. Heuristic Algorithms

- In optimization, there exist exact and heuristic algorithms.
- Let's look at the classical Traveling Salesperson Problem (TSP)[6,113,153,157,251,261].



Exact vs. Heuristic Algorithms

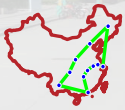
- In optimization, there exist exact and heuristic algorithms.
- Let's look at the classical Traveling Salesperson Problem (TSP)[6,113,153,157,251,261].
  - Clearly, there is (at least) one shortest tour.



Exact vs. Heuristic Algorithms

- In optimization, there exist exact and heuristic algorithms.
- Let's look at the classical Traveling Salesperson Problem (TSP)[6,113,153,157,251,261].
  - Clearly, there is (at least) one shortest tour.

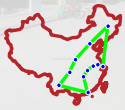
getting the optimal solution
for a TSP



Exact vs. Heuristic Algorithms

- In optimization, there exist exact and heuristic algorithms.
- Let's look at the classical Traveling Salesperson Problem (TSP)[6,113,153,157,251,261].
  - Clearly, there is (at least) one shortest tour.
  - Theory proofs that the worst-case runtime needed to find this tour grows exponentially with the number n of cities we want to visit.[1,49,55,139,152]

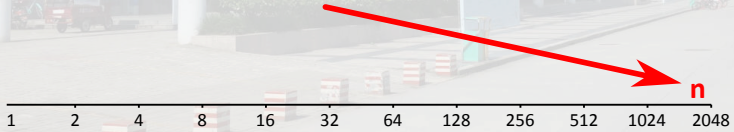
getting the optimal solution
for a TSP



Exact vs. Heuristic Algorithms

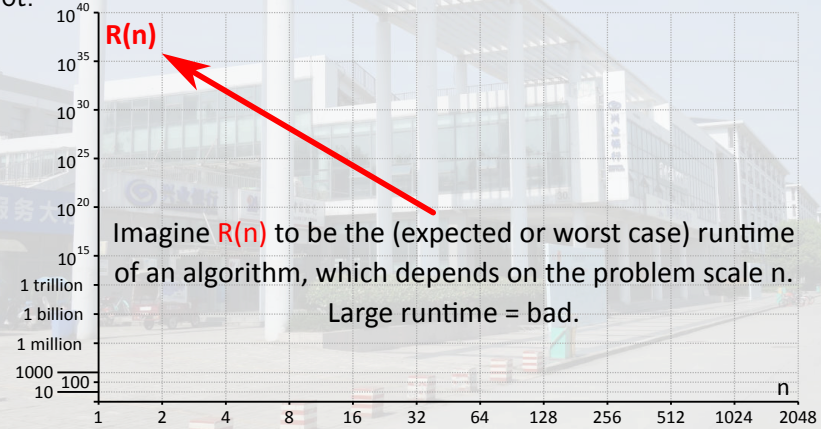
- In optimization, there exist exact and heuristic algorithms.
- Let's look at the classical Traveling Salesperson Problem (TSP)[6,113,153,157,251,261].
- What does exponential growth mean?
- Let's say we have a number of cities n.

The problem scale n usually corresponding to the number of decision variables.



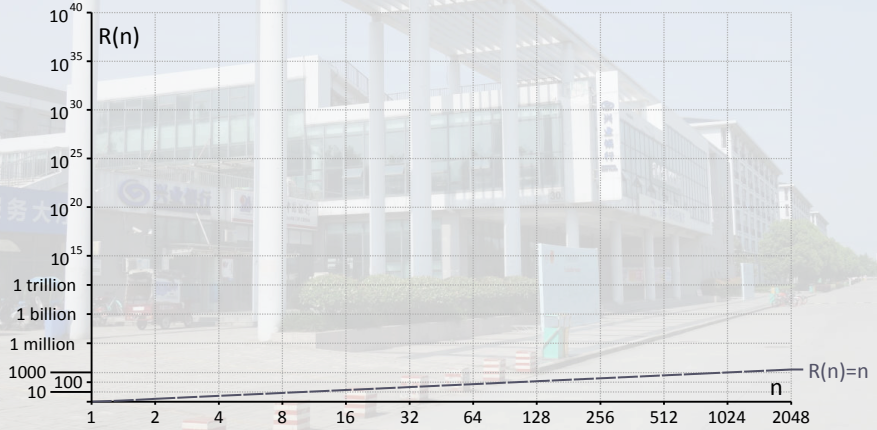
Exact vs. Heuristic Algorithms

- In optimization, there exist exact and heuristic algorithms.
- Let's look at the classical Traveling Salesperson Problem (TSP)[6,113,153,157,251,261].
- Let's say we have a number of cities n and a runtime as a function $R(n)$ in this log-log plot.



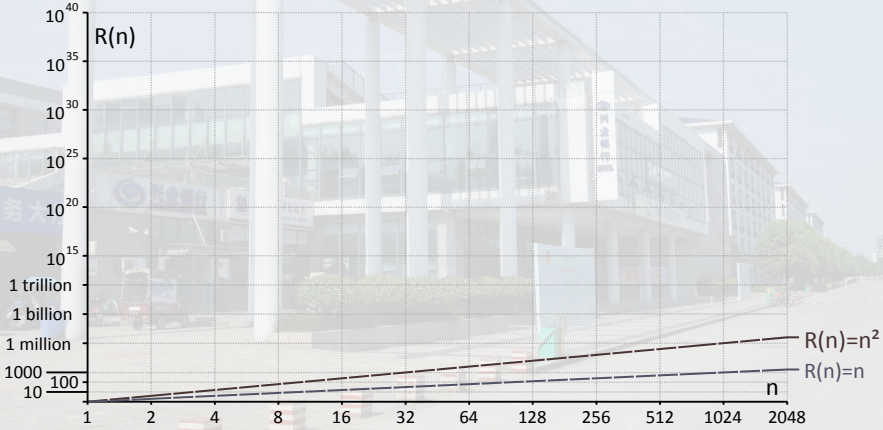
Exact vs. Heuristic Algorithms

- In optimization, there exist exact and heuristic algorithms.
- Let's look at the classical Traveling Salesperson Problem (TSP)[6,113,153,157,251,261].
- A linear function means that the runtime $R(n)$ grows slowly with n.



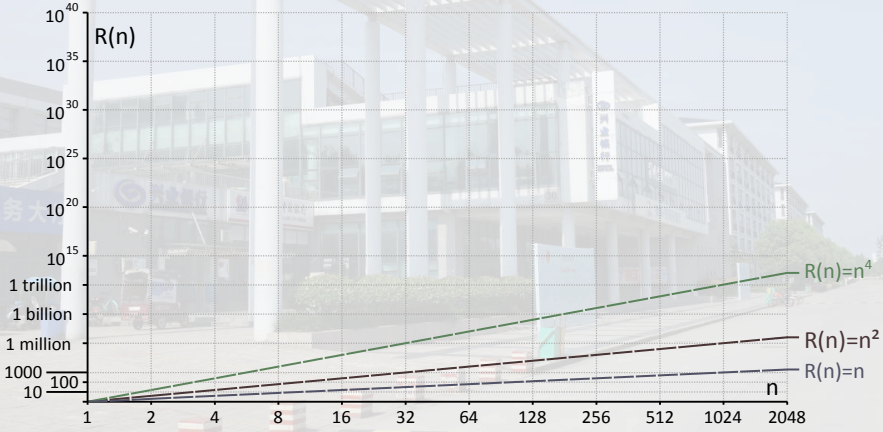
Exact vs. Heuristic Algorithms

- In optimization, there exist exact and heuristic algorithms.
- Let's look at the classical Traveling Salesperson Problem (TSP)[6,113,153,157,251,261].
- A quadratic function (a straight line in log-log plots) is also OK.



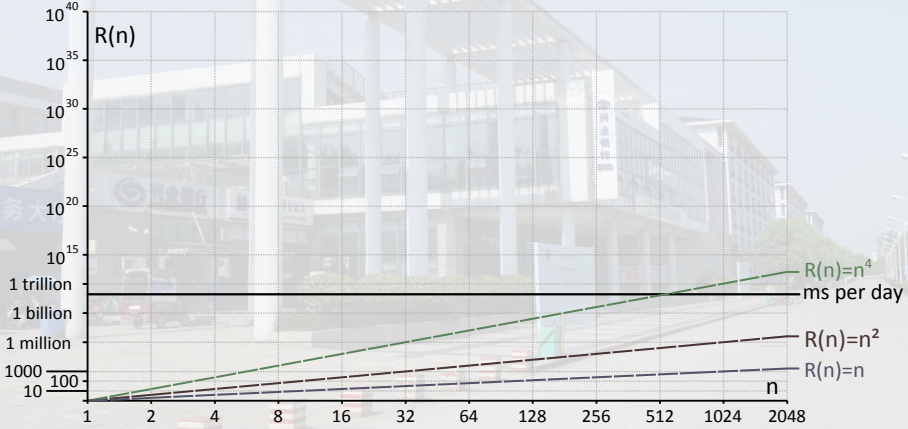
Exact vs. Heuristic Algorithms

- In optimization, there exist exact and heuristic algorithms.
- Let's look at the classical Traveling Salesperson Problem (TSP)[6,113,153,157,251,261].
- A quartic function $R(n) = n^4$ gets quite large for growing n.



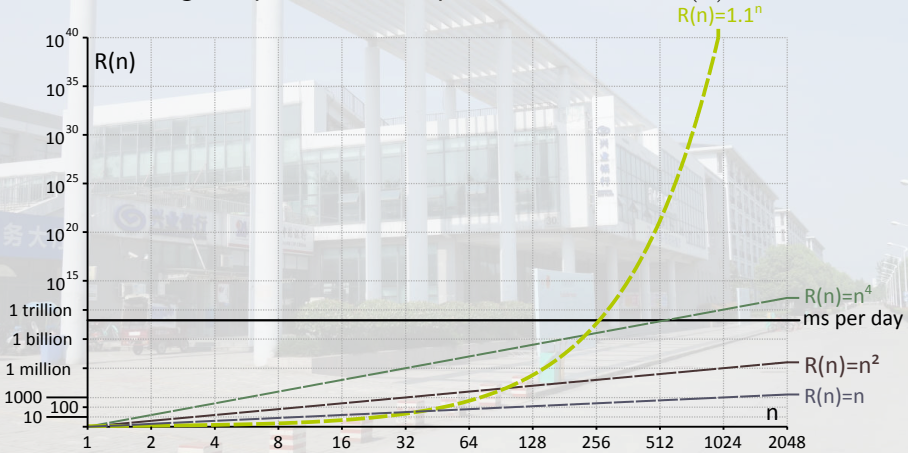
Exact vs. Heuristic Algorithms

- In optimization, there exist exact and heuristic algorithms.
- Let's look at the classical Traveling Salesperson Problem (TSP)[6,113,153,157,251,261].
- A quartic function exceeds the number of milliseconds per day at $n \approx 512$.



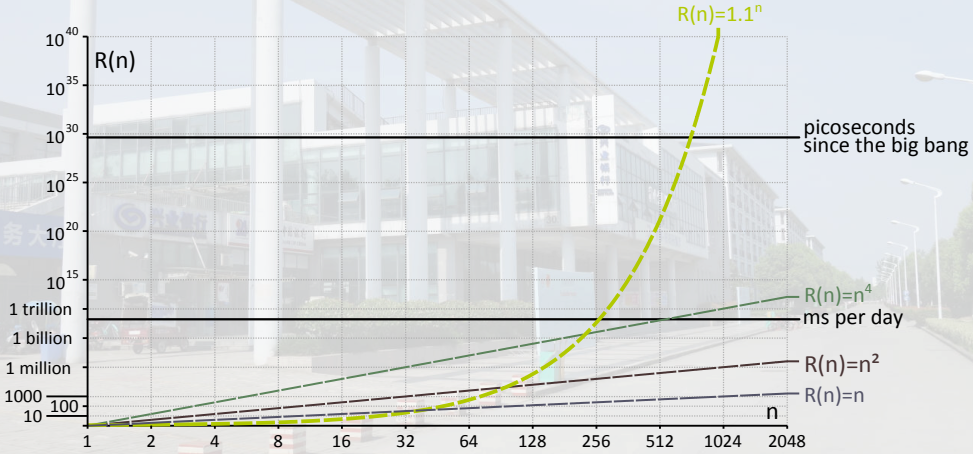
Exact vs. Heuristic Algorithms

- In optimization, there exist exact and heuristic algorithms.
- Let's look at the classical Traveling Salesperson Problem (TSP)[6,113,153,157,251,261].
- But this is nothing compared to the exponential function $R(n) = 1.1^n \ldots$



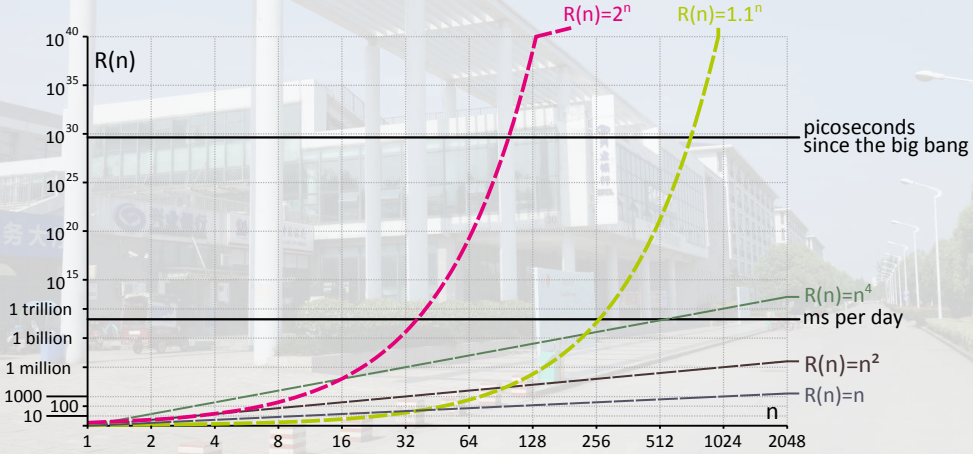
Exact vs. Heuristic Algorithms

- In optimization, there exist exact and heuristic algorithms.
- Let's look at the classical Traveling Salesperson Problem (TSP)[6,113,153,157,251,261].
- A runtime of 1.1^n becomes infeasible for $n > 512$.



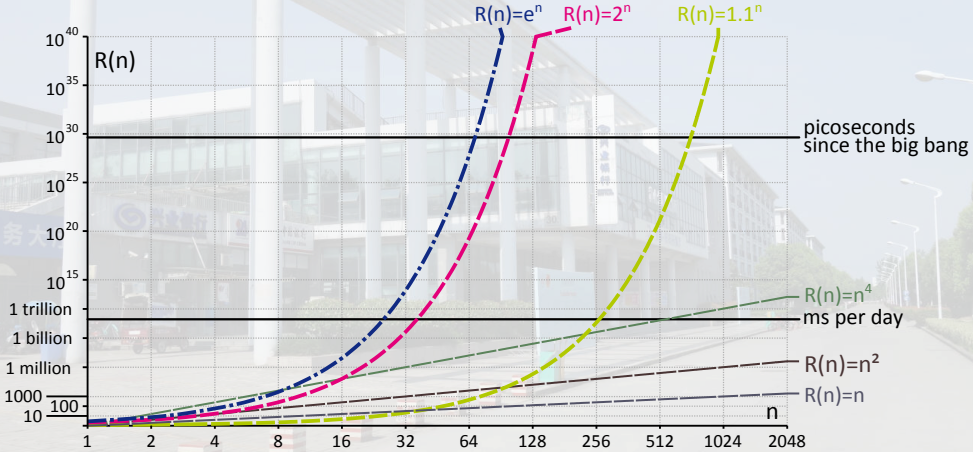
Exact vs. Heuristic Algorithms

- In optimization, there exist exact and heuristic algorithms.
- Let's look at the classical Traveling Salesperson Problem (TSP)[6,113,153,157,251,261].
- For larger bases, the runtime grows even faster.



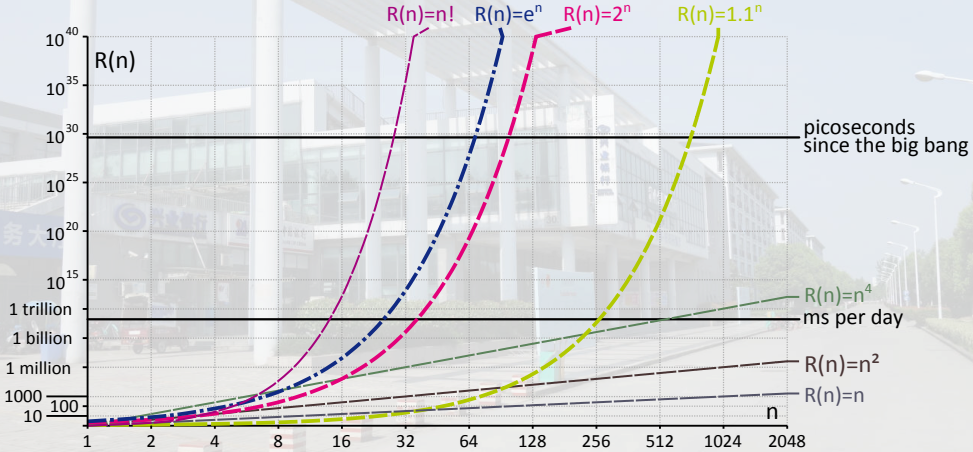
Exact vs. Heuristic Algorithms

- In optimization, there exist exact and heuristic algorithms.
- Let's look at the classical Traveling Salesperson Problem (TSP)[6,113,153,157,251,261].
- For larger bases, the runtime grows even faster.



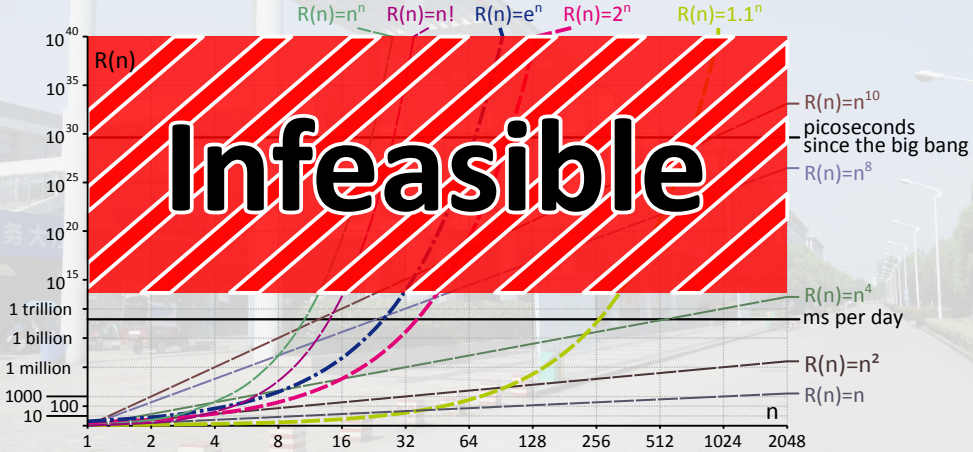
Exact vs. Heuristic Algorithms

- In optimization, there exist exact and heuristic algorithms.
- Let's look at the classical Traveling Salesperson Problem (TSP)[6,113,153,157,251,261].
- If we would enumerate all possible tours of n cities in a TSP, that would be $n!$.



Exact vs. Heuristic Algorithms

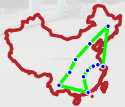
- In optimization, there exist exact and heuristic algorithms.
- Let's look at the classical Traveling Salesperson Problem (TSP)[6,113,153,157,251,261].
- If we would enumerate all possible tours of n cities in a TSP, that would be $n!$.



Exact vs. Heuristic Algorithms

- In optimization, there exist exact and heuristic algorithms.
- Let's look at the classical Traveling Salesperson Problem (TSP)[6,113,153,157,251,261].

getting the optimal solution
for a TSP may take too long



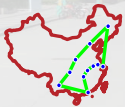
consumed runtime: very much / too (?) long

Exact vs. Heuristic Algorithms

- In optimization, there exist exact and heuristic algorithms.
- Let's look at the classical Traveling Salesperson Problem (TSP)[6,113,153,157,251,261].
  - But we can find just some tour very quickly.



getting the optimal solution
for a TSP may take too long



very little / fast — consumed runtime — very much / too (?) long

Exact vs. Heuristic Algorithms

- In optimization, there exist exact and heuristic algorithms.
- Let's look at the classical Traveling Salesperson Problem (TSP)[6,113,153,157,251,261].
  - But we can find just some tour very quickly.
  - Of course the quality of that tour will be lower: the tour will be longer than the best one.



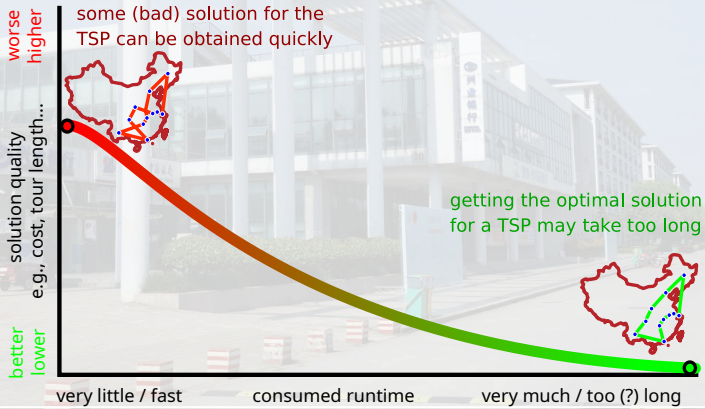
Exact vs. Heuristic Algorithms

- In optimization, there exist exact and heuristic algorithms.
- Let's look at the classical Traveling Salesperson Problem (TSP)[6,113,153,157,251,261].
  - Of course the quality of that tour will be lower.



Exact vs. Heuristic Algorithms

- In optimization, there exist exact and heuristic algorithms.
- Let's look at the classical Traveling Salesperson Problem (TSP)[6,113,153,157,251,261].
  - Is there something in between?



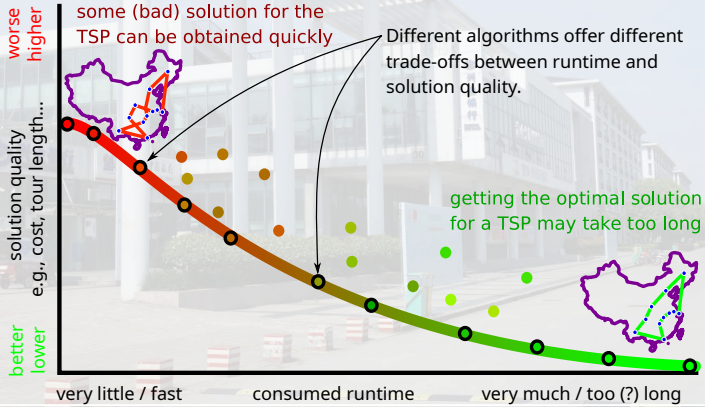
Exact vs. Heuristic Algorithms

- In optimization, there exist exact and heuristic algorithms.
- Let's look at the classical Traveling Salesperson Problem (TSP)[6,113,153,157,251,261].
  - Is there something in between?
  - (Meta-)Heuristic optimization algorithms try to find solutions which are as good as possible as fast as possible.



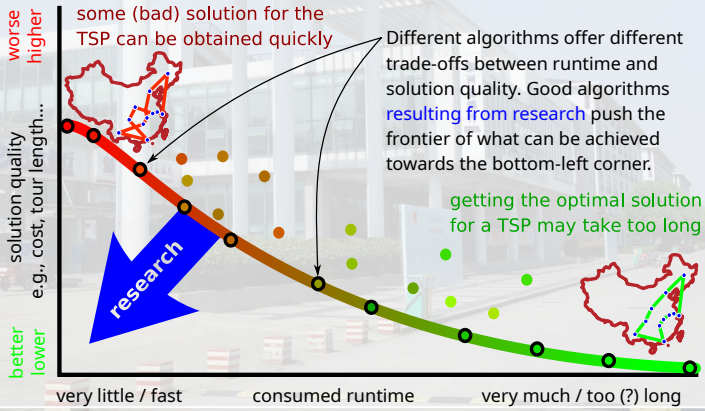
Exact vs. Heuristic Algorithms

- In optimization, there exist exact and heuristic algorithms.
- Let's look at the classical Traveling Salesperson Problem (TSP)[6,113,153,157,251,261].
  - (Meta-)Heuristic optimization algorithms try to find solutions which are as good as possible as fast as possible.
  - Optimization often means to make a trade-off between solution quality and runtime.



Exact vs. Heuristic Algorithms

- In optimization, there exist exact and heuristic algorithms.
- Let's look at the classical Traveling Salesperson Problem (TSP)[6,113,153,157,251,261].
  - Optimization often means to make a trade-off between solution quality and runtime.



Views on Performance and Time



Views on Performance

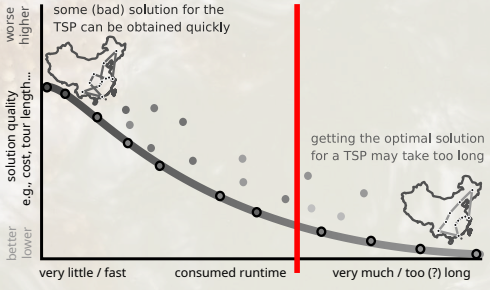
- Runtime and solution quality in optimization are intertwined and should never be considered separately.

Views on Performance

- Runtime and solution quality in optimization are intertwined and should never be considered separately.
- There are two main views on what performance is[115,116,254,256].

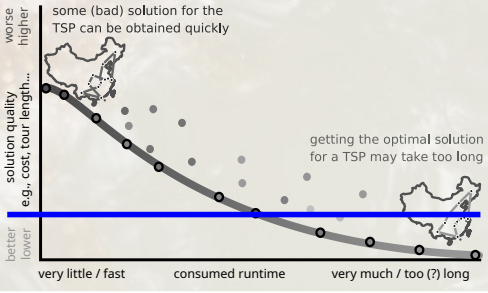
Views on Performance

- Runtime and solution quality in optimization are intertwined and should never be considered separately.
- There are two main views on what performance is[115,116,254,256]:
  1. Solution quality reached after a certain runtime



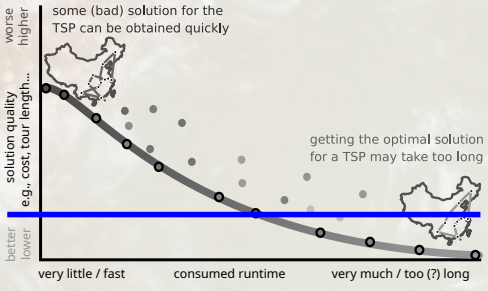
Views on Performance

- Runtime and solution quality in optimization are intertwined and should never be considered separately.
- There are two main views on what performance is[115,116,254,256]:
  1. Solution quality reached after a certain runtime
  2. Runtime to reach a certain solution quality



Views on Performance

- Runtime and solution quality in optimization are intertwined and should never be considered separately.
- There are two main views on what performance is[115,116,254,256]:
  1. Solution quality reached after a certain runtime
  2. Runtime to reach a certain solution quality

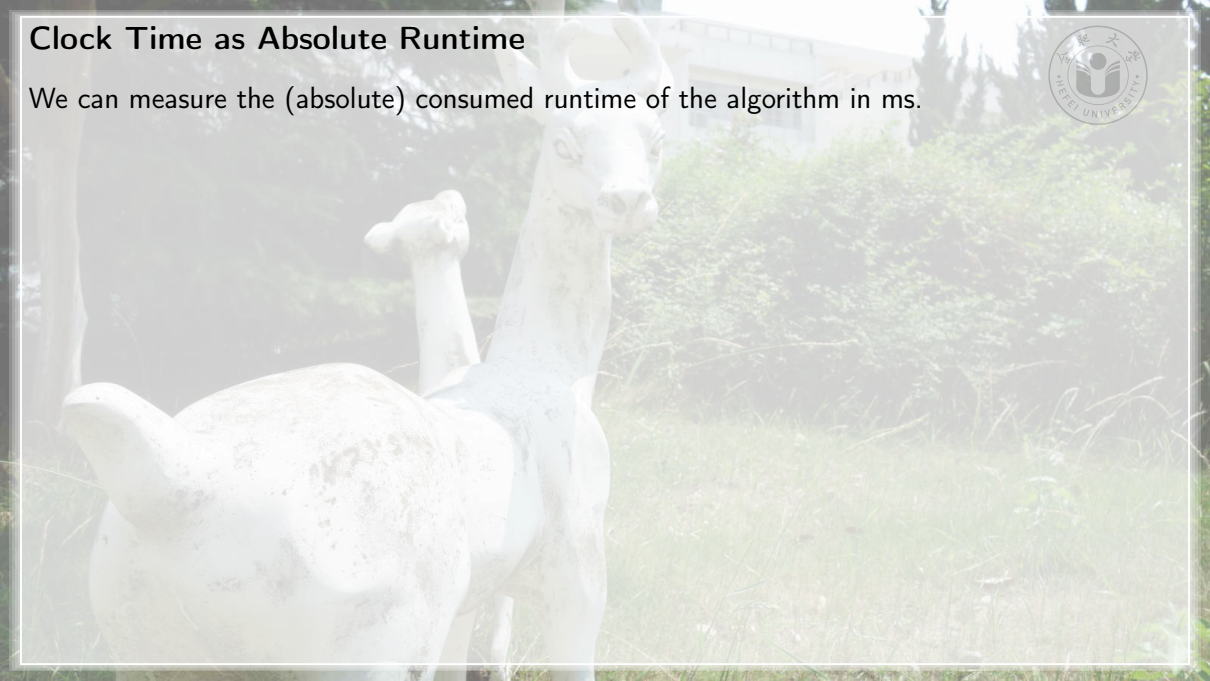


What is Runtime?

- What actually is runtime?

Clock Time as Absolute Runtime

We can measure the (absolute) consumed runtime of the algorithm in ms.



Clock Time as Absolute Runtime

We can measure the (absolute) consumed runtime of the algorithm implementation in ms.



Clock Time as Absolute Runtime

We can measure the (absolute) consumed runtime of the algorithm implementation in ms.

- Advantages

Clock Time as Absolute Runtime

We can measure the (absolute) consumed runtime of the algorithm implementation in ms.

- Advantages:
  - Results in many works reported in this format

Clock Time as Absolute Runtime

We can measure the (absolute) consumed runtime of the algorithm implementation in ms.

- Advantages:
  - Results in many works reported in this format
  - A quantity that makes physical sense

Clock Time as Absolute Runtime

We can measure the (absolute) consumed runtime of the algorithm implementation in ms.

- Advantages:
  - Results in many works reported in this format
  - A quantity that makes physical sense
  - Includes all "hidden complexities" of an algorithm implementation (memory management, matrix operations, data structures, ...)

Clock Time as Absolute Runtime

We can measure the (absolute) consumed runtime of the algorithm implementation in ms.

- Advantages:
  - Results in many works reported in this format
  - A quantity that makes physical sense
  - Includes all "hidden complexities" of an algorithm implementation (memory management, matrix operations, data structures, ...)
- Disadvantages

Clock Time as Absolute Runtime



We can measure the (absolute) consumed runtime of the algorithm implementation in ms.

- Advantages:
  - Results in many works reported in this format
  - A quantity that makes physical sense
  - Includes all "hidden complexities" of an algorithm implementation (memory management, matrix operations, data structures, ...)
- Disadvantages:
  - Strongly machine dependent and inherently incomparable over different machines

Clock Time as Absolute Runtime

We can measure the (absolute) consumed runtime of the algorithm implementation in ms.

- Advantages:
  - Results in many works reported in this format
  - A quantity that makes physical sense
  - Includes all "hidden complexities" of an algorithm implementation (memory management, matrix operations, data structures, ...)
- Disadvantages:
  - Strongly machine dependent and inherently incomparable over different machines
  - Measurements are only valuable for a few years

Clock Time as Absolute Runtime



We can measure the (absolute) consumed runtime of the algorithm implementation in ms.

- Advantages:
  - Results in many works reported in this format
  - A quantity that makes physical sense
  - Includes all "hidden complexities" of an algorithm implementation (memory management, matrix operations, data structures, ...)
- Disadvantages:
  - Strongly machine dependent and inherently incomparable over different machines
  - Measurements are only valuable for a few years
  - Can be biased by "outside effects," e.g., OS, scheduling, other processes, I/O, swapping, ...

Clock Time as Absolute Runtime



We can measure the (absolute) consumed runtime of the algorithm implementation in ms.

- Advantages:
  - Results in many works reported in this format
  - A quantity that makes physical sense
  - Includes all "hidden complexities" of an algorithm implementation (memory management, matrix operations, data structures, ...)
- Disadvantages:
  - Strongly machine dependent and inherently incomparable over different machines
  - Measurements are only valuable for a few years
  - Can be biased by "outside effects," e.g., OS, scheduling, other processes, I/O, swapping, ...
- Hardware, software, OS, programming language, etc. all have nothing to do with the optimization algorithm itself and are relevant only in a specific application...

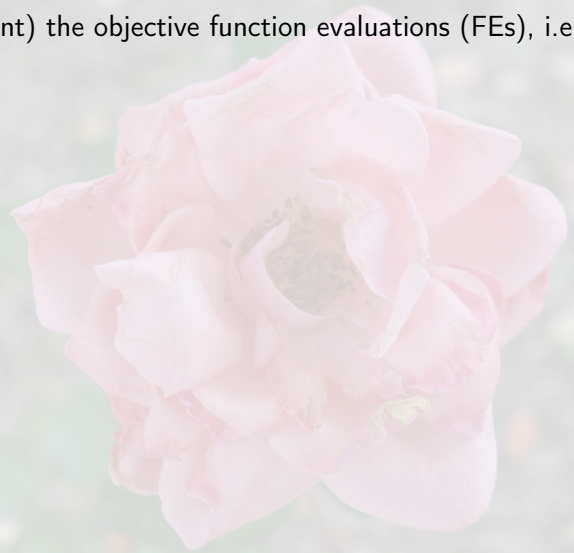
Clock Time as Absolute Runtime



We can measure the (absolute) consumed runtime of the algorithm implementation in ms.

- Advantages:
  - Results in many works reported in this format
  - A quantity that makes physical sense
  - Includes all "hidden complexities" of an algorithm implementation (memory management, matrix operations, data structures, ...)
- Disadvantages:
  - Strongly machine dependent and inherently incomparable over different machines
  - Measurements are only valuable for a few years
  - Can be biased by "outside effects," e.g., OS, scheduling, other processes, I/O, swapping, ...
- Hardware, software, OS, programming language, etc. all have nothing to do with the optimization algorithm itself and are relevant only in a specific application...
- ...for research they may be less interesting, while for a specific application they do matter.

Objective Function Evaluations: FEs



We can measure (count) the objective function evaluations (FEs), i.e., the number of tested candidate solutions.

Objective Function Evaluations: FEs



We can measure (count) the objective function evaluations (FEs), i.e., the number of tested candidate solutions.

- Advantages

Objective Function Evaluations: FEs



We can measure (count) the objective function evaluations (FEs), i.e., the number of tested candidate solutions.

- Advantages:
  - Results in many works reported in this format (or FEs can be deduced)

Objective Function Evaluations: FEs



We can measure (count) the objective function evaluations (FEs), i.e., the number of tested candidate solutions.

- Advantages:
  - Results in many works reported in this format (or FEs can be deduced)
  - Machine-independent, theory-related measure

Objective Function Evaluations: FEs



We can measure (count) the objective function evaluations (FEs), i.e., the number of tested candidate solutions.

- Advantages:
  - Results in many works reported in this format (or FEs can be deduced)
  - Machine-independent, theory-related measure
  - Cannot be influenced by "outside effects"

Objective Function Evaluations: FEs



We can measure (count) the objective function evaluations (FEs), i.e., the number of tested candidate solutions.

- Advantages:
  - Results in many works reported in this format (or FEs can be deduced)
  - Machine-independent, theory-related measure
  - Cannot be influenced by "outside effects"
  - In many optimization problems, computing the objective value is the most time consuming task

Objective Function Evaluations: FEs

We can measure (count) the objective function evaluations (FEs), i.e., the number of tested candidate solutions.

- Advantages:
  - Results in many works reported in this format (or FEs can be deduced)
  - Machine-independent, theory-related measure
  - Cannot be influenced by "outside effects"
  - In many optimization problems, computing the objective value is the most time consuming task
- Disadvantages

Objective Function Evaluations: FEs



We can measure (count) the objective function evaluations (FEs), i.e., the number of tested candidate solutions.

- Advantages:
  - Results in many works reported in this format (or FEs can be deduced)
  - Machine-independent, theory-related measure
  - Cannot be influenced by "outside effects"
  - In many optimization problems, computing the objective value is the most time consuming task
- Disadvantages:
  - No clear relationship to real runtime

Objective Function Evaluations: FEs

We can measure (count) the objective function evaluations (FEs), i.e., the number of tested candidate solutions.

- Advantages:
  - Results in many works reported in this format (or FEs can be deduced)
  - Machine-independent, theory-related measure
  - Cannot be influenced by "outside effects"
  - In many optimization problems, computing the objective value is the most time consuming task
- Disadvantages:
  - No clear relationship to real runtime
  - Does not contain "hidden complexities" of algorithm

Objective Function Evaluations: FEs

We can measure (count) the objective function evaluations (FEs), i.e., the number of tested candidate solutions.

- Advantages:
  - Results in many works reported in this format (or FEs can be deduced)
  - Machine-independent, theory-related measure
  - Cannot be influenced by "outside effects"
  - In many optimization problems, computing the objective value is the most time consuming task
- Disadvantages:
  - No clear relationship to real runtime
  - Does not contain "hidden complexities" of algorithm
  - 1 FE: very different costs in different situations![251]

Objective Function Evaluations: FEs



We can measure (count) the objective function evaluations (FEs), i.e., the number of tested candidate solutions.

- Advantages:
  - Results in many works reported in this format (or FEs can be deduced)
  - Machine-independent, theory-related measure
  - Cannot be influenced by "outside effects"
  - In many optimization problems, computing the objective value is the most time consuming task
- Disadvantages:
  - No clear relationship to real runtime
  - Does not contain "hidden complexities" of algorithm
  - 1 FE: very different costs in different situations![251]
    - When applying a local search that swaps two cities in each move to the TSP, 1 FE can be done in $\mathcal{O}(1)$[251].

Objective Function Evaluations: FEs

We can measure (count) the objective function evaluations (FEs), i.e., the number of tested candidate solutions.

- Advantages:
  - Results in many works reported in this format (or FEs can be deduced)
  - Machine-independent, theory-related measure
  - Cannot be influenced by "outside effects"
  - In many optimization problems, computing the objective value is the most time consuming task
- Disadvantages:
  - No clear relationship to real runtime
  - Does not contain "hidden complexities" of algorithm
  - 1 FE: very different costs in different situations![251]
    - When applying a local search that swaps two cities in each move to the TSP, 1 FE can be done in $\mathcal{O}(1)$[251].
    - When applying Ant Colony Optimization (ACO) instead, each FE takes $\mathcal{O}(n^2)$[251].

Objective Function Evaluations: FEs

We can measure (count) the objective function evaluations (FEs), i.e., the number of tested candidate solutions.

- Advantages:
  - Results in many works reported in this format (or FEs can be deduced)
  - Machine-independent, theory-related measure
  - Cannot be influenced by "outside effects"
  - In many optimization problems, computing the objective value is the most time consuming task
- Disadvantages:
  - No clear relationship to real runtime
  - Does not contain "hidden complexities" of algorithm
  - 1 FE: very different costs in different situations![251]
    - When applying a local search that swaps two cities in each move to the TSP, 1 FE can be done in $\mathcal{O}(1)$[251].
    - When applying Ant Colony Optimization (ACO) instead, each FE takes $\mathcal{O}(n^2)$[251].
- Relevant for comparing algorithms, but not so much for the practical application or comparing implementations.

Do not count generations

- In an evolutionary algorithm (EA)[10,247], in each generation (= iteration), a set of new solution is created and evaluated.

Do not count generations



- In an evolutionary algorithm (EA)[10,247], in each generation (= iteration), a set of new solution is created and evaluated.
- Traditionally, the number of generations passed until some goal was reached was used in the EA community.

Do not count generations



- In an evolutionary algorithm (EA)[10,247], in each generation (= iteration), a set of new solution is created and evaluated.
- Traditionally, the number of generations passed until some goal was reached was used in the EA community.
- Do not use the number of *generations* in your EA as time measure! Instead count the FEs.

Do not count generations



- In an evolutionary algorithm (EA)[10,247], in each generation (= iteration), a set of new solution is created and evaluated.
- Traditionally, the number of generations passed until some goal was reached was used in the EA community.
- Do not use the number of *generations* in your EA as time measure! Instead count the FEs, because:
- The "number of generations" are not really comparable for different population sizes or with algorithms that do not use populations.

Do not count generations



- In an evolutionary algorithm (EA)[10,247], in each generation (= iteration), a set of new solution is created and evaluated.
- Traditionally, the number of generations passed until some goal was reached was used in the EA community.
- Do not use the number of *generations* in your EA as time measure! Instead count the FEs, because:
- The "number of generations" are not really comparable for different population sizes or with algorithms that do not use populations.
- Often, the mapping between generations and FEs is not clear

Do not count generations



- In an evolutionary algorithm (EA)[10,247], in each generation (= iteration), a set of new solution is created and evaluated.
- Traditionally, the number of generations passed until some goal was reached was used in the EA community.
- Do not use the number of *generations* in your EA as time measure! Instead count the FEs, because:
- The "number of generations" are not really comparable for different population sizes or with algorithms that do not use populations.
- Often, the mapping between generations and FEs is not clear, for example
  - Do you evaluate offspring solutions that are identical to their parents?

Do not count generations



- In an evolutionary algorithm (EA)[10,247], in each generation (= iteration), a set of new solution is created and evaluated.
- Traditionally, the number of generations passed until some goal was reached was used in the EA community.
- Do not use the number of *generations* in your EA as time measure! Instead count the FEs, because:
- The "number of generations" are not really comparable for different population sizes or with algorithms that do not use populations.
- Often, the mapping between generations and FEs is not clear, for example
  - Do you evaluate offspring solutions that are identical to their parents?
  - Is a local search involved that refines some or all solutions in the population?

Do not count generations



- In an evolutionary algorithm (EA)[10,247], in each generation (= iteration), a set of new solution is created and evaluated.
- Traditionally, the number of generations passed until some goal was reached was used in the EA community.
- Do not use the number of *generations* in your EA as time measure! Instead count the FEs, because:
- The "number of generations" are not really comparable for different population sizes or with algorithms that do not use populations.
- Often, the mapping between generations and FEs is not clear, for example
  - Do you evaluate offspring solutions that are identical to their parents?
  - Is a local search involved that refines some or all solutions in the population?
  - In a $(\mu + \lambda)$-EA, is the first population of size $\mu + \lambda$, λ, or μ?

Do not count generations



- In an evolutionary algorithm (EA)[10,247], in each generation (= iteration), a set of new solution is created and evaluated.
- Traditionally, the number of generations passed until some goal was reached was used in the EA community.
- Do not use the number of *generations* in your EA as time measure! Instead count the FEs, because:
- The "number of generations" are not really comparable for different population sizes or with algorithms that do not use populations.
- Often, the mapping between generations and FEs is not clear, for example
  - Do you evaluate offspring solutions that are identical to their parents?
  - Is a local search involved that refines some or all solutions in the population?
  - In a $(\mu + \lambda)$-EA, is the first population of size $\mu + \lambda$, λ, or μ?
  - What if the population size changes adaptively?

Do not count generations



- In an evolutionary algorithm (EA)[10,247], in each generation (= iteration), a set of new solution is created and evaluated.
- Traditionally, the number of generations passed until some goal was reached was used in the EA community.
- Do not use the number of *generations* in your EA as time measure! Instead count the FEs, because:
- The "number of generations" are not really comparable for different population sizes or with algorithms that do not use populations.
- Often, the mapping between generations and FEs is not clear, for example
  - Do you evaluate offspring solutions that are identical to their parents?
  - Is a local search involved that refines some or all solutions in the population?
  - In a $(\mu + \lambda)$-EA, is the first population of size $\mu + \lambda$, λ, or μ?
  - What if the population size changes adaptively?
- I suggest to prefer FEs over generations if you want to count algorithm steps.

Runtime



- I suggest to always measure both the consumed FEs and the runtime in milliseconds.

Runtime



- I suggest to always measure both the consumed FEs and the runtime in milliseconds.
- Anyway, with what we have learned, we can rewrite the two views by choosing a time measure[115,254]

Runtime



- I suggest to always measure both the consumed FEs and the runtime in milliseconds.
- Anyway, with what we have learned, we can rewrite the two views by choosing a time measure[115,254], e.g.:
  1. Solution quality reached after a certain number of FEs

Runtime



- I suggest to always measure both the consumed FEs and the runtime in milliseconds.
- Anyway, with what we have learned, we can rewrite the two views by choosing a time measure[115,254], e.g.:
  1. Solution quality reached after a certain number of FEs
  2. Milliseconds needed to reach a certain solution quality

Solution Quality

- Common measure of solution quality: Objective function value of best solution discovered.

Solution Quality

- Common measure of solution quality: Objective function value of best solution discovered.
- Rewrite the two views[115,254]

Solution Quality



- Common measure of solution quality: Objective function value of best solution discovered.
- Rewrite the two views[115,254]:
  1. Best objective function value reached after a certain number of milliseconds

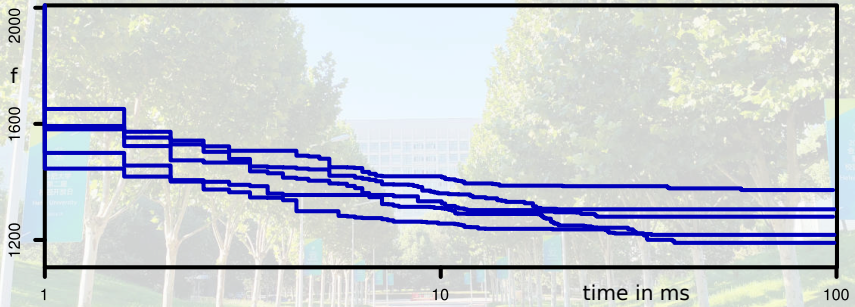
Solution Quality



- Common measure of solution quality: Objective function value of best solution discovered.
- Rewrite the two views[115,254]:
  1. Best objective function value reached after a certain number of milliseconds
  2. Number of FEs needed to reach a certain objective function value

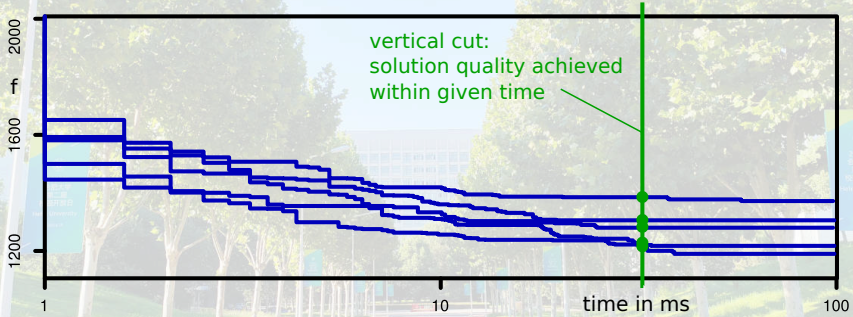
Views on Performance

- Which one is the "better" view on performance?



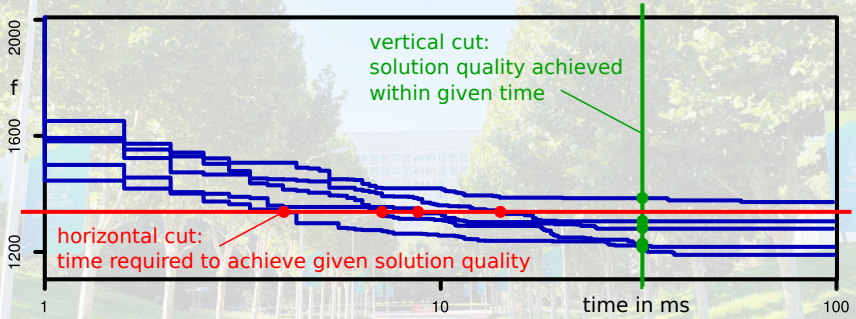
Views on Performance

- Which one is the "better" view on performance?
  1. Best objective function value reached after a certain number of FEs



Views on Performance

- Which one is the "better" view on performance?
  1. Best objective function value reached after a certain number of FEs
  2. Number of FEs needed to reach a certain objective function value



Views on Performance

- Which one is the "better" view on performance?
  1. Best objective function value reached after a certain number of FEs
  2. Number of FEs needed to reach a certain objective function value
- This question is still debated in research...

Which view is better?

- Number of FEs needed to reach a certain objective function value
- Preferred by, e.g., the BBOB/COCO benchmark suite[115]

Which view is better?

- Number of FEs needed to reach a certain objective function value
- Preferred by, e.g., the BBOB/COCO benchmark suite[115]:
  - Measuring the time needed to reach a target function value allows meaningful statements such as "Algorithm A is two/ten/hundred times faster than Algorithm B in solving this problem."

Which view is better?



- Number of FEs needed to reach a certain objective function value
- Preferred by, e.g., the BBOB/COCO benchmark suite[115]:
  - Measuring the time needed to reach a target function value allows meaningful statements such as "Algorithm A is two/ten/hundred times faster than Algorithm B in solving this problem."
  - However, there is no interpretable meaning to the fact that Algorithm A reaches a function value that is two/ten/hundred times smaller than the one reached by Algorithm B.

Which view is better?



- Number of FEs needed to reach a certain objective function value
- Preferred by, e.g., the BBOB/COCO benchmark suite[115]:
  - Measuring the time needed to reach a target function value allows meaningful statements such as "Algorithm A is two/ten/hundred times faster than Algorithm B in solving this problem."
  - However, there is no interpretable meaning to the fact that Algorithm A reaches a function value that is two/ten/hundred times smaller than the one reached by Algorithm B.
  - "Benchmarking Theory Perspective"

Which view is better?

- Number of FEs needed to reach a certain objective function value
- Preferred by, e.g., the BBOB/COCO benchmark suite[115]:
  - Measuring the time needed to reach a target function value allows meaningful statements such as "Algorithm A is two/ten/hundred times faster than Algorithm B in solving this problem."
  - However, there is no interpretable meaning to the fact that Algorithm A reaches a function value that is two/ten/hundred times smaller than the one reached by Algorithm B.
  - "Benchmarking Theory Perspective"
- Sometimes problematic: What if one run does not reach the goal quality?



Which view is better?

- Number of FEs needed to reach a certain objective function value
- Preferred by, e.g., the BBOB/COCO benchmark suite[115]:
  - Measuring the time needed to reach a target function value allows meaningful statements such as "Algorithm A is two/ten/hundred times faster than Algorithm B in solving this problem."
  - However, there is no interpretable meaning to the fact that Algorithm A reaches a function value that is two/ten/hundred times smaller than the one reached by Algorithm B.
  - "Benchmarking Theory Perspective"
- Sometimes problematic: What if one run does not reach the goal quality?
- Then, alternative measures need to be computed, such as the ERT[8,207] or PAR2 and PAR10[30,140].



Which view is better?

- Best objective function value reached after a certain number of FEs

Which view is better?

- Best objective function value reached after a certain number of FEs
- Preferred by many benchmark suites such as[233].

Which view is better?

- Best objective function value reached after a certain number of FEs
- Preferred by many benchmark suites such as[233].
- Practice Perspective: Best results achievable with given time budget wins.

Which view is better?

- Best objective function value reached after a certain number of FEs
- Preferred by many benchmark suites such as[233].
- Practice Perspective: Best results achievable with given time budget wins.
- This perspective maybe less suitable for scientific benchmarking, but surely is useful in practice.

Which view is better?

- Best objective function value reached after a certain number of FEs
- Preferred by many benchmark suites such as[233].
- Practice Perspective: Best results achievable with given time budget wins.
- This perspective maybe less suitable for scientific benchmarking, but surely is useful in practice.
- "How good is the tour for the TSP that we can find in 5 minutes with our algorithm?"

Which view is better?

- Best objective function value reached after a certain number of FEs
- Preferred by many benchmark suites such as[233].
- Practice Perspective: Best results achievable with given time budget wins.
- This perspective maybe less suitable for scientific benchmarking, but surely is useful in practice.
- "How good is the tour for the TSP that we can find in 5 minutes with our algorithm?"
- Always well-defined, because vertical cuts can always be reached.

Views on Performance



- No official consensus on which view is “better.”

Views on Performance



- No official consensus on which view is “better.”
- This also strongly depends on the situation.

Views on Performance



- No official consensus on which view is "better."
- This also strongly depends on the situation.
- If we can actually always solve the problem to a "natural" goal quality (e.g., to optimality), then we should prefer the horizontal cut (time-to-target) method.

Views on Performance



- No official consensus on which view is "better."
- This also strongly depends on the situation.
- If we can actually always solve the problem to a "natural" goal quality (e.g., to optimality), then we should prefer the horizontal cut (time-to-target) method.
- If we have clear application requirements specifying a fixed budget, then we should prefer the fixed-budget approach.

Views on Performance



- No official consensus on which view is "better."
- This also strongly depends on the situation.
- If we can actually always solve the problem to a "natural" goal quality (e.g., to optimality), then we should prefer the horizontal cut (time-to-target) method.
- If we have clear application requirements specifying a fixed budget, then we should prefer the fixed-budget approach.
- Otherwise, the best approach may be: Evaluate algorithm according to both methods.[251,254,256]

Views on Performance



- No official consensus on which view is "better."
- This also strongly depends on the situation.
- If we can actually always solve the problem to a "natural" goal quality (e.g., to optimality), then we should prefer the horizontal cut (time-to-target) method.
- If we have clear application requirements specifying a fixed budget, then we should prefer the fixed-budget approach.
- Otherwise, the best approach may be: Evaluate algorithm according to both methods.[251,254,256]
- Maybe cast a net of several horizontal and vertical cuts, to get a better picture...

Determining Target Values



- How to determine the right maximum FEs or target function values?

Determining Target Values

- How to determine the right maximum FEs or target function values?
  1. from the constraints of a practical application

Determining Target Values

- How to determine the right maximum FEs or target function values?
  1. from the constraints of a practical application
  2. from studies in literature regarding similar or the same problem

Determining Target Values

- How to determine the right maximum FEs or target function values?
  1. from the constraints of a practical application
  2. from studies in literature regarding similar or the same problem
  3. from simple or well-known algorithms

Determining Target Values

- How to determine the right maximum FEs or target function values?
  1. from the constraints of a practical application
  2. from studies in literature regarding similar or the same problem
  3. from simple or well-known algorithms
  4. from experience

Determining Target Values

- How to determine the right maximum FEs or target function values?
  1. from the constraints of a practical application
  2. from studies in literature regarding similar or the same problem
  3. from simple or well-known algorithms
  4. from experience
  5. from prior, small-scale experiments

Determining Target Values

- How to determine the right maximum FEs or target function values?
  1. from the constraints of a practical application
  2. from studies in literature regarding similar or the same problem
  3. from simple or well-known algorithms
  4. from experience
  5. from prior, small-scale experiments
  6. based on known results or well-accepted bounds

Statistical Measures



Problem Instances and Randomized Algorithms



- For each optimization problem (like the TSP) there are several instances (e.g., different sets of cities that need to be visited).

Problem Instances and Randomized Algorithms



- For each optimization problem (like the TSP) there are several instances (e.g., different sets of cities that need to be visited).
  - Some instances will be easy, some will be hard.

Problem Instances and Randomized Algorithms



- For each optimization problem (like the TSP) there are several instances (e.g., different sets of cities that need to be visited).
  - Some instances will be easy, some will be hard.
  - We always must use multiple different problem instances to get reliable results.

Problem Instances and Randomized Algorithms



- For each optimization problem (like the TSP) there are several instances (e.g., different sets of cities that need to be visited).
  - Some instances will be easy, some will be hard.
  - We always must use multiple different problem instances to get reliable results.
  - Performance indicators need to be computed for each instance and also summarized over several instances.

Problem Instances and Randomized Algorithms



- For each optimization problem (like the TSP) there are several instances (e.g., different sets of cities that need to be visited).
  - Some instances will be easy, some will be hard.
  - We always must use multiple different problem instances to get reliable results.
  - Performance indicators need to be computed for each instance and also summarized over several instances.
- Special situation: Randomized Algorithms

Problem Instances and Randomized Algorithms



- For each optimization problem (like the TSP) there are several instances (e.g., different sets of cities that need to be visited).
  - Some instances will be easy, some will be hard.
  - We always must use multiple different problem instances to get reliable results.
  - Performance indicators need to be computed for each instance and also summarized over several instances.
- Special situation: Randomized Algorithms:
  - Performance values cannot be given as an “absolute” value!

Problem Instances and Randomized Algorithms



- For each optimization problem (like the TSP) there are several instances (e.g., different sets of cities that need to be visited).
  - Some instances will be easy, some will be hard.
  - We always must use multiple different problem instances to get reliable results.
  - Performance indicators need to be computed for each instance and also summarized over several instances.
- Special situation: Randomized Algorithms:
  - Performance values cannot be given as an "absolute" value!
  - 1 run = 1 application of an optimization algorithm to a problem, runs are independent from all prior runs.

Problem Instances and Randomized Algorithms



- For each optimization problem (like the TSP) there are several instances (e.g., different sets of cities that need to be visited).
  - Some instances will be easy, some will be hard.
  - We always must use multiple different problem instances to get reliable results.
  - Performance indicators need to be computed for each instance and also summarized over several instances.
- Special situation: Randomized Algorithms:
  - Performance values cannot be given as an "absolute" value!
  - 1 run = 1 application of an optimization algorithm to a problem, runs are independent from all prior runs.
  - Results can be different for each run!

Problem Instances and Randomized Algorithms



- For each optimization problem (like the TSP) there are several instances (e.g., different sets of cities that need to be visited).
  - Some instances will be easy, some will be hard.
  - We always must use multiple different problem instances to get reliable results.
  - Performance indicators need to be computed for each instance and also summarized over several instances.
- Special situation: Randomized Algorithms:
  - Performance values cannot be given as an "absolute" value!
  - 1 run = 1 application of an optimization algorithm to a problem, runs are independent from all prior runs.
  - Results can be different for each run!
  - Executing a randomized algorithm one time does not give reliable information.

Problem Instances and Randomized Algorithms



- For each optimization problem (like the TSP) there are several instances (e.g., different sets of cities that need to be visited).
  - Some instances will be easy, some will be hard.
  - We always must use multiple different problem instances to get reliable results.
  - Performance indicators need to be computed for each instance and also summarized over several instances.
- Special situation: Randomized Algorithms:
  - Performance values cannot be given as an "absolute" value!
  - 1 run = 1 application of an optimization algorithm to a problem, runs are independent from all prior runs.
  - Results can be different for each run!
  - Executing a randomized algorithm one time does not give reliable information.
  - Statistical evaluation over sets of runs necessary.

Important Distinction

- Crucial Difference: distribution and sample

Important Distinction

- Crucial Difference: distribution and sample
- A sample is what we *measure*.

Important Distinction

- Crucial Difference: distribution and sample
- A sample is what we *measure*.
- A distribution is the asymptotic result of the ideal process.

Important Distinction



- Crucial Difference: distribution and sample
- A sample is what we *measure*.
- A distribution is the asymptotic result of the ideal process.
- Statistical parameters of the distribution can be estimated from a sample.

Important Distinction

- Crucial Difference: distribution and sample
- A sample is what we *measure*.
- A distribution is the asymptotic result of the ideal process.
- Statistical parameters of the distribution can be estimated from a sample.
- Example: Dice Throw



Important Distinction

- Crucial Difference: distribution and sample
- A sample is what we *measure*.
- A distribution is the asymptotic result of the ideal process.
- Statistical parameters of the distribution can be estimated from a sample.
- Example: Dice Throw
- How likely is it to roll a 1, 2, 3, 4, 5, or 6?



Important Distinction

# throws	number	f(1)	f(2)	f(3)	f(4)	f(5)	f(6)
1	5	0.0000	0.0000	0.0000	0.0000	1.0000	0.0000



Important Distinction

# throws	number	f(1)	f(2)	f(3)	f(4)	f(5)	f(6)
1	5	0.0000	0.0000	0.0000	0.0000	1.0000	0.0000
2	4	0.0000	0.0000	0.0000	0.5000	0.5000	0.0000



Important Distinction

# throws	number	f(1)	f(2)	f(3)	f(4)	f(5)	f(6)
1	5	0.0000	0.0000	0.0000	0.0000	1.0000	0.0000
2	4	0.0000	0.0000	0.0000	0.5000	0.5000	0.0000
3	1	0.3333	0.0000	0.0000	0.3333	0.3333	0.0000



Important Distinction

# throws	number	f(1)	f(2)	f(3)	f(4)	f(5)	f(6)
1	5	0.0000	0.0000	0.0000	0.0000	1.0000	0.0000
2	4	0.0000	0.0000	0.0000	0.5000	0.5000	0.0000
3	1	0.3333	0.0000	0.0000	0.3333	0.3333	0.0000
4	4	0.2500	0.0000	0.0000	0.5000	0.2500	0.0000



Important Distinction

# throws	number	f(1)	f(2)	f(3)	f(4)	f(5)	f(6)
1	5	0.0000	0.0000	0.0000	0.0000	1.0000	0.0000
2	4	0.0000	0.0000	0.0000	0.5000	0.5000	0.0000
3	1	0.3333	0.0000	0.0000	0.3333	0.3333	0.0000
4	4	0.2500	0.0000	0.0000	0.5000	0.2500	0.0000
5	3	0.2000	0.0000	0.2000	0.4000	0.2000	0.0000



Important Distinction

# throws	number	f(1)	f(2)	f(3)	f(4)	f(5)	f(6)
1	5	0.0000	0.0000	0.0000	0.0000	1.0000	0.0000
2	4	0.0000	0.0000	0.0000	0.5000	0.5000	0.0000
3	1	0.3333	0.0000	0.0000	0.3333	0.3333	0.0000
4	4	0.2500	0.0000	0.0000	0.5000	0.2500	0.0000
5	3	0.2000	0.0000	0.2000	0.4000	0.2000	0.0000
6	3	0.1667	0.0000	0.3333	0.3333	0.1667	0.0000



Important Distinction

# throws	number	f(1)	f(2)	f(3)	f(4)	f(5)	f(6)
1	5	0.0000	0.0000	0.0000	0.0000	1.0000	0.0000
2	4	0.0000	0.0000	0.0000	0.5000	0.5000	0.0000
3	1	0.3333	0.0000	0.0000	0.3333	0.3333	0.0000
4	4	0.2500	0.0000	0.0000	0.5000	0.2500	0.0000
5	3	0.2000	0.0000	0.2000	0.4000	0.2000	0.0000
6	3	0.1667	0.0000	0.3333	0.3333	0.1667	0.0000
7	2	0.1429	0.1429	0.2857	0.2857	0.1429	0.0000
8	1	0.2500	0.1250	0.2500	0.2500	0.1250	0.0000
9	4	0.2222	0.1111	0.2222	0.3333	0.1111	0.0000
10	2	0.2000	0.2000	0.2000	0.3000	0.1000	0.0000



Important Distinction

# throws	number	f(1)	f(2)	f(3)	f(4)	f(5)	f(6)
1	5	0.0000	0.0000	0.0000	0.0000	1.0000	0.0000
2	4	0.0000	0.0000	0.0000	0.5000	0.5000	0.0000
3	1	0.3333	0.0000	0.0000	0.3333	0.3333	0.0000
4	4	0.2500	0.0000	0.0000	0.5000	0.2500	0.0000
5	3	0.2000	0.0000	0.2000	0.4000	0.2000	0.0000
6	3	0.1667	0.0000	0.3333	0.3333	0.1667	0.0000
7	2	0.1429	0.1429	0.2857	0.2857	0.1429	0.0000
8	1	0.2500	0.1250	0.2500	0.2500	0.1250	0.0000
9	4	0.2222	0.1111	0.2222	0.3333	0.1111	0.0000
10	2	0.2000	0.2000	0.2000	0.3000	0.1000	0.0000
11	6	0.1818	0.1818	0.1818	0.2727	0.0909	0.0909
12	3	0.1667	0.1667	0.2500	0.2500	0.0833	0.0833



Important Distinction

# throws	number	f(1)	f(2)	f(3)	f(4)	f(5)	f(6)
1	5	0.0000	0.0000	0.0000	0.0000	1.0000	0.0000
2	4	0.0000	0.0000	0.0000	0.5000	0.5000	0.0000
3	1	0.3333	0.0000	0.0000	0.3333	0.3333	0.0000
4	4	0.2500	0.0000	0.0000	0.5000	0.2500	0.0000
5	3	0.2000	0.0000	0.2000	0.4000	0.2000	0.0000
6	3	0.1667	0.0000	0.3333	0.3333	0.1667	0.0000
7	2	0.1429	0.1429	0.2857	0.2857	0.1429	0.0000
8	1	0.2500	0.1250	0.2500	0.2500	0.1250	0.0000
9	4	0.2222	0.1111	0.2222	0.3333	0.1111	0.0000
10	2	0.2000	0.2000	0.2000	0.3000	0.1000	0.0000
11	6	0.1818	0.1818	0.1818	0.2727	0.0909	0.0909
12	3	0.1667	0.1667	0.2500	0.2500	0.0833	0.0833
100	...	0.1900	0.2100	0.1500	0.1600	0.1200	0.1700
1'000	...	0.1700	0.1670	0.1620	0.1670	0.1570	0.1770
10'000	...	0.1682	0.1699	0.1680	0.1661	0.1655	0.1623
100'000	...	0.1671	0.1649	0.1664	0.1676	0.1668	0.1672
1'000'000	...	0.1673	0.1663	0.1662	0.1673	0.1666	0.1664



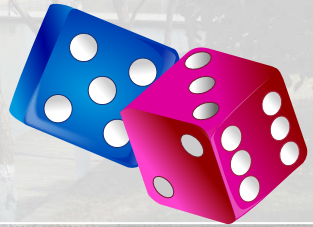
Important Distinction

# throws	number	f(1)	f(2)	f(3)	f(4)	f(5)	f(6)
1	5	0.0000	0.0000	0.0000	0.0000	1.0000	0.0000
2	4	0.0000	0.0000	0.0000	0.5000	0.5000	0.0000
3	1	0.3333	0.0000	0.0000	0.3333	0.3333	0.0000
4	4	0.2500	0.0000	0.0000	0.5000	0.2500	0.0000
5	3	0.2000	0.0000	0.2000	0.4000	0.2000	0.0000
6	3	0.1667	0.0000	0.3333	0.3333	0.1667	0.0000
7	2	0.1429	0.1429	0.2857	0.2857	0.1429	0.0000
8	1	0.2500	0.1250	0.2500	0.2500	0.1250	0.0000
9	4	0.2222	0.1111	0.2222	0.3333	0.1111	0.0000
10	2	0.2000	0.2000	0.2000	0.3000	0.1000	0.0000
11	6	0.1818	0.1818	0.1818	0.2727	0.0909	0.0909
12	3	0.1667	0.1667	0.2500	0.2500	0.0833	0.0833
100	...	0.1900	0.2100	0.1500	0.1600	0.1200	0.1700
1'000	...	0.1700	0.1670	0.1620	0.1670	0.1570	0.1770
10'000	...	0.1682	0.1699	0.1680	0.1661	0.1655	0.1623
100'000	...	0.1671	0.1649	0.1664	0.1676	0.1668	0.1672
1'000'000	...	0.1673	0.1663	0.1662	0.1673	0.1666	0.1664
10'000'000	...	0.1667	0.1667	0.1666	0.1668	0.1667	0.1665
100'000'000	...	0.1667	0.1666	0.1666	0.1667	0.1667	0.1667
1'000'000'000	...	0.1667	0.1667	0.1667	0.1667	0.1667	0.1667



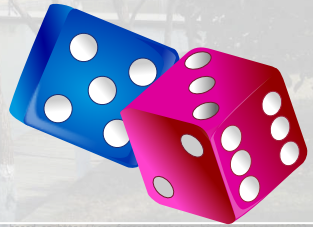
Important Distinction

- Crucial Difference: distribution and sample
- A sample is what we *measure*.
- A distribution is the asymptotic result of the ideal process.
- Statistical parameters of the distribution can be estimated from a sample.
- Example: Dice Throw
- How likely is it to roll a 1, 2, 3, 4, 5, or 6?
- All statistically determined parameters are just estimates based on measurements.



Important Distinction

- Crucial Difference: distribution and sample
- A sample is what we *measure*.
- A distribution is the asymptotic result of the ideal process.
- Statistical parameters of the distribution can be estimated from a sample.
- Example: Dice Throw
- How likely is it to roll a 1, 2, 3, 4, 5, or 6?
- All statistically determined parameters are just estimates based on measurements.
- The parameters of a random process cannot be measured directly, but only be estimated from multiple measures.



Statistical Measures
Measures of the Average

Measures of the Average

- Assume that we have obtained a sample $A = (a_0, a_1, \ldots, a_{n-1})$ of n observations from an experiment.

Measures of the Average



- Assume that we have obtained a sample $A = (a_0, a_1, \ldots, a_{n-1})$ of n observations from an experiment, e.g., we have measured the qualities a_i of the best discovered solutions of $n = 101$ independent runs of an optimization algorithm.

Measures of the Average



- Assume that we have obtained a sample $A = (a_0, a_1, \ldots, a_{n-1})$ of n observations from an experiment, e.g., we have measured the qualities a_i of the best discovered solutions of $n = 101$ independent runs of an optimization algorithm.
- We usually want to reduce this set of numbers to a single value which can give us an impression of what the "average outcome" (or result quality is).

Measures of the Average



- Assume that we have obtained a sample $A = (a_0, a_1, \ldots, a_{n-1})$ of n observations from an experiment, e.g., we have measured the qualities a_i of the best discovered solutions of $n = 101$ independent runs of an optimization algorithm.
- We usually want to reduce this set of numbers to a single value which can give us an impression of what the "average outcome" (or result quality is).
- Three of the most common options for doing so, for estimating the "center" of a distribution, are the arithmetic mean, the median, and the geometric mean.

Arithmetic Mean

Definition: Arithmetic Mean

The arithmetic mean $\mathrm{mean}(A)$ is an estimate of the expected value of a distribution from which a dataset was sampled.

Arithmetic Mean

Definition: Arithmetic Mean

The arithmetic mean $\mathrm{mean}(A)$ is an estimate of the expected value of a distribution from which a dataset was sampled. It is computed on data sample $A = (a_0, a_1, \ldots, a_{n-1})$ as the sum of all n elements a_i in the sample data A divided by the total number n of values.

Arithmetic Mean

Definition: Arithmetic Mean

The arithmetic mean $\mathrm{mean}(A)$ is an estimate of the expected value of a distribution from which a dataset was sampled. It is computed on data sample $A = (a_0, a_1, \ldots, a_{n-1})$ as the sum of all n elements a_i in the sample data A divided by the total number n of values.

$$\mathrm{mean}(A) = \frac{1}{n} \sum_{i=0}^{n-1} a_i \tag{1}$$

Sample Median

Definition: Median

The median $\mathrm{median}(A)$ is the value separating the bigger-valued half from the smaller-valued half of a data sample.

Sample Median

Definition: Median

The median $\mathrm{median}(A)$ is the value separating the bigger-valued half from the smaller-valued half of a data sample. Its estimate is the value right in the middle of a *sorted* data sample $A = (a_0, a_1, \ldots, a_{n-1})$ where $a_{i-1} \leq a_i \; \forall i \in 1 \ldots (n-1)$.

Sample Median

Definition: Median

The median $\mathrm{median}(A)$ is the value separating the bigger-valued half from the smaller-valued half of a data sample. Its estimate is the value right in the middle of a *sorted* data sample $A = (a_0, a_1, \ldots, a_{n-1})$ where $a_{i-1} \leq a_i \; \forall i \in 1 \ldots (n-1)$.

$$\mathrm{median}(A) = \begin{cases} a_{\frac{n-1}{2}} & \text{if } n \text{ is odd} \\ \frac{1}{2}\left(a_{\frac{n}{2}-1} + a_{\frac{n}{2}}\right) & \text{otherwise} \end{cases} \quad \text{if } a_{i-1} \leq a_i \; \forall i \in 1 \ldots (n-1) \qquad (2)$$

Outliers

- Sometimes the data contains outliers[112,171].

Outliers

- Sometimes the data contains outliers[112,171], i.e., observations which are much different from the other measurements.

Outliers

- Sometimes the data contains outliers[112,171], i.e., observations which are much different from the other measurements.
- They may represent measurement errors or observations which have been been disturbed by unusual effects.

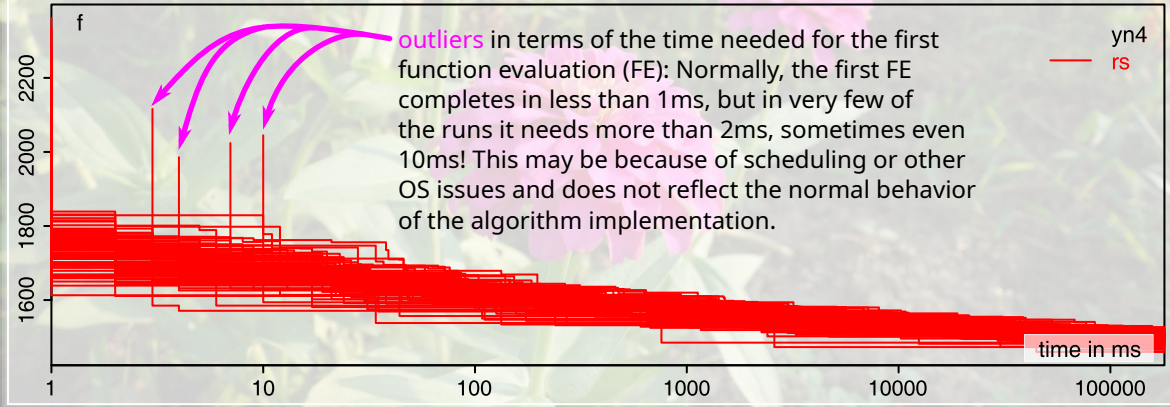
Outliers



- Sometimes the data contains outliers[112,171], i.e., observations which are much different from the other measurements.
- They may represent measurement errors or observations which have been been disturbed by unusual effects.
- For example, maybe the operating system was updating itself during a run of one of our algorithms and, thus, took away some of the computation budget.

Outliers

- For example, maybe the operating system was updating itself during a run of one of our algorithms and, thus, took away some of the computation budget.
- In my experiments here, there are sometimes outliers in the time that it takes to create and evaluate the first candidate solution.



Outliers



- For example, maybe the operating system was updating itself during a run of one of our algorithms and, thus, took away some of the computation budget.
- In my experiments here, there are sometimes outliers in the time that it takes to create and evaluate the first candidate solution.
- But outliers are actually important. So I say this right now. I will also say it again later. But I am afraid that you may tune out during the following example. So remember: Outliers are important. Anyway...

Example for Data Samples w/o Outlier

- Two sets of data samples A and B with $n_a = n_b = 19$ values.

$$
\begin{aligned}
A &= (1, 3, 4, 4, 4, 5, 6, 6, 6, 6, 7, 7, 9, 9, 9, 10, 11, 12, 14) \\
B &= (1, 3, 4, 4, 4, 5, 6, 6, 6, 6, 7, 7, 9, 9, 9, 10, 11, 12, 10'008)
\end{aligned}
$$

Example for Data Samples w/o Outlier

- Two sets of data samples A and B with $n_a = n_b = 19$ values.

$$
\begin{aligned}
A &= (1, 3, 4, 4, 4, 5, 6, 6, 6, 6, 7, 7, 9, 9, 9, 10, 11, 12, 14) \\
B &= (1, 3, 4, 4, 4, 5, 6, 6, 6, 6, 7, 7, 9, 9, 9, 10, 11, 12, 10'008)
\end{aligned}
$$

- We find that

Example for Data Samples w/o Outlier

- Two sets of data samples A and B with $n_a = n_b = 19$ values.

$$
\begin{aligned}
A &= (1, 3, 4, 4, 4, 5, 6, 6, 6, 6, 7, 7, 9, 9, 9, 10, 11, 12, 14) \\
B &= (1, 3, 4, 4, 4, 5, 6, 6, 6, 6, 7, 7, 9, 9, 9, 10, 11, 12, 10'008)
\end{aligned}
$$

- We find that
  - $\mathrm{mean}(A) = \frac{1}{19} \sum_{i=0}^{18} a_i = \frac{133}{19} = 7$

Example for Data Samples w/o Outlier

- Two sets of data samples A and B with $n_a = n_b = 19$ values.

$$
\begin{aligned}
A &= (1, 3, 4, 4, 4, 5, 6, 6, 6, 6, 7, 7, 9, 9, 9, 10, 11, 12, 14) \\
B &= (1, 3, 4, 4, 4, 5, 6, 6, 6, 6, 7, 7, 9, 9, 9, 10, 11, 12, 10'008)
\end{aligned}
$$

- We find that
  - $\text{mean}(A) = \frac{1}{19} \sum_{i=0}^{18} a_i = \frac{133}{19} = 7$ and
  - $\text{mean}(B) = \frac{1}{19} \sum_{i=0}^{18} b_i = \frac{10'127}{19} = 553$

Example for Data Samples w/o Outlier

- Two sets of data samples A and B with $n_a = n_b = 19$ values.

$$A = (1, 3, 4, 4, 4, 5, 6, 6, 6, 6, 7, 7, 9, 9, 9, 10, 11, 12, 14)$$
$$B = (1, 3, 4, 4, 4, 5, 6, 6, 6, 6, 7, 7, 9, 9, 9, 10, 11, 12, 10'008)$$

- We find that
  - $\text{mean}(A) = \frac{1}{19} \sum_{i=0}^{18} a_i = \frac{133}{19} = 7$ and
  - $\text{mean}(B) = \frac{1}{19} \sum_{i=0}^{18} b_i = \frac{10'127}{19} = 553$, while
  - $\text{median}(A) = a_9 = 6$

Example for Data Samples w/o Outlier

- Two sets of data samples A and B with $n_a = n_b = 19$ values.

$$A = (1, 3, 4, 4, 4, 5, 6, 6, 6, 6, 7, 7, 9, 9, 9, 10, 11, 12, 14)$$
$$B = (1, 3, 4, 4, 4, 5, 6, 6, 6, 6, 7, 7, 9, 9, 9, 10, 11, 12, 10'008)$$

- We find that
  - $\text{mean}(A) = \frac{1}{19} \sum_{i=0}^{18} a_i = \frac{133}{19} = 7$ and
  - $\text{mean}(B) = \frac{1}{19} \sum_{i=0}^{18} b_i = \frac{10'127}{19} = 553$, while
  - $\text{median}(A) = a_9 = 6$ and
  - $\text{median}(B) = b_9 = 6$.

Example for Data Samples w/o Outlier

- Two sets of data samples A and B with $n_a = n_b = 19$ values.

$$
\begin{aligned}
A &= (1, 3, 4, 4, 4, 5, 6, 6, 6, 6, 7, 7, 9, 9, 9, 10, 11, 12, 14) \\
B &= (1, 3, 4, 4, 4, 5, 6, 6, 6, 6, 7, 7, 9, 9, 9, 10, 11, 12, 10'008)
\end{aligned}
$$

- We find that
  - $\text{mean}(A) = \frac{1}{19} \sum_{i=0}^{18} a_i = \frac{133}{19} = 7$ and
  - $\text{mean}(B) = \frac{1}{19} \sum_{i=0}^{18} b_i = \frac{10'127}{19} = 553$, while
  - $\text{median}(A) = a_9 = 6$ and
  - $\text{median}(B) = b_9 = 6$.
- The median is not affected by the outliers.

Example for Data Samples w/o Outlier

- Two sets of data samples A and B with $n_a = n_b = 19$ values.

$$A = (1, 3, 4, 4, 4, 5, 6, 6, 6, 6, 7, 7, 9, 9, 9, 10, 11, 12, 14)$$
$$B = (1, 3, 4, 4, 4, 5, 6, 6, 6, 6, 7, 7, 9, 9, 9, 10, 11, 12, 10'008)$$

- We find that
  - $\text{mean}(A) = \frac{1}{19}\sum_{i=0}^{18} a_i = \frac{133}{19} = 7$ and
  - $\text{mean}(B) = \frac{1}{19}\sum_{i=0}^{18} b_i = \frac{10'127}{19} = 553$, while
  - $\text{median}(A) = a_9 = 6$ and
  - $\text{median}(B) = b_9 = 6$.
- The median is not affected by the outliers.
- $\text{mean}(B) = 553$ is a value completely different from anything that actually occurs in B...
  ...it gives us a completely wrong impression.

Outliers can be important!

- If you think about it, where could outliers in our experiments come from?

Outliers can be important!

- If you think about it, where could outliers in our experiments come from?
  1. The operating systems scheduling or other strange effects could mess with our timing.

Outliers can be important!

- If you think about it, where could outliers in our experiments come from?
  1. The operating systems scheduling or other strange effects could mess with our timing. This could cause worse results.

Outliers can be important!

- If you think about it, where could outliers in our experiments come from?
  1. The operating systems scheduling or other strange effects could mess with our timing. This could cause worse results. But usually this is already it.

Outliers can be important!

- If you think about it, where could outliers in our experiments come from?
  1. The operating systems scheduling or other strange effects could mess with our timing. This could cause worse results. But usually this is already it.
  2. Unless your objective function is noisy, e.g., if you measure some physical quantity, or the objective function involves randomized simulations, there are hardly any other "outside" effects that could mess up our results!

Outliers can be important!

- If you think about it, where could outliers in our experiments come from?
  1. The operating systems scheduling or other strange effects could mess with our timing. This could cause worse results. But usually this is already it.
  2. Unless your objective function is noisy, e.g., if you measure some physical quantity, or the objective function involves randomized simulations, there are hardly any other "outside" effects that could mess up our results!
- Instead, most likely there could be
  - bugs in our code!

Outliers can be important!

- If you think about it, where could outliers in our experiments come from?
  1. The operating systems scheduling or other strange effects could mess with our timing. This could cause worse results. But usually this is already it.
  2. Unless your objective function is noisy, e.g., if you measure some physical quantity, or the objective function involves randomized simulations, there are hardly any other "outside" effects that could mess up our results!
- Instead, most likely there could be
  - bugs in our code!
    - Bugs in our code are *the* most important number one reason for outliers!

Outliers can be important!

- If you think about it, where could outliers in our experiments come from?
  1. The operating systems scheduling or other strange effects could mess with our timing. This could cause worse results. But usually this is already it.
  2. Unless your objective function is noisy, e.g., if you measure some physical quantity, or the objective function involves randomized simulations, there are hardly any other "outside" effects that could mess up our results!
- Instead, most likely there could be
  - bugs in our code!
    - Bugs in our code are *the* most important number one reason for outliers!
    - Yes, also in your code! (Btw: Please use unit test[195,214,236].)

Outliers can be important!

- If you think about it, where could outliers in our experiments come from?
  1. The operating systems scheduling or other strange effects could mess with our timing. This could cause worse results. But usually this is already it.
  2. Unless your objective function is noisy, e.g., if you measure some physical quantity, or the objective function involves randomized simulations, there are hardly any other "outside" effects that could mess up our results!
- Instead, most likely there could be
  - bugs in our code!
    - Bugs in our code are *the* most important number one reason for outliers!
    - Yes, also in your code! (Btw: Please use unit test[195,214,236].)
  - Or: bad (but rare) worst-case behaviors of our algorithm!

Outliers can be important!

- If you think about it, where could outliers in our experiments come from?
  1. The operating systems scheduling or other strange effects could mess with our timing. This could cause worse results. But usually this is already it.
  2. Unless your objective function is noisy, e.g., if you measure some physical quantity, or the objective function involves randomized simulations, there are hardly any other "outside" effects that could mess up our results!
- Instead, most likely there could be
  - bugs in our code!
    - Bugs in our code are *the* most important number one reason for outliers!
    - Yes, also in your code! (Btw: Please use unit test[195,214,236].)
  - Or: bad (but rare) worst-case behaviors of our algorithm!

    Imagine that:

Outliers can be important!

- If you think about it, where could outliers in our experiments come from?
  1. The operating systems scheduling or other strange effects could mess with our timing. This could cause worse results. But usually this is already it.
  2. Unless your objective function is noisy, e.g., if you measure some physical quantity, or the objective function involves randomized simulations, there are hardly any other "outside" effects that could mess up our results!
- Instead, most likely there could be
  - bugs in our code!
    - Bugs in our code are *the* most important number one reason for outliers!
    - Yes, also in your code! (Btw: Please use unit test[195,214,236].)
  - Or: bad (but rare) worst-case behaviors of our algorithm!

    Imagine that: Your algorithm can actually solve the TSP or Maximum Satisfiability (MaxSAT) problem in polynomial time on 90% of all instances...

Outliers can be important!

- If you think about it, where could outliers in our experiments come from?
  1. The operating systems scheduling or other strange effects could mess with our timing. This could cause worse results. But usually this is already it.
  2. Unless your objective function is noisy, e.g., if you measure some physical quantity, or the objective function involves randomized simulations, there are hardly any other "outside" effects that could mess up our results!
- Instead, most likely there could be
  - bugs in our code!
    - Bugs in our code are *the* most important number one reason for outliers!
    - Yes, also in your code! (Btw: Please use unit test[195,214,236].)
  - Or: bad (but rare) worst-case behaviors of our algorithm!

    Imagine that: Your algorithm can actually solve the TSP or Maximum Satisfiability (MaxSAT) problem in polynomial time on 90% of all instances... ...but on 10%, it needs exponential time.

Outliers can be important!

- If you think about it, where could outliers in our experiments come from?
  1. The operating systems scheduling or other strange effects could mess with our timing. This could cause worse results. But usually this is already it.
  2. Unless your objective function is noisy, e.g., if you measure some physical quantity, or the objective function involves randomized simulations, there are hardly any other "outside" effects that could mess up our results!
- Instead, most likely there could be
  - bugs in our code!
    - Bugs in our code are *the* most important number one reason for outliers!
    - Yes, also in your code! (Btw: Please use unit test[195,214,236].)
  - Or: bad (but rare) worst-case behaviors of our algorithm!

    Imagine that: Your algorithm can actually solve the TSP or Maximum Satisfiability (MaxSAT) problem in polynomial time on 90% of all instances... ...but on 10%, it needs exponential time. If you just look at the median runtime, you may think you discovered something awesome.

Outliers can be important!

- If you think about it, where could outliers in our experiments come from?
  1. The operating systems scheduling or other strange effects could mess with our timing. This could cause worse results. But usually this is already it.
  2. Unless your objective function is noisy, e.g., if you measure some physical quantity, or the objective function involves randomized simulations, there are hardly any other "outside" effects that could mess up our results!
- Instead, most likely there could be
  - bugs in our code!
    - Bugs in our code are *the* most important number one reason for outliers!
    - Yes, also in your code! (Btw: Please use unit test[195,214,236].)
  - Or: bad (but rare) worst-case behaviors of our algorithm!

    Imagine that: Your algorithm can actually solve the TSP or Maximum Satisfiability (MaxSAT) problem in polynomial time on 90% of all instances... ...but on 10%, it needs exponential time. If you just look at the median runtime, you may think you discovered something awesome. Actually, this is quite common...

Outliers can be important!

- If you t
  1. Th
     co
  2. Ur
     ob
     efl
- Instead
  - bu
  - Or

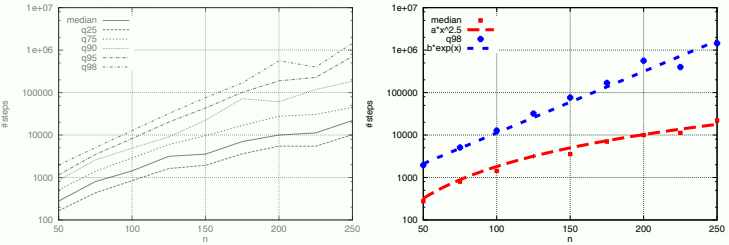


Figure 17. *Left:* Scaling of instance hardness with problem size for WalkSAT, approx. optimal noise, applied to Random-3-SAT test-sets. *Right:* Functional approximations of median and 0.98 percentile; the median seems to grow polynomially with n while the 0.98 percentile clearly shows exponential growth.

(Taken from the paper *"Local Search Algorithms for SAT: An Empirical Evaluation"* by Hoos and Stützle, coloring added manually[122], copyright Springer.)

Outliers can be important!

- If you think about it, where could outliers in our experiments come from?
  1. The operating systems scheduling or other strange effects could mess with our timing. This could cause worse results. But usually this is already it.
  2. Unless your objective function is noisy, e.g., if you measure some physical quantity, or the objective function involves randomized simulations, there are hardly any other "outside" effects that could mess up our results!
- Instead, most likely there could be
  - bugs in our code!
    - Bugs in our code are *the* most important number one reason for outliers!
    - Yes, also in your code! (Btw: Please use unit test[195,214,236].)
  - Or: bad (but rare) worst-case behaviors of our algorithm!

    Imagine that: Your algorithm can actually solve the TSP or Maximum Satisfiability (MaxSAT) problem in polynomial time on 90% of all instances... ...but on 10%, it needs exponential time. If you just look at the median runtime, you may think you discovered something awesome. Actually, this is quite common...
- Thus, we may actually want that outliers influence our statistics...

Geometric Mean

OK, arithmetic mean, median ... but what about the geometric mean?

Geometric Mean



OK, arithmetic mean, median ... but what about the geometric mean?

Definition: Geometric Mean

The geometric mean $\mathrm{geom}(A)$ is the nth root of the product of n positive values.

Geometric Mean

OK, arithmetic mean, median ... but what about the geometric mean?

Definition: Geometric Mean

The geometric mean $\mathrm{geom}(A)$ is the nth root of the product of n positive values.

$$\mathrm{geom}(A) = \sqrt[n]{\prod_{i=0}^{n-1} a_i} \tag{3}$$

(4)

Geometric Mean



OK, arithmetic mean, median ... but what about the geometric mean?

Definition: Geometric Mean

The geometric mean $\mathrm{geom}(A)$ is the nth root of the product of n positive values.

$$\mathrm{geom}(A) = \sqrt[n]{\prod_{i=0}^{n-1} a_i} \tag{3}$$

$$\mathrm{geom}(A) = \exp\left(\frac{1}{n}\sum_{i=0}^{n-1} \log a_i\right) \tag{4}$$

Normalized Data



- Often, our data is somehow normalized.

Normalized Data



- Often, our data is somehow normalized.
- Let's say we solve the problem instances I_1 to I_3 with the different algorithms A_1 to A_3.

Normalized Data



- Often, our data is somehow normalized.
- Let's say we solve the problem instances I_1 to I_3 with the different algorithms A_1 to A_3.
- We measure the required runtimes as follows:

Normalized Data



- Often, our data is somehow normalized.
- Let's say we solve the problem instances I_1 to I_3 with the different algorithms A_1 to A_3.
- We measure the required runtimes as follows:

	A_1	A_2	A_3
I_1			
I_2			
I_3			

Normalized Data

- Often, our data is somehow normalized.
- Let's say we solve the problem instances I_1 to I_3 with the different algorithms A_1 to A_3.
- We measure the required runtimes as follows:

	A_1	A_2	A_3
I_1	10 s		
I_2			
I_3			

Normalized Data

- Often, our data is somehow normalized.
- Let's say we solve the problem instances I_1 to I_3 with the different algorithms A_1 to A_3.
- We measure the required runtimes as follows:

	A_1	A_2	A_3
I_1	10 s		
I_2	20 s		
I_3			

Normalized Data

- Often, our data is somehow normalized.
- Let's say we solve the problem instances I_1 to I_3 with the different algorithms A_1 to A_3.
- We measure the required runtimes as follows:

	A_1	A_2	A_3
I_1	10 s		
I_2	20 s		
I_3	40 s		

Normalized Data

- Often, our data is somehow normalized.
- Let's say we solve the problem instances I_1 to I_3 with the different algorithms A_1 to A_3.
- We measure the required runtimes as follows:

	A_1	A_2	A_3
I_1	10 s	20 s	
I_2	20 s		
I_3	40 s		

Normalized Data

- Often, our data is somehow normalized.
- Let's say we solve the problem instances I_1 to I_3 with the different algorithms A_1 to A_3.
- We measure the required runtimes as follows:

	A_1	A_2	A_3
I_1	10 s	20 s	
I_2	20 s	40 s	
I_3	40 s		

Normalized Data

- Often, our data is somehow normalized.
- Let's say we solve the problem instances I_1 to I_3 with the different algorithms A_1 to A_3.
- We measure the required runtimes as follows:

	A_1	A_2	A_3
I_1	10 s	20 s	
I_2	20 s	40 s	
I_3	40 s	10 s	

Normalized Data

- Often, our data is somehow normalized.
- Let's say we solve the problem instances I_1 to I_3 with the different algorithms A_1 to A_3.
- We measure the required runtimes as follows:

	A_1	A_2	A_3
I_1	10 s	20 s	40 s
I_2	20 s	40 s	
I_3	40 s	10 s	

Normalized Data



- Often, our data is somehow normalized.
- Let's say we solve the problem instances I_1 to I_3 with the different algorithms A_1 to A_3.
- We measure the required runtimes as follows:

	A_1	A_2	A_3
I_1	10 s	20 s	40 s
I_2	20 s	40 s	10 s
I_3	40 s	10 s	

Normalized Data



- Often, our data is somehow normalized.
- Let's say we solve the problem instances I_1 to I_3 with the different algorithms A_1 to A_3.
- We measure the required runtimes as follows:

	A_1	A_2	A_3
I_1	10 s	20 s	40 s
I_2	20 s	40 s	10 s
I_3	40 s	10 s	20 s

Normalized Data



- Often, our data is somehow normalized.
- We measure the required runtimes as follows:
- The arithmetic mean values are the same.

	A_1	A_2	A_3
I_1	10 s	20 s	40 s
I_2	20 s	40 s	10 s
I_3	40 s	10 s	20 s
mean:	23.33 s	23.33 s	23.33 s

Normalized Data



- Often, our data is somehow normalized.
- The arithmetic mean and the median values are the same.

	A_1	A_2	A_3
I_1	10 s	20 s	40 s
I_2	20 s	40 s	10 s
I_3	40 s	10 s	20 s
mean:	23.33 s	23.33 s	23.33 s
median:	20.00 s	20.00 s	20.00 s

Normalized Data



- Often, our data is somehow normalized.
- The arithmetic mean, the median, and the geometric mean values are the same.

	A_1	A_2	A_3
I_1	10 s	20 s	40 s
I_2	20 s	40 s	10 s
I_3	40 s	10 s	20 s
mean:	23.33 s	23.33 s	23.33 s
median:	20.00 s	20.00 s	20.00 s
geom:	20.00 s	20.00 s	20.00 s

Normalized Data



- Often, our data is somehow normalized.
- The arithmetic mean, the median, and the geometric mean values are the same.
- We can conclude that the three algorithms offer the same performance in average over these benchmark instances.

	A_1	A_2	A_3
I_1	10 s	20 s	40 s
I_2	20 s	40 s	10 s
I_3	40 s	10 s	20 s
mean:	23.33 s	23.33 s	23.33 s
median:	20.00 s	20.00 s	20.00 s
geom:	20.00 s	20.00 s	20.00 s

Normalized Data



- Often, our data is somehow normalized.
- We can conclude that the three algorithms offer the same performance in average over these benchmark instances.
- But often the measured numbers "look messier" and are harder to compare at first glance.

	A_1	A_2	A_3
I_1	10 s	20 s	40 s
I_2	20 s	40 s	10 s
I_3	40 s	10 s	20 s
mean:	23.33 s	23.33 s	23.33 s
median:	20.00 s	20.00 s	20.00 s
geom:	20.00 s	20.00 s	20.00 s

Normalized Data

- Often, our data is somehow normalized.
- But often the measured numbers "look messier" and are harder to compare at first glance.
- So often we want to normalize them by picking one algorithm as "standard" and dividing them by its measurements.

	A_1	A_2	A_3
I_1	10 s	20 s	40 s
I_2	20 s	40 s	10 s
I_3	40 s	10 s	20 s
mean:	23.33 s	23.33 s	23.33 s
median:	20.00 s	20.00 s	20.00 s
geom:	20.00 s	20.00 s	20.00 s

Normalized Data



- Often, our data is somehow normalized.
- But often the measured numbers "look messier" and are harder to compare at first glance.
- So often we want to normalize them by picking one algorithm as "standard" and dividing them by its measurements.
- Let's say A_1 was a well-known heuristic, maybe we even took its results from a paper, and we want to use it as baseline for comparison and normalize our data by it.

	A_1	A_2	A_3
I_1	10 s	20 s	40 s
I_2	20 s	40 s	10 s
I_3	40 s	10 s	20 s
mean:	23.33 s	23.33 s	23.33 s
median:	20.00 s	20.00 s	20.00 s
geom:	20.00 s	20.00 s	20.00 s

Normalized Data



- Often, our data is somehow normalized.
- So often we want to normalize them by picking one algorithm as "standard" and dividing them by its measurements.
- Let's say A_1 was a well-known heuristic, maybe we even took its results from a paper, and we want to use it as baseline for comparison and normalize our data by it.

	A_1	A_2	A_3
I_1	10 s	20 s	40 s
I_2	20 s	40 s	10 s
I_3	40 s	10 s	20 s
mean:	23.33 s	23.33 s	23.33 s
median:	20.00 s	20.00 s	20.00 s
geom:	20.00 s	20.00 s	20.00 s

Normalized Data



- Often, our data is somehow normalized.
- Let's say A_1 was a well-known heuristic, maybe we even took its results from a paper, and we want to use it as baseline for comparison and normalize our data by it.
- OK, so we get this table with normalized values, which allow us to make sense of the data at first glance.

	A_1	A_2	A_3
I_1	10 s	20 s	40 s
I_2	20 s	40 s	10 s
I_3	40 s	10 s	20 s
mean:	23.33 s	23.33 s	23.33 s
median:	20.00 s	20.00 s	20.00 s
geom:	20.00 s	20.00 s	20.00 s

	A_1	A_2	A_3
I_1	1.00	2.00	4.00
I_2	1.00	2.00	0.50
I_3	1.00	0.25	0.50

Normalized Data

- Often, our data is somehow normalized.
- OK, so we get this table with normalized values, which allow us to make sense of the data at first glance.
- If we now compute the arithmetic mean

	A_1	A_2	A_3
I_1	10 s	20 s	40 s
I_2	20 s	40 s	10 s
I_3	40 s	10 s	20 s
mean:	23.33 s	23.33 s	23.33 s
median:	20.00 s	20.00 s	20.00 s
geom:	20.00 s	20.00 s	20.00 s

	A_1	A_2	A_3
I_1	1.00	2.00	4.00
I_2	1.00	2.00	0.50
I_3	1.00	0.25	0.50
mean:	1.00	1.42	1.67

Normalized Data



- Often, our data is somehow normalized.
- OK, so we get this table with normalized values, which allow us to make sense of the data at first glance.
- If we now compute the arithmetic mean, then A_1 is best

	A_1	A_2	A_3
I_1	10 s	20 s	40 s
I_2	20 s	40 s	10 s
I_3	40 s	10 s	20 s
mean:	23.33 s	23.33 s	23.33 s
median:	20.00 s	20.00 s	20.00 s
geom:	20.00 s	20.00 s	20.00 s

	A_1	A_2	A_3
I_1	1.00	2.00	4.00
I_2	1.00	2.00	0.50
I_3	1.00	0.25	0.50
mean:	1.00	1.42	1.67

Normalized Data

- Often, our data is somehow normalized.
- OK, so we get this table with normalized values, which allow us to make sense of the data at first glance.
- If we now compute the arithmetic mean, then A_1 is best and A_3 looks worst.

	A_1	A_2	A_3
I_1	10 s	20 s	40 s
I_2	20 s	40 s	10 s
I_3	40 s	10 s	20 s
mean:	23.33 s	23.33 s	23.33 s
median:	20.00 s	20.00 s	20.00 s
geom:	20.00 s	20.00 s	20.00 s

	A_1	A_2	A_3
I_1	1.00	2.00	4.00
I_2	1.00	2.00	0.50
I_3	1.00	0.25	0.50
mean:	1.00	1.42	1.67

Normalized Data

- Often, our data is somehow normalized.
- OK, so we get this table with normalized values, which allow us to make sense of the data at first glance.
- If we now compute the arithmetic mean, then A_1 is best and A_3 looks worst.
- According to the median

	A_1	A_2	A_3
I_1	10 s	20 s	40 s
I_2	20 s	40 s	10 s
I_3	40 s	10 s	20 s
mean:	23.33 s	23.33 s	23.33 s
median:	20.00 s	20.00 s	20.00 s
geom:	20.00 s	20.00 s	20.00 s

	A_1	A_2	A_3
I_1	1.00	2.00	4.00
I_2	1.00	2.00	0.50
I_3	1.00	0.25	0.50
mean:	1.00	1.42	1.67
median:	1.00	2.00	0.50

Normalized Data

- Often, our data is somehow normalized.
- If we now compute the arithmetic mean, then A_1 is best and A_3 looks worst.
- According to the median, A_3 is best

	A_1	A_2	A_3
I_1	10 s	20 s	40 s
I_2	20 s	40 s	10 s
I_3	40 s	10 s	20 s
mean:	23.33 s	23.33 s	23.33 s
median:	20.00 s	20.00 s	20.00 s
geom:	20.00 s	20.00 s	20.00 s

	A_1	A_2	A_3
I_1	1.00	2.00	4.00
I_2	1.00	2.00	0.50
I_3	1.00	0.25	0.50
mean:	1.00	1.42	1.67
median:	1.00	2.00	0.50

Normalized Data

- Often, our data is somehow normalized.
- If we now compute the arithmetic mean, then A_1 is best and A_3 looks worst.
- According to the median, A_3 is best and A_2 is worst!

	A_1	A_2	A_3
I_1	10 s	20 s	40 s
I_2	20 s	40 s	10 s
I_3	40 s	10 s	20 s
mean:	23.33 s	23.33 s	23.33 s
median:	20.00 s	20.00 s	20.00 s
geom:	20.00 s	20.00 s	20.00 s

	A_1	A_2	A_3
I_1	1.00	2.00	4.00
I_2	1.00	2.00	0.50
I_3	1.00	0.25	0.50
mean:	1.00	1.42	1.67
median:	1.00	2.00	0.50

Normalized Data



- Often, our data is somehow normalized.
- If we now compute the arithmetic mean, then A_1 is best and A_3 looks worst.
- According to the median, A_3 is best and A_2 is worst!
- Only the geometric mean still indicates that the algorithms perform the same...

	A_1	A_2	A_3
I_1	10 s	20 s	40 s
I_2	20 s	40 s	10 s
I_3	40 s	10 s	20 s
mean:	23.33 s	23.33 s	23.33 s
median:	20.00 s	20.00 s	20.00 s
geom:	20.00 s	20.00 s	20.00 s

	A_1	A_2	A_3
I_1	1.00	2.00	4.00
I_2	1.00	2.00	0.50
I_3	1.00	0.25	0.50
mean:	1.00	1.42	1.67
median:	1.00	2.00	0.50
geom:	1.00	1.00	1.00

Normalized Data

- Often, our data is somehow normalized.
- Hm.

	A_1	A_2	A_3
I_1	10 s	20 s	40 s
I_2	20 s	40 s	10 s
I_3	40 s	10 s	20 s
mean:	23.33 s	23.33 s	23.33 s
median:	20.00 s	20.00 s	20.00 s
geom:	20.00 s	20.00 s	20.00 s

	A_1	A_2	A_3
I_1	1.00	2.00	4.00
I_2	1.00	2.00	0.50
I_3	1.00	0.25	0.50
mean:	1.00	1.42	1.67
median:	1.00	2.00	0.50
geom:	1.00	1.00	1.00

Normalized Data



- Often, our data is somehow normalized.
- Hm. OK, then let's normalize using the results of A_2 instead.

	A_1	A_2	A_3
I_1	10 s	20 s	40 s
I_2	20 s	40 s	10 s
I_3	40 s	10 s	20 s
mean:	23.33 s	23.33 s	23.33 s
median:	20.00 s	20.00 s	20.00 s
geom:	20.00 s	20.00 s	20.00 s

Normalized Data



- Often, our data is somehow normalized.
- Hm. OK, then let's normalize using the results of A_2 instead.
- OK, so we get this table with normalized values.

	A_1	A_2	A_3
I_1	10 s	20 s	40 s
I_2	20 s	40 s	10 s
I_3	40 s	10 s	20 s
mean:	23.33 s	23.33 s	23.33 s
median:	20.00 s	20.00 s	20.00 s
geom:	20.00 s	20.00 s	20.00 s

	A_1	A_2	A_3
I_1	0.50	1.00	2.00
I_2	0.50	1.00	0.25
I_3	4.00	1.00	2.00

Normalized Data



- Often, our data is somehow normalized.
- OK, so we get this table with normalized values.
- If we now compute the arithmetic mean

	A_1	A_2	A_3
I_1	10 s	20 s	40 s
I_2	20 s	40 s	10 s
I_3	40 s	10 s	20 s
mean:	23.33 s	23.33 s	23.33 s
median:	20.00 s	20.00 s	20.00 s
geom:	20.00 s	20.00 s	20.00 s

	A_1	A_2	A_3
I_1	0.50	1.00	2.00
I_2	0.50	1.00	0.25
I_3	4.00	1.00	2.00
mean:	1.67	1.00	1.42

Normalized Data



- Often, our data is somehow normalized.
- OK, so we get this table with normalized values.
- If we now compute the arithmetic mean, then A_2 is best

	A_1	A_2	A_3
I_1	10 s	20 s	40 s
I_2	20 s	40 s	10 s
I_3	40 s	10 s	20 s
mean:	23.33 s	23.33 s	23.33 s
median:	20.00 s	20.00 s	20.00 s
geom:	20.00 s	20.00 s	20.00 s

	A_1	A_2	A_3
I_1	0.50	1.00	2.00
I_2	0.50	1.00	0.25
I_3	4.00	1.00	2.00
mean:	1.67	1.00	1.42

Normalized Data

- Often, our data is somehow normalized.
- OK, so we get this table with normalized values.
- If we now compute the arithmetic mean, then A_2 is best and A_1 looks worst.

	A_1	A_2	A_3
I_1	10 s	20 s	40 s
I_2	20 s	40 s	10 s
I_3	40 s	10 s	20 s
mean:	23.33 s	23.33 s	23.33 s
median:	20.00 s	20.00 s	20.00 s
geom:	20.00 s	20.00 s	20.00 s

	A_1	A_2	A_3
I_1	0.50	1.00	2.00
I_2	0.50	1.00	0.25
I_3	4.00	1.00	2.00
mean:	1.67	1.00	1.42

Normalized Data

- Often, our data is somehow normalized.
- OK, so we get this table with normalized values.
- If we now compute the arithmetic mean, then A_2 is best and A_1 looks worst.
- According to the median

	A_1	A_2	A_3
I_1	10 s	20 s	40 s
I_2	20 s	40 s	10 s
I_3	40 s	10 s	20 s
mean:	23.33 s	23.33 s	23.33 s
median:	20.00 s	20.00 s	20.00 s
geom:	20.00 s	20.00 s	20.00 s

	A_1	A_2	A_3
I_1	0.50	1.00	2.00
I_2	0.50	1.00	0.25
I_3	4.00	1.00	2.00
mean:	1.67	1.00	1.42
median:	0.50	1.00	2.00

Normalized Data

- Often, our data is somehow normalized.
- If we now compute the arithmetic mean, then A_2 is best and A_1 looks worst.
- According to the median, A_1 is best

	A_1	A_2	A_3
I_1	10 s	20 s	40 s
I_2	20 s	40 s	10 s
I_3	40 s	10 s	20 s
mean:	23.33 s	23.33 s	23.33 s
median:	20.00 s	20.00 s	20.00 s
geom:	20.00 s	20.00 s	20.00 s

	A_1	A_2	A_3
I_1	0.50	1.00	2.00
I_2	0.50	1.00	0.25
I_3	4.00	1.00	2.00
mean:	1.67	1.00	1.42
median:	0.50	1.00	2.00

Normalized Data

- Often, our data is somehow normalized.
- If we now compute the arithmetic mean, then A_2 is best and A_1 looks worst.
- According to the median, A_1 is best and A_3 is worst!

	A_1	A_2	A_3
I_1	10 s	20 s	40 s
I_2	20 s	40 s	10 s
I_3	40 s	10 s	20 s
mean:	23.33 s	23.33 s	23.33 s
median:	20.00 s	20.00 s	20.00 s
geom:	20.00 s	20.00 s	20.00 s

	A_1	A_2	A_3
I_1	0.50	1.00	2.00
I_2	0.50	1.00	0.25
I_3	4.00	1.00	2.00
mean:	1.67	1.00	1.42
median:	0.50	1.00	2.00

Normalized Data



- Often, our data is somehow normalized.
- If we now compute the arithmetic mean, then A_2 is best and A_1 looks worst.
- According to the median, A_1 is best and A_3 is worst!
- Only the geometric mean still indicates that the algorithms perform the same...

	A_1	A_2	A_3
I_1	10 s	20 s	40 s
I_2	20 s	40 s	10 s
I_3	40 s	10 s	20 s
mean:	23.33 s	23.33 s	23.33 s
median:	20.00 s	20.00 s	20.00 s
geom:	20.00 s	20.00 s	20.00 s

	A_1	A_2	A_3
I_1	0.50	1.00	2.00
I_2	0.50	1.00	0.25
I_3	4.00	1.00	2.00
mean:	1.67	1.00	1.42
median:	0.50	1.00	2.00
geom:	1.00	1.00	1.00

Normalized Data



- Often, our data is somehow normalized.
- Only the geometric mean still indicates that the algorithms perform the same...
- The geometric mean is the only meaningful average if we have normalized data![94]

Normalized Data



- Often, our data is somehow normalized.
- The geometric mean is the only meaningful average if we have normalized data![94]
- And we very often have normalized data.

Normalized Data



- Often, our data is somehow normalized.
- The geometric mean is the only meaningful average if we have normalized data![94]
- And we very often have normalized data.
- For example, at least half of the papers on the Job Shop Scheduling Problem (JSSP) normalize the result qualities they obtain on benchmark instances with the Best Known Solutions (BKSes)[248].

Normalized Data



- Often, our data is somehow normalized.
- The geometric mean is the only meaningful average if we have normalized data![94]
- And we very often have normalized data.
- For example, at least half of the papers on the Job Shop Scheduling Problem (JSSP) normalize the result qualities they obtain on benchmark instances with the Best Known Solutions (BKSes) and then compute the arithmetic mean[248].

Arithmetic Mean vs. Median vs. Geometric Mean



- Most publications report arithmetic mean results, many report median results, almost none report geometric means.

Arithmetic Mean vs. Median vs. Geometric Mean



- Most publications report arithmetic mean results, many report median results, almost none report geometric means.
- The median is more robust against outliers compared to the arithmetic mean.

Arithmetic Mean vs. Median vs. Geometric Mean



- Most publications report arithmetic mean results, many report median results, almost none report geometric means.
- The median is more robust against outliers compared to the arithmetic mean, however, in normal application scenarios, there are very few acceptable reasons for outliers.

Arithmetic Mean vs. Median vs. Geometric Mean



- Most publications report arithmetic mean results, many report median results, almost none report geometric means.
- The median is more robust against outliers compared to the arithmetic mean, however, in normal application scenarios, there are very few acceptable reasons for outliers.
- We therefore want to know both the arithmetic mean and the median.

Arithmetic Mean vs. Median vs. Geometric Mean



- Most publications report arithmetic mean results, many report median results, almost none report geometric means.
- The median is more robust against outliers compared to the arithmetic mean, however, in normal application scenarios, there are very few acceptable reasons for outliers.
- We therefore want to know both the arithmetic mean and the median:
  - If the arithmetic mean is much worse than the median, then

Arithmetic Mean vs. Median vs. Geometric Mean



- Most publications report arithmetic mean results, many report median results, almost none report geometric means.
- The median is more robust against outliers compared to the arithmetic mean, however, in normal application scenarios, there are very few acceptable reasons for outliers.
- We therefore want to know both the arithmetic mean and the median:
  - If the arithmetic mean is much worse than the median, then
    - maybe we have a bug in our code that only sometimes has an impact.

Arithmetic Mean vs. Median vs. Geometric Mean



- Most publications report arithmetic mean results, many report median results, almost none report geometric means.
- The median is more robust against outliers compared to the arithmetic mean, however, in normal application scenarios, there are very few acceptable reasons for outliers.
- We therefore want to know both the arithmetic mean and the median:
  - If the arithmetic mean is much worse than the median, then
    - maybe we have a bug in our code that only sometimes has an impact or
    - our algorithm has a bad worst-case behavior (which is also good to know).

Arithmetic Mean vs. Median vs. Geometric Mean



- Most publications report arithmetic mean results, many report median results, almost none report geometric means.
- The median is more robust against outliers compared to the arithmetic mean, however, in normal application scenarios, there are very few acceptable reasons for outliers.
- We therefore want to know both the arithmetic mean and the median:
  - If the arithmetic mean is much worse than the median, then
    - maybe we have a bug in our code that only sometimes has an impact or
    - our algorithm has a bad worst-case behavior (which is also good to know).
  - If the median is much worse than the mean, then the mean is too optimistic, i.e., most of the time we should expect worse results.

Arithmetic Mean vs. Median vs. Geometric Mean



- Most publications report arithmetic mean results, many report median results, almost none report geometric means.
- The median is more robust against outliers compared to the arithmetic mean, however, in normal application scenarios, there are very few acceptable reasons for outliers.
- We therefore want to know both the arithmetic mean and the median:
  - If the arithmetic mean is much worse than the median, then
    - maybe we have a bug in our code that only sometimes has an impact or
    - our algorithm has a bad worst-case behavior (which is also good to know).
  - If the median is much worse than the mean, then the mean is too optimistic, i.e., most of the time we should expect worse results.
- If there are outliers, the value of the arithmetic mean itself may be very different from any actually observed value, while the median is (almost always) similar to some actual measurements.

Arithmetic Mean vs. Median vs. Geometric Mean



- Most publications report arithmetic mean results, many report median results, almost none report geometric means.
- The median is more robust against outliers compared to the arithmetic mean, however, in normal application scenarios, there are very few acceptable reasons for outliers.
- We therefore want to know both the arithmetic mean and the median.

Arithmetic Mean vs. Median vs. Geometric Mean



- Most publications report arithmetic mean results, many report median results, almost none report geometric means.
- The median is more robust against outliers compared to the arithmetic mean, however, in normal application scenarios, there are very few acceptable reasons for outliers.
- We therefore want to know both the arithmetic mean and the median.
- Often, our data is implicitly or explicitly normalized.

Arithmetic Mean vs. Median vs. Geometric Mean



- Most publications report arithmetic mean results, many report median results, almost none report geometric means.
- The median is more robust against outliers compared to the arithmetic mean, however, in normal application scenarios, there are very few acceptable reasons for outliers.
- We therefore want to know both the arithmetic mean and the median.
- Often, our data is implicitly or explicitly normalized, e.g.,
  - if we divide result qualities by results of well-known heuristics or BKSes or

Arithmetic Mean vs. Median vs. Geometric Mean



- Most publications report arithmetic mean results, many report median results, almost none report geometric means.
- The median is more robust against outliers compared to the arithmetic mean, however, in normal application scenarios, there are very few acceptable reasons for outliers.
- We therefore want to know both the arithmetic mean and the median.
- Often, our data is implicitly or explicitly normalized, e.g.,
  - if we divide result qualities by results of well-known heuristics or BKSes or
  - if we normalize the runtime using another algorithm as standard.

Arithmetic Mean vs. Median vs. Geometric Mean



- Most publications report arithmetic mean results, many report median results, almost none report geometric means.
- The median is more robust against outliers compared to the arithmetic mean, however, in normal application scenarios, there are very few acceptable reasons for outliers.
- We therefore want to know both the arithmetic mean and the median.
- Often, our data is implicitly or explicitly normalized, e.g.,
  - if we divide result qualities by results of well-known heuristics or BKSes or
  - if we normalize the runtime using another algorithm as standard.
- Then, the arithmetic mean and median can be very misleading and the geometric mean must be computed.

Arithmetic Mean vs. Median vs. Geometric Mean



- Most publications report arithmetic mean results, many report median results, almost none report geometric means.
- The median is more robust against outliers compared to the arithmetic mean, however, in normal application scenarios, there are very few acceptable reasons for outliers.
- We therefore want to know both the arithmetic mean and the median.
- Often, our data is implicitly or explicitly normalized, e.g.,
  - if we divide result qualities by results of well-known heuristics or BKSes or
  - if we normalize the runtime using another algorithm as standard.
- Then, the arithmetic mean and median can be very misleading and the geometric mean must be computed.
- I think: On raw data, compute all three measures of average, and pay special attention to the one looking the worst.

Arithmetic Mean vs. Median vs. Geometric Mean



- Most publications report arithmetic mean results, many report median results, almost none report geometric means.
- The median is more robust against outliers compared to the arithmetic mean, however, in normal application scenarios, there are very few acceptable reasons for outliers.
- We therefore want to know both the arithmetic mean and the median.
- Often, our data is implicitly or explicitly normalized, e.g.,
  - if we divide result qualities by results of well-known heuristics or BKSes or
  - if we normalize the runtime using another algorithm as standard.
- Then, the arithmetic mean and median can be very misleading and the geometric mean must be computed.
- I think: On raw data, compute all three measures of average, and pay special attention to the one looking the worst. On normalized data, compute the geometric mean.

Arithmetic Mean vs. Median vs. Geometric Mean



- Most publications report arithmetic mean results, many report median results, almost none report geometric means.
- The median is more robust against outliers compared to the arithmetic mean, however, in normal application scenarios, there are very few acceptable reasons for outliers.
- We therefore want to know both the arithmetic mean and the median.
- Often, our data is implicitly or explicitly normalized, e.g.,
  - if we divide result qualities by results of well-known heuristics or BKSes or
  - if we normalize the runtime using another algorithm as standard.
- Then, the arithmetic mean and median can be very misleading and the geometric mean must be computed.
- I think: On raw data, compute all three measures of average, and pay special attention to the one looking the worst. On normalized data, compute the geometric mean, but also consider the arithmetic mean and median *if and only if they make **your** algorithm look worse*.

Statistical Measures
Measures of the Spread

Measures of the Spread

- The average gives us a good impression about the central value or location of a distribution.

Measures of the Spread

- The average gives us a good impression about the central value or location of a distribution.
- It does not tell us much about the range of the data.

Measures of the Spread



- The average gives us a good impression about the central value or location of a distribution.
- It does not tell us much about the range of the data.
- We do not know whether the data we have measured is very similar to the median or whether it may differ very much from the mean.

Measures of the Spread



- The average gives us a good impression about the central value or location of a distribution.
- It does not tell us much about the range of the data.
- We do not know whether the data we have measured is very similar to the median or whether it may differ very much from the mean.
- An average alone is not very meaningful – if we known nothing about the range of the data.

Measures of the Spread



- The average gives us a good impression about the central value or location of a distribution.
- It does not tell us much about the range of the data.
- We do not know whether the data we have measured is very similar to the median or whether it may differ very much from the mean.
- An average alone is not very meaningful – if we known nothing about the range of the data.
- We can therefore compute a measure of dispersion, i.e., a value that tells us whether the observations are stretched and spread far or squeezed tight around the center.

Sample Variance

Definition: Variance

The variance of a distribution is the expectation of the squared deviation of the underlying random variable from its expected value.

Sample Variance

Definition: Variance

The variance of a distribution is the expectation of the squared deviation of the underlying random variable from its expected value.

Definition: Sample Variance

The variance $\mathrm{var}(A)$ of a data sample $A = (a_0, a_1, \ldots, a_{n-1})$ with n observations can be estimated as:

$$\mathrm{var}(A) = \frac{1}{n-1} \sum_{i=0}^{n-1} (a_i - \mathrm{mean}(A))^2 = \frac{1}{n-1} \left[\left(\sum_{i=0}^{n-1} a_i^2 \right) - \frac{1}{n} \left(\sum_{i=0}^{n-1} a_i \right)^2 \right]$$

Standard Deviation

Definition: Sample Standard Deviation

The standard deviation $\mathrm{sd}(A)$ of a data sample $A = (a_0, a_1, \ldots, a_{n-1})$ with n observations is the square root of the estimated variance $\mathrm{var}(A)$.

$$\mathrm{sd}(A) = \sqrt{\mathrm{var}(A)}$$

Standard Deviation

- Small standard deviations indicate that the observations tend to be similar to the mean.

Standard Deviation

- Small standard deviations indicate that the observations tend to be similar to the mean.
- Large standard deviations indicate that they tend to be far from the mean.

Standard Deviation

- Small standard deviations indicate that the observations tend to be similar to the mean.
- Large standard deviations indicate that they tend to be far from the mean.
- Small standard deviations in optimization results and runtime indicate that the algorithm is reliable.

Standard Deviation



- Small standard deviations indicate that the observations tend to be similar to the mean.
- Large standard deviations indicate that they tend to be far from the mean.
- Small standard deviations in optimization results and runtime indicate that the algorithm is reliable.
- Large standard deviations indicate unreliable algorithms.

Standard Deviation



- Small standard deviations indicate that the observations tend to be similar to the mean.
- Large standard deviations indicate that they tend to be far from the mean.
- Small standard deviations in optimization results and runtime indicate that the algorithm is reliable.
- Large standard deviations indicate unreliable algorithms, but may also offer a potential that could be exploited.

Standard Deviation



- Small standard deviations indicate that the observations tend to be similar to the mean.
- Large standard deviations indicate that they tend to be far from the mean.
- Small standard deviations in optimization results and runtime indicate that the algorithm is reliable.
- Large standard deviations indicate unreliable algorithms, but may also offer a potential that could be exploited: Given enough time, we can restart algorithms several times and expect to get different (and thus sometimes better) solutions.

Quantiles

Definition: Sample Quantile

The q-quantiles are the cut points that divide a sorted data sample $A = (a_0, a_1, \ldots, a_{n-1})$ where $a_{i-1} \leq a_i \ \forall i \in 1 \ldots (n-1)$ into q equally-sized parts.

Quantiles

Definition: Sample Quantile

The q-quantiles are the cut points that divide a sorted data sample $A = (a_0, a_1, \ldots, a_{n-1})$ where $a_{i-1} \leq a_i \ \forall i \in 1 \ldots (n-1)$ into q equally-sized parts. $\text{quantile}_q^k(A)$ be the kth q-quantile, with $k \in 1 \ldots (q-1)$, i.e., there are $q-1$ of the q-quantiles.

$$
\begin{aligned}
h &= (n-1)\tfrac{k}{q} \\
\text{quantile}_q^k(A) &= \begin{cases} a_h & \text{if } h \text{ is integer} \\ a_{\lfloor h \rfloor} + (h - \lfloor h \rfloor) * \left(a_{\lfloor h \rfloor + 1} - a_{\lfloor h \rfloor}\right) & \text{otherwise} \end{cases}
\end{aligned}
$$

Quantiles

Definition: Sample Quantile

The q-quantiles are the cut points that divide a sorted data sample $A = (a_0, a_1, \ldots, a_{n-1})$ where $a_{i-1} \leq a_i \ \forall i \in 1 \ldots (n-1)$ into q equally-sized parts. $\text{quantile}_q^k(A)$ be the kth q-quantile, with $k \in 1 \ldots (q-1)$, i.e., there are $q-1$ of the q-quantiles.

$$
\begin{aligned}
h &= (n-1)\tfrac{k}{q} \\
\text{quantile}_q^k(A) &= \begin{cases} a_h & \text{if } h \text{ is integer} \\ a_{\lfloor h \rfloor} + (h - \lfloor h \rfloor) * \left(a_{\lfloor h \rfloor + 1} - a_{\lfloor h \rfloor}\right) & \text{otherwise} \end{cases}
\end{aligned}
$$

- Quantiles are a generalized form of the median.

Quantiles

Definition: Sample Quantile

The q-quantiles are the cut points that divide a sorted data sample $A = (a_0, a_1, \ldots, a_{n-1})$ where $a_{i-1} \leq a_i \ \forall i \in 1 \ldots (n-1)$ into q equally-sized parts. $\text{quantile}_q^k(A)$ be the kth q-quantile, with $k \in 1 \ldots (q-1)$, i.e., there are $q-1$ of the q-quantiles.

$$
\begin{aligned}
h &= (n-1)\tfrac{k}{q} \\
\text{quantile}_q^k(A) &= \begin{cases} a_h & \text{if } h \text{ is integer} \\ a_{\lfloor h \rfloor} + (h - \lfloor h \rfloor) * \left(a_{\lfloor h \rfloor + 1} - a_{\lfloor h \rfloor}\right) & \text{otherwise} \end{cases}
\end{aligned}
$$

- Quantiles are a generalized form of the median.
- The $\text{quantile}_1^2(A)$ is the median of A

Quantiles

Definition: Sample Quantile

The q-quantiles are the cut points that divide a sorted data sample $A = (a_0, a_1, \ldots, a_{n-1})$ where $a_{i-1} \leq a_i \ \forall i \in 1 \ldots (n-1)$ into q equally-sized parts. $\text{quantile}_q^k(A)$ be the kth q-quantile, with $k \in 1 \ldots (q-1)$, i.e., there are $q-1$ of the q-quantiles.

$$
\begin{aligned}
h &= (n-1)\tfrac{k}{q} \\
\text{quantile}_q^k(A) &= \begin{cases} a_h & \text{if } h \text{ is integer} \\ a_{\lfloor h \rfloor} + (h - \lfloor h \rfloor) * \left(a_{\lfloor h \rfloor + 1} - a_{\lfloor h \rfloor}\right) & \text{otherwise} \end{cases}
\end{aligned}
$$

- Quantiles are a generalized form of the median.
- The $\text{quantile}_1^2(A)$ is the median of A
- 4-quantiles are called *quartiles*.

Quantiles

Definition: Sample Quantile

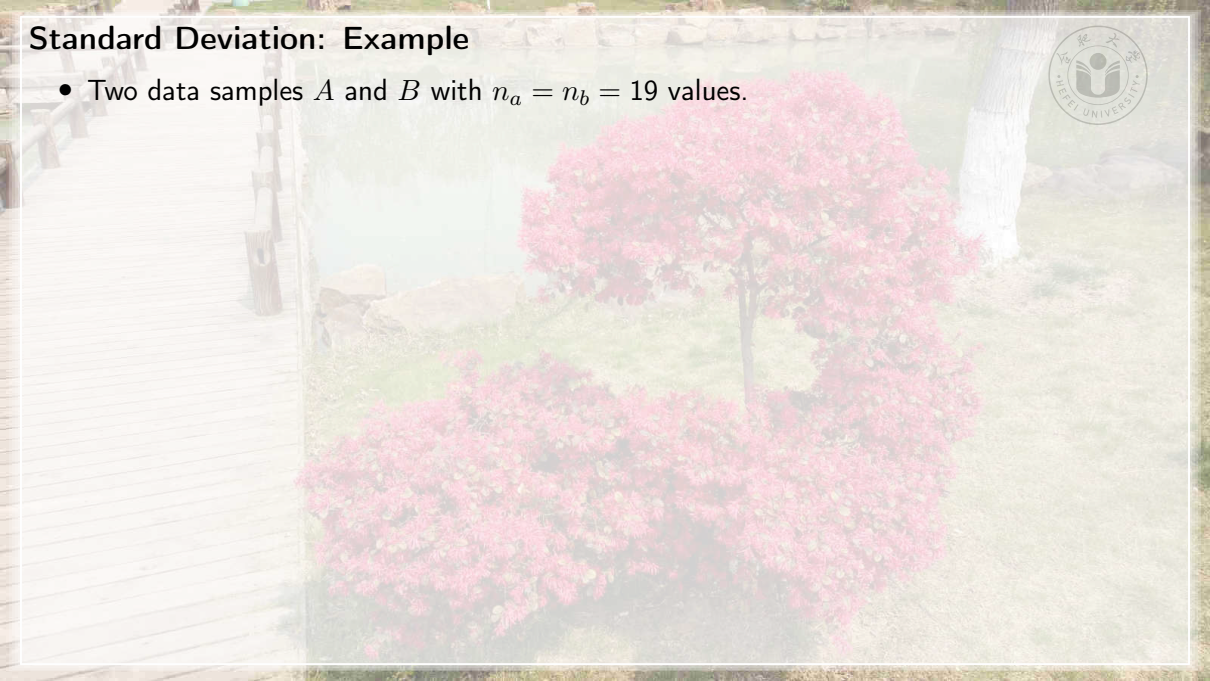
The q-quantiles are the cut points that divide a sorted data sample $A = (a_0, a_1, \ldots, a_{n-1})$ where $a_{i-1} \leq a_i \ \forall i \in 1 \ldots (n-1)$ into q equally-sized parts. $\text{quantile}_q^k(A)$ be the kth q-quantile, with $k \in 1 \ldots (q-1)$, i.e., there are $q-1$ of the q-quantiles.

$$
\begin{aligned}
h &= (n-1)\tfrac{k}{q} \\
\text{quantile}_q^k(A) &= \begin{cases} a_h & \text{if } h \text{ is integer} \\ a_{\lfloor h \rfloor} + (h - \lfloor h \rfloor) * \left(a_{\lfloor h \rfloor + 1} - a_{\lfloor h \rfloor}\right) & \text{otherwise} \end{cases}
\end{aligned}
$$

- Quantiles are a generalized form of the median.
- The $\text{quantile}_1^2(A)$ is the median of A
- 4-quantiles are called *quartiles*.
- We often consider *percentiles* or write things like "98% quantile" or "0.98 percentile" or "98% percentile" meaning $\text{quantile}_{100}^{98}$.

Standard Deviation: Example

- Two data samples A and B with $n_a = n_b = 19$ values.



Standard Deviation: Example

- Two data samples A and B with $n_a = n_b = 19$ values.

$$
\begin{aligned}
A &= (1, 3, 4, 4, 4, 5, 6, 6, 6, 6, 7, 7, 9, 9, 9, 10, 11, 12, 14) \\
\mathrm{mean}(A) &= 7 \\
B &= (1, 3, 4, 4, 4, 5, 6, 6, 6, 6, 7, 7, 9, 9, 9, 10, 11, 12, 10'008) \\
\mathrm{mean}(B) &= 533
\end{aligned}
$$

Standard Deviation: Example

- Two data samples A and B with $n_a = n_b = 19$ values.

$$
\begin{aligned}
A &= (1, 3, 4, 4, 4, 5, 6, 6, 6, 6, 7, 7, 9, 9, 9, 10, 11, 12, 14) \\
\text{mean}(A) &= 7 \\
B &= (1, 3, 4, 4, 4, 5, 6, 6, 6, 6, 7, 7, 9, 9, 9, 10, 11, 12, 10'008) \\
\text{mean}(B) &= 533 \\
\text{var}(A) &= \frac{1}{19-1} \sum_{i=1}^{19} (a_i - \text{mean}(A))^2 = \frac{198}{18} = 11 \\
\text{var}(B) &= \frac{1}{19-1} \sum_{i=1}^{19} (b_i - \text{mean}(B))^2 = \frac{94'763'306}{18} \approx 5'264'628
\end{aligned}
$$

Standard Deviation: Example

- Two data samples A and B with $n_a = n_b = 19$ values.

$$
\begin{aligned}
A &= (1, 3, 4, 4, 4, 5, 6, 6, 6, 6, 7, 7, 9, 9, 9, 10, 11, 12, 14) \\
\text{mean}(A) &= 7 \\
B &= (1, 3, 4, 4, 4, 5, 6, 6, 6, 6, 7, 7, 9, 9, 9, 10, 11, 12, 10'008) \\
\text{mean}(B) &= 533 \\
\text{var}(A) &= \frac{1}{19-1} \sum_{i=1}^{19} (a_i - \text{mean}(A))^2 = \frac{198}{18} = 11 \\
\text{var}(B) &= \frac{1}{19-1} \sum_{i=1}^{19} (b_i - \text{mean}(B))^2 = \frac{94'763'306}{18} \approx 5'264'628 \\
\text{sd}(A) &= \sqrt{\text{var}\, A} = \sqrt{11} \approx 3.3 \\
\text{sd}(B) &= \sqrt{\text{var}\, B} = \sqrt{\frac{94'763'306}{18}} \approx 2294
\end{aligned}
$$

Standard Deviation: Example

- Two data samples A and B with $n_a = n_b = 19$ values.

$$
\begin{aligned}
A &= (1, 3, 4, 4, 4, 5, 6, 6, 6, 6, 7, 7, 9, 9, 9, 10, 11, 12, 14) \\
B &= (1, 3, 4, 4, 4, 5, 6, 6, 6, 6, 7, 7, 9, 9, 9, 10, 11, 12, 10'008) \\
\operatorname{var}(A) &= \frac{1}{19-1} \sum_{i=1}^{19} (a_i - \operatorname{mean}(A))^2 = \frac{198}{18} = 11 \\
\operatorname{var}(B) &= \frac{1}{19-1} \sum_{i=1}^{19} (b_i - \operatorname{mean}(B))^2 = \frac{94'763'306}{18} \approx 5'264'628 \\
\operatorname{sd}(A) &= \sqrt{\operatorname{var} A} = \sqrt{11} \approx 3.3 \\
\operatorname{sd}(B) &= \sqrt{\operatorname{var} B} = \sqrt{\frac{94'763'306}{18}} \approx 2294
\end{aligned}
$$

- Being based on the arithmetic mean, the variance and standard deviation are heavily influenced by outliers – with all pros and cons coming with that...

Quantiles: Example

- Two data samples A and B with $n_a = n_b = 19$ values.

Quantiles: Example

- Two data samples A and B with $n_a = n_b = 19$ values.

$$A = (1, 3, 4, 4, 4, 5, 6, 6, 6, 6, 7, 7, 9, 9, 9, 10, 11, 12, 14)$$
$$B = (1, 3, 4, 4, 4, 5, 6, 6, 6, 6, 7, 7, 9, 9, 9, 10, 11, 12, 10'008)$$

Quantiles: Example

- Two data samples A and B with $n_a = n_b = 19$ values.

$$
\begin{aligned}
A &= (1, 3, 4, 4, 4, 5, 6, 6, 6, 6, 7, 7, 9, 9, 9, 10, 11, 12, 14) \\
B &= (1, 3, 4, 4, 4, 5, 6, 6, 6, 6, 7, 7, 9, 9, 9, 10, 11, 12, 10'008) \\
\text{quantile}_4^1(A) &= \text{quantile}_4^1(B) = 4.5 \\
\text{quantile}_4^3(A) &= \text{quantile}_4^3(B) = 9
\end{aligned}
$$

Quantiles: Example

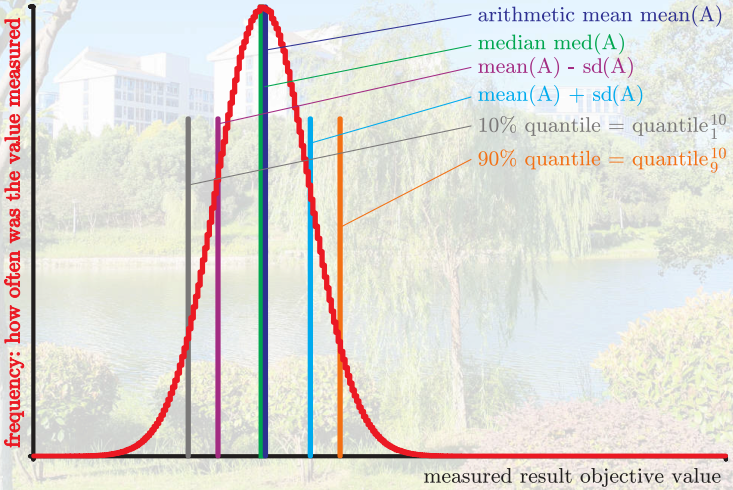
- Two data samples A and B with $n_a = n_b = 19$ values.

$$
\begin{aligned}
A &= (1, 3, 4, 4, 4, 5, 6, 6, 6, 6, 7, 7, 9, 9, 9, 10, 11, 12, 14) \\
B &= (1, 3, 4, 4, 4, 5, 6, 6, 6, 6, 7, 7, 9, 9, 9, 10, 11, 12, 10'008) \\
\text{quantile}_4^1(A) &= \text{quantile}_4^1(B) = 4.5 \\
\text{quantile}_4^3(A) &= \text{quantile}_4^3(B) = 9
\end{aligned}
$$

- Being generalizations of the median, the quantiles are little influenced by outliers – with all pros and cons coming with that...

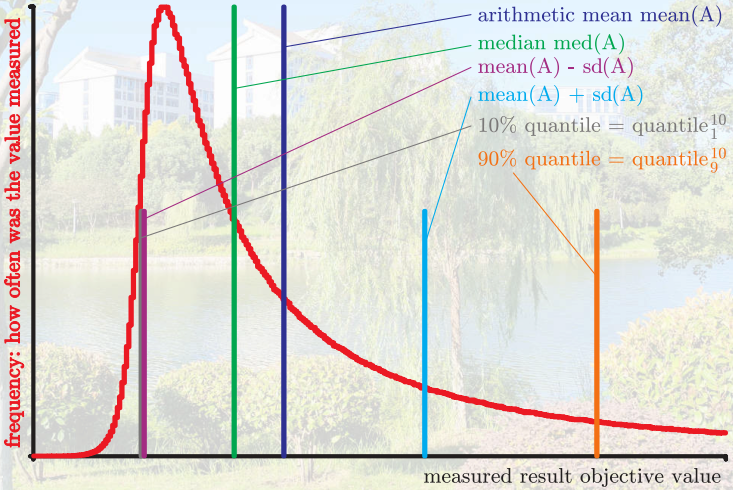
Further Example

- The implicit assumption that $\text{mean} \pm \text{sd}$ is a meaningful range is not always true!



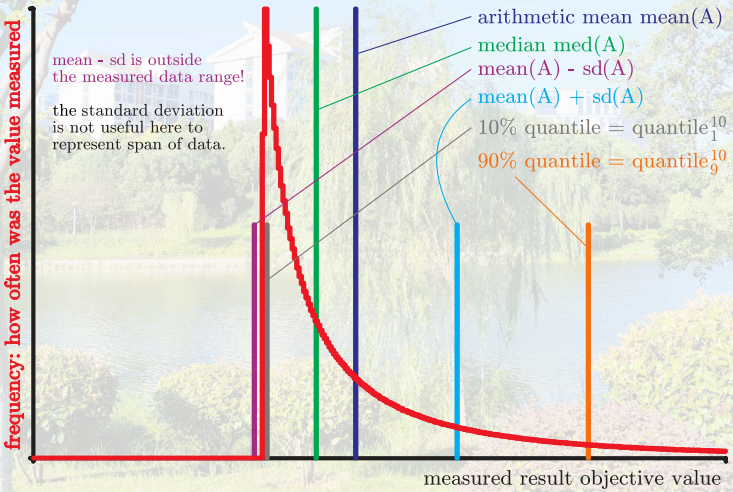
Further Example

- The implicit assumption that $\text{mean} \pm \text{sd}$ is a meaningful range is not always true!



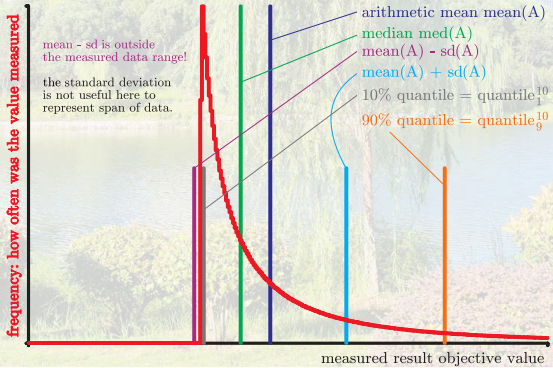
Further Example

- The implicit assumption that $\text{mean} \pm \text{sd}$ is a meaningful range is not always true!



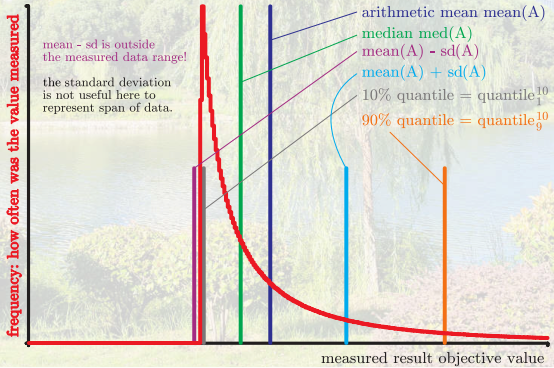
Further Example

- The implicit assumption that $\text{mean} \pm \text{sd}$ is a meaningful range is not always true!
- Such a shape is possible in optimization!



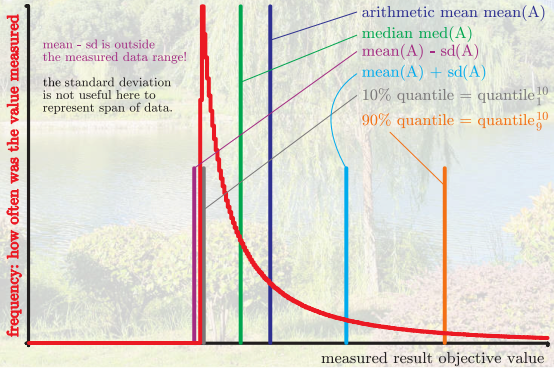
Further Example

- The implicit assumption that $\text{mean} \pm \text{sd}$ is a meaningful range is not always true!
- Such a shape is possible in optimization:
  - The global optimum marks a lower bound for the possible objective values.



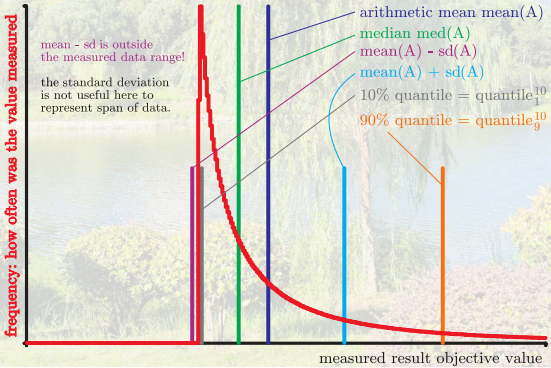
Further Example

- The implicit assumption that $\text{mean} \pm \text{sd}$ is a meaningful range is not always true!
- Such a shape is possible in optimization:
  - The global optimum marks a lower bound for the possible objective values.
  - A good algorithm often returns results which are close-to-optimal.



Further Example

- The implicit assumption that $\text{mean} \pm \text{sd}$ is a meaningful range is not always true!
- Such a shape is possible in optimization:
  - The global optimum marks a lower bound for the possible objective values.
  - A good algorithm often returns results which are close-to-optimal.
  - There may be a long tail of few but significantly worse runs.



Further Example



- The implicit assumption that $\mathrm{mean} \pm \mathrm{sd}$ is a meaningful range is not always true!
- Such a shape is possible in optimization:
  - The global optimum marks a lower bound for the possible objective values.
  - A good algorithm often returns results which are close-to-optimal.
  - There may be a long tail of few but significantly worse runs.
  - A statement such as *"For this TSP instance, our algorithm can find tours with a length of* 100 ± 120 *km."* makes little sense...

Statistical Comparisons

Introduction

- We can now, e.g., perform 20 runs each with two different optimization algorithms on one problem instance and compute the medians of a performance indicator.

Introduction

- We can now, e.g., perform 20 runs each with two different optimization algorithms on one problem instance and compute the medians of a performance indicator.
- Likely, they will be different.

Introduction

- We can now, e.g., perform 20 runs each with two different optimization algorithms on one problem instance and compute the medians of a performance indicator.
- Likely, they will be different.
- For one of the two algorithms, the results will be better.

Introduction

- We can now, e.g., perform 20 runs each with two different optimization algorithms on one problem instance and compute the medians of a performance indicator.
- Likely, they will be different.
- For one of the two algorithms, the results will be better.
- What does this mean?

Introduction

- We can now, e.g., perform 20 runs each with two different optimization algorithms on one problem instance and compute the medians of a performance indicator.
- Likely, they will be different.
- For one of the two algorithms, the results will be better.
- What does this mean?
- It means that one of the two algorithms is better.

Introduction

- We can now, e.g., perform 20 runs each with two different optimization algorithms on one problem instance and compute the medians of a performance indicator.
- Likely, they will be different.
- For one of the two algorithms, the results will be better.
- What does this mean?
- It means that one of the two algorithms is better with a certain probability.

Introduction

- We can now, e.g., perform 20 runs each with two different optimization algorithms on one problem instance and compute the medians of a performance indicator.
- Likely, they will be different.
- For one of the two algorithms, the results will be better.
- What does this mean?
- It means that one of the two algorithms is better with a certain probability.
- If we say *"A is better than B,"* we have a certain probability p to be wrong.

Introduction

- We can now, e.g., perform 20 runs each with two different optimization algorithms on one problem instance and compute the medians of a performance indicator.
- Likely, they will be different.
- For one of the two algorithms, the results will be better.
- What does this mean?
- It means that one of the two algorithms is better with a certain probability.
- If we say *"A is better than B,"* we have a certain probability p to be wrong.
- The statement *"A is better than B"* makes only sense after we have decided about an upper bound α for the acceptable error probability p! (and if $p < \alpha$, obviously)

Statistical Tests

- Compare two data samples $A = (a_1, a_2, \dots)$ and $B = (b_1, b_2, \dots)$ and

Statistical Tests

- Compare two data samples $A = (a_1, a_2, \dots)$ and $B = (b_1, b_2, \dots)$ and
- get a result (e.g., *"The median of A is bigger than the median of B"*) together with an error probability p that the conclusion is wrong.

Statistical Tests

- Compare two data samples $A = (a_1, a_2, \dots)$ and $B = (b_1, b_2, \dots)$ and
- get a result (e.g., *"The median of A is bigger than the median of B"*) together with an error probability p that the conclusion is wrong.
- If p is less than a previously chosen significance level (upper bound) α, we can accept the conclusion.

Statistical Tests

- Compare two data samples $A = (a_1, a_2, \dots)$ and $B = (b_1, b_2, \dots)$ and
- get a result (e.g., *"The median of A is bigger than the median of B"*) together with an error probability p that the conclusion is wrong.
- If p is less than a previously chosen significance level (upper bound) α, we can accept the conclusion.
- Otherwise, the observation is not significant.

Statistical Tests

- Compare two data samples $A = (a_1, a_2, \dots)$ and $B = (b_1, b_2, \dots)$ and
- get a result (e.g., *"The median of A is bigger than the median of B"*) together with an error probability p that the conclusion is wrong.
- If p is less than a previously chosen significance level (upper bound) α, we can accept the conclusion.
- Otherwise, the observation is not significant and must be ignored.

Statistical Tests

- Compare two data samples $A = (a_1, a_2, \dots)$ and $B = (b_1, b_2, \dots)$ and
- get a result (e.g., *"The median of A is bigger than the median of B"*) together with an error probability p that the conclusion is wrong.
- If p is less than a previously chosen significance level (upper bound) α, we can accept the conclusion.
- Otherwise, the observation is not significant and must be ignored.
- But how can we arrive at such statements? How can we even estimate a probability to be wrong?

Statistical Tests

- Compare two data samples $A = (a_1, a_2, \dots)$ and $B = (b_1, b_2, \dots)$ and
- get a result (e.g., *"The median of A is bigger than the median of B"*) together with an error probability p that the conclusion is wrong.
- If p is less than a previously chosen significance level (upper bound) α, we can accept the conclusion.
- Otherwise, the observation is not significant and must be ignored.
- But how can we arrive at such statements? How can we even estimate a probability to be wrong?
- Disclaimer: I am not a mathematician. What follows are simplified explanations of concepts.

Example for Underlying Idea

- Coin flip game: We flip a coin. If it is heads, I give you 1 RMB, if it is tails, you give me 1 RMB.

Example for Underlying Idea

- Coin flip game: We flip a coin. If it is heads, I give you 1 RMB, if it is tails, you give me 1 RMB.



heads

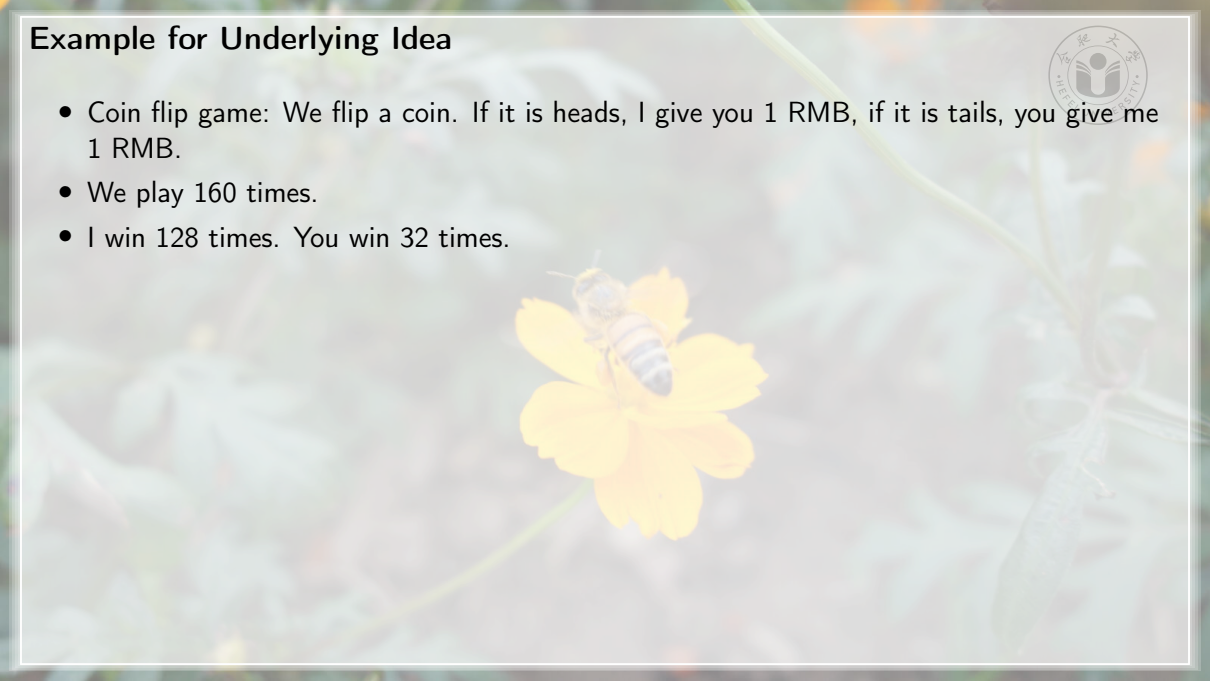


tails

Example for Underlying Idea

- Coin flip game: We flip a coin. If it is heads, I give you 1 RMB, if it is tails, you give me 1 RMB.
- We play 160 times.

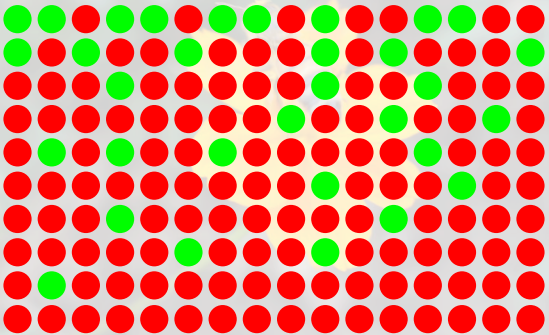
Example for Underlying Idea



- Coin flip game: We flip a coin. If it is heads, I give you 1 RMB, if it is tails, you give me 1 RMB.
- We play 160 times.
- I win 128 times. You win 32 times.

Example for Underlying Idea

- Coin flip game: We flip a coin. If it is heads, I give you 1 RMB, if it is tails, you give me 1 RMB.
- We play 160 times.
- I win 128 times. You win 32 times.



Example for Underlying Idea



- Coin flip game: We flip a coin. If it is heads, I give you 1 RMB, if it is tails, you give me 1 RMB.
- We play 160 times.
- I win 128 times. You win 32 times.
- Did I cheat? Is my coin "fixed?" (i.e., is your chance to win $\neq 0.5$)

Example for Underlying Idea



- Coin flip game: We flip a coin. If it is heads, I give you 1 RMB, if it is tails, you give me 1 RMB.
- We play 160 times.
- I win 128 times. You win 32 times.
- Did I cheat? Is my coin "fixed?" (i.e., is your chance to win $\neq 0.5$)
- Assumption: I cheat. (alternative hypothesis H_1)

Example for Underlying Idea



- Coin flip game: We flip a coin. If it is heads, I give you 1 RMB, if it is tails, you give me 1 RMB.
- We play 160 times.
- I win 128 times. You win 32 times.
- Did I cheat? Is my coin "fixed?" (i.e., is your chance to win $\neq 0.5$)
- Assumption: I cheat. (alternative hypothesis H_1)
- It is impossible to compute my winning probability if I cheated...

Example for Underlying Idea



- Coin flip game: We flip a coin. If it is heads, I give you 1 RMB, if it is tails, you give me 1 RMB.
- We play 160 times.
- I win 128 times. You win 32 times.
- Did I cheat? Is my coin "fixed?" (i.e., is your chance to win $\neq 0.5$)
- Assumption: I cheat. (alternative hypothesis H_1)
- It is impossible to compute my winning probability if I cheated...
- Counter-Assumption: I did not cheat. (null hypothesis H_0)

Example for Underlying Idea



- Coin flip game: We flip a coin. If it is heads, I give you 1 RMB, if it is tails, you give me 1 RMB.
- We play 160 times.
- I win 128 times. You win 32 times.
- Did I cheat? Is my coin "fixed?" (i.e., is your chance to win $\neq 0.5$)
- Assumption: I cheat. (alternative hypothesis H_1)
- It is impossible to compute my winning probability if I cheated...
- Counter-Assumption: I did not cheat. (null hypothesis H_0)
- How likely is it that I win at least 128 times if I did not cheat?

Example for Underlying Idea



- Coin flip game: We flip a coin. If it is heads, I give you 1 RMB, if it is tails, you give me 1 RMB.
- We play 160 times.
- I win 128 times. You win 32 times.
- Did I cheat? Is my coin "fixed?" (i.e., is your chance to win $\neq 0.5$)
- Assumption: I cheat. (alternative hypothesis H_1)
- It is impossible to compute my winning probability if I cheated...
- Counter-Assumption: I did not cheat. (null hypothesis H_0)
- How likely is it that I win at least 128 times if I did not cheat?
- (What we will do right now is called *binomial test*.)

Example for Underlying Idea

- How likely is it that I win at least 128 times if I did not cheat?

Example for Underlying Idea

- How likely is it that I win at least 128 times if I did not cheat?
- Then, the probabilities for heads and tails are $q = P(\texttt{head}) = P(\texttt{tail}) = 0.5$.

Example for Underlying Idea

- How likely is it that I win at least 128 times if I did not cheat?
- Then, the probabilities for heads and tails are $q = P(\texttt{head}) = P(\texttt{tail}) = 0.5$.
- Flipping a coin n times is a Bernoulli Process

Example for Underlying Idea

- How likely is it that I win at least 128 times if I did not cheat?
- Then, the probabilities for heads and tails are $q = P(\texttt{head}) = P(\texttt{tail}) = 0.5$.
- Flipping a coin n times is a Bernoulli Process
- The probability $P(k|n)$ to flip $k \in 0..n$ times heads (or tails) is thus:

$$P(k|n) = \binom{n}{k} 0.5^k * (1 - 0.5)^{n-k} = \binom{n}{k} 0.5^k * 0.5^{n-k} = \binom{n}{k} \frac{1}{2^n}$$

Example for Underlying Idea

- How likely is it that I win at least 128 times if I did not cheat?
- Then, the probabilities for heads and tails are $q = P(\texttt{head}) = P(\texttt{tail}) = 0.5$.
- Flipping a coin n times is a Bernoulli Process
- The probability $P(k|n)$ to flip $k \in 0..n$ times heads (or tails) is thus:

$$P(k|n) = \binom{n}{k} 0.5^k * (1 - 0.5)^{n-k} = \binom{n}{k} 0.5^k * 0.5^{n-k} = \binom{n}{k} \frac{1}{2^n}$$

- For winning at least $z = 128$ times, we need to compute:

$$P(k \geq z|n) \quad = \quad \sum_{i=z}^{n} P(i|n)$$

Example for Underlying Idea

- How likely is it that I win at least 128 times if I did not cheat?
- Then, the probabilities for heads and tails are $q = P(\texttt{head}) = P(\texttt{tail}) = 0.5$.
- Flipping a coin n times is a Bernoulli Process
- The probability $P(k|n)$ to flip $k \in 0..n$ times heads (or tails) is thus:

$$P(k|n) = \binom{n}{k} 0.5^k * (1 - 0.5)^{n-k} = \binom{n}{k} 0.5^k * 0.5^{n-k} = \binom{n}{k} \frac{1}{2^n}$$

- For winning at least $z = 128$ times, we need to compute:

$$P(k \geq z|n) = \sum_{i=z}^{n} P(i|n) = \sum_{i=128}^{160} P(i|160) = \sum_{i=128}^{160} \left[\binom{160}{i} \frac{1}{2^{160}} \right]$$

Example for Underlying Idea

- How likely is it that I win at least 128 times if I did not cheat?
- Then, the probabilities for heads and tails are $q = P(\texttt{head}) = P(\texttt{tail}) = 0.5$.
- Flipping a coin n times is a Bernoulli Process
- The probability $P(k|n)$ to flip $k \in 0..n$ times heads (or tails) is thus:

$$P(k|n) = \binom{n}{k} 0.5^k * (1 - 0.5)^{n-k} = \binom{n}{k} 0.5^k * 0.5^{n-k} = \binom{n}{k} \frac{1}{2^n}$$

- For winning at least $z = 128$ times, we need to compute:

$$P(k \geq z|n) = \sum_{i=z}^{n} P(i|n) = \frac{1}{2^{160}} \sum_{i=128}^{160} \binom{160}{i}$$

Example for Underlying Idea

- How likely is it that I win at least 128 times if I did not cheat?
- Then, the probabilities for heads and tails are $q = P(\texttt{head}) = P(\texttt{tail}) = 0.5$.
- Flipping a coin n times is a Bernoulli Process
- The probability $P(k|n)$ to flip $k \in 0..n$ times heads (or tails) is thus:

$$P(k|n) = \binom{n}{k} 0.5^k * (1 - 0.5)^{n-k} = \binom{n}{k} 0.5^k * 0.5^{n-k} = \binom{n}{k} \frac{1}{2^n}$$

- For winning at least $z = 128$ times, we need to compute:

$$\begin{aligned} P(k \geq z|n) &= \sum_{i=z}^{n} P(i|n) = \frac{1}{2^{160}} \sum_{i=128}^{160} \binom{160}{i} \\ &= \frac{1'538'590'628'148'134'280'316'221'828'039'113}{365'375'409'332'725'729'550'921'208'179'070'754'913'983'135'744} \end{aligned}$$

Example for Underlying Idea

- How likely is it that I win at least 128 times if I did not cheat?
- Then, the probabilities for heads and tails are $q = P(\texttt{head}) = P(\texttt{tail}) = 0.5$.
- Flipping a coin n times is a Bernoulli Process
- The probability $P(k|n)$ to flip $k \in 0..n$ times heads (or tails) is thus:

$$P(k|n) = \binom{n}{k} 0.5^k * (1 - 0.5)^{n-k} = \binom{n}{k} 0.5^k * 0.5^{n-k} = \binom{n}{k} \frac{1}{2^n}$$

- For winning at least $z = 128$ times, we need to compute:

$$P(k \geq z|n) = \sum_{i=z}^{n} P(i|n) = \frac{1}{2^{160}} \sum_{i=128}^{160} \binom{160}{i} \approx \frac{1.539 * 10^{33}}{3.654 * 10^{47}}$$

Example for Underlying Idea

- How likely is it that I win at least 128 times if I did not cheat?
- Then, the probabilities for heads and tails are $q = P(\texttt{head}) = P(\texttt{tail}) = 0.5$.
- Flipping a coin n times is a Bernoulli Process
- The probability $P(k|n)$ to flip $k \in 0..n$ times heads (or tails) is thus:

$$P(k|n) = \binom{n}{k} 0.5^k * (1 - 0.5)^{n-k} = \binom{n}{k} 0.5^k * 0.5^{n-k} = \binom{n}{k} \frac{1}{2^n}$$

- For winning at least $z = 128$ times, we need to compute:

$$\begin{aligned} P(k \geq z|n) &= \sum_{i=z}^{n} P(i|n) = \frac{1}{2^{160}} \sum_{i=128}^{160} \binom{160}{i} \approx \frac{1.539 * 10^{33}}{3.654 * 10^{47}} \\ &\approx 0.00000000000000421098571 \end{aligned}$$

Example for Underlying Idea

- How likely is it that I win at least 128 times if I did not cheat?
- Then, the probabilities for heads and tails are $q = P(\texttt{head}) = P(\texttt{tail}) = 0.5$.
- Flipping a coin n times is a Bernoulli Process
- The probability $P(k|n)$ to flip $k \in 0..n$ times heads (or tails) is thus:

$$P(k|n) = \binom{n}{k} 0.5^k * (1 - 0.5)^{n-k} = \binom{n}{k} 0.5^k * 0.5^{n-k} = \binom{n}{k} \frac{1}{2^n}$$

- For winning at least $z = 128$ times, we need to compute:

$$\begin{aligned} P(k \geq z|n) &= \sum_{i=z}^{n} P(i|n) = \frac{1}{2^{160}} \sum_{i=128}^{160} \binom{160}{i} \approx \frac{1.539 * 10^{33}}{3.654 * 10^{47}} \\ &\approx 4.211 \cdot 10^{-15} \end{aligned}$$

Example for Underlying Idea

- Question: How likely is it that I win at least 128 times if I did not cheat?

Example for Underlying Idea



- Question: How likely is it that I win at least 128 times if I did not cheat?
- If the coin was an ideal coin, the chance that I win at least 128 out of 160 times is about $4 \cdot 10^{-15}$.

Example for Underlying Idea



- Question: How likely is it that I win at least 128 times if I did not cheat?
- If the coin was an ideal coin, the chance that I win at least 128 out of 160 times is about $4 \cdot 10^{-15}$.
- If you claim that I cheat, your chance to be wrong is about $4 \cdot 10^{-15}$.

Example for Underlying Idea

- Question: How likely is it that I win at least 128 times if I did not cheat?
- If the coin was an ideal coin, the chance that I win at least 128 out of 160 times is about $4 \cdot 10^{-15}$.
- If you claim that I cheat, your chance to be wrong is about $4 \cdot 10^{-15}$.
- Thus, if we cannot accept a chance p to be wrong higher than a significance level $\alpha = 1\%$, we can still say:

The observation is significant, I did likely cheat.

A More Specific Example for Tests



- We want to compare two algorithms $\mathcal{A}$ and $\mathcal{B}$ on a given problem instance.

A More Specific Example for Tests



- We want to compare two algorithms $\mathcal{A}$ and $\mathcal{B}$ on a given problem instance.
- We have conducted a small experiment and measured objective values of their final results in a few runs in form of the two data sets A and B, respectively:

$$\begin{aligned} A &= (2, 5, 6, 7, 9, 10) \\ B &= (1, 3, 4, 8) \end{aligned}$$

A More Specific Example for Tests



- We want to compare two algorithms $\mathcal{A}$ and $\mathcal{B}$ on a given problem instance.
- We have conducted a small experiment and measured objective values of their final results in a few runs in form of the two data sets A and B, respectively:

$$\begin{aligned} A &= (2, 5, 6, 7, 9, 10) \\ B &= (1, 3, 4, 8) \end{aligned}$$

- From this, we can estimate the arithmetic means:

A More Specific Example for Tests



- We want to compare two algorithms $\mathcal{A}$ and $\mathcal{B}$ on a given problem instance.
- We have conducted a small experiment and measured objective values of their final results in a few runs in form of the two data sets A and B, respectively:

$$\begin{aligned} A &= (2, 5, 6, 7, 9, 10) \\ B &= (1, 3, 4, 8) \end{aligned}$$

- From this, we can estimate the arithmetic means:

$$\begin{aligned} \mathrm{mean}(A) &= \frac{39}{6} = 6.5 \\ \mathrm{mean}(B) &= \frac{16}{4} = 4 \end{aligned}$$

A More Specific Example

$$\begin{aligned} \text{mean}(A) &= \frac{39}{6} = 6.5 \\ \text{mean}(B) &= \frac{16}{4} = 4 \end{aligned}$$

- It looks like algorithm $\mathcal{B}$ may produce the smaller objective values.

A More Specific Example

$$\begin{aligned} \text{mean}(A) &= \frac{39}{6} = 6.5 \\ \text{mean}(B) &= \frac{16}{4} = 4 \end{aligned}$$

- It looks like algorithm $\mathcal{B}$ may produce the smaller objective values.
- But is this assumption justified based on the data we have?

A More Specific Example

$$\begin{aligned} \text{mean}(A) &= \frac{39}{6} = 6.5 \\ \text{mean}(B) &= \frac{16}{4} = 4 \end{aligned}$$

- It looks like algorithm $\mathcal{B}$ may produce the smaller objective values.
- But is this assumption justified based on the data we have?
- Is the difference between $\text{mean}(A)$ and $\text{mean}(B)$ significant at a threshold of $\alpha = 2\%$?

A More Specific Example



- If $\mathcal{B}$ is truly better than $\mathcal{A}$, which is our hypothesis H_1, then we cannot calculate anything.

A More Specific Example



- If $\mathcal{B}$ is truly better than $\mathcal{A}$, which is our hypothesis H_1, then we cannot calculate anything.
- Let us therefore assume as null hypothesis H_0 the observed difference did just happen by chance and, well, $\mathcal{A} \equiv \mathcal{B}$.

A More Specific Example



- If $\mathcal{B}$ is truly better than $\mathcal{A}$, which is our hypothesis H_1, then we cannot calculate anything.
- Let us therefore assume as null hypothesis H_0 the observed difference did just happen by chance and, well, $\mathcal{A} \equiv \mathcal{B}$.
- Then, this would mean that the data samples A and B stem from the same algorithm (as $\mathcal{A} \equiv \mathcal{B}$).

A More Specific Example



- If $\mathcal{B}$ is truly better than $\mathcal{A}$, which is our hypothesis H_1, then we cannot calculate anything.
- Let us therefore assume as null hypothesis H_0 the observed difference did just happen by chance and, well, $\mathcal{A} \equiv \mathcal{B}$.
- Then, this would mean that the data samples A and B stem from the same algorithm (as $\mathcal{A} \equiv \mathcal{B}$).
- The division into the two sets would only be artificial, an artifact of our experimental design.

A More Specific Example



- If $\mathcal{B}$ is truly better than $\mathcal{A}$, which is our hypothesis H_1, then we cannot calculate anything.
- Let us therefore assume as null hypothesis H_0 the observed difference did just happen by chance and, well, $\mathcal{A} \equiv \mathcal{B}$.
- Then, this would mean that the data samples A and B stem from the same algorithm (as $\mathcal{A} \equiv \mathcal{B}$).
- The division into the two sets would only be artificial, an artifact of our experimental design.
- Instead of having two data samples, we only have one, namely the union set O with 10 elements:

A More Specific Example



- If $\mathcal{B}$ is truly better than $\mathcal{A}$, which is our hypothesis H_1, then we cannot calculate anything.
- Let us therefore assume as null hypothesis H_0 the observed difference did just happen by chance and, well, $\mathcal{A} \equiv \mathcal{B}$.
- Then, this would mean that the data samples A and B stem from the same algorithm (as $\mathcal{A} \equiv \mathcal{B}$).
- The division into the two sets would only be artificial, an artifact of our experimental design.
- Instead of having two data samples, we only have one, namely the union set O with 10 elements:

$$O = A \cup B = (1, 2, 3, 4, 5, 6, 7, 8, 9, 10)$$

A More Specific Example

$$O = A \cup B = (1, 2, 3, 4, 5, 6, 7, 8, 9, 10)$$

- Any division C into two sets with 4 and 6 elements has the same probability

A More Specific Example

$$O = A \cup B = (1, 2, 3, 4, 5, 6, 7, 8, 9, 10)$$

- Any division C into two sets with 4 and 6 elements has the same probability
- $|O| = 10$

A More Specific Example

$$O = A \cup B = (1, 2, 3, 4, 5, 6, 7, 8, 9, 10)$$

- Any division C into two sets with 4 and 6 elements has the same probability
- $|O| = 10$
- There are $\binom{10}{4} = 210$ different ways to draw 4 (or 6) elements from O

A More Specific Example

$$O = A \cup B = (1, 2, 3, 4, 5, 6, 7, 8, 9, 10)$$

- Any division C into two sets with 4 and 6 elements has the same probability
- $|O| = 10$
- There are $\binom{10}{4} = 210$ different ways to draw 4 (or 6) elements from O
- If H_0 holds, all have the same probability

A More Specific Example

$$O = A \cup B = (1, 2, 3, 4, 5, 6, 7, 8, 9, 10)$$

- Any division C into two sets with 4 and 6 elements has the same probability
- $|O| = 10$
- There are $\binom{10}{4} = 210$ different ways to draw 4 (or 6) elements from O
- If H_0 holds, all have the same probability
- Let's use a Python[250] program to test the combinations

A More Specific Example

```
"""Enumerate all combinations of numbers 1 to 10."""
mean_lower_or_equal_to_4 = 0  # how often did we find a mean <= 4
total_combinations       = 0  # total number of tested combinations

for i in range(1, 11):              # i goes from 1 to 10
    for j in range(1, i):           # j goes from 1 to i - 1
        for k in range(1, j):       # k goes from 1 to j - 1
            for l in range(1, k):   # l goes from 1 to k - 1
                if ((i + j + k + l) / 4) <= 4:  # check for extreme case
                    mean_lower_or_equal_to_4 += 1  # count extreme case
                total_combinations += 1   # count all combinations

print(f" combinations with mean <= 4: {mean_lower_or_equal_to_4}")
print(f"total number of combinations: {total_combinations}")
```

A More Specific Example



```
"""Enumerate all combinations of numbers 1 to 10."""
mean_lower_or_equal_to_4 = 0  # how often did we find a mean <= 4
total_combinations       = 0  # total number of tested combinations

for i in range(1, 11):              # i goes from 1 to 10
    for j in range(1, i):           # j goes from 1 to i - 1
        for k in range(1, j):       # k goes from 1 to j - 1
            for l in range(1, k):   # l goes from 1 to k - 1
                if ((i + j + k + l) / 4) <= 4:  # check for extreme case
                    mean_lower_or_equal_to_4 += 1  # count extreme case
                total_combinations += 1   # count all combinations

print(f" combinations with mean <= 4: {mean_lower_or_equal_to_4}")
print(f"total number of combinations: {total_combinations}")
```

```
 combinations with mean <= 4: 27
total number of combinations: 210
```

A More Specific Example

$$O = A \cup B = (1, 2, 3, 4, 5, 6, 7, 8, 9, 10)$$

- Any division C into two sets with 4 and 6 elements has the same probability
- $|O| = 10$
- There are $\binom{10}{4} = 210$ different ways to draw 4 (or 6) elements from O
- If H_0 holds, all have the same probability
- There are 27 such combinations with a mean of less or equal 4.

A More Specific Example

$$O = A \cup B = (1, 2, 3, 4, 5, 6, 7, 8, 9, 10)$$

- Any division C into two sets with 4 and 6 elements has the same probability
- $|O| = 10$
- There are $\binom{10}{4} = 210$ different ways to draw 4 (or 6) elements from O
- If H_0 holds, all have the same probability
- There are 27 such combinations with a mean of less or equal 4.
- The probability p to observe a situation at least as extreme as A and B under H_0 is thus:

A More Specific Example

$$O = A \cup B = (1, 2, 3, 4, 5, 6, 7, 8, 9, 10)$$

- Any division C into two sets with 4 and 6 elements has the same probability
- $|O| = 10$
- There are $\binom{10}{4} = 210$ different ways to draw 4 (or 6) elements from O
- If H_0 holds, all have the same probability
- There are 27 such combinations with a mean of less or equal 4.
- The probability p to observe a situation at least as extreme as A and B under H_0 is thus:

$$p = \frac{\#\text{cases } C : \text{mean}(C) \leq \text{mean}(B)}{\#\text{all cases}} = \frac{27}{210} = \frac{9}{70} \approx 0.1286$$

A More Specific Example

- Extreme cases into the other direction are the same, because if $\mathrm{mean}(B) \leq 4$ then $\mathrm{mean}(A) \geq 6.5$ for any division $A \cup B = O$ and vice versa.

$$O \quad = \quad A \cup B = (1, 2, 3, 4, 5, 6, 7, 8, 9, 10)$$

A More Specific Example

- Extreme cases into the other direction are the same, because if $\mathrm{mean}(B) \leq 4$ then $\mathrm{mean}(A) \geq 6.5$ for any division $A \cup B = O$ and vice versa:

$$O = A \cup B = (1, 2, 3, 4, 5, 6, 7, 8, 9, 10)$$

$$\sum_{\forall o \in O} o = \sum_{o=1}^{10} o = \frac{10(10+1)}{2} = 55$$

A More Specific Example

- Extreme cases into the other direction are the same, because if $\mathrm{mean}(B) \leq 4$ then $\mathrm{mean}(A) \geq 6.5$ for any division $A \cup B = O$ and vice versa:

$$O = A \cup B = (1, 2, 3, 4, 5, 6, 7, 8, 9, 10)$$

$$\sum_{\forall o \in O} o = \sum_{o=1}^{10} o = \frac{10(10+1)}{2} = 55$$

$$\mathrm{mean}(B) = \left(\frac{1}{4} \sum_{\forall b \in B} b\right) \leq 4 \implies \left(\sum_{\forall b \in B} b\right) \leq 4 * 4 \leq 16$$

A More Specific Example

- Extreme cases into the other direction are the same, because if $\mathrm{mean}(B) \leq 4$ then $\mathrm{mean}(A) \geq 6.5$ for any division $A \cup B = O$ and vice versa:

$$O = A \cup B = (1, 2, 3, 4, 5, 6, 7, 8, 9, 10)$$

$$\sum_{\forall o \in O} o = \sum_{o=1}^{10} o = \frac{10(10+1)}{2} = 55$$

$$\mathrm{mean}(B) = \left(\frac{1}{4} \sum_{\forall b \in B} b\right) \leq 4 \implies \left(\sum_{\forall b \in B} b\right) \leq 4 * 4 \leq 16$$

$$O = A \cup B \implies \sum_{\forall a \in A} a = \left(\sum_{\forall o \in O} o\right) - \left(\sum_{\forall b \in B} b\right)$$

A More Specific Example

- Extreme cases into the other direction are the same, because if $\text{mean}(B) \leq 4$ then $\text{mean}(A) \geq 6.5$ for any division $A \cup B = O$ and vice versa:

$$O = A \cup B = (1, 2, 3, 4, 5, 6, 7, 8, 9, 10)$$

$$\sum_{\forall o \in O} o = \sum_{o=1}^{10} o = \frac{10(10+1)}{2} = 55$$

$$\text{mean}(B) = \left(\frac{1}{4} \sum_{\forall b \in B} b\right) \leq 4 \implies \left(\sum_{\forall b \in B} b\right) \leq 4 * 4 \leq 16$$

$$O = A \cup B \implies \sum_{\forall a \in A} a = \left(\sum_{\forall o \in O} o\right) - \left(\sum_{\forall b \in B} b\right)$$

$$\sum_{\forall b \in B} b \leq 16 \implies \left(\sum_{\forall a \in A} a\right) \geq 55 - 16 \geq 39$$

A More Specific Example

- Extreme cases into the other direction are the same, because if $\text{mean}(B) \leq 4$ then $\text{mean}(A) \geq 6.5$ for any division $A \cup B = O$ and vice versa:

$$\sum_{\forall o \in O} o = \sum_{o=1}^{10} o = \frac{10(10+1)}{2} = 55$$

$$\text{mean}(B) = \left(\frac{1}{4} \sum_{\forall b \in B} b\right) \leq 4 \implies \left(\sum_{\forall b \in B} b\right) \leq 4 * 4 \leq 16$$

$$O = A \cup B \implies \sum_{\forall a \in A} a = \left(\sum_{\forall o \in O} o\right) - \left(\sum_{\forall b \in B} b\right)$$

$$\sum_{\forall b \in B} b \leq 16 \implies \left(\sum_{\forall a \in A} a\right) \geq 55 - 16 \geq 39$$

$$\text{mean}(A) = \frac{1}{6}\left(\sum_{\forall a \in A} a\right)$$

A More Specific Example

- Extreme cases into the other direction are the same, because if $\text{mean}(B) \leq 4$ then $\text{mean}(A) \geq 6.5$ for any division $A \cup B = O$ and vice versa:

$$\text{mean}(B) = \left(\frac{1}{4} \sum_{\forall b \in B} b\right) \leq 4 \implies \left(\sum_{\forall b \in B} b\right) \leq 4 * 4 \leq 16$$

$$O = A \cup B \implies \sum_{\forall a \in A} a = \left(\sum_{\forall o \in O} o\right) - \left(\sum_{\forall b \in B} b\right)$$

$$\sum_{\forall b \in B} b \leq 16 \implies \left(\sum_{\forall a \in A} a\right) \geq 55 - 16 \geq 39$$

$$\text{mean}(A) = \frac{1}{6}\left(\sum_{\forall a \in A} a\right)$$

$$\text{mean}(B) \leq 4 \Rightarrow \text{mean}(A) \geq \frac{39}{6} \geq 6.5$$

A More Specific Example

- Extreme cases into the other direction are the same, because if $\text{mean}(B) \leq 4$ then $\text{mean}(A) \geq 6.5$ for any division $A \cup B = O$ and vice versa:

$$\text{mean}(B) = \left(\frac{1}{4}\sum_{\forall b \in B} b\right) \leq 4 \quad \Longrightarrow \quad \left(\sum_{\forall b \in B} b\right) \leq 4 * 4 \leq 16$$

$$O = A \cup B \quad \Longrightarrow \quad \sum_{\forall a \in A} a = \left(\sum_{\forall o \in O} o\right) - \left(\sum_{\forall b \in B} b\right)$$

$$\sum_{\forall b \in B} b \leq 16 \quad \Longrightarrow \quad \left(\sum_{\forall a \in A} a\right) \geq 55 - 16 \geq 39$$

$$\text{mean}(A) \quad = \quad \frac{1}{6}\left(\sum_{\forall a \in A} a\right)$$

$$\text{mean}(B) \leq 4 \quad \Rightarrow \quad \text{mean}(A) \geq \frac{39}{6} \geq 6.5$$

- So – of course – we could have also done the test the other way around with the same result!

A More Specific Example

- The probability p to observe a constallation at least as extreme as A or B under H_0 is thus:

$$p = \frac{\#\text{cases } C : \text{mean}(C) \leq \text{mean}(B)}{\#\text{all cases}} = \frac{27}{210} = \frac{9}{70} \approx 0.1286$$

A More Specific Example

- The probability p to observe a constallation at least as extreme as A or B under H_0 is thus:

$$p = \frac{\#\text{cases } C : \text{mean}(C) \leq \text{mean}(B)}{\#\text{all cases}} = \frac{27}{210} = \frac{9}{70} \approx 0.1286$$

- If we claim that A and B are from distributions with means as different as observed...

A More Specific Example

- The probability p to observe a constallation at least as extreme as A or B under H_0 is thus:

$$p = \frac{\#\text{cases } C : \text{mean}(C) \leq \text{mean}(B)}{\#\text{all cases}} = \frac{27}{210} = \frac{9}{70} \approx 0.1286$$

- If we claim that A and B are from distributions with means as different as observed...
- ...we are wrong with probability $p \approx 0.13$

A More Specific Example

- The probability p to observe a constallation at least as extreme as A or B under H_0 is thus:

$$p = \frac{\#\text{cases } C : \text{mean}(C) \leq \text{mean}(B)}{\#\text{all cases}} = \frac{27}{210} = \frac{9}{70} \approx 0.1286$$

- If we claim that A and B are from distributions with means as different as observed...
- ...we are wrong with probability $p \approx 0.13$
- At a significance level of $\alpha = 2\%$, the means of A and B are not significantly different! ($2\% < 0.13$)

A More Specific Example

- The probability p to observe a constallation at least as extreme as A or B under H_0 is thus:

$$p = \frac{\#\text{cases } C : \text{mean}(C) \leq \text{mean}(B)}{\#\text{all cases}} = \frac{27}{210} = \frac{9}{70} \approx 0.1286$$

- If we claim that A and B are from distributions with means as different as observed...
- ...we are wrong with probability $p \approx 0.13$
- At a significance level of $\alpha = 2\%$, the means of A and B are not significantly different! ($2\% < 0.13$)
- Actually: This here is an example for an *Randomization Test*[36,83].

A More Specific Example

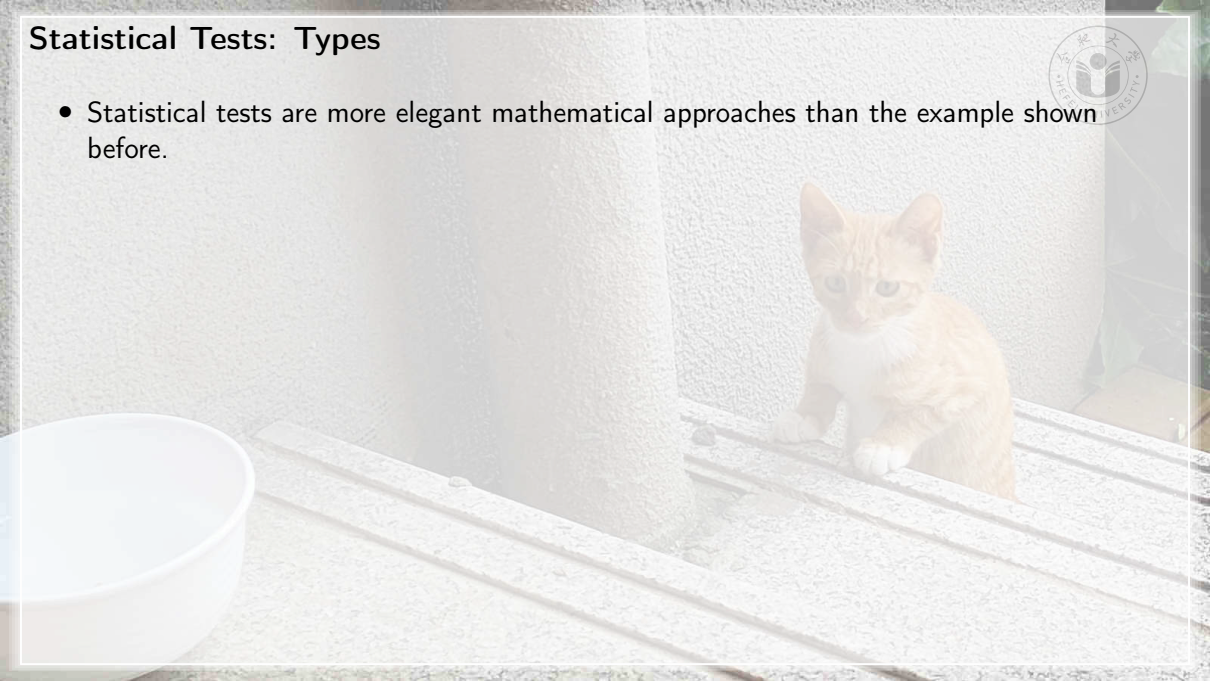
- The probability p to observe a constallation at least as extreme as A or B under H_0 is thus:

$$p = \frac{\#\text{cases } C : \text{mean}(C) \leq \text{mean}(B)}{\#\text{all cases}} = \frac{27}{210} = \frac{9}{70} \approx 0.1286$$

- If we claim that A and B are from distributions with means as different as observed...
- ...we are wrong with probability $p \approx 0.13$
- At a significance level of $\alpha = 2\%$, the means of A and B are not significantly different! ($2\% < 0.13$)
- Actually: This here is an example for an *Randomization Test*[36,83].
- The method here is only feasible for small sample sets, real tests are more sophisticated

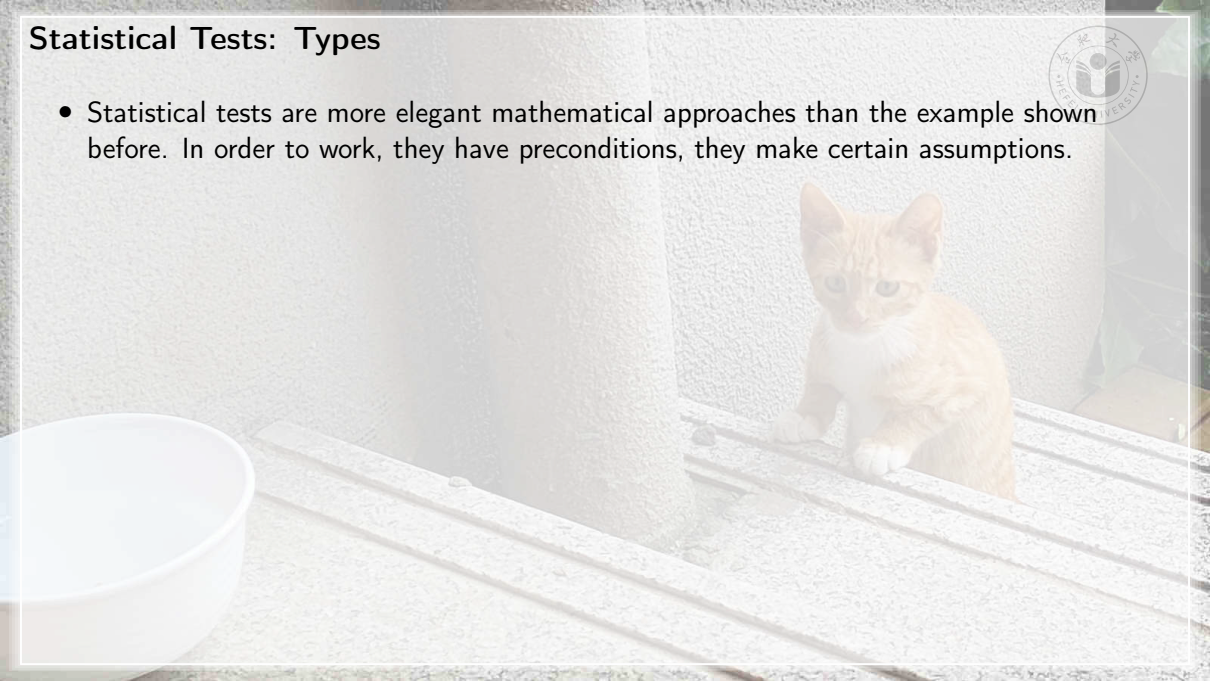
Statistical Tests: Types

- Statistical tests are more elegant mathematical approaches than the example shown before.



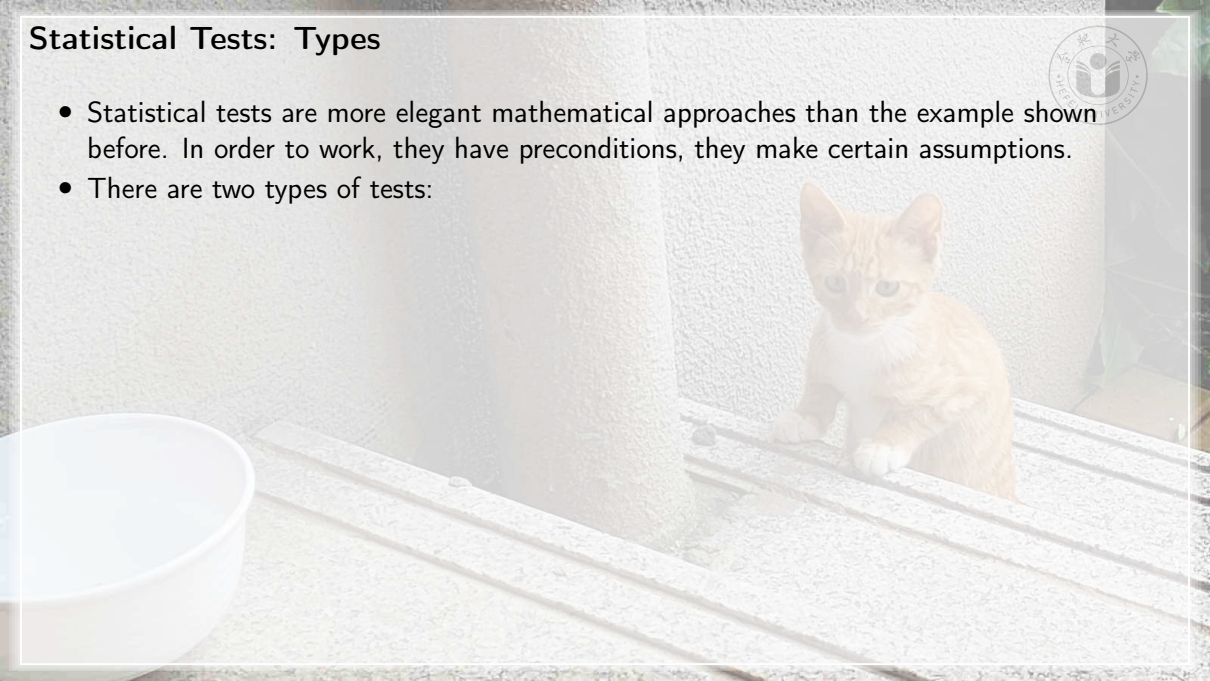
Statistical Tests: Types

- Statistical tests are more elegant mathematical approaches than the example shown before. In order to work, they have preconditions, they make certain assumptions.



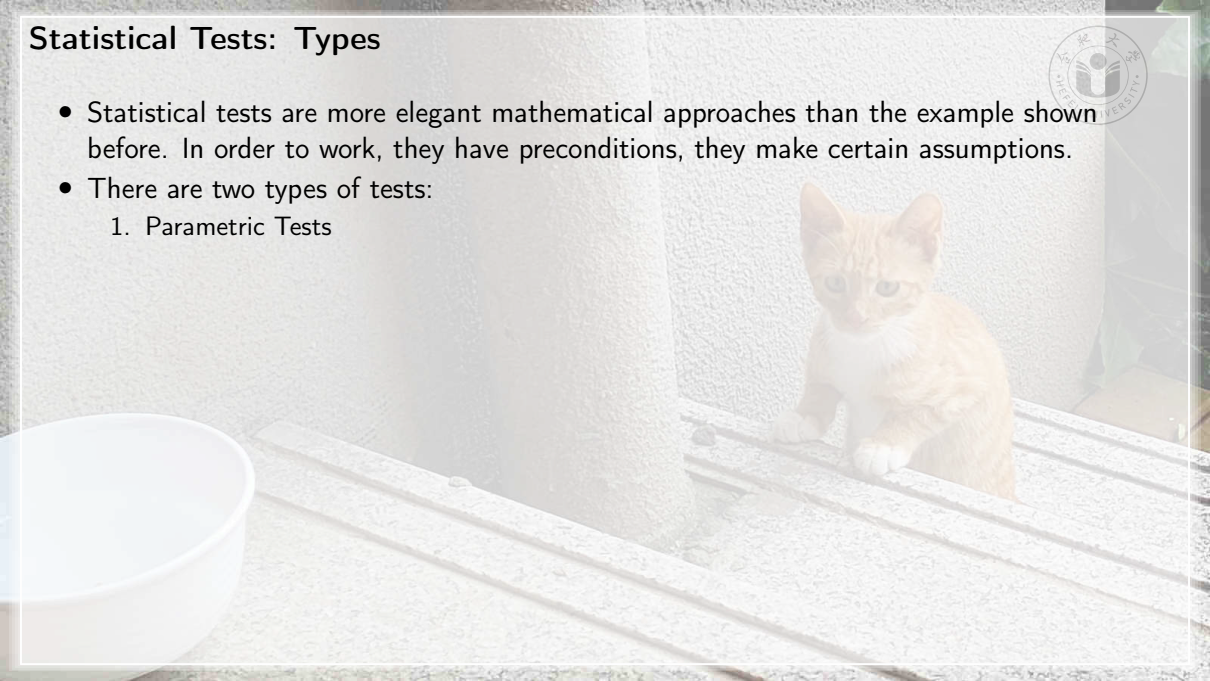
Statistical Tests: Types

- Statistical tests are more elegant mathematical approaches than the example shown before. In order to work, they have preconditions, they make certain assumptions.
- There are two types of tests:



Statistical Tests: Types

- Statistical tests are more elegant mathematical approaches than the example shown before. In order to work, they have preconditions, they make certain assumptions.
- There are two types of tests:
  1. Parametric Tests



Statistical Tests: Types

- Statistical tests are more elegant mathematical approaches than the example shown before. In order to work, they have preconditions, they make certain assumptions.
- There are two types of tests:
  1. Parametric Tests
     - Assume that the data samples follow a certain distribution

Statistical Tests: Types

- Statistical tests are more elegant mathematical approaches than the example shown before. In order to work, they have preconditions, they make certain assumptions.
- There are two types of tests:
  1. Parametric Tests
     - Assume that the data samples follow a certain distribution
     - Examples[44]: t-test (assumes normal distribution)

Statistical Tests: Types

- Statistical tests are more elegant mathematical approaches than the example shown before. In order to work, they have preconditions, they make certain assumptions.
- There are two types of tests:
  1. Parametric Tests
     - Assume that the data samples follow a certain distribution
     - Examples[44]: t-test (assumes normal distribution)
     - The distribution of the data we measure is unknown...

Statistical Tests: Types

- Statistical tests are more elegant mathematical approaches than the example shown before. In order to work, they have preconditions, they make certain assumptions.
- There are two types of tests:
  1. Parametric Tests
     - Assume that the data samples follow a certain distribution
     - Examples[44]: t-test (assumes normal distribution)
     - The distribution of the data we measure is unknown...
     - ...and usually not normal nor symmetric (see the further quantiles/stddev plot example).

Statistical Tests: Types

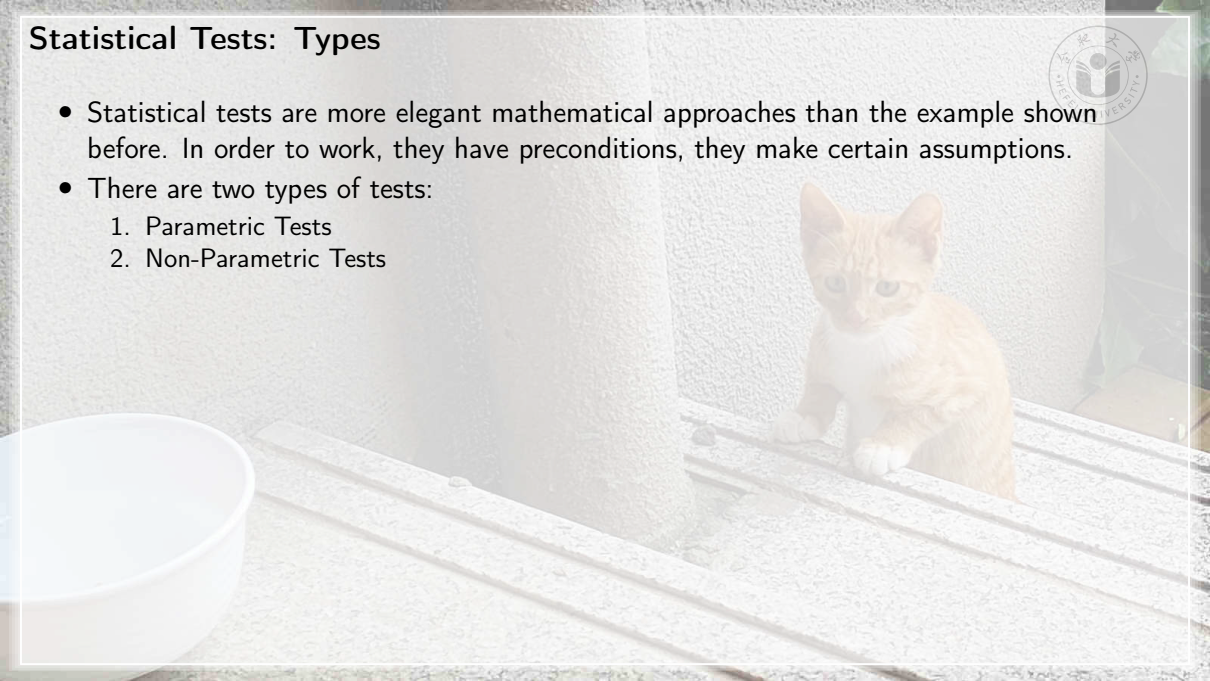
- Statistical tests are more elegant mathematical approaches than the example shown before. In order to work, they have preconditions, they make certain assumptions.
- There are two types of tests:
  1. Parametric Tests
     - Assume that the data samples follow a certain distribution
     - Examples[44]: t-test (assumes normal distribution)
     - The distribution of the data we measure is unknown...
     - ...and usually not normal nor symmetric (see the further quantiles/stddev plot example).
     - The condition for using such tests often cannot be met (known distribution)

Statistical Tests: Types

- Statistical tests are more elegant mathematical approaches than the example shown before. In order to work, they have preconditions, they make certain assumptions.
- There are two types of tests:
  1. Parametric Tests
     - Assume that the data samples follow a certain distribution
     - Examples[44]: t-test (assumes normal distribution)
     - The distribution of the data we measure is unknown...
     - ...and usually not normal nor symmetric (see the further quantiles/stddev plot example).
     - The condition for using such tests often cannot be met (known distribution)
     - Parametric tests should usually not be used here!

Statistical Tests: Types

- Statistical tests are more elegant mathematical approaches than the example shown before. In order to work, they have preconditions, they make certain assumptions.
- There are two types of tests:
  1. Parametric Tests
  2. Non-Parametric Tests



Statistical Tests: Types

- Statistical tests are more elegant mathematical approaches than the example shown before. In order to work, they have preconditions, they make certain assumptions.
- There are two types of tests:
  1. Parametric Tests
  2. Non-Parametric Tests
     - Make few assumption about the distribution from which the data was sampled.

Statistical Tests: Types

- Statistical tests are more elegant mathematical approaches than the example shown before. In order to work, they have preconditions, they make certain assumptions.
- There are two types of tests:
  1. Parametric Tests
  2. Non-Parametric Tests
     - Make few assumption about the distribution from which the data was sampled.
     - Examples[247]: the Wilcoxon rank sum test with continuity correction (also called Mann-Whitney U test[16,121,172,222]), Fisher's Exact Test[93], the Sign Test[114,222], the Randomization Test[36,83], and Wilcoxon's Signed Rank Test[264].

Statistical Tests: Types

- Statistical tests are more elegant mathematical approaches than the example shown before. In order to work, they have preconditions, they make certain assumptions.
- There are two types of tests:
  1. Parametric Tests
  2. Non-Parametric Tests
     - Make few assumption about the distribution from which the data was sampled.
     - Examples[247]: the Wilcoxon rank sum test with continuity correction (also called Mann-Whitney U test[16,121,172,222]), Fisher's Exact Test[93], the Sign Test[114,222], the Randomization Test[36,83], and Wilcoxon's Signed Rank Test[264].
     - These tests are more robust (less assumptions)

Statistical Tests: Types

- Statistical tests are more elegant mathematical approaches than the example shown before. In order to work, they have preconditions, they make certain assumptions.
- There are two types of tests:
  1. Parametric Tests
  2. Non-Parametric Tests
     - Make few assumption about the distribution from which the data was sampled.
     - Examples[247]: the Wilcoxon rank sum test with continuity correction (also called Mann-Whitney U test[16,121,172,222]), Fisher's Exact Test[93], the Sign Test[114,222], the Randomization Test[36,83], and Wilcoxon's Signed Rank Test[264].
     - These tests are more robust (less assumptions)
     - This usually is the kind of test we want to use.

Statistical Tests: Types

- Statistical tests are more elegant mathematical approaches than the example shown before. In order to work, they have preconditions, they make certain assumptions.
- There are two types of tests:
  1. Parametric Tests
  2. Non-Parametric Tests
     - Make few assumption about the distribution from which the data was sampled.
     - Examples[247]: the Wilcoxon rank sum test with continuity correction (also called Mann-Whitney U test[16,121,172,222]), Fisher's Exact Test[93], the Sign Test[114,222], the Randomization Test[36,83], and Wilcoxon's Signed Rank Test[264].
     - These tests are more robust (less assumptions)
     - This usually is the kind of test we want to use.
     - They work similar to the previous test example, but with larger sample sizes

Statistical Tests: Types

- Statistical tests are more elegant mathematical approaches than the example shown before. In order to work, they have preconditions, they make certain assumptions.
- There are two types of tests:
  1. Parametric Tests
  2. Non-Parametric Tests
     - Make few assumption about the distribution from which the data was sampled.
     - Examples[247]: the Wilcoxon rank sum test with continuity correction (also called Mann-Whitney U test[16,121,172,222]), Fisher's Exact Test[93], the Sign Test[114,222], the Randomization Test[36,83], and Wilcoxon's Signed Rank Test[264].
     - These tests are more robust (less assumptions)
     - This usually is the kind of test we want to use.
     - They work similar to the previous test example, but with larger sample sizes
     - Often, the most suitable test is the Mann-Whitney U test.

A fair warning



- There are many algorithms and even more configuration parameters.

A fair warning



- There are many algorithms and even more configuration parameters.
- All kinds of algorithm modules and parameters have some kind of impact on the performance.

A fair warning



- There are many algorithms and even more configuration parameters.
- All kinds of algorithm modules and parameters have some kind of impact on the performance.
- If I have two different algorithms $\mathcal{A}$ and $\mathcal{B}$, logic dictates that their performance is also different.

A fair warning



- There are many algorithms and even more configuration parameters.
- All kinds of algorithm modules and parameters have some kind of impact on the performance.
- If I have two different algorithms $\mathcal{A}$ and $\mathcal{B}$, logic dictates that their performance is also different.
- But is this difference usually significant?

A fair warning



- There are many algorithms and even more configuration parameters.
- All kinds of algorithm modules and parameters have some kind of impact on the performance.
- If I have two different algorithms $\mathcal{A}$ and $\mathcal{B}$, logic dictates that their performance is also different.
- But is this difference usually significant?
- From the viewpoint of statistics: Probably yes.

A fair warning



- There are many algorithms and even more configuration parameters.
- All kinds of algorithm modules and parameters have some kind of impact on the performance.
- If I have two different algorithms $\mathcal{A}$ and $\mathcal{B}$, logic dictates that their performance is also different.
- But is this difference usually significant?
- From the viewpoint of statistics: Probably yes.
- If I just conduct enough runs, maybe thousands, or millions, than even a difference of 0.001% in performance will pass a test as significant.

A fair warning



- There are many algorithms and even more configuration parameters.
- All kinds of algorithm modules and parameters have some kind of impact on the performance.
- If I have two different algorithms $\mathcal{A}$ and $\mathcal{B}$, logic dictates that their performance is also different.
- But is this difference usually significant?
- From the viewpoint of statistics: Probably yes.
- If I just conduct enough runs, maybe thousands, or millions, than even a difference of 0.001% in performance will pass a test as significant.
- To be practically significant, the measured difference of results should be large enough and statistically significant already with few runs, say, 11 or 21, not just with ≥ 100 runs.

Compare $N \geq 2$ Algorithms



- For comparing $N \geq 2$ algorithms, we can compare any two algorithms with each other

Compare $N \geq 2$ Algorithms



- For comparing $N \geq 2$ algorithms, we can compare any two algorithms with each other
- N Algorithms $\Rightarrow k = N(N-1)/2$ statistical tests (e.g., Mann-Whitney U)

Compare $N \geq 2$ Algorithms



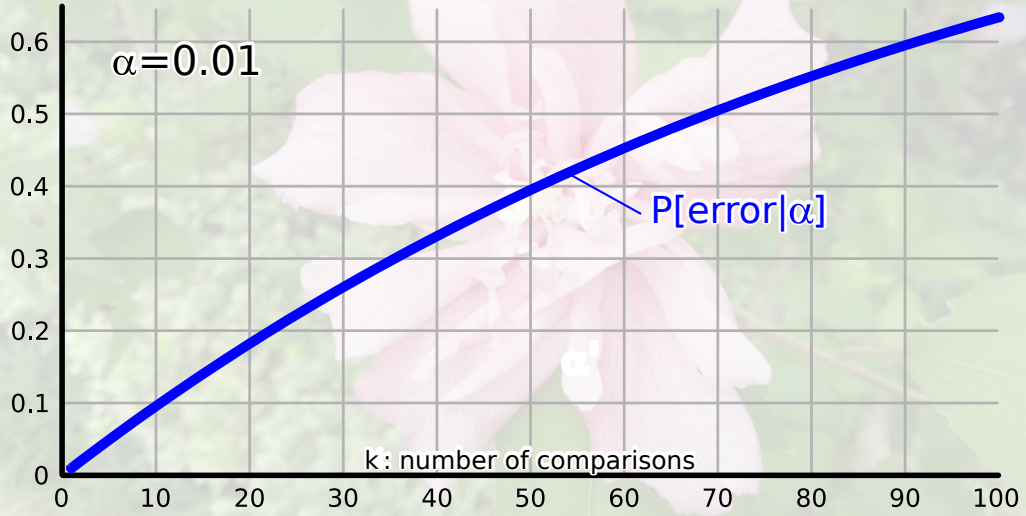
- For comparing $N \geq 2$ algorithms, we can compare any two algorithms with each other
- N Algorithms $\Rightarrow k = N(N-1)/2$ statistical tests (e.g., Mann-Whitney U)
- k tests and each with error proability α

Compare $N \geq 2$ Algorithms



- For comparing $N \geq 2$ algorithms, we can compare any two algorithms with each other
- N Algorithms $\Rightarrow k = N(N-1)/2$ statistical tests (e.g., Mann-Whitney U)
- k tests and each with error proability $\alpha \Longrightarrow$ total probability E to make at least one error $E = 1 - ((1-\alpha)^k)$

Compare $N \geq 2$ Algorithms



Compare $N \geq 2$ Algorithms



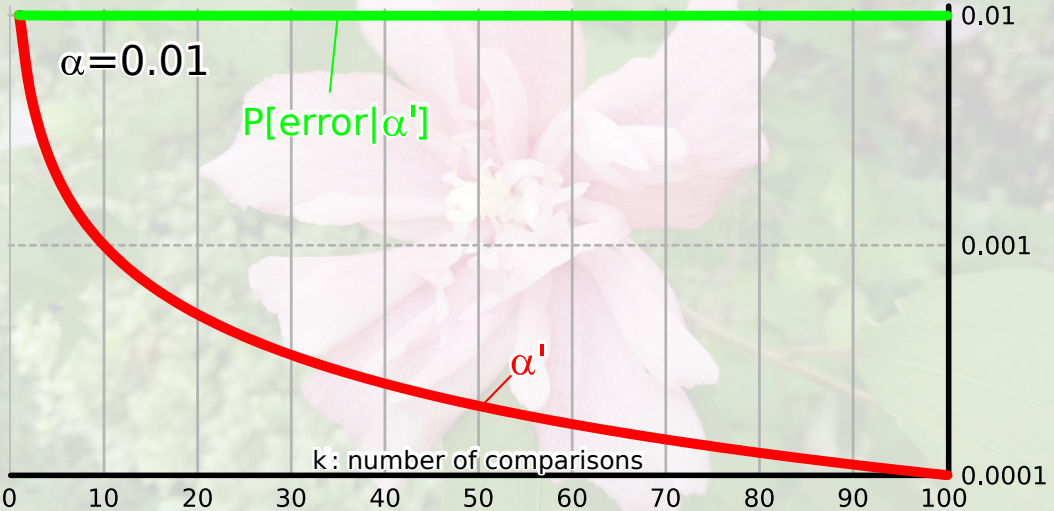
- For comparing $N \geq 2$ algorithms, we can compare any two algorithms with each other
- N Algorithms $\Rightarrow k = N(N-1)/2$ statistical tests
- k tests and each with error proability $\alpha \Longrightarrow$ total probability E to make at least one error $E = 1 - ((1-\alpha)^k)$
- Correction needed

Compare $N \geq 2$ Algorithms



- For comparing $N \geq 2$ algorithms, we can compare any two algorithms with each other
- N Algorithms $\Rightarrow k = N(N-1)/2$ statistical tests
- k tests and each with error proability $\alpha \Longrightarrow$ total probability E to make at least one error $E = 1 - ((1-\alpha)^k)$
- Correction needed: Bonferroni correction[35,180,217]: Use $\alpha' = \alpha/k$ as significance level instead of α, then the overall probability E to make an error will remain $E \leq \alpha$.

Compare $N \geq 2$ Algorithms



Compare $N \geq 2$ Algorithms



- For comparing $N \geq 2$ algorithms, we can compare any two algorithms with each other
- N Algorithms $\Rightarrow k = N(N-1)/2$ statistical tests
- k tests and each with error proability $\alpha \implies$ total probability E to make at least one error $E = 1 - ((1-\alpha)^k)$
- Correction needed: Bonferroni correction[35,180,217]: Use $\alpha' = \alpha/k$ as significance level instead of α, then the overall probability E to make an error will remain $E \leq \alpha$.
- The Bonferroni correction is very conservative and not suitable when checking independent parameters[201], but for comparing optimization algorithm, it does the trick. Being conservative is good.

Testing is Not Enough



The question of termination

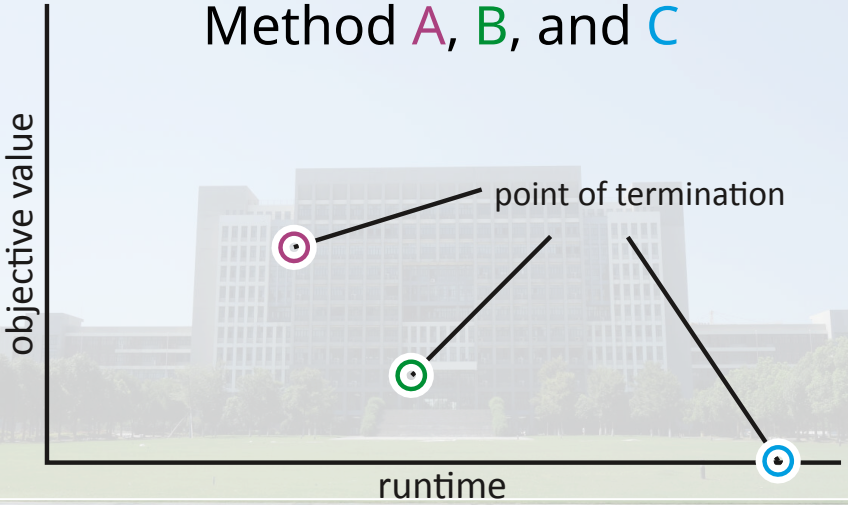
- Literature usually reports tuples "(instance, result, runtime)"

The question of termination



- Literature usually reports tuples "(instance, result, runtime)"
- Papers often use different termination criteria

The question of termination



The question of termination

- Literature usually reports tuples "(instance, result, runtime)"
- Problem: Papers often use different termination criteria

The question of termination

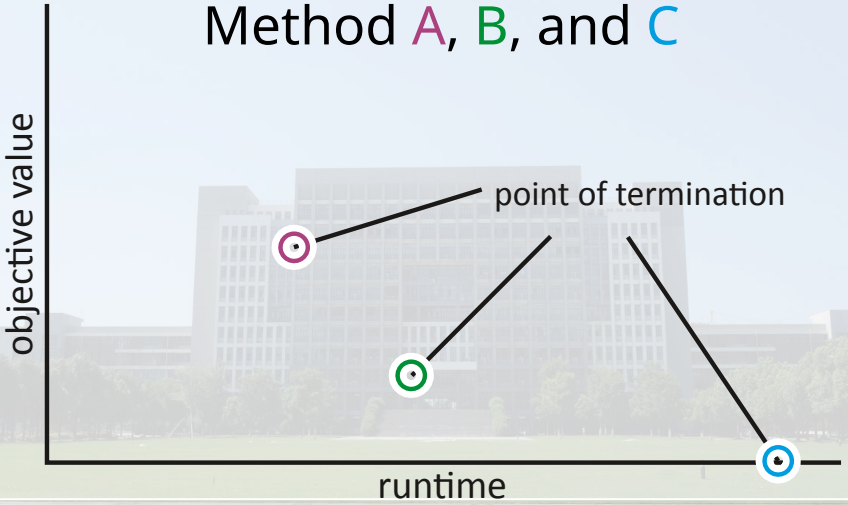
- Literature usually reports tuples "(instance, result, runtime)"
- Problem: Papers often use different termination criteria
- Anytime Algorithms[34]

The question of termination

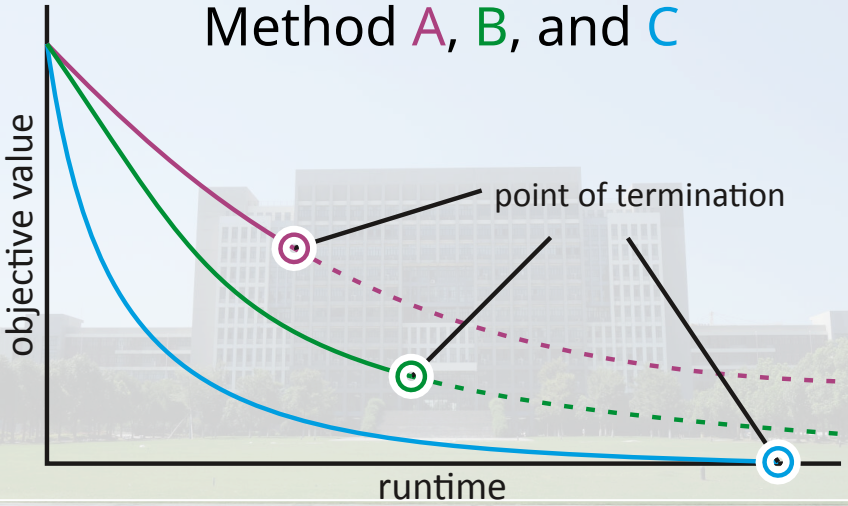


- Literature usually reports tuples "(instance, result, runtime)"
- Problem: Papers often use different termination criteria
- Anytime Algorithms[34]: Always have approximate solution, refine it iteratively

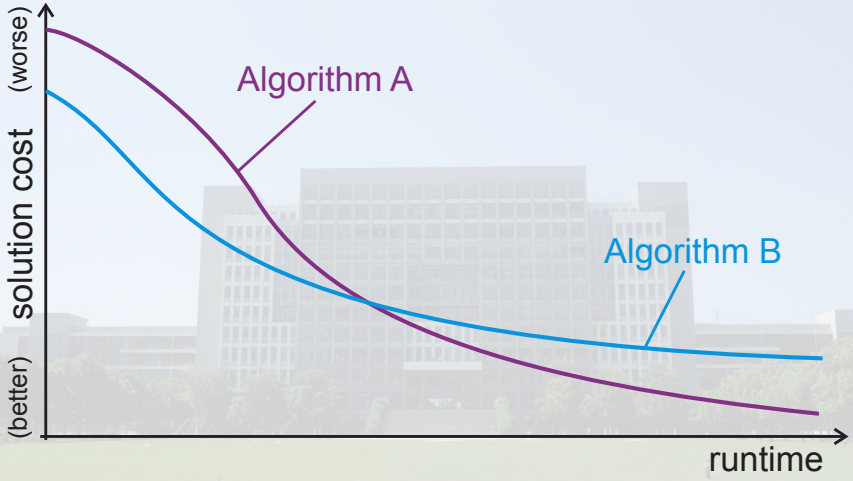
The question of termination



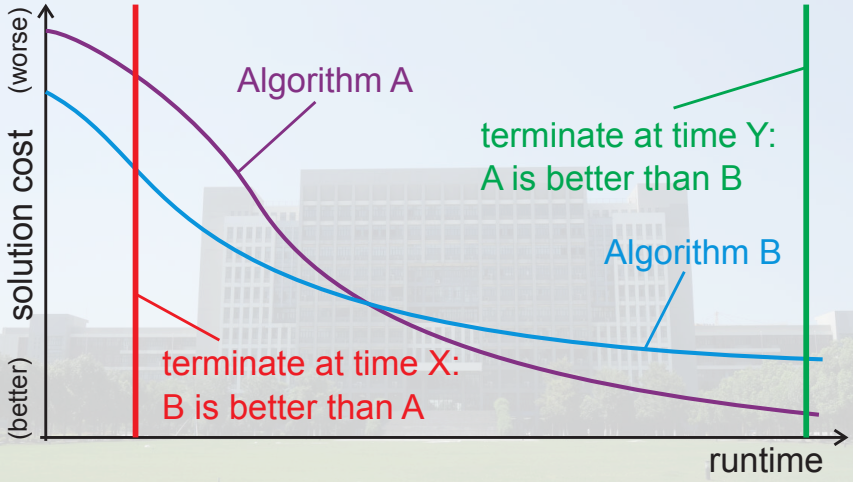
The question of termination



The question of termination



The question of termination



The question of termination



- Literature usually reports tuples "(instance, result, runtime)"
- Problem: Papers often use different termination criteria
- Anytime Algorithms[34]: Always have approximate solution, refine it iteratively
- One measure point per run or instance does not tell the whole story!

The question of termination



- Literature usually reports tuples "(instance, result, runtime)"
- Problem: Papers often use different termination criteria
- Anytime Algorithms[34]: Always have approximate solution, refine it iteratively
- One measure point per run or instance does not tell the whole story!
- Using statistical tests cannot solve this issue (still: at one point in time).

The question of termination



- Literature usually reports tuples "(instance, result, runtime)"
- Problem: Papers often use different termination criteria
- Anytime Algorithms[34]: Always have approximate solution, refine it iteratively
- One measure point per run or instance does not tell the whole story!
- Using statistical tests cannot solve this issue (still: at one point in time).
- We should have the "whole performance curves!"

The question of termination



- Literature usually reports tuples "(instance, result, runtime)"
- Problem: Papers often use different termination criteria
- Anytime Algorithms[34]: Always have approximate solution, refine it iteratively
- One measure point per run or instance does not tell the whole story!
- Using statistical tests cannot solve this issue (still: at one point in time).
- We should have the "whole performance curves!" ... ideally mean or median curves over several runs!

Other Stuff

New Algorithms and Problems



- There are many papers that introduce two things at the same time: a new optimization problem and a new algorithm.

New Algorithms and Problems



- There are many papers that introduce two things at the same time: a new optimization problem and a new algorithm. I think this is not a good idea.

New Algorithms and Problems



- There are many papers that introduce two things at the same time: a new optimization problem and a new algorithm. I think this is not a good idea.
- If we introduce a new optimization algorithm, we should test it on well-known, well-established benchmark problems.

New Algorithms and Problems



- There are many papers that introduce two things at the same time: a new optimization problem and a new algorithm. I think this is not a good idea.
- If we introduce a new optimization algorithm, we should test it on well-known, well-established benchmark problems. For such problems, results from other well-known and well-established algorithms exist – so we can compare our algorithm to them and investigate its performance objectively.

New Algorithms and Problems



- There are many papers that introduce two things at the same time: a new optimization problem and a new algorithm. I think this is not a good idea.
- If we introduce a new optimization algorithm, we should test it on well-known, well-established benchmark problems. For such problems, results from other well-known and well-established algorithms exist – so we can compare our algorithm to them and investigate its performance objectively.
- If we introduce a new optimization problem, we should apply well-known and well-established algorithms to it.

New Algorithms and Problems



- There are many papers that introduce two things at the same time: a new optimization problem and a new algorithm. I think this is not a good idea.
- If we introduce a new optimization algorithm, we should test it on well-known, well-established benchmark problems. For such problems, results from other well-known and well-established algorithms exist – so we can compare our algorithm to them and investigate its performance objectively.
- If we introduce a new optimization problem, we should apply well-known and well-established algorithms to it. This way we get proper baseline results and can understand whether the problem is hard for the state-of-the-art and/or how far this state-of-the-art allows us to go.

New Algorithms and Problems



- There are many papers that introduce two things at the same time: a new optimization problem and a new algorithm. I think this is not a good idea.
- If we introduce a new optimization algorithm, we should test it on well-known, well-established benchmark problems. For such problems, results from other well-known and well-established algorithms exist – so we can compare our algorithm to them and investigate its performance objectively.
- If we introduce a new optimization problem, we should apply well-known and well-established algorithms to it. This way we get proper baseline results and can understand whether the problem is hard for the state-of-the-art and/or how far this state-of-the-art allows us to go.
- If we have two "moving parts" at the same time, it is hard to understand whether an algorithm is good and whether a problem is hard.

New Algorithms and Problems



- There are many papers that introduce two things at the same time: a new optimization problem and a new algorithm. I think this is not a good idea.
- If we introduce a new optimization algorithm, we should test it on well-known, well-established benchmark problems. For such problems, results from other well-known and well-established algorithms exist – so we can compare our algorithm to them and investigate its performance objectively.
- If we introduce a new optimization problem, we should apply well-known and well-established algorithms to it. This way we get proper baseline results and can understand whether the problem is hard for the state-of-the-art and/or how far this state-of-the-art allows us to go.
- If we have two "moving parts" at the same time, it is hard to understand whether an algorithm is good and whether a problem is hard.
- If you have an own new algorithm on a new problem and use other algorithms for comparision, you might be tempted to just use the most basic configurations of these algorithms.

New Algorithms and Problems



- There are many papers that introduce two things at the same time: a new optimization problem and a new algorithm. I think this is not a good idea.
- If we introduce a new optimization algorithm, we should test it on well-known, well-established benchmark problems. For such problems, results from other well-known and well-established algorithms exist – so we can compare our algorithm to them and investigate its performance objectively.
- If we introduce a new optimization problem, we should apply well-known and well-established algorithms to it. This way we get proper baseline results and can understand whether the problem is hard for the state-of-the-art and/or how far this state-of-the-art allows us to go.
- If we have two "moving parts" at the same time, it is hard to understand whether an algorithm is good and whether a problem is hard.
- If you have an own new algorithm on a new problem and use other algorithms for comparision, you might be tempted to just use the most basic configurations of these algorithms. Then your algorithm might look good, while it actually is not.

New Algorithms and Problems



- There are many papers that introduce two things at the same time: a new optimization problem and a new algorithm. I think this is not a good idea.
- If we introduce a new optimization algorithm, we should test it on well-known, well-established benchmark problems. For such problems, results from other well-known and well-established algorithms exist – so we can compare our algorithm to them and investigate its performance objectively.
- If we introduce a new optimization problem, we should apply well-known and well-established algorithms to it. This way we get proper baseline results and can understand whether the problem is hard for the state-of-the-art and/or how far this state-of-the-art allows us to go.
- If we have two "moving parts" at the same time, it is hard to understand whether an algorithm is good and whether a problem is hard.
- If you have an own new algorithm on a new problem and use other algorithms for comparision, you might be tempted to just use the most basic configurations of these algorithms. Then your algorithm might look good, while it actually is not.
- Know the standard benchmark instances for your field!

Reproducibility



- Your experiments should be well-documented and reproducible.

Reproducibility



- Your experiments should be well-documented and reproducible.
- In the ideal case, someone else can run your code and get the same results.

Reproducibility



- Your experiments should be well-documented and reproducible.
- In the ideal case, someone else can run your code and get the same results.
- For this purpose, you should make your code available, e.g., put it on GitHub or zenodo.org.

Reproducibility



- Your experiments should be well-documented and reproducible.
- In the ideal case, someone else can run your code and get the same results.
- For this purpose, you should make your code available, e.g., put it on GitHub or zenodo.org.
- If your experiments are time-consuming, also make sure to properly store all your results in human- and machine-readable form, ideally in a comma-separated values (CSV) format.

Reproducibility



- Your experiments should be well-documented and reproducible.
- In the ideal case, someone else can run your code and get the same results.
- For this purpose, you should make your code available, e.g., put it on GitHub or zenodo.org.
- If your experiments are time-consuming, also make sure to properly store all your results in human- and machine-readable form, ideally in a comma-separated values (CSV) format.
- You should make an archive such that a) I can directly run the same experiments that you did and b) also have all the data and tools to create the same statistics and figures.

Reproducibility



- Your experiments should be well-documented and reproducible.
- In the ideal case, someone else can run your code and get the same results.
- For this purpose, you should make your code available, e.g., put it on GitHub or zenodo.org.
- If your experiments are time-consuming, also make sure to properly store all your results in human- and machine-readable form, ideally in a comma-separated values (CSV) format.
- You should make an archive such that a) I can directly run the same experiments that you did and b) also have all the data and tools to create the same statistics and figures.
- But what if someone finds an error in work?

Reproducibility



- Your experiments should be well-documented and reproducible.
- In the ideal case, someone else can run your code and get the same results.
- For this purpose, you should make your code available, e.g., put it on GitHub or zenodo.org.
- If your experiments are time-consuming, also make sure to properly store all your results in human- and machine-readable form, ideally in a comma-separated values (CSV) format.
- You should make an archive such that a) I can directly run the same experiments that you did and b) also have all the data and tools to create the same statistics and figures.
- But what if someone finds an error in work?
- That is OK.

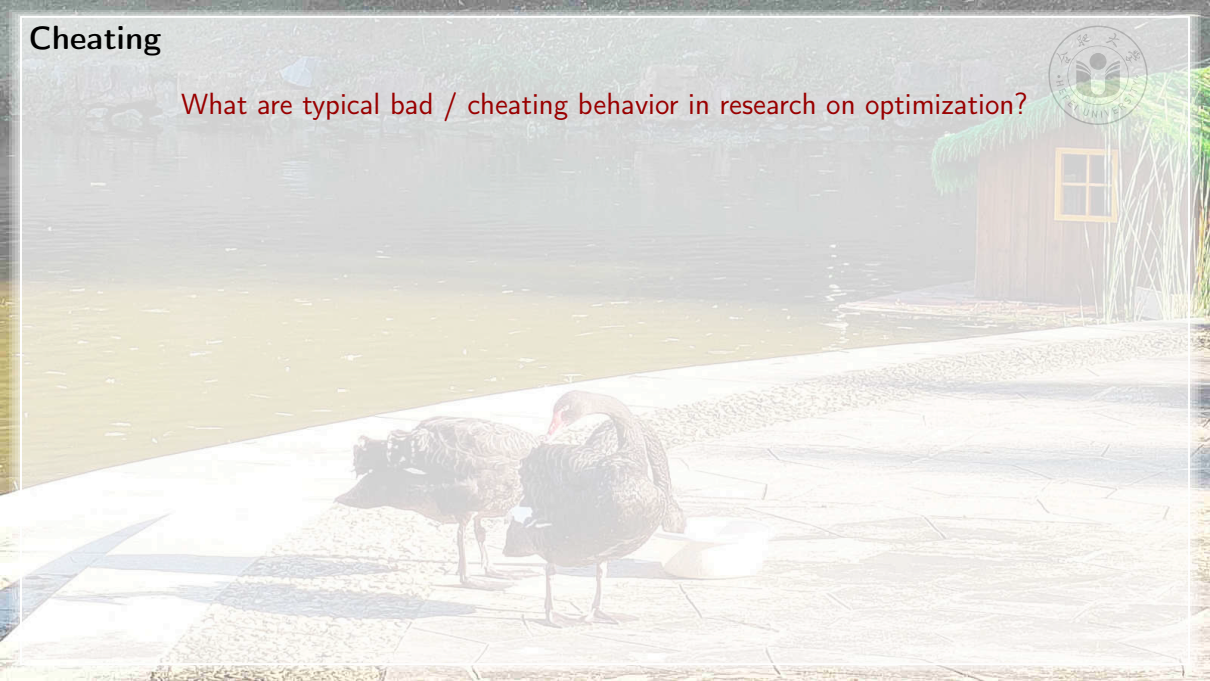
Reproducibility



- Your experiments should be well-documented and reproducible.
- In the ideal case, someone else can run your code and get the same results.
- For this purpose, you should make your code available, e.g., put it on GitHub or zenodo.org.
- If your experiments are time-consuming, also make sure to properly store all your results in human- and machine-readable form, ideally in a comma-separated values (CSV) format.
- You should make an archive such that a) I can directly run the same experiments that you did and b) also have all the data and tools to create the same statistics and figures.
- But what if someone finds an error in work?
- That is OK.
- Better they find it in your code that you voluntarily provided than after going through significant re-implementation effort...

Cheating

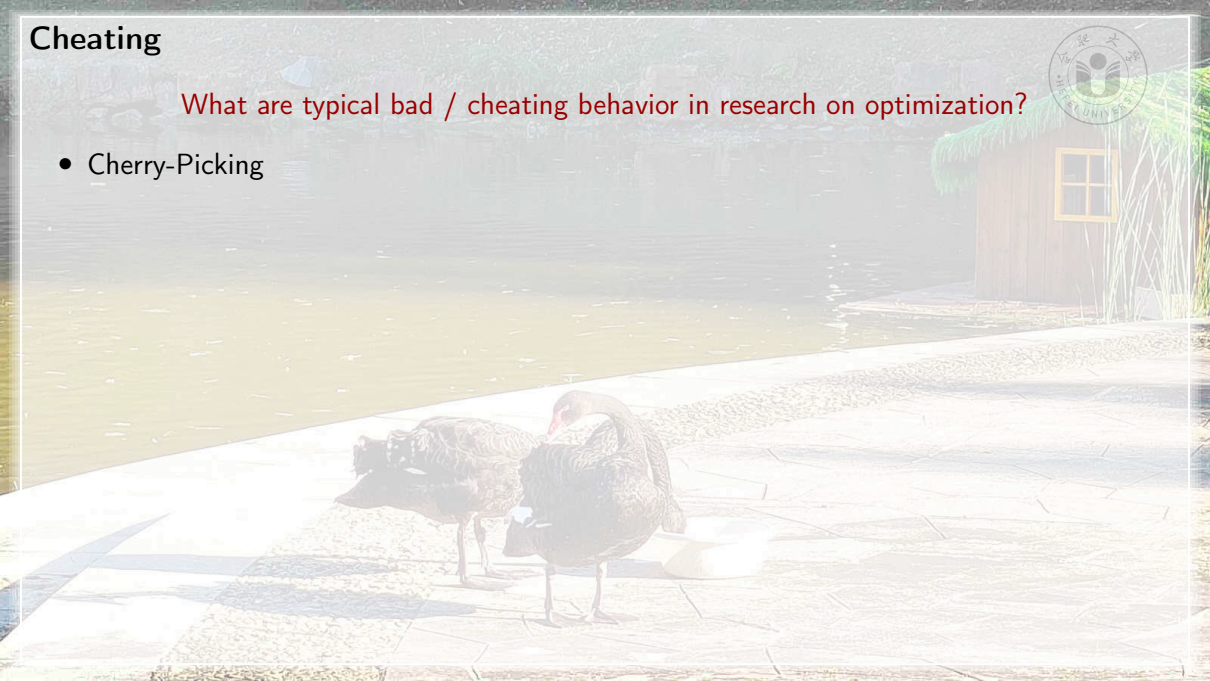
What are typical bad / cheating behavior in research on optimization?



Cheating

What are typical bad / cheating behavior in research on optimization?

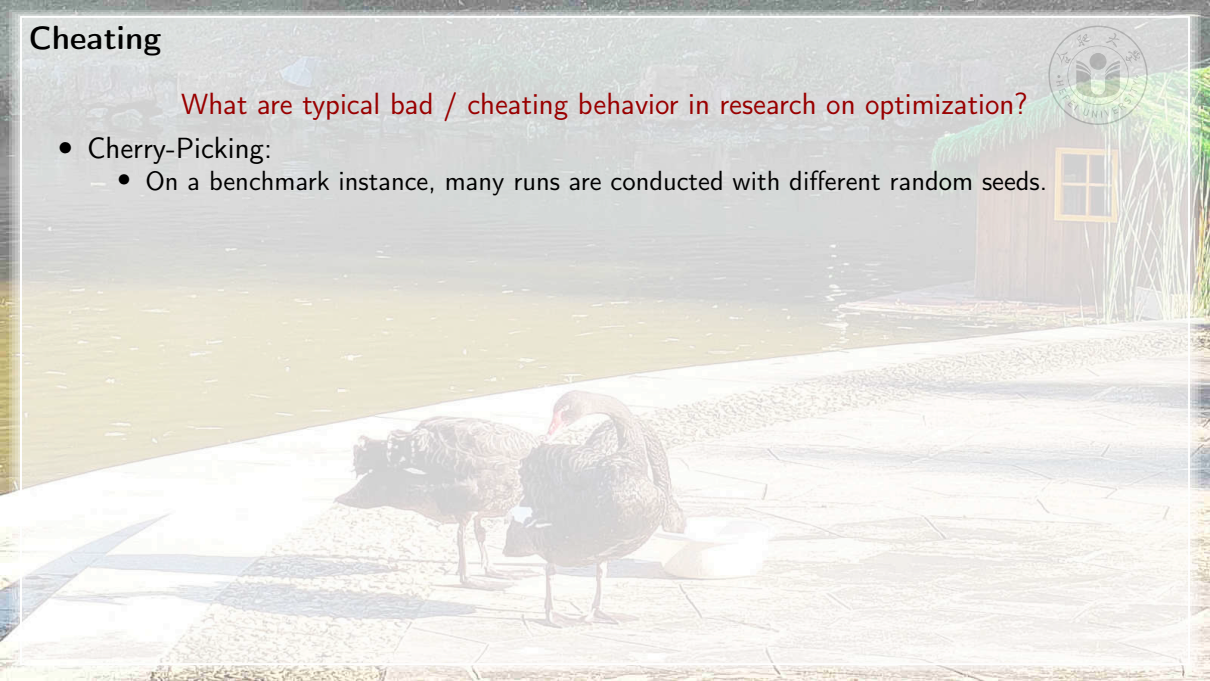
- Cherry-Picking



Cheating

What are typical bad / cheating behavior in research on optimization?

- Cherry-Picking:
  - On a benchmark instance, many runs are conducted with different random seeds.



Cheating



What are typical bad / cheating behavior in research on optimization?

- Cherry-Picking:
  - On a benchmark instance, many runs are conducted with different random seeds. But only the 10 with the best results are reported.

Cheating



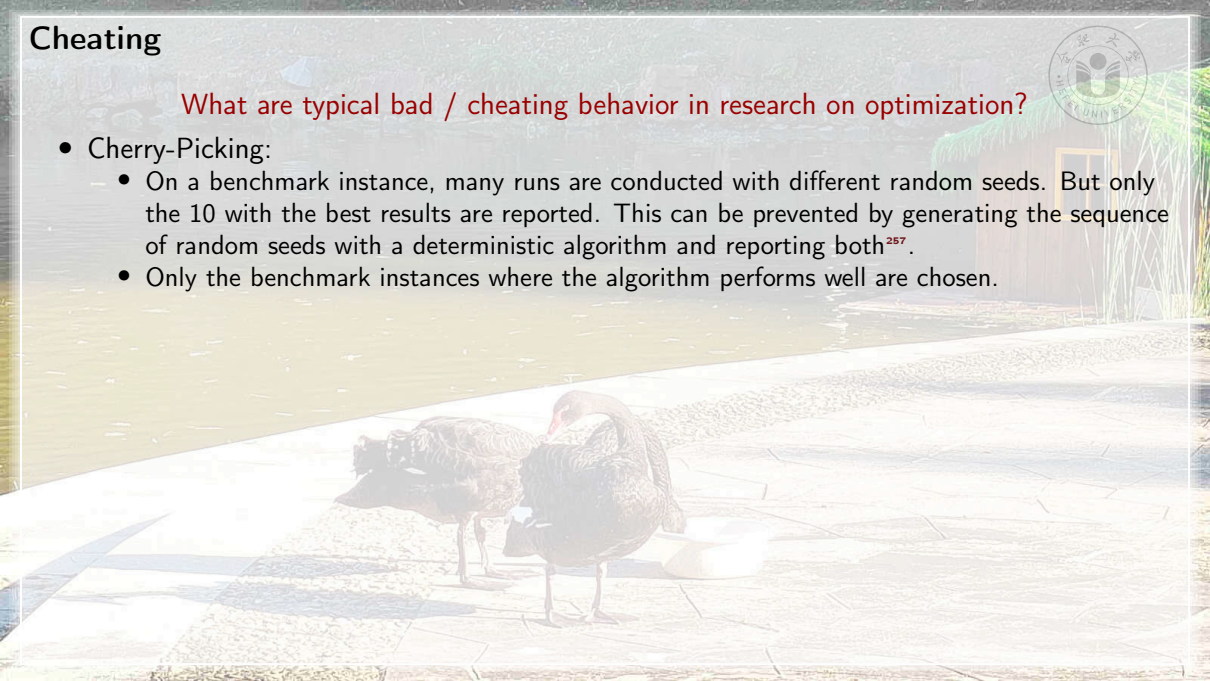
What are typical bad / cheating behavior in research on optimization?

- Cherry-Picking:
  - On a benchmark instance, many runs are conducted with different random seeds. But only the 10 with the best results are reported. This can be prevented by generating the sequence of random seeds with a deterministic algorithm and reporting both[257].

Cheating

What are typical bad / cheating behavior in research on optimization?

- Cherry-Picking:
  - On a benchmark instance, many runs are conducted with different random seeds. But only the 10 with the best results are reported. This can be prevented by generating the sequence of random seeds with a deterministic algorithm and reporting both[257].
  - Only the benchmark instances where the algorithm performs well are chosen.



Cheating



What are typical bad / cheating behavior in research on optimization?

- Cherry-Picking:
  - On a benchmark instance, many runs are conducted with different random seeds. But only the 10 with the best results are reported. This can be prevented by generating the sequence of random seeds with a deterministic algorithm and reporting both[257].
  - Only the benchmark instances where the algorithm performs well are chosen. Be wary of statements such as "We now present the results of our algorithm on 10 of the TSPLib instances." (TSPLib has more than 100...)

Cheating

What are typical bad / cheating behavior in research on optimization?

- Cherry-Picking:
  - On a benchmark instance, many runs are conducted with different random seeds. But only the 10 with the best results are reported. This can be prevented by generating the sequence of random seeds with a deterministic algorithm and reporting both[257].
  - Only the benchmark instances where the algorithm performs well are chosen. Be wary of statements such as "We now present the results of our algorithm on 10 of the TSPLib instances." (TSPLib has more than 100...)
  - Weak algorithms are chosen for comparison.

Cheating



What are typical bad / cheating behavior in research on optimization?

- Cherry-Picking:
  - On a benchmark instance, many runs are conducted with different random seeds. But only the 10 with the best results are reported. This can be prevented by generating the sequence of random seeds with a deterministic algorithm and reporting both[257].
  - Only the benchmark instances where the algorithm performs well are chosen. Be wary of statements such as "We now present the results of our algorithm on 10 of the TSPLib instances." (TSPLib has more than 100...)
  - Weak algorithms are chosen for comparison. Comparison must always be done with the state-of-the-art on the specific problem at hand.

Cheating



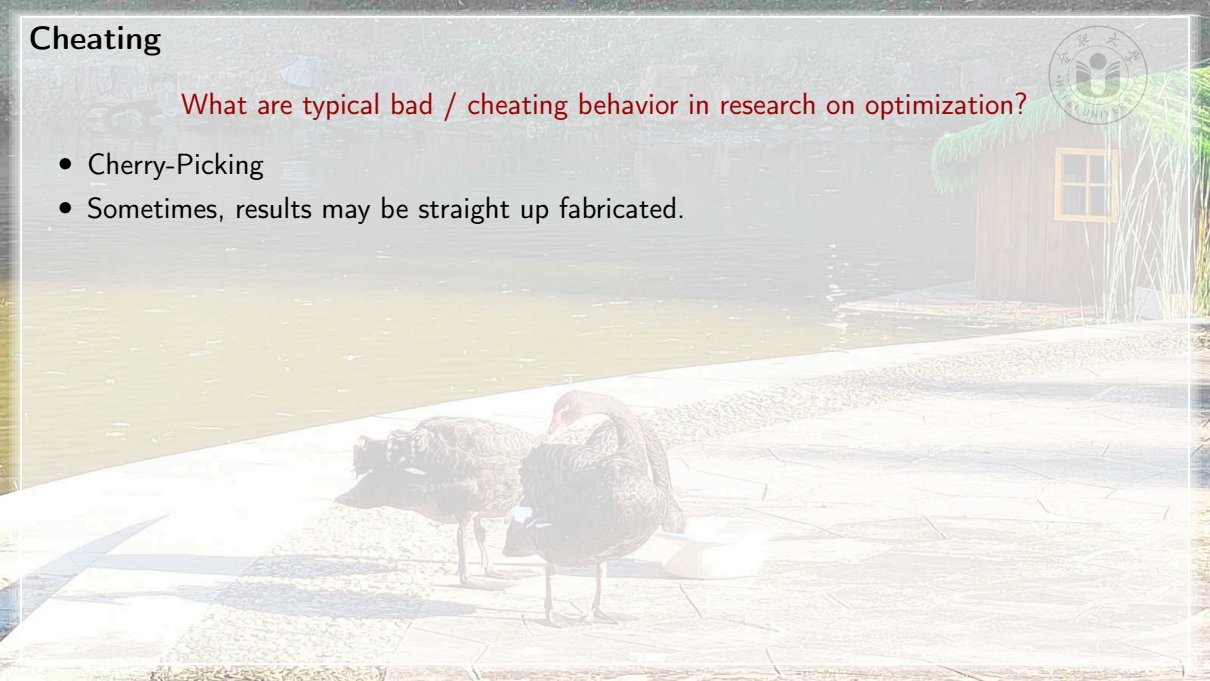
What are typical bad / cheating behavior in research on optimization?

- Cherry-Picking:
  - On a benchmark instance, many runs are conducted with different random seeds. But only the 10 with the best results are reported. This can be prevented by generating the sequence of random seeds with a deterministic algorithm and reporting both[257].
  - Only the benchmark instances where the algorithm performs well are chosen. Be wary of statements such as "We now present the results of our algorithm on 10 of the TSPLib instances." (TSPLib has more than 100...)
  - Weak algorithms are chosen for comparison. Comparison must always be done with the state-of-the-art on the specific problem at hand. Be wary of statements such as "We compare our algorithm with the Standard Genetic Algorithm (SGA)." (because the SGA is usually not the SOTA)

Cheating

What are typical bad / cheating behavior in research on optimization?

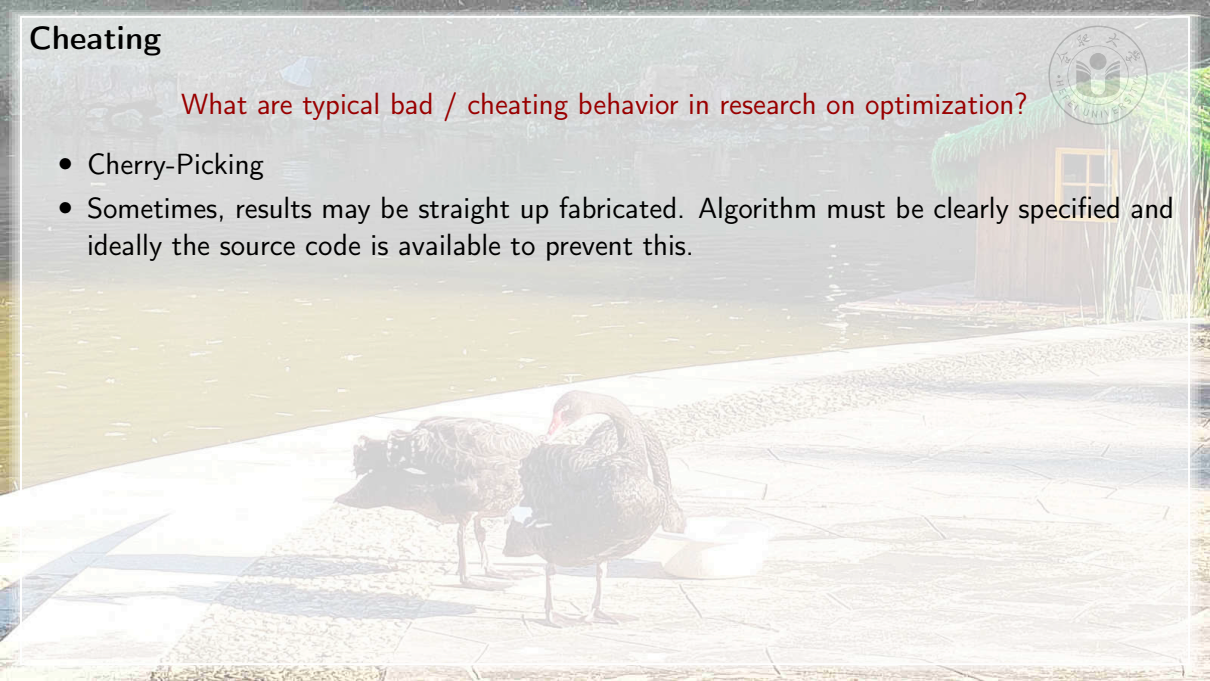
- Cherry-Picking
- Sometimes, results may be straight up fabricated.



Cheating

What are typical bad / cheating behavior in research on optimization?

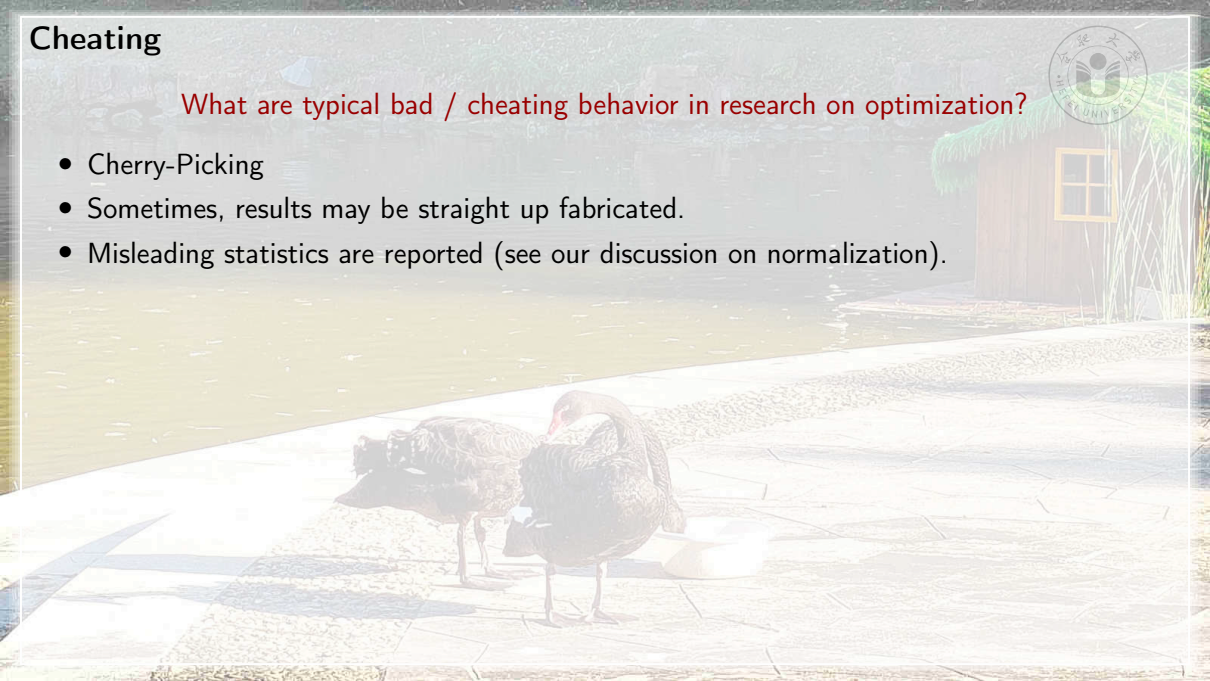
- Cherry-Picking
- Sometimes, results may be straight up fabricated. Algorithm must be clearly specified and ideally the source code is available to prevent this.



Cheating

What are typical bad / cheating behavior in research on optimization?

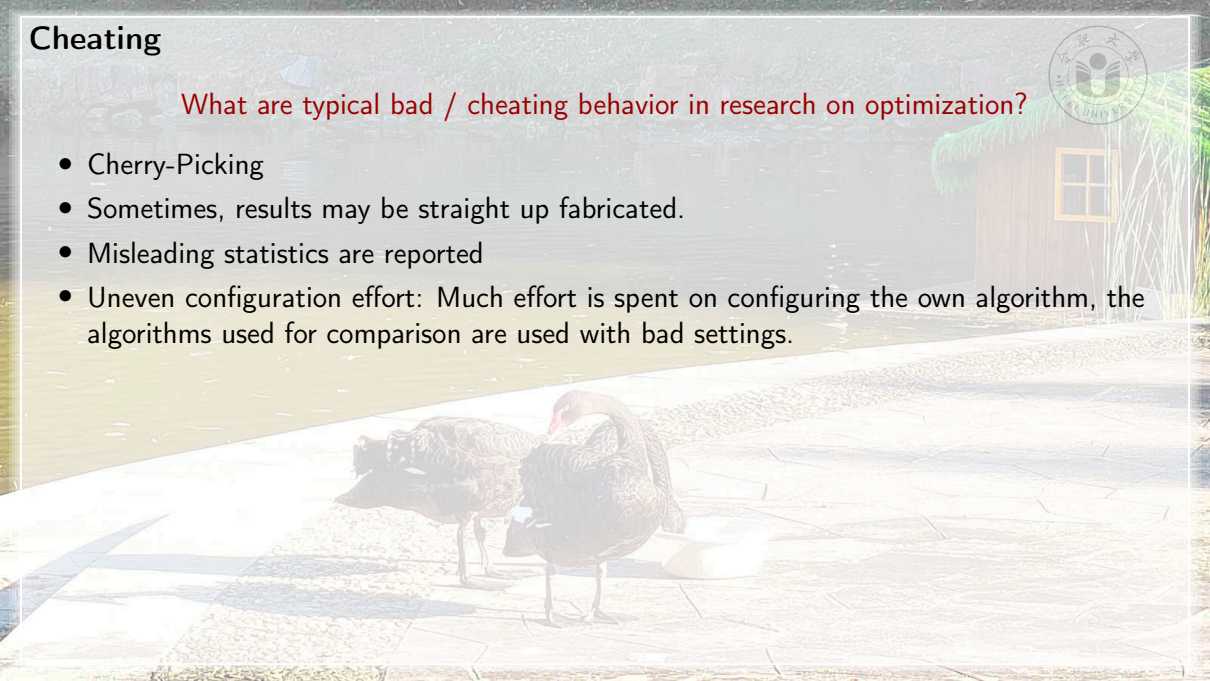
- Cherry-Picking
- Sometimes, results may be straight up fabricated.
- Misleading statistics are reported (see our discussion on normalization).



Cheating

What are typical bad / cheating behavior in research on optimization?

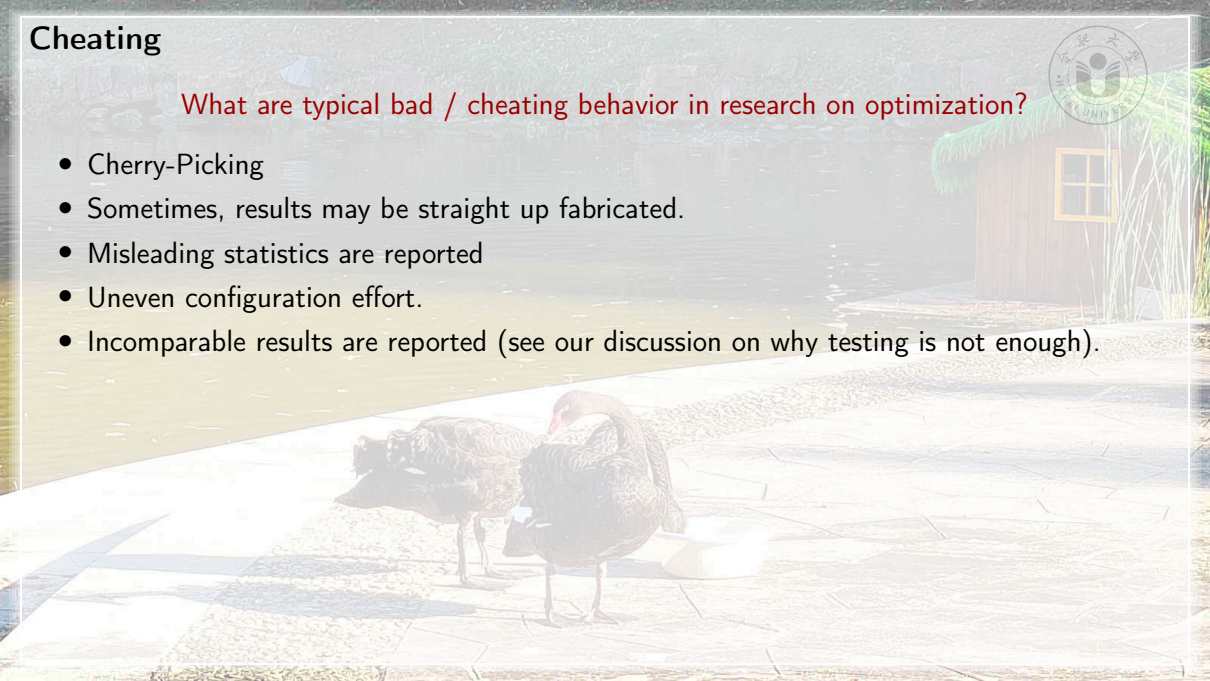
- Cherry-Picking
- Sometimes, results may be straight up fabricated.
- Misleading statistics are reported
- Uneven configuration effort: Much effort is spent on configuring the own algorithm, the algorithms used for comparison are used with bad settings.



Cheating

What are typical bad / cheating behavior in research on optimization?

- Cherry-Picking
- Sometimes, results may be straight up fabricated.
- Misleading statistics are reported
- Uneven configuration effort.
- Incomparable results are reported (see our discussion on why testing is not enough).



Cheating

What are typical bad / cheating behavior in research on optimization?

- Cherry-Picking
- Sometimes, results may be straight up fabricated.
- Misleading statistics are reported
- Uneven configuration effort.
- Incomparable results are reported.
- Misleading significance in test results (high α, many runs, no corrections).

Cheating

What are typical bad / cheating behavior in research on optimization?

- Cherry-Picking
- Sometimes, results may be straight up fabricated.
- Misleading statistics are reported
- Uneven configuration effort.
- Incomparable results are reported.
- Misleading significance in test results (high α, many runs, no corrections).

Reproducibility prevents cheating and misunderstandings!

Summary

Summary

- The optimization algorithms we consider in this lecture are randomized.

Summary

- The optimization algorithms we consider in this lecture are randomized.
- Comparing them must be done in a statistical way using data from multiple runs

Summary

- The optimization algorithms we consider in this lecture are randomized.
- Comparing them must be done in a statistical way using data from multiple runs
- Two views on performance

Summary

- The optimization algorithms we consider in this lecture are randomized.
- Comparing them must be done in a statistical way using data from multiple runs
- Two views on performance:
  1. best result after fixed number of FEs/runtime

Summary

- The optimization algorithms we consider in this lecture are randomized.
- Comparing them must be done in a statistical way using data from multiple runs
- Two views on performance:
  1. best result after fixed number of FEs/runtime
  2. number of FEs/runtime needed to get certain result

Summary

- The optimization algorithms we consider in this lecture are randomized.
- Comparing them must be done in a statistical way using data from multiple runs
- Two views on performance:
  1. best result after fixed number of FEs/runtime
  2. number of FEs/runtime needed to get certain result
- For every single algorithm/configuration, compute

Summary

- The optimization algorithms we consider in this lecture are randomized.
- Comparing them must be done in a statistical way using data from multiple runs
- Two views on performance:
  1. best result after fixed number of FEs/runtime
  2. number of FEs/runtime needed to get certain result
- For every single algorithm/configuration, compute:
  1. arithmetic and geometric mean and median of key performance indicators

Summary

- The optimization algorithms we consider in this lecture are randomized.
- Comparing them must be done in a statistical way using data from multiple runs
- Two views on performance:
  1. best result after fixed number of FEs/runtime
  2. number of FEs/runtime needed to get certain result
- For every single algorithm/configuration, compute:
  1. arithmetic and geometric mean and median of key performance indicators
  2. quartiles or top/bottom 1% quantile to get a feeling for the usual range of values

Summary

- The optimization algorithms we consider in this lecture are randomized.
- Comparing them must be done in a statistical way using data from multiple runs
- Two views on performance:
  1. best result after fixed number of FEs/runtime
  2. number of FEs/runtime needed to get certain result
- For every single algorithm/configuration, compute:
  1. arithmetic and geometric mean and median of key performance indicators
  2. quartiles or top/bottom 1% quantile to get a feeling for the usual range of values
  3. don't trust just arithmetic mean or standard deviation alone

Summary

- The optimization algorithms we consider in this lecture are randomized.
- Comparing them must be done in a statistical way using data from multiple runs
- Two views on performance:
  1. best result after fixed number of FEs/runtime
  2. number of FEs/runtime needed to get certain result
- For every single algorithm/configuration, compute:
  1. arithmetic and geometric mean and median of key performance indicators
  2. quartiles or top/bottom 1% quantile to get a feeling for the usual range of values
  3. don't trust just arithmetic mean or standard deviation alone
  4. geometric mean if the data is normalized

Summary

- The optimization algorithms we consider in this lecture are randomized.
- Comparing them must be done in a statistical way using data from multiple runs
- Two views on performance:
  1. best result after fixed number of FEs/runtime
  2. number of FEs/runtime needed to get certain result
- For every single algorithm/configuration, compute:
  1. arithmetic and geometric mean and median of key performance indicators
  2. quartiles or top/bottom 1% quantile to get a feeling for the usual range of values
  3. don't trust just arithmetic mean or standard deviation alone
  4. geometric mean if the data is normalized
- Use non-parametric statistical tests with corrections for multiple comparisons.

Summary

- The optimization algorithms we consider in this lecture are randomized.
- Comparing them must be done in a statistical way using data from multiple runs
- Two views on performance:
  1. best result after fixed number of FEs/runtime
  2. number of FEs/runtime needed to get certain result
- For every single algorithm/configuration, compute:
  1. arithmetic and geometric mean and median of key performance indicators
  2. quartiles or top/bottom 1% quantile to get a feeling for the usual range of values
  3. don't trust just arithmetic mean or standard deviation alone
  4. geometric mean if the data is normalized
- Use non-parametric statistical tests with corrections for multiple comparisons.
- Do not only collect one data sample per run, try to plot progress curves.

Summary

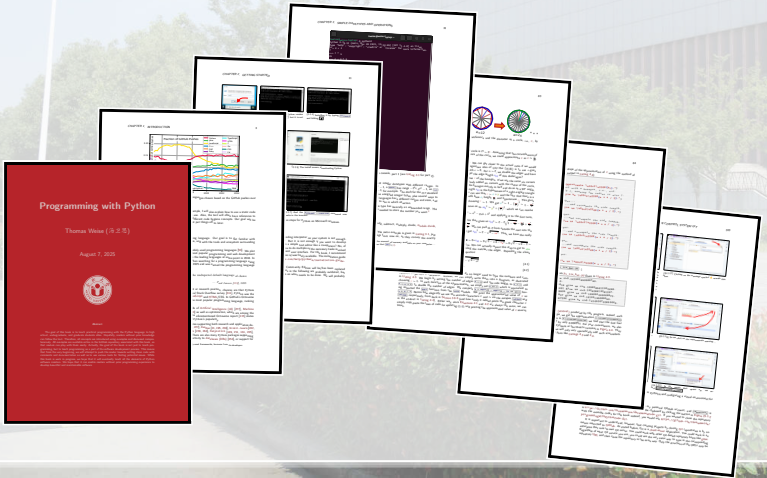
- The optimization algorithms we consider in this lecture are randomized.
- Comparing them must be done in a statistical way using data from multiple runs
- Two views on performance:
  1. best result after fixed number of FEs/runtime
  2. number of FEs/runtime needed to get certain result
- For every single algorithm/configuration, compute:
  1. arithmetic and geometric mean and median of key performance indicators
  2. quartiles or top/bottom 1% quantile to get a feeling for the usual range of values
  3. don't trust just arithmetic mean or standard deviation alone
  4. geometric mean if the data is normalized
- Use non-parametric statistical tests with corrections for multiple comparisons.
- Do not only collect one data sample per run, try to plot progress curves.
- Use well-known benchmarks, provide your source code!

Advertisement



Programming with Python

We have a freely available course book on *Programming with Python* at `https://thomasweise.github.io/programmingWithPython`, with focus on practical software development using the Python ecosystem of tools[250].



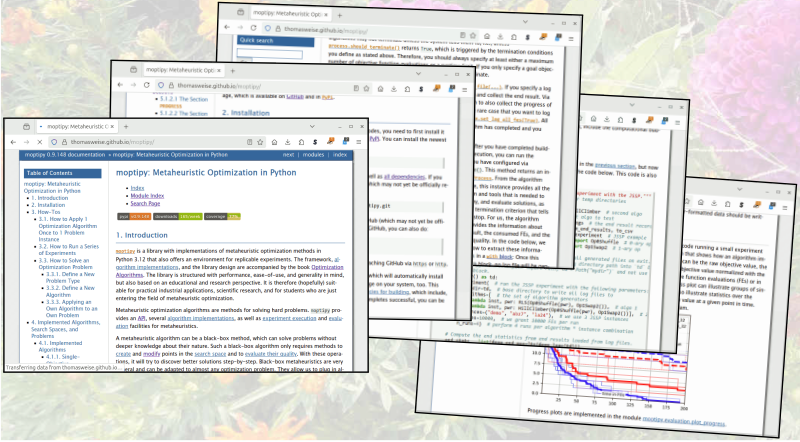
Databases

We have a freely available course book on *Databases* at `https://thomasweise.github.io/databases`, with actual practical examples using a real database management system (DBMS)[246].



Metaheuristic Optimization in Python: moptipy

We offer moptipy[257], a mature open source Python package for metaheuristic optimization, which implements several algorithms, can run self-documenting experiments in parallel and in a distributed fashion, and offers statistical evaluation tools.



谢谢您们！

Thank you!

Vielen Dank!



References I

[1] Scott Aaronson. "The Limits of Quantum Computers". *Scientific American* 298(3):62–69, Mar. 2008. New York, NY, USA: Scientific American, Inc., a division of Springer Nature Limited. ISSN: 1946-7087. doi:10.1038/scientificamerican0308-62. URL: http://www.cs.virginia.edu/~robins/The_Limits_of_Quantum_Computers.pdf (visited on 2021-08-10) (cit. on pp. 12–14).

[2] Joseph Adams, Egon Balas, and Daniel Zawack. "The Shifting Bottleneck Procedure for Job Shop Scheduling". *Management Science* 34(3):391–401, Mar. 1988. Catonsville, MD, USA: The Institute for Operations Research and the Management Sciences (INFORMS). ISSN: 0025-1909. doi:10.1287/mnsc.34.3.391 (cit. on p. 512).

[3] Hernán E. Aguirre and Keiki Takadama, eds. *20th Genetic and Evolutionary Computation Conference (GECCO'2018)*. July 15–19, 2018, Kyōto, Kyōto-fu, Japan. New York, NY, USA: Association for Computing Machinery (ACM), 2018. ISBN: 978-1-4503-5618-3. doi:10.1145/3205455.

[4] Miguel F. Anjos. *The QAPLIB Challenge*. Edinburgh, Scotland, UK: Miguel F. Anjos © University of Edinburgh, Nov. 19, 2023. URL: https://www.miguelanjos.com/projects/2999 (visited on 2026-09-28). The document lists the 28 instances of QAPLIB [40,42] that had not yet been solved to optimality by the time of its writing. (Cit. on p. 516).

[5] Denis Antipov and Benjamin Doerr. "Precise Runtime Analysis for Plateaus". In: *15th International Conference on Parallel Problem Solving from Nature (PPSN XV)*. Vol. 2. Sept. 8–12, 2018, Coimbra, Portugal. Ed. by Anne Auger, Carlos M. Fonseca, Nuno Lourenço, Penousal Machado, Luís Paquete, and L. Darrell Whitley. Vol. 11102 of Lecture Notes in Computer Science (LNCS). Cham, Switzerland: Springer, 2018, pp. 117–128. ISSN: 0302-9743. ISBN: 978-3-319-99258-7. doi:10.1007/978-3-319-99259-4_10 (cit. on p. 515).

[6] David Lee Applegate, Robert E. Bixby, Vašek Chvátal, and William John Cook. *The Traveling Salesman Problem: A Computational Study*. 2nd ed. Vol. 17 of Princeton Series in Applied Mathematics. Princeton, NJ, USA: Princeton University Press, 2007. ISBN: 978-0-691-12993-8. This book also discusses the Concorde TSP solver. (Cit. on pp. 10–34, 509, 518).

[7] David Lee Applegate and William John Cook. "A Computational Study of the Job-Shop Scheduling Problem". *ORSA Journal on Computing* 3(2):149–156, May 1991. Catonsville, MD, USA: The Institute for Operations Research and the Management Sciences (INFORMS). ISSN: 0899-1499. doi:10.1287/ijoc.3.2.149. The JSSP instances used were generated in Bonn in 1986. (Cit. on p. 512).

References II

[8] Anne Auger and Nikolaus Hansen. "Performance Evaluation of an Advanced Local Search Evolutionary Algorithm". In: *IEEE Congress on Evolutionary Computation (CEC'2005)*. Sept. 2–4, 2005, Edinburgh, Scotland, UK. Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE), 2005, pp. 1777–1784. ISBN: 978-0-7803-9363-9. doi:10.1109/CEC.2005.1554903. URL: http://www.cmap.polytechnique.fr/~nikolaus.hansen/cec2005localcmaes.pdf (visited on 2025-07-25) (cit. on pp. 89–94).

[9] Paul Gustav Heinrich Bachmann. *Die Analytische Zahlentheorie / Dargestellt von Paul Bachmann*. Vol. Zweiter Theil of Zahlentheorie: Versuch einer Gesamtdarstellung dieser Wissenschaft in ihren Haupttheilen. Leipzig, Sachsen, Germany: B. G. Teubner, 1894. ISBN: 978-1-4181-6963-3. URL: http://gallica.bnf.fr/ark:/12148/bpt6k994750 (visited on 2023-12-13) (cit. on p. 524).

[10] Thomas Bäck, David B. Fogel, and Zbigniew "Zbyszek" Michalewicz, eds. *Handbook of Evolutionary Computation*. Bristol, England, UK: IOP Publishing Ltd and Oxford, Oxfordshire, England, UK: Oxford University Press, 1997. ISBN: 978-0-7503-0392-7 (cit. on pp. 67–76, 510, 517).

[11] Thomas Bartz-Beielstein, Carola Doerr, Daan van den Berg, Jakob Bossek, Sowmya Chandrasekaran, Tome Eftimov, Andreas Fischbach, Pascal Kerschke, William La Cava, Manuel López-Ibáñez, Katherine Mary Malan, Jason Hall Moore, Boris Naujoks, Patryk Orzechowski, Vanessa Volz, Markus Wagner, and Thomas Weise (汤卫思). "Benchmarking in Optimization: Best Practice and Open Issues". (abs/2007.03488), Dec. 18, 2020. doi:10.48550/arXiv.2007.03488. URL: https://arxiv.org/abs/2007.03488 (visited on 2025-07-25). arXiv:2007.03488v2 [cs.NE] 16 Dec 2020 (cit. on pp. 4–9, 510).

[12] Roberto Battiti and Giampietro Tecchiolli. "Local Search with Memory: Benchmarking RTS". *OR Spectrum: Quantitative Approaches in Management. Formerly Operations-Research-Spektrum* 17:67–86, June 1995. London, England, UK: Springer Nature Limited. ISSN: 0171-6468. doi:10.1007/BF01719249 (cit. on p. 517).

[13] Roberto Battiti and Giampietro Tecchiolli. "Simulated Annealing and Tabu Search in the Long Run: A Comparison on QAP Tasks". *Computers & Mathematics with Applications* 28(6):1–8, Sept. 1994. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0898-1221. doi:10.1016/0898-1221(94)00147-2 (cit. on pp. 517, 518).

[14] Roberto Battiti and Giampietro Tecchiolli. "The Reactive Tabu Search". 6(2):126–140, May 1994. doi:10.1287/ijoc.6.2.126. Applications of Reactive Tabu Search (RTS) to the 0-1 Knapsack and Quadratic Assignment Problem (QAP). (Cit. on pp. 517, 518).

References III

[15] Roberto Battiti and Giampietro Tecchiolli. "Training Neural Nets with the Reactive Tabu Search". *IEEE Transactions on Neural Networks* 6(5):1185–1200, Sept. 1995. Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE). ISSN: 1045-9227. doi:10.1109/72.410361. See also "Training Neural Nets with the Reactive Tabu Search, Technical Report UTM 421, Trento, Alto Adige/Südtirol, Italy: University of Trento, 11 1993" (cit. on p. 517).

[16] Daniel F. Bauer. "Constructing Confidence Sets Using Rank Statistics". *Journal of the American Statistical Association (J AM STAT ASSOC)* 67(339):687–690, Sept. 1972. London, England, UK: Taylor and Francis Ltd. for Alexandria, VA, USA: American Statistical Association (ASA). ISSN: 0162-1459. doi:10.1080/01621459.1972.10481279 (cit. on pp. 376–381).

[17] James C. Bean. "Genetic Algorithms and Random Keys for Sequencing and Optimization". *ORSA Journal on Computing* 6(2):107–220, Spr. 1994. Catonsville, MD, USA: The Institute for Operations Research and the Management Sciences (INFORMS). ISSN: 0899-1499. doi:10.1287/ijoc.6.2.154. Applies the Random Keys encoding to scheduling and the QAP. See [18]. (Cit. on pp. 470, 509, 516).

[18] James C. Bean. *Genetics and Random Keys for Sequencing and Optimization*. Tech. rep. 92-43. Ann Arbor, MI, USA: University of Michigan, Department of Industrial & Operations Engineering, June 30, 1992. URL: https://www.math.wm.edu/~rrkinc/homework/RdmKeyGA.pdf (visited on 2026-10-01). See [17]. (Cit. on pp. 470, 509, 516).

[19] John Edward Beasley. "OR-Library: Distributing Test Problems by Electronic Mail". *The Journal of the Operational Research Society (JORS)* 41:1069–1072, Nov. 1990. London, England, UK: Taylor and Francis Ltd. and London, England, UK: Springer Nature Limited. ISSN: 0160-5682. doi:10.1057/jors.1990.166. See the website [20] (cit. on pp. 470, 512, 515).

[20] John Edward Beasley. *Welcome to OR-Library*. Uxbridge, Middlesex, England, UK: Brunel University of London, June 1990–Feb. 2018. URL: https://people.brunel.ac.uk/~mastjjb/jeb/info.html (visited on 2026-09-30). See the article [19]. Contains instances of the JSSP, 1D- and 2D-Packing, Knapsack, periodic TSPs, amongst others. (Cit. on pp. 470, 512, 515).

[21] Kent L. Beck. *JUnit Pocket Guide*. Sebastopol, CA, USA: O'Reilly Media, Inc., Sept. 2004. ISBN: 978-0-596-00743-0 (cit. on p. 520).

References IV

[22] Andreas Beham, Michael Affenzeller, and Erik Pitzer. "Metaheuristic Algorithms for the Quadratic Assignment Problem: Performance and Comparison". In: *Innovative Technologies in Management and Science*. Ed. by Ryszard Klempous and Jan Nikodem. Vol. 10 of Topics in Intelligent Engineering and Informatics (TIEI). Cham, Switzerland: Springer, 2015, pp. 171–190. ISSN: 2193-9411. ISBN: 978-3-319-12651-7. doi:10.1007/978-3-319-12652-4_10. Algorithms like stochastic local search (SLS)/iterated local search (ILS), Simulated Annealing (SA), Tabu Search (TS), Variable Neighborhood Search (VNS), EAs, Memetic Algorithms (MAs) are compared and, with the exception of SA and EA, all perform best on some instance groups of QAPLIB. (Cit. on p. 516).

[23] Richard Ernest Bellman. *Dynamic Programming*. Princeton Landmarks in Mathematics and Physics. Princeton, NJ, USA: Princeton University Press, 1957–2003. ISBN: 978-0-691-14668-3 (cit. on p. 523).

[24] Bengt-Erik Bengtsson. "Packing Rectangular Pieces— A Heuristic Approach". *The Computer Journal* 25(3):353–357, Aug. 1982. Oxford, Oxfordshire, England, UK: Oxford University Press. ISSN: 0010-4620. doi:10.1093/comjnl/25.3.353 (cit. on p. 508).

[25] Tim Berners-Lee. *Re: Qualifiers on Hypertext links...* Geneva, Switzerland: World Wide Web project, European Organization for Nuclear Research (CERN) and Newsgroups: alt.hypertext, Aug. 6, 1991. URL: https://www.w3.org/People/Berners-Lee/1991/08/art-6484.txt (visited on 2025-02-05) (cit. on p. 521).

[26] Tim Berners-Lee, Larry Masinter, and Mark P. McCahill. *Uniform Resource Locators (URL)*. Request for Comments (RFC) 1738. Wilmington, DE, USA: Internet Engineering Task Force (IETF), Dec. 1994. URL: https://www.ietf.org/rfc/rfc1738.txt (visited on 2025-02-05) (cit. on p. 520).

[27] Muhammed Beyaz, Tansel Dökeroğlu, and Ahmet Cosar. "Robust Hyper-Heuristic Algorithms for the Offline Oriented/Non-Oriented 2D Bin Packing Problems". *Applied Soft Computing (ASOC)* 36:236–245, Nov. 2015. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 1568-4946. doi:10.1016/j.asoc.2015.06.063 (cit. on p. 508).

[28] Christian Bierwirth. "A Generalized Permutation Approach to Job Shop Scheduling with Genetic Algorithms". *OR Spectrum: Quantitative Approaches in Management. Formerly Operations-Research-Spektrum* 17(2–3):87–92, June 1995. London, England, UK: Springer Nature Limited. ISSN: 0171-6468. doi:10.1007/BF01719250. URL: https://www.researchgate.net/publication/240263036 (visited on 2026-09-30). Proposes a permutations-with-repetitions based encoding for the JSSP, see also [29]. (Cit. on pp. 472, 482, 512, 522).

References V

[29] Christian Bierwirth, Dirk C. Mattfeld, and Herbert Kopfer. "On Permutation Representations for Scheduling Problems". In: *4th International Conference on Parallel Problem Solving from Nature (PPSN IV)*. Sept. 22–24, 1996, Berlin, Germany. Ed. by Hans-Michael Voigt, Werner Ebeling, Ingo Rechenberg, and Hans-Paul Schwefel. Vol. 1141 of Lecture Notes in Computer Science (LNCS). Berlin/Heidelberg, Germany: Springer-Verlag GmbH Germany, 1996, pp. 310–318. ISSN: 0302-9743. ISBN: 978-3-540-61723-5. doi:10.1007/3-540-61723-X_995. URL: https://www.researchgate.net/publication/2753293 (visited on 2026-09-30). Proposes a permutations-with-repetitions based encoding for the JSSP, see also [28]. (Cit. on pp. 471, 482, 512, 522).

[30] Bernd Bischl, Pascal Kerschke, Lars Kotthoff, Thomas Marius Lindauer, Yuri Malitsky, Alexandre Fréchette, Holger H. Hoos, Frank Hutter, Kevin Leyton-Brown, Kevin Tierney, and Joaquin Vanschoren. "ASlib: A Benchmark Library for Algorithm Selection". 237:41–58, Aug. 2016. doi:10.1016/j.artint.2016.04.003 (cit. on pp. 89–94).

[31] Jacek Błażewicz, Wolfgang Domschke, and Erwin Pesch. "The Job Shop Scheduling Problem: Conventional and New Solution Techniques". *European Journal of Operational Research* 93(1):1–33, Aug. 1996. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0377-2217. doi:10.1016/0377-2217(95)00362-2 (cit. on p. 512).

[32] Joshua Bloch. *Effective Java*. Reading, MA, USA: Addison-Wesley Professional, May 2008. ISBN: 978-0-321-35668-0 (cit. on p. 512).

[33] Christian Blum and Verena Schmid. "Solving the 2D Bin Packing Problem by Means of a Hybrid Evolutionary Algorithm". In: *International Conference on Computational Science (ICCS'2013)*. June 5–7, 2013, Barcelona, Spain. Ed. by Vassil Alexandrov, Michael Lees, Valeria V. Krzhizhanovskaya, Jack J. Dongarra, and Peter M. A. Sloot. Vol. 18 of Procedia Computer Science. Amsterdam, The Netherlands: Elsevier B.V., 2013, pp. 899–908. ISSN: 1877-0509. doi:10.1016/j.procs.2013.05.255. An EA for the oriented two-dimensional Bin Packing Problem with free cuts (2BP|O|F) using the heuristic by Wong and Lee [266]. (Cit. on p. 508).

[34] Mark S. Boddy and Thomas L. Dean. *Solving Time-Dependent Planning Problems*. Tech. rep. CS-89-03. Providence, RI, USA: Brown University, Department of Computer Science, Feb. 1989. URL: ftp://ftp.cs.brown.edu/pub/techreports/89/cs89-03.pdf (visited on 2025-07-25) (cit. on pp. 399, 400, 402–404, 409–412).

[35] Carlo Emilio Bonferroni. "Il calcolo delle assicurazioni su gruppi di teste". In: *Studi in Onore del Professore Salvatore Ortu Carboni*. Rome, Italy: Tipografia del Senato, 1935, pp. 13–60 (cit. on pp. 392, 394, 395, 397).

References VI

[36] Jürgen Bortz, Gustav Adolf Lienert, and Klaus Boehnke. *Verteilungsfreie Methoden in der Biostatistik*. 3rd ed. Springer-Lehrbuch. Berlin/Heidelberg, Germany: Springer-Verlag GmbH Germany, 2008. ISBN: 978-3-445-11034-3. doi:10.1007/978-3-540-74707-9 (cit. on pp. 359–364, 376–381).

[37] Borko Bošković and Janez Brest. "Computational Search of Long Skew-Symmetric Binary Sequences with High Merit Factors". *MENDEL – Soft Computing Journal* 28(2):17–24, Dec. 2022. Brno, Czech Republic: Brno University of Technology (Vysoké učení technické v Brně). ISSN: 1803-3814. doi:10.13164/mendel.2022.2.017 (cit. on p. 513).

[38] Borko Bošković, Franc Brglez, and Janez Brest. "Low-Autocorrelation Binary Sequences: On Improved Merit Factors and Runtime Predictions to Achieve Them". *Applied Soft Computing (ASOC)* 56:262–285, July 2017. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 1568-4946. doi:10.1016/J.ASOC.2017.02.024 (cit. on p. 513).

[39] Janez Brest and Borko Bošković. "A Heuristic Algorithm for a Low Autocorrelation Binary Sequence Problem with Odd Length and High Merit Factor". *IEEE Access* 6:4127–4134, Jan. 2018. Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE). ISSN: 2169-3536. doi:10.1109/ACCESS.2018.2789916 (cit. on p. 513).

[40] Rainer E. Burkard, Eranda Çela, Stefan E. Karisch, Franz Rendl, Miguel F. Anjos, and Peter M. Hahn. *QAPLIB – A Quadratic Assignment Problem Library – Problem Instances and Solutions*. Edinburgh, Scotland, UK: University of Edinburgh, Apr. 20, 2012–Mar. 31, 2022. doi:10.7488/ds/3428. This is the 2012 status of the QAPLIB dataset from [42] (cit. on pp. 468, 473, 516).

[41] Rainer E. Burkard, Eranda Çela, Panos Miltiades Pardalos, and Leonidas S. Pitsoulis. "The Quadratic Assignment Problem". In: *Handbook of Combinatorial Optimization*. Ed. by Panos Miltiades Pardalos, Ding-Zhu Du, and Ronald Lewis Graham. 1st ed. Boston, MA, USA: Springer, 1998, pp. 1713–1809. ISBN: 978-1-4613-7987-4. doi:10.1007/978-1-4613-0303-9_27 (cit. on p. 516).

[42] Rainer E. Burkard, Stefan E. Karisch, and Franz Rendl. "QAPLIB – A Quadratic Assignment Problem Library". *Journal of Global Optimization* 10(4):391–403, June 1997. London, England, UK: Springer Nature Limited. ISSN: 0925-5001. doi:10.1023/A:1008293323270. This article introduces QAPLIB, a set of benchmark problems for the QAP, now hosted at [40]. (Cit. on pp. 468, 473, 516).

References VII

[43] Rainer E. Burkard and Josef Offermann. "Entwurf von Schreibmaschinentastaturen mittels quadratischer Zuordnungsprobleme". *Zeitschrift für Operations Research (ZOR). now Mathematical Methods of Operations Research (MMOR)* 21:B121–B132, Aug. 1977. Heidelberg, Baden-Württemberg, Germany: Physica-Verlag; now London, England, UK: Springer Nature Limited. ISSN: 0340-9422. doi:10.1007/BF01918175. Keyboard layout modeled as QAPs. (Cit. on p. 516).

[44] Shaun Burke. "Missing Values, Outliers, Robust Statistics & Non-Parametric Methods". *LC·GC Europe* 1(2):19–24, Jan. 2001. Cranbury, NJ, USA: MJH Life Sciences. ISSN: 1471-6577. URL: https://www.researchgate.net/profile/Marcel-Severijnen/post/Source-apportionment-of-atmospheric-Hg/attachment/5c3cbf323843b0067550831f/AS%3A715017976557570%401547484978416/download/Missing+Values%2C+Outliers%2C+Robust+Statistics+%26+Non-parametric+Methods+LCGC+Eur+Burke+2001+-+4+de+4.pdf (visited on 2025-07-25). Online Supplement (cit. on pp. 369–374).

[45] Eduardo Carvalho Pinto and Carola Doerr. *Towards a More Practice-Aware Runtime Analysis of Evolutionary Algorithms*. arXiv.org: Computing Research Repository (CoRR) abs/1812.00493. Ithaca, NY, USA: Cornell Universiy Library, Dec. 3, 2018. doi:10.48550/arXiv.1812.00493. URL: https://arxiv.org/abs/1812.00493 (visited on 2025-08-08). arXiv:1812.00493v1 [cs.NE] 3 Dec 2018 (cit. on p. 507).

[46] Vladimír Černý. "Thermodynamical Approach to the Traveling Salesman Problem: An Efficient Simulation Algorithm". *Journal of Optimization Theory and Applications* 45(1):41–51, Jan. 1985. New York, NY, USA: Springer Science+Business Media, LLC. ISSN: 0022-3239. doi:10.1007/BF00940812 (cit. on p. 517).

[47] Noureddine Chabini and Rachid Beguenane. "FPGA-Based Designs of the Factorial Function". In: *IEEE Canadian Conference on Electrical and Computer Engineering (CCECE'2022)*. Sept. 18–20, 2022, Halifax, NS, Canada. Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE), 2022, pp. 16–20. ISBN: 978-1-6654-8432-9. doi:10.1109/CCECE49351.2022.9918302 (cit. on p. 521).

[48] Tony F. Chan, Gene H. Golub, and Randall J. LeVeque. "Algorithms for Computing the Sample Variance: Analysis and Recommendations". *The American Statistician* 37(3):242–247, Aug. 1983. London, England, UK: Taylor and Francis Ltd. ISSN: 0003-1305. doi:10.2307/2683386. URL: https://engineering.yale.edu/application/files/3817/3714/7395/tr222.pdf (visited on 2026-10-30) (cit. on pp. 504, 525).

References VIII

[49] Bo Chen, Chris N. Potts, and Gerhard J. Woeginger. "A Review of Machine Scheduling: Complexity, Algorithms and Approximability". In: *Handbook of Combinatorial Optimization*. Ed. by Panos Miltiades Pardalos, Ding-Zhu Du, and Ronald Lewis Graham. 1st ed. Boston, MA, USA: Springer, 1998, pp. 1493–1641. ISBN: 978-1-4613-7987-4. doi:10.1007/978-1-4613-0303-9_25. See also pages 21–169 in volume 3/3 by Norwell, MA, USA: Kluwer Academic Publishers. Discusses, amongst others, that the JSSP is $\mathcal{NP}$-hard. (Cit. on pp. 12–14, 512, 524).

[50] Stephen Chen, James Montgomery, and Antonio Bolufé-Röhler. "Measuring the Curse of Dimensionality and its Effects on Particle Swarm Optimization and Differential Evolution". *Applied Intelligence – The International Journal of Research on Intelligent Systems for Real Life Complex Problems* 42:514–526, Apr. 2015. London, England, UK: Springer Nature Limited. ISSN: 0924-669X. doi:10.1007/s10489-014-0613-2 (cit. on p. 523).

[51] Jiayang Chen (陈嘉阳), Zhize Wu (吴志泽), Sarah Louise Thomson, and Thomas Weise (汤卫思). "Frequency Fitness Assignment: Optimization Without Bias for Good Solution Outperforms Randomized Local Search on the Quadratic Assignment Problem". In: *16th International Joint Conference on Computational Intelligence (IJCCI'24)*. Nov. 20–22, 2024, Porto, Portugal. Ed. by Francesco Marcelloni, Kurosh Madani, Niki van Stein, and Joaquim Filipe. Porto, Portugal: SciTePress: Science and Technology Publications, Lda, 2024, pp. 27–37. ISSN: 2184-3236. ISBN: 978-989-758-721-4. doi:10.5220/0012888600003837 (cit. on pp. 510, 514, 516).

[52] T. C. Edwin Cheng, Bo Peng, and Zhipeng Lü. "A Hybrid Evolutionary Algorithm to Solve the Job Shop Scheduling Problem". *Annals of Operations Research* 242(2):223–237, July 2016. London, England, UK: Springer Nature Limited. ISSN: 0254-5330. doi:10.1007/s10479-013-1332-5. A MA utlizing TS (cit. on p. 514).

[53] Philippe Chrétienne, Edward G. Coffman, Jan Karel Lenstra, and Zhen Liu, eds. *Scheduling Theory and Its Applications*. Chichester, West Sussex, England, UK: John Wiley and Sons Ltd., Sept. 1995. ISBN: 978-0-471-94059-3 (cit. on p. 512).

[54] Nestor M. Cid-Garcia and Yasmin A. Rios-Solis. "Positions and Covering: A Two-Stage Methodology to Obtain Optimal Solutions for the 2D-Bin Packing Problem". *PLoS ONE* 15(4):e0229358, Apr. 2020. San Francisco, CA, USA: Public Library of Science (PLOS). ISSN: 1932-6203. doi:10.1371/journal.pone.0229358. An exact method for the two-dimensional Bin Packing Problem with free cuts and rotation (2BP|R|F) and the 2BP|O|F. (Cit. on p. 508).

References IX

[55] Stephen Arthur Cook. "The Complexity of Theorem-Proving Procedures". In: *Third Annual ACM Symposium on Theory of Computing (STOC'1971)*. May 3–5, 1971, Shaker Heights, OH, USA. Ed. by Michael A. Harrison, Ranan B. Banerji, and Jeffrey D. Ullman. New York, NY, USA: Association for Computing Machinery (ACM), 1971, pp. 151–158. ISBN: 978-1-4503-7464-4. doi:10.1145/800157.805047 (cit. on pp. 12–14, 514, 524).

[56] William John Cook. *Concorde TSP Solver*. Waterloo, ON, Canada: University of Waterloo, 1997. URL: https://math.uwaterloo.ca/tsp/concorde (visited on 2026-09-26) (cit. on p. 509).

[57] William John Cook. *In Pursuit of the Traveling Salesman: Mathematics at the Limits of Computation*. Princeton, NJ, USA: Princeton University Press, 2012–Nov. 9, 2014. ISBN: 978-0-691-16352-9 (cit. on pp. 509, 513).

[58] William John Cook. *World TSP*. Waterloo, ON, Canada: University of Waterloo, 2013–Oct. 25, 2025. URL: http://www.math.uwaterloo.ca/tsp/world (visited on 2026-01-03) (cit. on pp. 513, 518).

[59] Thomas H. Cormen, Charles E. Leiserson, Ronald Linn Rivest, and Clifford Stein. "Asymptotic Notation". In: *Introduction to Algorithms*. 3rd ed. Cambridge, MA, USA: MIT Press, 2009. Chap. 3.1, pp. 43–53. ISBN: 978-0-262-03384-8. Discusses Θ, $\mathcal{O}$, Ω, o, and ω (cit. on pp. 524, 526).

[60] "csv – CSV File Reading and Writing". In: *Python 3 Documentation. The Python Standard Library*. Beaverton, OR, USA: Python Software Foundation (PSF), 2001–2025. URL: https://docs.python.org/3/library/csv.html (visited on 2024-11-14) (cit. on p. 509).

[61] Cemre Çubukçuoğlu, Pirouz Nourian, M. Fatih Tasgetiren, I. Sevil Sariyildiz, and Shervin Azadi. "Hospital Layout Design Renovation as a Quadratic Assignment Problem with Geodesic Distances". *Journal of Building Engineering* 44:102952, Dec. 2021. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 2352-7102. doi:10.1016/j.jobe.2021.102952 (cit. on p. 516).

[62] Kenneth Alan De Jong. *Evolutionary Computation: A Unified Approach*. Vol. 4 of Complex Adaptive Systems. Cambridge, MA, USA: MIT Press, 2006. ISBN: 978-0-262-04194-2. URL: https://www.researchgate.net/publication/220740669 (visited on 2025-08-08) (cit. on p. 517).

[63] *Definition of Operations Research*. University of Western Ontario, London, ON, Canada: International Federation of Operational Research Societies (IFORS), 2020. URL: https://www.ifors.org/what-is-or (visited on 2026-01-01) (cit. on p. 515).

References X

[64] Paul Deitel, Harvey Deitel, and Abbey Deitel. *Internet & World Wide Web: How to Program*. 5th ed. Hoboken, NJ, USA: Pearson Education, Inc., Nov. 2011. ISBN: 978-0-13-299045-5 (cit. on p. 521).

[65] Maxence Delorme, Manuel Iori, and Silvano Martello. "Bin Packing and Cutting Stock Problems: Mathematical Models and Exact Algorithms". *European Journal of Operational Research* 255(1):1–20, Nov. 2016. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0377-2217. doi:10.1016/j.ejor.2016.04.030 (cit. on p. 509).

[66] Ebru Demirkol, Sanjay V. Mehta, and Reha Uzsoy. "Benchmarks for Shop Scheduling Problems". *European Journal of Operational Research* 109(1):137–141, Aug. 1998. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0377-2217. doi:10.1016/S0377-2217(97)00019-2 (cit. on p. 512).

[67] Miroslav Dimitrov. *New Classes of Binary Sequences with High Merit Factor*. arXiv.org: Computing Research Repository (CoRR). Ithaca, NY, USA: Cornell Universiy Library, June 24, 2022. doi:10.48550/arXiv.2206.12070. URL: https://arxiv.org/pdf/2206.12070 (visited on 2026-09-21). arXiv:2206.12070v1 [cs.IT] 24 Jun 2022 (cit. on p. 513).

[68] Miroslav Dimitrov, Tsonka Baitcheva, and Nikolay Nikolov. "Efficient Generation of Low Autocorrelation Binary Sequences". *IEEE Signal Processing Letters* 27:341–345, 2020. Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE). ISSN: 1558-2361. doi:10.1109/LSP.2020.2972127 (cit. on p. 513).

[69] Slobodan Dmitrović. *Modern C for Absolute Beginners: A Friendly Introduction to the C Programming Language*. New York, NY, USA: Apress Media, LLC, Mar. 2024. ISBN: 979-8-8688-0224-9 (cit. on p. 509).

[70] Benjamin Doerr, Carola Doerr, and Franziska Ebel. "From Black-Box Complexity to Designing New Genetic Algorithms". *Theoretical Computer Science* 567:87–104, Feb. 2015. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0304-3975. doi:10.1016/J.TCS.2014.11.028 (cit. on p. 515).

[71] Carola Doerr, Furong Ye, Naama Horesh, Hao Wang, and Ofer M. ShirThomas Bäck. "Benchmarking Discrete Optimization Heuristics with IOHprofiler". In: *21st Genetic and Evolutionary Computation Conference Companion (GECCO'2019)*. July 13–17, 2019, Prague, Czech Republic. Ed. by Manuel López-Ibáñez, Anne Auger, and Thomas Stützle. New York, NY, USA: Association for Computing Machinery (ACM), 2019, pp. 1798–1806. ISBN: 978-1-4503-6748-6. doi:10.1145/3319619.3326810. See also [72,73] (cit. on p. 478).

References XI

[72] Carola Doerr, Furong Ye, Naama Horesh, Hao Wang, and Ofer M. ShirThomas Bäck. "Benchmarking Discrete Optimization Heuristics with IOHprofiler". *Applied Soft Computing (ASOC)* 88:106027, Mar. 2020. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 1568-4946. doi:10.1016/j.asoc.2019.106027. See also [71,73] (cit. on pp. 477, 478, 507, 512, 515).

[73] Carola Doerr, Furong Ye, Naama Horesh, Hao Wang, and Ofer M. ShirThomas Bäck. "Benchmarking Discrete Optimization Heuristics with IOHprofiler". (abs/1912.09237), Jan. 31, 2021. doi:10.48550/arXiv.1912.09237. URL: https://arxiv.org/abs/1912.09237 (visited on 2024-06-27). arXiv:1912.09237v2 [cs.NE] 31 Jan 2021, see also [71,72] (cit. on pp. 477, 478).

[74] Marco Dorigo, Mauro Birattari, and Thomas Stützle. "Ant Colony Optimization". *IEEE Computational Intelligence Magazine (CIM)* 1(4):28–39, Nov. 2006. Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE). ISSN: 1556-603X. doi:10.1109/MCI.2006.329691. URL: https://www.researchgate.net/publication/308953674 (visited on 2025-07-25) (cit. on p. 508).

[75] Marco Dorigo, Vittorio Maniezzo, and Alberto Colorni. "Ant System: Optimization by a Colony of Cooperating Agents". *IEEE Transactions on Systems, Man, and Cybernetics – Part B: Cybernetics* 26(1):29–41, Feb. 1996. Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE). ISSN: 1083-4419. doi:10.1109/3477.484436. URL: https://www.researchgate.net/publication/5589170 (visited on 2025-07-26) (cit. on p. 508).

[76] Marco Dorigo and Thomas Stützle. *Ant Colony Optimization*. Cambridge, MA, USA: MIT Press, 2004. ISBN: 978-0-262-04219-2 (cit. on p. 508).

[77] Johann Dréo, Alain Pétrowski, Patrick Siarry, and Éric D. Taillard. *Metaheuristics for Hard Optimization – Methods and Case Studies*. Berlin/Heidelberg, Germany: Springer-Verlag GmbH Germany, 2006. ISBN: 978-3-540-23022-9. doi:10.1007/3-540-30966-7. Discusses SA, TS, EAs, and ACO. (Cit. on p. 516).

[78] Stefan Droste, Thomas Jansen, and Ingo Wegener. "On the Analysis of the $(1+1)$ Evolutionary Algorithm". *Theoretical Computer Science* 276(1-2):51–81, Apr. 2002. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0304-3975. doi:10.1016/S0304-3975(01)00182-7 (cit. on pp. 507, 513, 518).

[79] Stefan Droste, Thomas Jansen, and Ingo Wegener. "Upper and Lower Bounds for Randomized Search Heuristics in Black-Box Optimization". *Theoretical Computer Science* 39(4):525–544, July 2006. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0304-3975. doi:10.1007/S00224-004-1177-Z (cit. on p. 515).

References XII

[80] Jacques Dutka. "The Early History of the Factorial Function". *Archive for History of Exact Sciences* 43(3):225–249, Sept. 1991. Berlin/Heidelberg, Germany: Springer-Verlag GmbH Germany. ISSN: 0003-9519. doi:10.1007/BF00389433. Communicated by Umberto Bottazzini (cit. on p. 521).

[81] Harald Dyckhoff and Ute Finke. *Cutting and Packing in Production and Distribution: A Typology and Bibliography*. Contributions to Management Science. Heidelberg, Baden-Württemberg, Germany: Physica-Verlag, 1992. ISSN: 1431-1941. ISBN: 978-3-642-63487-1. doi:10.1007/978-3-642-58165-6 (cit. on p. 509).

[82] Kelly Easton, George L. Nemhauser, and Michael A. Trick. "The Traveling Tournament Problem Description and Benchmarks". In: *7th International Conference on Principles and Practice of Constraint Programming (CP'01)*. Nov. 26–Dec. 1, 2001, Paphos, Cyprus. Ed. by Toby Walsh. Vol. 2239 of Lecture Notes in Computer Science (LNCS). Berlin/Heidelberg, Germany: Springer-Verlag GmbH Germany, 2001, pp. 580–584. ISSN: 0302-9743. ISBN: 978-3-540-42863-3. doi:10.1007/3-540-45578-7_43 (cit. on p. 519).

[83] Eugene S. Edgington. *Randomization Tests*. 3rd ed. Boca Raton, FL, USA: CRC Press, Inc., 1995. ISBN: 978-0-8247-9669-3 (cit. on pp. 359–364, 376–381).

[84] Alwalid N. Elshafei. "Hospital Layout as a Quadratic Assignment Problem". *The Journal of the Operational Research Society (JORS)* 28(1):167–179, Apr. 1977. London, England, UK: Taylor and Francis Ltd. and London, England, UK: Springer Nature Limited. ISSN: 0160-5682. doi:10.1057/jors.1977.29 (cit. on p. 516).

[85] Bernhard Eschermann and Hans-Joachim Wunderlich. "Optimized Synthesis of Self-Testable Finite State Machines". In: *20th International Symposium Fault-Tolerant Computing (FTCS'1990): Digest of Papers*. June 26–28, 1990, Newcastle Upon Tyne, England, UK. Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE), 1990. ISBN: 978-0-8186-2051-5. doi:10.1109/FTCS.1990.89393 (cit. on p. 516).

[86] Thomas A. Feo and Mauricio G. C. Resende. "Greedy Randomized Adaptive Search Procedures". *Journal of Global Optimization* 6(2):109–133, Mar. 1995. London, England, UK: Springer Nature Limited. ISSN: 0925-5001. doi:10.1007/BF01096763. Introduces Greedy Randomized Adaptive Search Procedures (GRASPs) and applies them to the set covering problem, among others, and lists many problem domains. (Cit. on p. 511).

[87] Luca Ferrari and Enrico Pirozzi. *Learn PostgreSQL*. 2nd ed. Birmingham, England, UK: Packt Publishing Ltd, Oct. 2023. ISBN: 978-1-83763-564-1 (cit. on p. 516).

References XIII

[88] Paola Festa and Mauricio G. C. Resende. "An Annotated Bibliography of GRASP—Part I: Algorithms". *International Transactions on Operational Research (ITOR)* 16(1):1–24, Jan. 2009. Chichester, West Sussex, England, UK: John Wiley and Sons Ltd. ISSN: 0969-6016. doi:10.1111/j.1475-3995.2009.00663.x. See also [89] (cit. on pp. 480, 511).

[89] Paola Festa and Mauricio G. C. Resende. "An Annotated Bibliography of GRASP—Part II: Applications". *International Transactions on Operational Research (ITOR)* 16(2):131–172, Mar. 2009. Chichester, West Sussex, England, UK: John Wiley and Sons Ltd. ISSN: 0969-6016. doi:10.1111/j.1475-3995.2009.00664.x. See also [88] (cit. on pp. 480, 511).

[90] Simon Fischer. "A Polynomial Upper Bound for a Mutation-Based Algorithm on the Two-Dimensional Ising Model". In: *6th Genetic and Evolutionary Computation Conference (GECCO'2004), Part I*. June 26–30, 2004, Seattle, WA, USA. Ed. by Kalyanmoy Deb, Riccardo Poli, Wolfgang Banzhaf, Hans-Georg Beyer, Edmund K. Burke, Paul J. Darwen, Dipankar Dasgupta, Dario Floreano, James A. Foster, Mark Harman, Owen Holland, Pier Luca Lanzi, Lee Spector, Andrea Tettamanzi, Dirk Thierens, and Andrew M. Tyrrell. Vol. 3102 of Lecture Notes in Computer Science (LNCS). Berlin/Heidelberg, Germany: Springer-Verlag GmbH Germany, 2004, pp. 1100–1112. ISSN: 0302-9743. ISBN: 978-3-540-22344-3. doi:10.1007/978-3-540-24854-5_108 (cit. on p. 512).

[91] Simon Fischer and Ingo Wegener. "The One-Dimensional Ising Model: Mutation versus Recombination". *Theoretical Computer Science* 344(2-3):208–225, Nov. 2005. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0304-3975. doi:10.1016/J.TCS.2005.04.002 (cit. on p. 512).

[92] Henry Fisher and Gerald L. Thompson. "Probabilistic Learning Combinations of Local Job-Shop Scheduling Rules". In: *Industrial Scheduling*. Ed. by John F. Muth and Gerald L. Thompson. Englewood Cliffs, NJ, USA: Prentice-Hall, 1963. Chap. 3.2, pp. 225–251 (cit. on p. 512).

[93] Sir Ronald Aylmer Fisher. "On the Interpretation of χ^2 from Contingency Tables, and the Calculation of P". *Journal of the Royal Statistical Society* 85:87–94, 1922. Chichester, West Sussex, England, UK: Blackwell Publishing Ltd for London, England, UK: Royal Statistical Society (RSS). ISSN: 0035-9238. URL: http://hdl.handle.net/2440/15173 (visited on 2021-08-10) (cit. on pp. 376–381).

[94] Philip J. Fleming and John J. Wallace. "How Not to Lie with Statistics: The Correct Way to Summarize Benchmark Results". *Communications of the ACM (CACM)* 29(3):218–221, Mar. 1986. New York, NY, USA: Association for Computing Machinery (ACM). ISSN: 0001-0782. doi:10.1145/5666.5673. This paper also presents that the geometric mean is the only meaningful average measure when dealing with normalized data (as opposed to the arithmetic mean or median (cit. on pp. 188–230, 522).

References XIV

[95] Charles Fleurent and Jacques A. Ferland. "Genetic Hybrids for the Quadratic Assignment Problem". In: *Quadratic Assignment and Related Problems, Proceedings of a DIMACS Workshop*. May 20–21, 1993, New Brunswick, NJ, USA: Rutgers University. Ed. by Panos Miltiades Pardalos and Henry Wolkowicz. Providence, RI, USA: American Mathematical Society (AMS), 1994, pp. 173–187. ISBN: 978-0-8218-6607-8. A MA using the TS from [231] as local search performs well. (Cit. on pp. 514, 516).

[96] Tobias Friedrich, Pietro Simone Oliveto, Dirk Sudholt, and Carsten Witt. "Analysis of Diversity-Preserving Mechanisms for Global Exploration". *Evolutionary Computation* 17(4):455–476, Win. 2009. Cambridge, MA, USA: MIT Press. ISSN: 1063-6560. doi:10.1162/evco.2009.17.4.17401 (cit. on p. 519).

[97] Tobias Friedrich, Francesco Quinzan, and Markus Wagner. "Escaping Large Deceptive Basins of Attraction with Heavy-Tailed Mutation Operators". In: *20th Genetic and Evolutionary Computation Conference (GECCO'2018)*. July 15–19, 2018, Kyōto, Kyōto-fu, Japan. Ed. by Hernán E. Aguirre and Keiki Takadama. New York, NY, USA: Association for Computing Machinery (ACM), 2018, pp. 293–300. ISBN: 978-1-4503-5618-3. doi:10.1145/3205455.3205515 (cit. on pp. 513, 519).

[98] José E. Gallardo, Carlos Cotta, and Antonio J. Fernández. "A Memetic Algorithm for the Low Autocorrelation Binary Sequence Problem". In: *9th Genetic and Evolutionary Computation Conference (GECCO'2007)*. July 7–11, 2007, London, England, UK. Ed. by Hod Lipson. New York, NY, USA: Association for Computing Machinery (ACM), 2007, pp. 1226–1233. ISBN: 978-1-59593-697-4. doi:10.1145/1276958.1277195 (cit. on p. 513).

[99] Michael R. Garey and David Stifler Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. New York, NY, USA: W. H. Freeman and Company, 1979. ISBN: 978-0-7167-1045-5 (cit. on p. 518).

[100] Mitsuo Gen, Yasuhiro Tsujimura, and Erika Kubota. "Solving Job-Shop Scheduling Problems by Genetic Algorithm". In: *Humans, Information and Technology: Proceedings of the 1994 IEEE International Conference on Systems, Man and Cybernetics (SMC'1994)*. Vol. 2. Oct. 2–5, 1994, San Antonio, TX, USA. Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE), 1994, pp. 1577–1582. ISBN: 978-0-7803-2129-8. doi:10.1109/ICSMC.1994.400072. Proposes a permutations-with-repetitions based encoding for the JSSP. (Cit. on pp. 482, 512, 522).

[101] Fred Glover. "Tabu Search – Part I". *ORSA Journal on Computing* 1(3):190–206, Sum. 1989. Catonsville, MD, USA: The Institute for Operations Research and the Management Sciences (INFORMS). ISSN: 0899-1499. doi:10.1287/ijoc.1.3.190 (cit. on p. 518).

References XV

[102] Fred Glover. "Tabu Search – Part II". *ORSA Journal on Computing* 2(1):4–32, Win. 1990. Catonsville, MD, USA: The Institute for Operations Research and the Management Sciences (INFORMS). ISSN: 0899-1499. doi:10.1287/ijoc.2.1.4 (cit. on p. 518).

[103] Fred Glover and Gary A. Kochenberger, eds. *Handbook of Metaheuristics*. Vol. 57 of International Series in Operations Research & Management Science (ISOR). Cham, Switzerland: Springer, 2003. ISSN: 0884-8289. ISBN: 978-0-306-48056-0. doi:10.1007/b101874 (cit. on p. 514).

[104] Fred Glover, Éric D. Taillard, and Dominique de Werra. "A User's Guide to Tabu Search". *Annals of Operations Research* 41(1):3–28, Mar. 1993. London, England, UK: Springer Nature Limited. ISSN: 0254-5330. doi:10.1007/BF02078647. Special issue on TS. (Cit. on p. 518).

[105] David Edward Goldberg. *Genetic Algorithms in Search, Optimization, and Machine Learning*. Redwood City, CA, USA: Addison Wesley Longman Publishing Co., Inc., 1989. ISBN: 978-0-201-15767-3 (cit. on p. 517).

[106] José Fernando Gonçalves and Mauricio G. C. Resende. "A Biased Random Key Genetic Algorithm for two-dimensional Bin Packing Problem (2BP) and 3D Bin Packing Problems". *International Journal of Production Economics (IJPE)* 145(2):500–510, Oct. 2013. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0925-5273. doi:10.1016/j.ijpe.2013.04.019. Applies the Biased Random Key Genetic Algorithm (BRKGA) to the 2BP|R|F and 2BP|O|F as well as 3D packing. Continuous vectors are translated to permutations and orientation data which is then the input of a placement heuristic. (Cit. on pp. 508, 509).

[107] José Fernando Gonçalves and Mauricio G. C. Resende. "An Extended Akers Graphical Method with a Biased Random-Key Genetic Algorithm for Job-Shop Scheduling". *International Transactions on Operational Research (ITOR)* 21(2):215–246, Mar. 2014. Chichester, West Sussex, England, UK: John Wiley and Sons Ltd. ISSN: 0969-6016. doi:10.1111/itor.12044. URL: http://mauricio.resende.info/doc/brkga-jss2011.pdf (visited on 2026-09-30). The BRKGA basically is a MA using a continuous encoding of permutations that are decoded to schedules using the method from [28,29,100,219] and then refined via a TS. (Cit. on pp. 509, 512).

[108] José Fernando Gonçalves and Mauricio G. C. Resende. "Biased Random-Key Genetic Algorithms for Combinatorial Optimization". *Journal of Heuristics* 17:487–525, Oct. 2010. London, England, UK: Springer Nature Limited. ISSN: 1381-1231. doi:10.1007/s10732-010-9143-1 (cit. on p. 509).

References XVI

[109] Michael T. Goodrich. *A Gentle Introduction to NP-Completeness*. Irvine, CA, USA: University of California, Irvine, Apr. 2022. URL: https://ics.uci.edu/~goodrich/teach/cs165/notes/NPComplete.pdf (visited on 2025-08-01) (cit. on p. 524).

[110] Ronald Lewis Graham, Eugene Leighton Lawler, Jan Karel Lenstra, and Alexander Hendrik George Rinnooy Kan. "Optimization and Approximation in Deterministic Sequencing and Scheduling: A Survey". Annals of Discrete Mathematics (5):287–326. Ed. by Peter Ladislaw Hammer, Ellis Lane Johnson, and Bernhard H. Korte, 1979. ISSN: 0167-5060. doi:10.1016/S0167-5060(08)70356-X. An early review on JSSPs and FSSPs. (Cit. on pp. 510, 512).

[111] Martin Grötschel, Michael Jünger, and Gerhard Reinelt. "Optimal Control of Plotting and Drilling Machines: A Case Study". *Zeitschrift für Operations Research (ZOR). now Mathematical Methods of Operations Research (MMOR)* 35(1):61–84, Jan. 1991. Heidelberg, Baden-Württemberg, Germany: Physica-Verlag; now London, England, UK: Springer Nature Limited. ISSN: 0340-9422. doi:10.1007/BF01415960. This paper tackles circuit board plotting and drilling problems by modelling (parts of) them as symmetric Traveling Salesperson Problems (STSPs). (Cit. on p. 518).

[112] Frank E. Grubbs. "Procedures for Detecting Outlying Observations in Samples". *Technometrics* 11(1):1–21, 1969. London, England, UK: Taylor and Francis Ltd. ISSN: 0040-1706. doi:10.1080/00401706.1969.10490657 (cit. on pp. 154–157).

[113] Gregory Z. Gutin and Abraham P. Punnen, eds. *The Traveling Salesman Problem and its Variations*. Vol. 12 of Combinatorial Optimization (COOP). New York, NY, USA: Springer New York, May 2002. ISSN: 1388-3011. doi:10.1007/b101971 (cit. on pp. 10–34, 518).

[114] Lorenz Gygax. *Statistik für Nutztierethologen – Einführung in die statistische Denkweise: Was ist, was macht ein statistischer Test?* 1.1. Zürich, Switzerland: Swiss Federal Veterinary Office (FVO), Centre for Proper Housing of Ruminants and Pigs, June 2003. URL: http://www.proximate-biology.ch/documents/introEtho.pdf (visited on 2025-07-25) (cit. on pp. 376–381).

[115] Nikolaus Hansen, Anne Auger, Steffen Finck, and Raymond Ros. *Real-Parameter Black-Box Optimization Benchmarking 2010: Experimental Setup*. Rapport de Recherche RR-7215. Institut National de Recherche en Informatique et en Automatique (INRIA), Mar. 9, 2010. URL: http://hal.inria.fr/inria-00462481 (visited on 2025-07-25). inria-00462481 (cit. on pp. 36–40, 77–84, 89–94).

[116] Nikolaus Hansen, Steffen Finck, and Raymond Ros. *COCO – COmparing Continuous Optimizers: The Documentation*. Tech. rep. RR-0409. May 2011. URL: http://hal.inria.fr/inria-00597334 (visited on 2025-07-25). inria-00597334 (cit. on pp. 36–40).

References XVII

[117] Pierre Hansen, Nenad Mladenović, and José Andrés Moreno Pérez. "Variable Neighbourhood Search: Methods and Applications. Invited Survey". *4OR* 6(4):319–360, Dec. 2008. ISSN: 1619-4500. doi:10.1007/s10288-008-0089-1. URL: https://www.researchgate.net/publication/225155945 (visited on 2026-10-02) (cit. on p. 521).

[118] Keld Helsgaun. "General k-opt Submoves for the Lin–Kernighan TSP Heuristic". *Mathematical Programming Computation* 1(2-3):119–163, July 2009. London, England, UK: Springer Nature Limited. ISSN: 1867-2949. doi:10.1007/s12532-009-0004-6. URL: https://www2.seas.gwu.edu/~simhaweb/champalg/tsp/papers/Helsgaun2.pdf (visited on 2026-09-26) (cit. on p. 513).

[119] Keld Helsgaun. "An Effective Implementation of the Lin-Kernighan Traveling Salesman Heuristic". 126(1):106–130, Oct. 2000. doi:10.1016/S0377-2217(99)00284-2. URL: http://akira.ruc.dk/~keld/research/LKH/LKH-2.0/DOC/LKH_REPORT.pdf (visited on 2026-09-26) (cit. on p. 513).

[120] André Henning. "Praktische Job-Shop Scheduling-Probleme". PhD thesis. Jena, Thüringen, Germany: Friedrich-Schiller-Universität Jena, Aug. 2002. URL: http://www.db-thueringen.de/servlets/DocumentServlet?id=873 (visited on 2026-09-30). Alternate url: https://nbn-resolving.org/urn:nbn:de:gbv:27-20060809-115700-4. Applies MA-like algorithms to the JSSP. (Cit. on pp. 512, 514).

[121] Myles Hollander and Douglas Alan Wolfe. *Nonparametric Statistical Methods*. Chichester, West Sussex, England, UK: John Wiley and Sons Ltd., 1973. ISBN: 978-0-471-40635-8 (cit. on pp. 376–381).

[122] Holger H. Hoos and Thomas Stützle. "Local Search Algorithms for SAT: An Empirical Evaluation". *Journal of Automated Reasoning* 24(4):421–481, May 2000. London, England, UK: Springer Nature Limited. ISSN: 0168-7433. doi:10.1023/A:1006350622830. URL: https://www.cs.ubc.ca/~hoos/Publ/jar00.pdf (visited on 2025-07-25) (cit. on p. 182).

[123] Holger H. Hoos and Thomas Stützle. *Stochastic Local Search: Foundations & Applications*. The Morgan Kaufmann Series in Artificial Intelligence. Amsterdam, The Netherlands: Elsevier B.V., 2005. ISBN: 978-1-55860-872-6. doi:10.1016/B978-1-55860-872-6.X5016-1 (cit. on pp. 514, 517).

[124] Holger H. Hoos and Thomas Stützle. *SATLIB – The Satisfiability Library*. 1.4.4. Vancouver, BC, Canada: The University of British Columbia (UBC), May 11, 2001. URL: https://www.cs.ubc.ca/~hoos/SATLIB/index-ubc.html (visited on 2026-09-20). See paper [125] (cit. on pp. 485, 517).

References XVIII

[125] Holger H. Hoos and Thomas Stützle. "SATLIB: An Online Resource for Research on SAT". In: *SAT2000 – Highlights of Satisfiability Research in the Year 2000*. Ed. by Ian Philip Gent, Toby Walsh, and Hans van Maaren. Vol. 63 of Frontiers in Artificial Intelligence and Applications. Amsterdam, The Netherlands: IOS Press BV, Jan. 2000, pp. 283–292. ISSN: 0922-6389. ISBN: 978-1-58603-061-2. See the website [124] (cit. on pp. 484, 517).

[126] John Hunt. *A Beginners Guide to Python 3 Programming*. 2nd ed. Undergraduate Topics in Computer Science (UTICS). Cham, Switzerland: Springer, 2023. ISSN: 1863-7310. ISBN: 978-3-031-35121-1. doi:10.1007/978-3-031-35122-8 (cit. on p. 516).

[127] *IEEE Standard for Floating-Point Arithmetic*. IEEE Std 754™-2019 (Revision of IEEE Std 754-2008). New York, NY, USA: Institute of Electrical and Electronics Engineers (IEEE), June 13, 2019 (cit. on p. 522).

[128] *Information Technology – Universal Coded Character Set (UCS)*. International Standard ISO/IEC 10646:2020. Geneva, Switzerland: International Organization for Standardization (ISO) and International Electrotechnical Commission (IEC), Dec. 2020 (cit. on p. 505).

[129] Manuel Iori, Vinícius Loti de Lima, Silvano Martello, Flávio K. Miyazaw, and Michele Monaci. "Exact Solution Techniques for Two-Dimensional Cutting and Packing". *European Journal of Operational Research* 289(2):399–415, Mar. 2021. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0377-2217. doi:10.1016/j.ejor.2020.06.050. See also arXiv:2004.12619v1 [math.OC] 27 Apr 2020 at doi:10.1016/j.ejor.2020.06.050. (Cit. on p. 508).

[130] Christoph Jabs, Dieter Vandesande, Jeremias Berg, Johannes Fichte, George Katsirelos, Ruben Martins, André Schidler, and Dominik Schreiber. *MaxSAT Evaluation: Complete Benchmark Collection*. 2026th ed. Geneva, Switzerland: zenodo (hosted by European Organization for Nuclear Research (CERN)), July 9, 2026. doi:10.5281/zenodo.21283148. All versions of this dataset can be reached directly via doi:10.5281/zenodo.21283147. See also https://github.com/maxsat-evaluations/data and https://maxsat-evaluations.github.io. (Cit. on p. 514).

[131] Dean Jacobs, Jan Prins, Peter Siegel, and Kenneth Wilson. "Monte Carlo Techniques in Code Optimization". In: *15th Annual Workshop on Microprogramming (MICRO 15)*. Oct. 5–7, 1982. Ed. by Joseph Allen Fisher, William J. Tracz, and Bill Hopkins. Palo Alto, CA, USA: Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE) and New York, NY, USA: Association for Computing Machinery (ACM), Oct. 1982, pp. 143–148. doi:10.5555/800036.800944. See [132] (cit. on p. 486).

References XIX

[132] Dean Jacobs, Jan Prins, Peter Siegel, and Kenneth Wilson. "Monte Carlo Techniques in Code Optimization". *ACM SIGMICRO Newsletter* 13(4):143–148, Dec. 1982. New York, NY, USA: Association for Computing Machinery (ACM). ISSN: 1050-916X. doi:10.1145/1014194.800944. See [131] (cit. on pp. 485, 517).

[133] Stefan Jakobs. "On Genetic Algorithms for the Packing of Polygons". *European Journal of Operational Research* 88(1):165–181, Jan. 1996. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0377-2217. doi:10.1016/0377-2217(94)00166-9 (cit. on p. 511).

[134] Wassim Jaziri, ed. *Tabu Search*. London, England, UK: IntechOpen Limited, Aug. 1, 2008. ISBN: 978-3-902613-34-9. doi:10.5772/65. URL: https://www.intechopen.com/books/6114 (visited on 2026-09-30) (cit. on p. 518).

[135] David Stifler Johnson, Gregory Z. Gutin, Lyle Andrew McGeoch, Anders Yeo, Weixiong Zhang, and Alexei Zverovitch. *Experimental Analysis of Heuristics for the ATSP*. Ed. by Gregory Z. Gutin and Abraham P. Punnen. New York, NY, USA, May 2002. doi:10.1007/0-306-48213-4_10. URL: https://archive.dimacs.rutgers.edu/Challenges/TSP/papers/atspchap.pdf (visited on 2026-09-25) (cit. on p. 518).

[136] David Stifler Johnson and Lyle Andrew McGeoch. *Experimental Analysis of Heuristics for the STSP*. Ed. by Gregory Z. Gutin and Abraham P. Punnen. New York, NY, USA, May 2002. doi:10.1007/0-306-48213-4_9. URL: https://archive.dimacs.rutgers.edu/Challenges/TSP/papers/stspchap.pdf (visited on 2026-09-25) (cit. on p. 518).

[137] David Stifler Johnson, Lyle Andrew McGeoch, Fred Glover, and Cesar Rego. *8th DIMACS Implementation Challenge: The Traveling Salesman Problem*. June 1, 2000–Nov. 19, 2013. URL: https://archive.dimacs.rutgers.edu/Challenges/TSP (visited on 2026-09-25) (cit. on p. 518).

[138] Selmer Martin Johnson. "Optimal Two- and Three-Stage Production Schedules with Setup Times Included". *Naval Research Logistics Quarterly* I(1):61–68, Mar. 1954. Chichester, West Sussex, England, UK: John Wiley and Sons Ltd. ISSN: 0894-069X. doi:10.1002/nav.3800010110. URL: https://www.rand.org/content/dam/rand/pubs/papers/2008/P402.pdf (visited on 2023-12-06) (cit. on p. 510).

References XX

[139] Richard "Dick" Manning Karp. "Reducibility Among Combinatorial Problems". In: *Complexity of Computer Computations. Proceedings of a symposium on the Complexity of Computer Computations, held March 20–22, 1972, at the IBM Thomas J. Watson Research Center, Yorktown Heights, New York, and sponsored by the Office of Naval Research, Mathematics Program, IBM World Trade Corporation, and the IBM Research Mathematical Sciences Department*. Mar. 20–22, 1972. Ed. by Raymond E. "Ray" Miller and James W. "Jim" Thatcher. New York, NY, USA: Springer New York, 1972, pp. 85–103. ISBN: 978-1-4684-2003-6. doi:10.1007/978-1-4684-2001-2_9 (cit. on pp. 12–14).

[140] Pascal Kerschke, Jakob Bossek, and Heike Trautmann. "Parameterization of State-of-the-Art Performance Indicators: A Robustness Study based on Inexact TSP Solvers". In: *20th Genetic and Evolutionary Computation Conference Companion (GECCO'2018)*. July 15–19, 2018, Kyōto, Kyōto-fu, Japan. Ed. by Hernán E. Aguirre and Keiki Takadama. New York, NY, USA: Association for Computing Machinery (ACM), 2018, pp. 1737–1744. ISBN: 978-1-4503-5764-7. doi:10.1145/3205651.3208233 (cit. on pp. 89–94).

[141] Scott Kirkpatrick, C. Daniel Gelatt, Jr., and Mario P. Vecchi. "Optimization by Simulated Annealing". *Science Magazine* 220(4598):671–680, May 13, 1983. Washington, D.C., USA: American Association for the Advancement of Science (AAAS). ISSN: 0036-8075. doi:10.1126/science.220.4598.671. URL: https://www.researchgate.net/publication/6026283 (visited on 2025-08-08) (cit. on p. 517).

[142] Robert Klein. *Scheduling of Resource-Constrained Projects*. Vol. 10 of Operations Research/Computer Science Interfaces Series (ORCS). New York, NY, USA: Springer New York, 2000. ISSN: 1387-666X. ISBN: 978-0-7923-8637-7. doi:10.1007/978-1-4615-4629-0 (cit. on p. 510).

[143] Donald Ervin Knuth. "Big Omicron and Big Omega and Big Theta". *ACM SIGACT News* 8(2):18–24, Apr.–June 1976. New York, NY, USA: Association for Computing Machinery (ACM). ISSN: 0163-5700. doi:10.1145/1008328.1008329 (cit. on p. 524).

[144] Donald Ervin Knuth. *Fundamental Algorithms*. 3rd ed. Vol. 1 of The Art of Computer Programming. Reading, MA, USA: Addison-Wesley Professional, 1997. ISBN: 978-0-201-89683-1 (cit. on p. 524).

[145] Donald Ervin Knuth. *Seminumerical Algorithms*. Vol. 2 of The Art of Computer Programming. Reading, MA, USA: Addison-Wesley Professional, Nov. 1997. ISBN: 978-0-201-89684-8 (cit. on pp. 504, 525).

[146] Donald Ervin Knuth. *Sorting and Searching*. Vol. 3 of The Art of Computer Programming. Reading, MA, USA: Addison-Wesley Professional, 1998. ISBN: 978-0-201-89685-5 (cit. on p. 523).

References XXI

[147] Tjalling Charles Koopmans and Martin Beckmann. "Assignment Problems and the Location of Economic Activities". *Econometrica* 25(1):53–76, 1957. New Haven, CT, USA: The Econometric Society and Chichester, West Sussex, England, UK: John Wiley and Sons Ltd. ISSN: 0012-9682. doi:10.2307/1907742 (cit. on p. 516).

[148] Jakob Krarup and Peter Mark Pruzan. "Computer-aided Layout Design". In: *Mathematical Programming in Use*. Ed. by Michel Louis Balinski and Claude Lemaréchal. Vol. 9 of. Berlin/Heidelberg, Germany: Springer-Verlag GmbH Germany, 1978. Chap. 6, pp. 75–94. ISSN: 0303-3929. ISBN: 978-3-642-00796-5. doi:10.1007/BFb0120827. Uses QAPs as model for designing architectural layouts. (Cit. on p. 516).

[149] Mark W. Krentel. "The Complexity of Optimization Problems". *Journal of Computer and System Sciences* 36(3):490–509, June 1988. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0022-0000. Explores the hardness (optimization) problems such TSP, Clique, and Coloring, where computing the optimal value is $\mathcal{NP}$-complete. (Cit. on p. 518).

[150] Edmund Landau. *Handbuch der Lehre von der Verteilung der Primzahlen*. Leipzig, Sachsen, Germany: B. G. Teubner, 1909. ISBN: 978-0-8218-2650-8 (cit. on p. 524).

[151] Eugene Leighton Lawler, Jan Karel Lenstra, and Alexander Hendrik George Rinnooy Kan. "Recent Developments in Deterministic Sequencing and Scheduling: A Survey". In: *Deterministic and Stochastic Scheduling: Proceedings of the NATO Advanced Study and Research Institute on Theoretical Approaches to Scheduling Problems*. June 6–17, 1981, Durham, England, UK. Ed. by Michael Anthony Howe Dempster, Jan Karel Lenstra, and Alexander Hendrik George Rinnooy Kan. Vol. 84 of Nato Science Series C: Mathematical and Physical Science (ASIC). London, England, UK: Springer Nature Limited, Apr. 1982, pp. 35–73. ISBN: 978-90-277-1397-1. doi:10.1007/978-94-009-7801-0_3. The JSSP is $\mathcal{NP}$-hard. (Cit. on p. 512).

[152] Eugene Leighton Lawler, Jan Karel Lenstra, Alexander Hendrik George Rinnooy Kan, and David B. Shmoys. "Sequencing and Scheduling: Algorithms and Complexity". In: *Production Planning and Inventory*. Ed. by Stephen C. Graves, Alexander Hendrik George Rinnooy Kan, and Paul H. Zipkin. Vol. IV of Handbooks of Operations Research and Management Science. Amsterdam, The Netherlands: Elsevier B.V., 1993. Chap. 9, pp. 445–522. ISSN: 0927-0507. ISBN: 978-0-444-87472-6. doi:10.1016/S0927-0507(05)80189-6. URL: http://alexandria.tue.nl/repository/books/339776.pdf (visited on 2023-12-06). Discusses, amongst others, that the JSSP is $\mathcal{NP}$-hard. (Cit. on pp. 12–14, 512, 524).

References XXII

[153] Eugene Leighton Lawler, Jan Karel Lenstra, Alexander Hendrik George Rinnooy Kan, and David B. Shmoys. *The Traveling Salesman Problem: A Guided Tour of Combinatorial Optimization*. Estimation, Simulation, and Control – Wiley-Interscience Series in Discrete Mathematics and Optimization. Chichester, West Sussex, England, UK: Wiley Interscience, Sept. 1985. ISSN: 0277-2698. ISBN: 978-0-471-90413-7 (cit. on pp. 10–34, 518).

[154] Stephen R. Lawrence. "Resource Constrained Project Scheduling: An Experimental Investigation of Heuristic Scheduling Techniques". PhD thesis. Pittsburgh, PA, USA: Graduate School of Industrial Administration (GSIA), Carnegie-Mellon University, 1984 (cit. on p. 512).

[155] Kent D. Lee and Steve Hubbard. *Data Structures and Algorithms with Python*. Undergraduate Topics in Computer Science (UTICS). Cham, Switzerland: Springer, 2015. ISSN: 1863-7310. ISBN: 978-3-319-13071-2. doi:10.1007/978-3-319-13072-9 (cit. on p. 516).

[156] Xueping Li and Kaike Zhang. "Single Batch Processing Machine Scheduling with Two-Dimensional Bin Packing Constraints". *International Journal of Production Economics (IJPE)* 196:113–121, Feb. 2018. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0925-5273. doi:10.1016/j.ijpe.2017.11.015. Mentions some scheduling tasks of Additive Manufacturing jobs can be viewed as instance of the 2BP|R|F. (Cit. on p. 508).

[157] Tianyu Liang (梁天宇), Zhize Wu (吴志泽), Jörg Lässig, Daan van den Berg, Sarah Louise Thomson, and Thomas Weise (汤卫思). "Addressing the Traveling Salesperson Problem with Frequency Fitness Assignment and Hybrid Algorithms". *Soft Computing* 28(17-18):9495–9508, July 2024. London, England, UK: Springer Nature Limited. ISSN: 1432-7643. doi:10.1007/S00500-024-09718-8 (cit. on pp. 10–34, 510, 514, 518).

[158] Tianyu Liang (梁天宇), Zhize Wu (吴志泽), Jörg Lässig, Daan van den Berg, and Thomas Weise (汤卫思). "Solving the Traveling Salesperson Problem using Frequency Fitness Assignment". In: *IEEE Symposium Series on Computational Intelligence (SSCI'2022)*. Dec. 4–7, 2022, Singapore. Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE), 2022. ISBN: 978-1-6654-8769-6. doi:10.1109/SSCI51031.2022.10022296 (cit. on pp. 510, 514).

References XXIII

[159] Tianyu Liang (梁天宇), Zhize Wu (吴志泽), Matthias Thürer, Markus Wagner, and Thomas Weise (汤卫思). "Generating Small Instances with Interesting Features for the Traveling Salesperson Problem". In: *16th International Joint Conference on Computational Intelligence (IJCCI'24)*. Nov. 20–22, 2024, Porto, Portugal. Ed. by Francesco Marcelloni, Kurosh Madani, Niki van Stein, and Joaquim Filipe. Porto, Portugal: SciTePress: Science and Technology Publications, Lda, 2024, pp. 173–180. ISSN: 2184-3236. ISBN: 978-989-758-721-4. doi:10.5220/0012888800003837 (cit. on p. 514).

[160] Shen Lin and Brian W. Kernighan. "An Effective Heuristic Algorithm for the Traveling-Salesman Problem". *Operations Research* 21(2):498–516, Mar.–Apr. 1973. Catonsville, MD, USA: The Institute for Operations Research and the Management Sciences (INFORMS). ISSN: 0030-364X. doi:10.1287/opre.21.2.498. URL: https://www.cs.princeton.edu/~bwk/btl.mirror/tsp.pdf (visited on 2026-09-26) (cit. on p. 513).

[161] Dequan Liu (刘德全) and Hongfei Teng (滕弘飞). "An Improved BL-Algorithm for EAGenetic Algorithm of the Orthogonal Packing of Rectangles". *European Journal of Operational Research* 112(2):413–420, Jan. 1999. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0377-2217. doi:10.1016/S0377-2217(97)00437-2 (cit. on p. 511).

[162] Andrea Lodi, Silvano Martello, and Michele Monaci. "Two-Dimensional Packing Problems: A Survey". *European Journal of Operational Research* 141(2):241–252, Sept. 2002. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0377-2217. doi:10.1016/S0377-2217(02)00123-6 (cit. on pp. 508, 509).

[163] Andrea Lodi, Silvano Martello, and Daniele Vigo. "Heuristic and Metaheuristic Approaches for a Class of Two-Dimensional Bin Packing Problems". *INFORMS Journal on Computing* 11(4):345–357, Aut. 1999. Catonsville, MD, USA: The Institute for Operations Research and the Management Sciences (INFORMS). ISSN: 1091-9856. doi:10.1287/ijoc.11.4.345 (cit. on p. 508).

[164] Andrea Lodi, Silvano Martello, and Daniele Vigo. "Recent Advances on Two-Dimensional Bin Packing Problems". *Discrete Applied Mathematics* 123(1–3):379–396, Nov. 2002. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0166-218X. doi:10.1016/S0166-218X(01)00347-X (cit. on pp. 508, 509).

[165] Eliane Maria Loiola, Nair Maria Maia de Abreu, Paulo Oswaldo Boaventura-Netto, Peter M. Hahn, and Tania Querido. "A Survey for the Quadratic Assignment Problem". *European Journal of Operational Research* 176(2):657–690, 2007. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0377-2217. doi:10.1016/j.ejor.2005.09.032 (cit. on p. 516).

References XXIV

[166] Mariana A. Londe, Luciana S. Pessoa, Carlos E. Andrade, and Mauricio G. C. Resende. *Biased Random-Key Genetic Algorithms: A Review*. arXiv.org: Computing Research Repository (CoRR) abs/2312.00961v2. Ithaca, NY, USA: Cornell Universiy Library, Dec. 6, 2023. doi:10.48550/arXiv.2312.00961. URL: https://arxiv.org/abs/2312.00961v2 (visited on 2024-06-27). arXiv:2312.00961v2 [cs.NE] 06 Dec 2023; See [167] (cit. on p. 491).

[167] Mariana A. Londe, Luciana S. Pessoa, Carlos E. Andrade, and Mauricio G. C. Resende. "Biased Random-Key Genetic Algorithms: A Review". *European Journal of Operational Research* 321(1):1–22, Feb. 2025. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0377-2217. doi:10.1016/j.ejor.2024.03.030. See [166] (cit. on pp. 491, 509).

[168] Marc Loy, Patrick Niemeyer, and Daniel Leuck. *Learning Java*. 5th ed. Sebastopol, CA, USA: O'Reilly Media, Inc., Mar. 2020. ISBN: 978-1-4920-5627-0 (cit. on p. 512).

[169] Peter Luschny. *A New Kind of Factorial Function*. Highland Park, NJ, USA: The OEIS Foundation Inc., Oct. 4, 2015. URL: https://oeis.org/A000142/a000142.pdf (visited on 2024-09-29) (cit. on p. 521).

[170] Mark Lutz. *Learning Python*. 6th ed. Sebastopol, CA, USA: O'Reilly Media, Inc., Mar. 2025. ISBN: 978-1-0981-7130-8 (cit. on p. 516).

[171] Gangadharrao Soundalyarao Maddala. *Introduction to Econometrics*. 2nd ed. New York, NY, USA: Macmillan Publishers, 1992. ISBN: 978-0-02-374545-4 (cit. on pp. 154–157).

[172] Henry B. Mann and Donald R. Whitney. "On a Test of whether One of Two Random Variables is Stochastically Larger than the Other". *The Annals of Mathematical Statistics (AOMS)* 18(1):50–60, Mar. 1947. Beachwood, OH, USA: Institute of Mathematical Statistics. ISSN: 0003-4851. doi:10.1214/aoms/1177730491. URL: http://projecteuclid.org/euclid.aoms/1177730491 (visited on 2025-07-25) (cit. on pp. 376–381).

[173] Reinhard Männer and Bernard Manderick, eds. *Parallel Problem Solving from Nature 2 (PPSN-II)*. Sept. 28–30, 1992, Brussels, Belgium. Amsterdam, The Netherlands: Elsevier B.V., 1992.

[174] Francesco Marcelloni, Kurosh Madani, Niki van Stein, and Joaquim Filipe, eds. *16th International Joint Conference on Computational Intelligence (IJCCI'24)*. Nov. 20–22, 2024, Porto, Portugal. Porto, Portugal: SciTePress: Science and Technology Publications, Lda, 2024. ISSN: 2184-3236. ISBN: 978-989-758-721-4. doi:10.5220/0000195000003837.

References XXV

[175] Silvano Martello and Daniele Vigo. "Exact Solution of the Two-Dimensional Finite Bin Packing Problem". 44(3):388–399, Mar. 1998. doi:10.1287/mnsc.44.3.388 (cit. on p. 508).

[176] Stephan Mertens. "Exhaustive Search for Low-Autocorrelation Binary Sequences". *Journal of Physics A: Mathematical and General* 29(18):L473, Sept. 1996. Bristol, England, UK: IOP Publishing Ltd. ISSN: 0305-4470. doi:10.1088/0305-4470/29/18/005 (cit. on p. 513).

[177] Peter Merz and Bernd Freisleben. "A Comparison of MAMemetic Algorithms, TSTabu Search, and ACOAnt Colonies for the QAPQuadratic Assignment Problem". In: *Congress on Evolutionary Computation (CEC'1999)*. July 6–9, 1999, Washington, D.C., USA. Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE), 1999, pp. 2063–2070. doi:10.1109/CEC.1999.785529 (cit. on p. 516).

[178] Zbigniew "Zbyszek" Michalewicz. *Genetic Algorithms + Data Structures = Evolution Programs*. Berlin/Heidelberg, Germany: Springer-Verlag GmbH Germany, 1996. ISBN: 978-3-540-58090-4. doi:10.1007/978-3-662-03315-9 (cit. on p. 517).

[179] Burkhard Militzer, Michele Zamparelli, and Dieter Beule. "Evolutionary Search for Low Autocorrelated Binary Sequences". *IEEE Transactions on Evolutionary Computation* 2(1):34–39, Apr. 1998. Los Alamitos, CA, USA: IEEE Computer Society. ISSN: 1089-778X. doi:10.1109/4235.728212 (cit. on p. 513).

[180] Rupert G. Miller, Jr. *Simultaneous Statistical Inference*. 2nd ed. Springer Series in Statistics. Berlin/Heidelberg, Germany: Springer-Verlag GmbH Germany, 1981. ISSN: 0172-7397. ISBN: 978-0-387-90548-8 (cit. on pp. 392, 394, 395, 397).

[181] Alfonsas Misevičius. "A Tabu Search Algorithm for the Quadratic Assignment Problem". *Computational Optimization and Applications* 30:95–111, Jan. 2005. London, England, UK: Springer Nature Limited. ISSN: 0926-6003. doi:10.1007/s10589-005-4562-x (cit. on p. 518).

[182] Alfonsas Misevičius. "A Modified Simulated Annealing Algorithm for the Quadratic Assignment Problem". *Informatica* 14(4):497–514, Jan. 2003. Vilnius, Lithuania: Vilnius University, Institute of Data Science and Digital Technologies (DMSTI), Lithuanian Academy of Sciences. ISSN: 0868-4952. doi:10.15388/Informatica.2003.037 (cit. on p. 517).

[183] Alfonsas Misevičius. *An Implementation of the Iterated Tabu Search Algorithm for the Quadratic Assignment Problem*. Working Paper. Kaunas, Lithuania: Kaunas University of Technology, 2008. See also [184] (cit. on p. 493).

References XXVI

[184] Alfonsas Misevičius. "An Implementation of the Iterated Tabu Search Algorithm for the Quadratic Assignment Problem". *OR Spectrum: Quantitative Approaches in Management. Formerly Operations-Research-Spektrum* 34(3):665–690, July 2012. London, England, UK: Springer Nature Limited. ISSN: 0171-6468. doi:10.1007/s00291-011-0274-z. See also [183] (cit. on pp. 492, 516, 518).

[185] Melanie Mitchell. *An Introduction to Genetic Algorithms*. Complex Adaptive Systems. Cambridge, MA, USA: MIT Press, Feb. 1998. ISBN: 978-0-262-13316-6. URL: http://boente.eti.br/fuzzy/ebook-fuzzy-mitchell.pdf (visited on 2025-08-08) (cit. on p. 517).

[186] Nenad Mladenović and Pierre Hansen. "Variable Neighborhood Search". *Computers & Operations Research* 24(11):1097–1100, Nov. 1997. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0305-0548. doi:10.1016/S0305-0548(97)00031-2 (cit. on p. 521).

[187] Wai Ho Mow, Ke-Lin Du, and Wei Hsiang Wu. "New Evolutionary Search for Long Low Autocorrelation Binary Sequences". *IEEE Transactions on Aerospace and Electronic Systems* 51(1):290–303, Jan. 2015. Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE). ISSN: 0018-9251. doi:10.1109/TAES.2014.130518 (cit. on p. 513).

[188] Heinz Mühlenbein. "How Genetic Algorithms Really Work: Mutation and Hillclimbing". In: *Parallel Problem Solving from Nature 2 (PPSN-II)*. Sept. 28–30, 1992, Brussels, Belgium. Ed. by Reinhard Männer and Bernard Manderick. Amsterdam, The Netherlands: Elsevier B.V., 1992, pp. 15–26. URL: https://www.researchgate.net/publication/284805798 (visited on 2025-08-15) (cit. on p. 515).

[189] Siegfried Nijssen and Thomas Bäck. "An Analysis of the Behavior of Simplified Evolutionary Algorithms on Trap Functions". *IEEE Transactions on Evolutionary Computation* 7(1):11–22, Feb. 2003. Los Alamitos, CA, USA: IEEE Computer Society. ISSN: 1089-778X. doi:10.1109/TEVC.2002.806169 (cit. on p. 518).

[190] Eugeniusz Nowicki and Czesław Smutnicki. "A Fast Taboo Search Algorithm for the Job Shop Problem". *Management Science* 42(6):783–938, June 1996. Catonsville, MD, USA: The Institute for Operations Research and the Management Sciences (INFORMS). ISSN: 0025-1909. doi:10.1287/mnsc.42.6.797 (cit. on p. 518).

[191] Regina O. Obe and Leo S. Hsu. *PostgreSQL: Up and Running*. 3rd ed. Sebastopol, CA, USA: O'Reilly Media, Inc., Oct. 2017. ISBN: 978-1-4919-6336-4 (cit. on p. 516).

[192] A. Jefferson Offutt. "Unit Testing Versus Integration Testing". In: *Test: Faster, Better, Sooner – IEEE International Test Conference (ITC'1991)*. Oct. 26–30, 1991, Nashville, TN, USA. Los Alamitos, CA, USA: IEEE Computer Society, 1991. Chap. Paper P2.3, pp. 1108–1109. ISSN: 1089-3539. ISBN: 978-0-8186-9156-0. doi:10.1109/TEST.1991.519784 (cit. on p. 520).

References XXVII

[193] Michael Olan. "Unit Testing: Test Early, Test Often". *Journal of Computing Sciences in Colleges (JCSC)* 19(2):319–328, Dec. 2003. New York, NY, USA: Association for Computing Machinery (ACM). ISSN: 1937-4771. doi:10.5555/948785.948830. URL: https://www.researchgate.net/publication/255673967 (visited on 2025-09-05) (cit. on p. 520).

[194] Tom Packebusch and Stephan Mertens. "Low Autocorrelation Binary Sequences". *Journal of Physics A: Mathematical and Theoretical* 49(16):165001, Mar. 2016. Bristol, England, UK: IOP Publishing Ltd. ISSN: 1751-8121. doi:10.1088/1751-8113/49/16/165001 (cit. on p. 513).

[195] Ashwin Pajankar. *Python Unit Test Automation: Automate, Organize, and Execute Unit Tests in Python*. New York, NY, USA: Apress Media, LLC, Dec. 2021. ISBN: 978-1-4842-7854-3 (cit. on pp. 168–183, 520).

[196] Panos Miltiades Pardalos, Ding-Zhu Du, and Ronald Lewis Graham, eds. *Handbook of Combinatorial Optimization*. 1st ed. Boston, MA, USA: Springer, 1998. ISBN: 978-1-4613-7987-4.

[197] Panos Miltiades Pardalos, Oleg V. Shylo, and Alkis Vazacopoulos. "Solving Job Shop Scheduling Problems Utilizing the Properties of Backbone and "Big Valley"". *Computational Optimization and Applications* 47(1):61–76, Sept. 2010. London, England, UK: Springer Nature Limited. ISSN: 0926-6003. doi:10.1007/s10589-008-9206-5. An algorithm similar to SA. (Cit. on pp. 512, 517).

[198] Francisco Parreño, Ramón Alvarez-Valdés, José Fernando Oliveira, and José Manuel Tamarit. "A Hybrid GRASP/VND Algorithm for Two- and Three-Dimensional Bin Packing". *Annals of Operations Research* 179(1):203–220, Sept. 2010. London, England, UK: Springer Nature Limited. ISSN: 0254-5330. doi:10.1007/s10479-008-0449-4 (cit. on pp. 508, 511, 520).

[199] Bo Peng, Zhipeng Lü, and T. C. Edwin Cheng. "A Tabu Search/Path Relinking Algorithm to Solve the Job Shop Scheduling Problem". *Computers & Operations Research* 53:154–164, Jan. 2015. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0305-0548. doi:10.1016/j.cor.2014.08.006. URL: http://arxiv.org/abs/1402.5613 (visited on 2026-09-30) (cit. on pp. 512, 518).

[200] Yasset Pérez-Riverol, Laurent Gatto, Rui Wang, Timo Sachsenberg, Julian Uszkoreit, Felipe da Veiga Leprevost, Christian Fufezan, Tobias Ternent, Stephen J. Eglen, Daniel S. Katz, Tom J. Pollard, Alexander Konovalov, Robert M. Flight, Kai Blin, and Juan Antonio Vizcaíno. "Ten Simple Rules for Taking Advantage of Git and GitHub". *PLOS Computational Biology* 12(7), July 14, 2016. San Francisco, CA, USA: Public Library of Science (PLOS). ISSN: 1553-7358. doi:10.1371/JOURNAL.PCBI.1004947 (cit. on p. 510).

References XXVIII

[201] Thomas V. Perneger. "What's wrong with Bonferroni adjustments". *British Medical Journal (BMJ)* (316):1236, Apr. 1998. ISSN: 1756-1833. doi:10.1136/bmj.316.7139.1236 (cit. on pp. 389–392, 394, 395, 397).

[202] Ferdinando Pezzella and Emanuela Merelli. "A Tabu Search Method Guided by Shifting Bottleneck for the Job Shop Scheduling Problem". *European Journal of Operational Research* 120(2):297–310, Jan. 2000. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0377-2217. doi:10.1016/S0377-2217(99)00158-7. URL: https://www2.cs.sfu.ca/CourseCentral/827/havens/papers/topic%5C%2310(JobShop)/Tabu%5C%20With%5C%20Shifting.pdf (visited on 2026-09-30) (cit. on p. 518).

[203] Martin Pincus. "Letter to the Editor – A Monte Carlo Method for the Approximate Solution of Certain Types of Constrained Optimization Problems". *Operations Research* 18(6):1225–1228, Nov.–Dec. 1970. Catonsville, MD, USA: The Institute for Operations Research and the Management Sciences (INFORMS). ISSN: 0030-364X. doi:10.1287/opre.18.6.1225 (cit. on p. 517).

[204] Marcelo Pinto, Cristóvão Silva, Matthias Thürer, and Samuel Moniz. "Survey in Operations Research and Management Science. Nesting and Scheduling Optimization of Additive Manufacturing Systems: Mapping the Territory". *Computers & Operations Research* 165(106592), Feb. 2024. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0305-0548. doi:10.1016/j.cor.2024.106592. Mentions some scheduling tasks of Additive Manufacturing jobs can be viewed as instance of the 2BP|R|F. (Cit. on p. 508).

[205] *PostgreSQL Essentials: Leveling Up Your Data Work*. Sebastopol, CA, USA: O'Reilly Media, Inc., Mar. 2024 (cit. on p. 516).

[206] S. D. Prestwich. *Improved Branch-and-Bound for Low Autocorrelation Binary Sequences*. arXiv.org: Computing Research Repository (CoRR) abs/1305.6187. Ithaca, NY, USA: Cornell Universiy Library, Apr. 3, 2024. doi:10.48550/arXiv.1305.6187. URL: https://arxiv.org/abs/1305.6187 (visited on 2013-07-23). arXiv:1305.6187v2 [cs.AI] 23 Jul 2013 (cit. on p. 513).

[207] Kenneth V. Price. "Differential Evolution vs. The Functions of the 2nd ICEO". In: *IEEE International Conference on Evolutionary Computation*. Indianapolis, IN, USA. Ed. by Russell C. "Russ" Eberhart, Peter J. Angeline, Thomas Bäck, Zbigniew "Zbyszek" Michalewicz, and Xin Yao (姚新). Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE), Apr. 13–16, 1997, pp. 153–157. ISBN: 978-0-7803-3949-1. doi:10.1109/ICEC.1997.592287 (cit. on pp. 89–94).

References XXIX

[208] *Programming Languages – C, Working Document of SC22/WG14*. International Standard ISO/3IEC9899:2017 C17 Ballot N2176. Geneva, Switzerland: International Organization for Standardization (ISO) and International Electrotechnical Commission (IEC), Nov. 2017. URL: https://files.lhmouse.com/standards/ISO%20C%20N2176.pdf (visited on 2024-06-29) (cit. on p. 509).

[209] Sanatan Rai and George Vairaktarakis. "NP-Complete Problems and Proof Methodology". In: *Encyclopedia of Optimization*. Ed. by Christodoulos A. Floudas and Panos Miltiades Pardalos. 2nd ed. Boston, MA, USA: Springer, Sept. 2008, pp. 2675–2682. ISBN: 978-0-387-74758-3. doi:10.1007/978-0-387-74759-0_462 (cit. on p. 524).

[210] Soraya B. Rana and L. Darrell Whitley. "Genetic Algorithm Behavior in the MAXSAT Domain". In: *5th International Conference on Parallel Problem Solving from Nature (PPSN V)*. Sept. 27–30, 1998, Amsterdam, The Netherlands. Ed. by Ágoston Endre Eiben, Thomas Bäck, Marc Schoenauer, and Hans-Paul Schwefel. Lecture Notes in Computer Science (LNCS). 1998, pp. 785–794. ISSN: 0302-9743. ISBN: 978-3-540-65078-2. doi:10.1007/BFb0056920 (cit. on p. 514).

[211] Gerhard Reinelt. "TSPLIB – A Traveling Salesman Problem Library". *ORSA Journal on Computing* 3(4):376–384, Aut. 1991. Catonsville, MD, USA: The Institute for Operations Research and the Management Sciences (INFORMS). ISSN: 0899-1499. doi:10.1287/ijoc.3.4.376 (cit. on pp. 518, 519).

[212] Gerhard Reinelt. *TSPLIB95*. Tech. rep. Heidelberg, Baden-Württemberg, Germany: Universität Heidelberg, Institut für Angewandte Mathematik, 1995–Apr. 28, 2016. URL: https://github.com/shredderzwj/TSPLIB (visited on 2026-09-25). The original Uniform Resource Locator (URL) of the repository has become defunct in 2026. Therefore, we provide the repository listed as main URL which contains the data. Additionally, you may also check https://github.com/thomasWeise/tspSuite of our own project *TSPSuite*, which contains all the TSPLib instances as well as Java code to load them. (Cit. on pp. 518, 519).

[213] Günter Rudolph. *Convergence Properties of Evolutionary Algorithms*. Vol. 35 of Forschungsergebnisse zur Informatik. Hamburg, Germany: Verlag Dr. Kovač, 1997. ISSN: 1435-6260. ISBN: 978-3-86064-554-3 (cit. on p. 513).

[214] Per Runeson. "A Survey of Unit Testing Practices". *IEEE Software* 23(4):22–29, July–Aug. 2006. Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE). ISSN: 0740-7459. doi:10.1109/MS.2006.91 (cit. on pp. 168–183, 520).

References XXX

[215] Sartaj Sahni and Teofilo Francisco Gonzalez Arce. "P-complete Approximation Problems". *Journal of the ACM (JACM)* 23(3):555–565, July 1976. New York, NY, USA: Association for Computing Machinery (ACM). ISSN: 0004-5411. doi:10.1145/321958.321975. URL: https://sites.cs.ucsb.edu/~teo/papers/JACM-PCom.pdf (visited on 2026-02-21) (cit. on p. 516).

[216] Guntram Scheithauer. *Introduction to Cutting and Packing Optimization: Problems, Modeling Approaches, Solution Methods*. Vol. 263 of International Series in Operations Research & Management Science (ISOR). Cham, Switzerland: Springer, 2018. ISSN: 0884-8289. ISBN: 978-3-319-64402-8. doi:10.1007/978-3-319-64403-5 (cit. on p. 509).

[217] Juliet Popper Shaffer. "Multiple Hypothesis Testing". *Annual Review of Psychology* 46(1):561–584, Jan. 1995. ISSN: 0066-4308. doi:10.1146/annurev.ps.46.020195.003021. URL: https://www.researchgate.net/publication/224817412 (cit. on pp. 392, 394, 395, 397).

[218] Yakov Shafranovich. *Common Format and MIME Type for Comma-Separated Values (CSV) Files*. Request for Comments (RFC) 4180. Wilmington, DE, USA: Internet Engineering Task Force (IETF), Oct. 2005. URL: https://www.ietf.org/rfc/rfc4180.txt (visited on 2025-02-05) (cit. on p. 509).

[219] Guoyong Shi, Hitoshi Iima, and Nobuo Sannomiya. "New Encoding Scheme for Solving Job Shop Problems by Genetic Algorithm". In: *35th IEEE Conference on Decision and Control (CDC'96)*. Vol. 4. Dec. 11–13, 1996. Kōbe, Hyōgo-ken, Japan: Institute of Electrical and Electronics Engineers (IEEE), 1997, pp. 4395–4400. ISBN: 978-0-7803-3590-5. doi:10.1109/CDC.1996.577484. URL: https://www.researchgate.net/publication/224238934 (visited on 2026-09-30). Proposes a permutations-with-repetitions based encoding for the JSSP. (Cit. on pp. 482, 512, 522).

[220] Oleg V. Shylo. *Job Shop Scheduling (Personal Homepage)*. Knoxville, TN, USA: Oleg V. Shylo, Aug. 2019. URL: http://optimizizer.com/jobshop.php (visited on 2026-09-30). Contains many JSSP benchmark instances. (Cit. on pp. 497, 512).

[221] Oleg V. Shylo and Hesam Shams. *Boosting Binary Optimization via Binary Classification: A Case Study of Job Shop Scheduling*. arXiv.org: Computing Research Repository (CoRR) abs/1808.10813. Proposes a TS using regression to adapt the tabu tenures for each variable of the solution vector of a JSSP separately. Many results are available in the GitHub repository https://github.com/quasiquasar/gta-jobshop-data; other results are published Shylo's website http://optimizizer.com/jobshop.php. Ithaca, NY, USA: Cornell Universiy Library, Aug. 31, 2018. doi:10.48550/arXiv.1808.10813. URL: https://arxiv.org/abs/1808.10813 (visited on 2026-09-30) (cit. on pp. 512, 518).

References XXXI

[222] Sidney Siegel and N. John Castellan Jr. *Nonparametric Statistics for The Behavioral Sciences*. Humanities, Social Sciences, & Language. New York, NY, USA: McGraw-Hill, 1988. ISBN: 978-0-07-057357-4 (cit. on pp. 376–381).

[223] Anna Skoulikari. *Learning Git*. Sebastopol, CA, USA: O'Reilly Media, Inc., May 2023. ISBN: 978-1-0981-3391-7 (cit. on p. 510).

[224] Hossein M. Soroush. "Scheduling in Stochastic Bicriteria Single Machine Systems with Set-up Times". *International Journal of Planning and Scheduling* 1(1–2):109–145, Jan. 2011. Geneva, Switzerland: Inderscience Publishers. ISSN: 2044-494X. doi:10.1504/IJPS.2011.044605. A scheduling problem is phrased as QAP. (Cit. on p. 516).

[225] Leon Steinberg. "The Backboard Wiring Problem: A Placement Algorithm". *SIAM Review* 3(1):37–50, Jan. 1961. Philadelphia, PA, USA: Society for Industrial and Applied Mathematics Publications (SIAM). ISSN: 0036-1445. doi:10.1137/1003003. The problem is modeled as a QAP (cit. on p. 516).

[226] Robert H. Storer, S. David Wu, and Renzo Vaccari. "New Search Spaces for Sequencing Problems with Application to Job Shop Scheduling". *Management Science* 38(10):1495–1509, Oct. 1992. Catonsville, MD, USA: The Institute for Operations Research and the Management Sciences (INFORMS). ISSN: 0025-1909. doi:10.1287/mnsc.38.10.1495 (cit. on p. 512).

[227] Thomas Strecker and Leonhard Hennig. "Automatic Layouting of Personalized Newspaper Pages". In: *Operations Research Proceedings 2008: Selected Papers of the Annual International Conference of the German Operations Research Society (GOR)*. Sept. 3–5, 2008, Augsburg, Bayern, Germany: University of Augsburg. Ed. by Bernhard Fleischmann, Karl-Heinz Borgwardt, Robert Klein, and Axel Tuma. Vol. 2008 of Operations Research Proceedings (ORP), Gesellschaft für Operations Research e.V., (GOR). Berlin/Heidelberg, Germany: Springer-Verlag GmbH Germany, 2009, pp. 469–474. ISSN: 0721-5924. doi:10.1007/978-3-642-00142-0_76 (cit. on p. 508).

[228] Thomas Stützle. "Iterated Local Search for the Quadratic Assignment Problem". *European Journal of Operational Research* 174(3):1519–1539, Nov. 2006. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0377-2217. doi:10.1016/j.ejor.2005.01.066 (cit. on pp. 511, 516).

[229] Éric D. Taillard. "Benchmarks for Basic Scheduling Problems". *European Journal of Operational Research* 64(2):278–285, Jan. 1993. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0377-2217. doi:10.1016/0377-2217(93)90182-M. URL: https://arodes.hes-so.ch/record/8224/files/Preprint.pdf (visited on 2026-02-20) (cit. on p. 512).

References XXXII

[230] Éric D. Taillard. "Comparison of Iterative Searches for the Quadratic Assignment Problem". *Location Science. Now incorporated into Computers & Operations Research* 3(2):87–105, Aug. 1995. Amsterdam, The Netherlands: Elsevier B.V. doi:10.1016/0966-8349(95)00008-6 (cit. on p. 518).

[231] Éric D. Taillard. "Robust Taboo Search for the Quadratic Assignment Problem". *Parallel Computing* 17(4–5):443–455, July 1991. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0167-8191. doi:10.1016/S0167-8191(05)80147-4 (cit. on pp. 481, 499, 516, 518).

[232] Éric D. Taillard and Luca M. Gambardella. *Adaptive Memories for the Quadratic Assignment Problem*. Tech. rep. IDSIA-87-97. Lugano, Switzerland: Istituto Dalle Molle Di Studi Sull Intelligenza Artificiale (IDSIA), Sept. 10, 1997. URL: https://mistic.iict-heig-vd.ch/taillard/articles.dir/TaillardGambardella1997.pdf (visited on 2026-09-28). A MA using TS from [231] as well as a ACO-based approach refining its solutions with a very simple SLS are presented. (Cit. on p. 516).

[233] Ke Tang (唐珂), Xiaodong Li, Ponnuthurai Nagaratnam Suganthan, Zhenyu Yang (杨振宇), and Thomas Weise (汤卫思). *Benchmark Functions for the CEC'2010 Special Session and Competition on Large-Scale Global Optimization*. Tech. rep. Hefei, Anhui, China (中国安徽省合肥市): University of Science and Technology of China (USTC, 中国科学技术大学), School of Computer Science and Technology (计算机科学与技术学院), Nature Inspired Computation and Applications Laboratory (NICAL), Jan. 8, 2010 (cit. on pp. 95–100).

[234] Alkin Tezuysal and Ibrar Ahmed. *Database Design and Modeling with PostgreSQL and MySQL*. Birmingham, England, UK: Packt Publishing Ltd, July 2024. ISBN: 978-1-80323-347-5 (cit. on p. 516).

[235] Dirk Thierens, David Edward Goldberg, and Angela Guimarães Pereira. "Domino Convergence, Drift, and the Temporal-Salience Structure of Problems". In: pp. 535–540. doi:10.1109/ICEC.1998.700085 (cit. on p. 507).

[236] George K. Thiruvathukal, Konstantin Läufer, and Benjamin Gonzalez. "Unit Testing Considered Useful". *Computing in Science & Engineering* 8(6):76–87, Nov.–Dec. 2006. Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE). ISSN: 1521-9615. doi:10.1109/MCSE.2006.124. URL: https://www.researchgate.net/publication/220094077 (visited on 2024-10-01) (cit. on pp. 168–183, 520).

References XXXIII

[237] Sarah Louise Thomson, Gabriela Ochoa, Daan van den Berg, Tianyu Liang (梁天宇), and Thomas Weise (汤卫思). "Entropy, Search Trajectories, and Explainability for Frequency Fitness Assignment". In: *Parallel Problem Solving from Nature (PPSN XVIII)*. Vol. 1. Sept. 14–18, 2024, Hagenberg, Mühlkreis, Austria. Ed. by Michael Affenzeller, Stephan M. Winkler, Anna V. Kononova, Heike Trautmann, Tea Tušar, Penousal Machado, and Thomas Bäck. Vol. 15148 of Lecture Notes in Computer Science (LNCS). Cham, Switzerland: Springer. ISSN: 0302-9743. ISBN: 978-3-031-70054-5. doi:10.1007/978-3-031-70055-2_23. Explores randomized local search (RLS), RLS with FFA (FRLS), and SA on the QAP. (Cit. on pp. 510, 514).

[238] Ulrich Wilhelm Thonemann and Andreas Bölte. *An Improved Simulated Annealing Algorithm for the Quadratic Assignment Problem*. Working Paper. Paderborn, Nordrhein-Westfalen, Germany: School of Business, Department of Production and Operations Research, University of Paderborn, 1994 (cit. on p. 517).

[239] Marco Tomassini. "Complex Networks Analysis of the Energy Landscape of the Low Autocorrelation Binary Sequences Problem". *Physica A: Statistical Mechanics and its Applications* 577:126089, Sept. 1, 2021. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0378-4371. doi:10.1016/j.physa.2021.126089. Lists the LABS as $\mathcal{NP}$-hard. (Cit. on p. 513).

[240] Mariot Tsitoara. *Beginning Git and GitHub: Version Control, Project Management and Teamwork for the New Developer*. New York, NY, USA: Apress Media, LLC, Mar. 2024. ISBN: 979-8-8688-0215-7 (cit. on pp. 510, 520).

[241] Daan van den Berg, Florian Braam, Mark Moes, Emiel Suilen, and Sandjai Bhulai. "Almost Squares in Almost Squares: Solving the Final Instance". In: *5th International Conference on Data Analytics (DATA ANALYTICS'2016)*. Oct. 9–13, 2016, Venice, Venezia, Veneto, Italy. Ed. by Sandjai Bhulai and Ivana Semanjski. Wilmington, DE, USA: International Academy, Research, and Industry Association (IARIA), 2016, pp. 69–74. ISSN: 2308-4464. ISBN: 978-1-61208-510-4. URL: https://math.vu.nl/~sbhulai/publications/data_analytics2016b.pdf (visited on 2026-10-01). This poses a very interesting 2BP|R|F problem, where "almost squars," i.e., rectangles with sizes $(i, i+1)$, must be placed in an almost square. (Cit. on p. 508).

[242] Clarissa Van Hoyweghen, David Edward Goldberg, and Bart Naudts. "From Twomax To The Ising Model: Easy And Hard Symmetrical Problems". In: *4th Genetic and Evolutionary Computation Conference (GECCO'2002)*. July 9–13, 2002, New York, NY, USA. Ed. by William B. Langdon, Erick Cantú-Paz, Keith E. Mathias, Rajkumar Roy, David Davis, Riccardo Poli, Karthik Balakrishnan, Vasant G. Honavar, Günter Rudolph, Joachim Wegener, Larry Bull, Mitchell A. Potter, Alan C. Schultz, Julian F. Miller, Edmund K. Burke, and Natasa Jonoska. Burlington, MA, USA/San Mateo, CA, USA: Morgan Kaufmann Publishers, 2002, pp. 626–633. ISBN: 978-1-55860-878-8 (cit. on p. 519).

References XXXIV

[243] Dieter Vandesande, Christoph Jabs, Jeremias Berg, Johannes Fichte, George Katsirelos, Ruben Martins, André Schidler, and Dominik Schreiber. *MaxSAT Evaluation 2026: Solvers, Benchmarks & Results*. v1. Geneva, Switzerland: zenodo (hosted by European Organization for Nuclear Research (CERN)), July 14, 2026. doi:10.5281/zenodo.21358014. See also https://maxsat-evaluations.github.io. (Cit. on p. 514).

[244] Kristian Verduin, Sarah Louise Thomson, and Daan van den Berg. "Too Constrained for Genetic Algorithms. Too Hard for Evolutionary Computing. The Traveling Tournament Problem". In: *15th International Joint Conference on Computational Intelligence (IJCCI'23)*. Nov. 13–15, 2023, Rome, Italy. Ed. by Niki van Stein, Francesco Marcelloni, H. K. Lam, Marie Cottrell, and Joaquim Filipe. Porto, Portugal: SciTePress: Science and Technology Publications, Lda, 2023, pp. 246–257. ISSN: 2184-3236. ISBN: 978-989-758-674-3. doi:10.5220/0012192100003595 (cit. on p. 519).

[245] Petr Vilím, Philippe Laborie, and Paul Shaw. "Failure-Directed Search for Constraint-Based Scheduling". In: *12th International Conference on the Integration of AI and OR Techniques in Constraint Programming (CPAIOR'2015)*. Ed. by Laurent Michel. Vol. 9075 of Theoretical Computer Science and General Issues (LNTCS), sub-series of Lecture Notes in Computer Science (LNCS). Barcelona, Spain: Springer, 2015, pp. 437–453. ISSN: 2512-2010. ISBN: 978-3-319-18007-6. doi:10.1007/978-3-319-18008-3_30. URL: https://www.researchgate.net/publication/280223626 (visited on 2026-09-30) (cit. on p. 512).

[246] Thomas Weise (汤卫思). *Databases*. Hefei, Anhui, China (中国安徽省合肥市): Hefei University (合肥大学), School of Artificial Intelligence and Big Data (人工智能与大数据学院), 2025. URL: https://thomasweise.github.io/databases (visited on 2025-01-05) (cit. on pp. 465, 510).

[247] Thomas Weise (汤卫思). *Global Optimization Algorithms – Theory and Application*. self-published, 2009. URL: https://www.researchgate.net/publication/200622167 (visited on 2025-07-25) (cit. on pp. 67–76, 376–381, 510, 514, 517).

[248] Thomas Weise (汤卫思). *jsspInstancesAndResults: Results, Data, and Instances of the Job Shop Scheduling Problem*. San Francisco, CA, USA: GitHub Inc, 2019–2020. URL: https://github.com/thomasWeise/jsspInstancesAndResults (visited on 2025-08-20) (cit. on pp. 188–230, 512).

[249] Thomas Weise (汤卫思). *Programming with C*. Hefei, Anhui, China (中国安徽省合肥市): Hefei University (合肥大学), School of Artificial Intelligence and Big Data (人工智能与大数据学院), 2026. URL: https://thomasweise.github.io/programmingWithC (visited on 2026-05-10) (cit. on p. 509).

References XXXV

[250] Thomas Weise (汤卫思). *Programming with Python*. Hefei, Anhui, China (中国安徽省合肥市): Hefei University (合肥大学), School of Artificial Intelligence and Big Data (人工智能与大数据学院), 2024–2026. URL: https://thomasweise.github.io/programmingWithPython (visited on 2025-01-05) (cit. on pp. 345, 464, 516).

[251] Thomas Weise (汤卫思), Raymond Chiong, Jörg Lässig, Ke Tang (唐珂), Shigeyoshi Tsutsui, Wenxiang Chen (陈文祥), Zbigniew "Zbyszek" Michalewicz, and Xin Yao (姚新). "Benchmarking Optimization Algorithms: An Open Source Framework for the Traveling Salesman Problem". *IEEE Computational Intelligence Magazine (CIM)* 9(3):40–52, Aug. 2014. Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE). ISSN: 1556-603X. doi:10.1109/MCI.2014.2326101. Applies a multitude of algorithms including SLSes, EAs, MAs, ACO, and EDAs to the TSPLib instances. (Cit. on pp. 10–34, 54–66, 101–106, 510, 518, 521).

[252] Thomas Weise (汤卫思), Xinlu Li (李新路), Yan Chen (陈岩), and Zhize Wu (吴志泽). "Solving Job Shop Scheduling Problems without using a Bias for Good Solutions". In: *23rd Genetic and Evolutionary Computation Conference (GECCO'2021), Companion Volume*. July 10–14, 2021, Lille, France. Ed. by Krzysztof Krawiec. New York, NY, USA: Association for Computing Machinery (ACM), 2021, pp. 1459–1466. ISBN: 978-1-4503-8351-6. doi:10.1145/3449726.3463124. Applies Frequency Fitness Assignment (FFA) to the JSSP. (Cit. on p. 522).

[253] Thomas Weise (汤卫思), Stefan Niemczyk, Hendrik Skubch, Roland Reichle, and Kurt Geihs. "A Tunable Model for Multi-Objective, Epistatic, Rugged, and Neutral Fitness Landscapes". In: *10th Genetic and Evolutionary Computation Conference (GECCO'2008)*. July 12–16, 2008, Atlanta, GA, USA. New York, NY, USA: Association for Computing Machinery (ACM), 2008, pp. 795–802. ISBN: 978-1-60558-130-9. doi:10.1145/1389095.1389252. The original paper introducing the W-Model (W-Model). (Cit. on p. 521).

[254] Thomas Weise (汤卫思), Li Niu (牛力), and Ke Tang (唐珂). "AOAB – Automated Optimization Algorithm Benchmarking". In: *12th Annual Conference Companion on Genetic and Evolutionary Computation (GECCO'2010)*. July 7–11, 2010, Portland, OR, USA. Ed. by Martin Pelikan and Jürgen Branke. New York, NY, USA: Association for Computing Machinery (ACM), 2010, pp. 1479–1486. ISBN: 978-1-4503-0073-5. doi:10.1145/1830761.1830763 (cit. on pp. 36–40, 77–84, 101–106, 510).

[255] Thomas Weise (汤卫思), Mingxu Wan (万明绪), Pu Wang (王璞), Ke Tang (唐珂), Alexandre Devert, and Xin Yao (姚新). "Frequency Fitness Assignment". *IEEE Transactions on Evolutionary Computation* 18(2):226–243, Apr. 2014. Los Alamitos, CA, USA: IEEE Computer Society. ISSN: 1089-778X. doi:10.1109/TEVC.2013.2251885. The article introducing FFA. (Cit. on p. 510).

References XXXVI

[256] Thomas Weise (汤卫思), Xiaofeng Wang (王晓峰), Qi Qi (齐琪), Bin Li (李斌), and Ke Tang (唐珂). "Automatically Discovering Clusters of Algorithm and Problem Instance Behaviors as well as their Causes from Experimental Data, Algorithm Setups, and Instance Features". *Applied Soft Computing (ASOC)* 73:366–382, Dec. 2018. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 1568-4946. doi:10.1016/J.ASOC.2018.08.030. Finds clusters of algorithm behavior on the W-Model. (Cit. on pp. 36–40, 101–106).

[257] Thomas Weise (汤卫思) and Zhize Wu (吴志泽). "Replicable Self-Documenting Experiments with Arbitrary Search Spaces and Algorithms". In: *25th Conference on Genetic and Evolutionary Computation (GECCO'2023), Companion Volume*. July 15–19, 2023, Lisbon, Portugal. Ed. by Sara Silva and Luís Paquete. New York, NY, USA: Association for Computing Machinery (ACM), 2023, pp. 1891–1899. ISBN: 979-8-4007-0120-7. doi:10.1145/3583133.3596306. Introduces the experiment execution and data collection facilities of moptipy. (Cit. on pp. 434–441, 466, 510, 514).

[258] Thomas Weise (汤卫思), Zhize Wu (吴志泽), Xinlu Li (李新路), and Yan Chen (陈岩). "Frequency Fitness Assignment: Making Optimization Algorithms Invariant under Bijective Transformations of the Objective Function Value". *IEEE Transactions on Evolutionary Computation* 25(2):307–319, Apr. 2021. Los Alamitos, CA, USA: IEEE Computer Society. ISSN: 1089-778X. doi:10.1109/TEVC.2020.3032090 (cit. on pp. 507, 510, 512, 513, 515, 518, 519).

[259] Thomas Weise (汤卫思), Zhize Wu (吴志泽), Xinlu Li (李新路), Yan Chen (陈岩), and Jörg Lässig. "Frequency Fitness Assignment: Optimization without Bias for Good Solutions can be Efficient". *IEEE Transactions on Evolutionary Computation* 27(4):980–992, 2023. Los Alamitos, CA, USA: IEEE Computer Society. ISSN: 1089-778X. doi:10.1109/TEVC.2022.3191698. Applies many FFA-based algorithm variants to problems such as OneMax, LeadingOnes, Linear Harmonic Function (LinHarm), TwoMax, Trap, Jump, Plateau, Ising Model, and N-Queens. (Cit. on pp. 507, 510, 513, 518, 519).

[260] Thomas Weise (汤卫思) and Zijun Wu (吴自军). "Difficult Features of Combinatorial Optimization Problems and the Tunable W-Model Benchmark Problem for Simulating Them". In: *20th Genetic and Evolutionary Computation Conference (GECCO'2018)*. July 15–19, 2018, Kyōto, Kyōto-fu, Japan. Ed. by Hernán E. Aguirre and Keiki Takadama. New York, NY, USA: Association for Computing Machinery (ACM), 2018. ISBN: 978-1-4503-5618-3. doi:10.1145/3205651.3208240 (cit. on p. 521).

[261] Thomas Weise (汤卫思), Yuezhong Wu (吴越钟), Raymond Chiong, Ke Tang (唐珂), and Jörg Lässig. "Global versus Local Search: The Impact of Population Sizes on Evolutionary Algorithm Performance". *Journal of Global Optimization* 66(3):511–534, Feb. 2016. London, England, UK: Springer Nature Limited. ISSN: 0925-5001. doi:10.1007/s10898-016-0417-5 (cit. on pp. 10–34, 518).

References XXXVII

[262] B. P. Welford. "Note on a Method for Calculating Corrected Sums of Squares and Products". *Technometrics* 4(3):419–420, Aug. 1962. London, England, UK: Taylor and Francis Ltd. ISSN: 0040-1706. doi:10.2307/1266577. This algorithm is often used to compute the sample variance [48,145] (cit. on p. 525).

[263] L. Darrell Whitley. "The GENITOR Algorithm and Selection Pressure: Why Rank-Based Allocation of Reproductive Trials is Best". In: *3rd International Conference on Genetic Algorithms (ICGA'1989)*. June 1989, Fairfax, VA, USA: George Mason University. Ed. by J. David Schaffer. Burlington, MA, USA/San Mateo, CA, USA: Morgan Kaufmann Publishers, 1989, pp. 116–123. ISBN: 978-1-55860-066-9. URL: https://www.researchgate.net/publication/2527551 (visited on 2025-08-08) (cit. on pp. 513, 517).

[264] Frank Wilcoxon. "Individual Comparisons by Ranking Methods". *Biometrics Bulletin* 1(6):80–83, Dec. 1945. Washington, D.C., USA: International Biometric Society. ISSN: 0099-4987 (cit. on pp. 376–381).

[265] James M. Wilson. "Gantt Charts: A Centenary Appreciation". *European Journal of Operational Research* 149(2):430–437, Sept. 2003. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0377-2217. doi:10.1016/S0377-2217(02)00769-5 (cit. on p. 510).

[266] L. Wong and Lai-Soon Lee. "Heuristic Placement Routines for Two-Dimensional Bin Packing Problem". 5(4):334–341, Dec. 2009. doi:/10.3844/jmssp.2009.334.341. URL: https://www.thescipub.com/pdf/jmssp.2009.334.341 (visited on 2026-10-02). A heuristic for the 2BP|R|F and the 2BP|O|F (cit. on p. 472).

[267] CAO Xiang (曹翔), Zhize Wu (吴志泽), Daan van den Berg, and Thomas Weise (汤卫思). "Randomized Local Search vs. NSGA-II vs. Frequency Fitness Assignment on The Traveling Tournament Problem". In: *16th International Joint Conference on Computational Intelligence (IJCCI'24)*. Nov. 20–22, 2024, Porto, Portugal. Ed. by Francesco Marcelloni, Kurosh Madani, Niki van Stein, and Joaquim Filipe. Porto, Portugal: SciTePress: Science and Technology Publications, Lda, 2024, pp. 38–49. ISSN: 2184-3236. ISBN: 978-989-758-721-4. doi:10.5220/0012891500003837 (cit. on pp. 514, 519, 522).

[268] Takeshi Yamada and Ryohei Nakano. "A Genetic Algorithm Applicable to Large-Scale Job-Shop Instances". In: *Parallel Problem Solving from Nature 2 (PPSN-II)*. Sept. 28–30, 1992, Brussels, Belgium. Ed. by Reinhard Männer and Bernard Manderick. Amsterdam, The Netherlands: Elsevier B.V., 1992, pp. 281–290. URL: https://www.researchgate.net/publication/220701684 (visited on 2026-09-30) (cit. on p. 512).

References XXXVIII

[269] Kinza Yasar and Craig S. Mullins. *Definition: Database Management System (DBMS)*. Newton, MA, USA: TechTarget, Inc., June 2024. URL: https://www.techtarget.com/searchdatamanagement/definition/database-management-system (visited on 2025-01-11) (cit. on p. 510).

[270] François Yergeau. *UTF-8, A Transformation Format of ISO 10646*. Request for Comments (RFC) 3629. Wilmington, DE, USA: Internet Engineering Task Force (IETF), Nov. 2003. URL: https://www.ietf.org/rfc/rfc3629.txt (visited on 2025-02-05). See Unicode and [128] (cit. on p. 520).

[271] Jiazheng Zeng (曾嘉政), Thomas Weise (汤卫思), Zhize Wu (吴志泽), and Markus Wagner. "When Frequency Fitness Assignment Fails: Trapped States in Frequency-Guided Local Search". In: *28th Genetic and Evolutionary Computation Conference (GECCO'2026)*. July 13–17, 2026, San José, Costa Rica. Ed. by Leonardo Trujillo and Ting Hu. New York, NY, USA: Association for Computing Machinery (ACM), 2026, pp. 233–242. ISBN: 979-8-4007-2487-9. doi:10.1145/3795095.3805098. Explores $(1+1)$ evolutionary algorithm ($(1+1)$ EA), $(1+1)$ EA with FFA ($(1+1)$ FEA), RLS, and FRLS as well as hybrid algorithms on a variety of discrete benchmarks such as OneMax, LeadingOnes, LinHarm, Binary Integer (BinInt), TwoMax, Trap, Jump, Plateau, Ising Model, N-Queens and also applies them to the search for Low-Autocorrelation Binary Sequences (LABS). (Cit. on pp. 507, 510, 512, 513, 515, 517–519).

[272] Chaoyong Zhang, Xinyu Shao, Yunqing Rao, and Haobo Qiu. "Some New Results on Tabu Search Algorithm Applied to the Job-Shop Scheduling Problem". In: *Tabu Search*. Ed. by Wassim Jaziri. London, England, UK: IntechOpen Limited, Aug. 1, 2008. Chap. 8. ISBN: 978-3-902613-34-9. doi:10.5772/5593. URL: https://www.intechopen.com/chapters/4596 (visited on 2026-09-30) (cit. on pp. 512, 518).

[273] Rui Zhao (赵睿), Tianyu Liang (梁天宇), Zhize Wu (吴志泽), Daan van den Berg, Matthias Thürer, and Thomas Weise (汤卫思). "Randomized Local Search on the 2D Rectangular Bin Packing Problem with Item Rotation". In: *26th Genetic and Evolutionary Computation Conference (GECCO'2024)*. July 14–18, 2024, Melbourne, VIC, Australia. Ed. by Xiaodong Li and Julia Handl. New York, NY, USA: Association for Computing Machinery (ACM), 2024, pp. 235–238. ISBN: 979-8-4007-0494-9. doi:10.1145/3638530.3654139 (cit. on pp. 508, 511, 514).

References XXXIX

[274] Rui Zhao (赵睿), Zhize Wu (吴志泽), Daan van den Berg, Matthias Thürer, Tianyu Liang (梁天宇), Ming Tan (檀明), and Thomas Weise (汤卫思). "Randomized Local Search for Two-Dimensional Bin Packing and a Negative Result for Frequency Fitness Assignment". In: *16th International Joint Conference on Computational Intelligence (IJCCI'24)*. Nov. 20–22, 2024, Porto, Portugal. Ed. by Francesco Marcelloni, Kurosh Madani, Niki van Stein, and Joaquim Filipe. Porto, Portugal: SciTePress: Science and Technology Publications, Lda, 2024, pp. 15–26. ISSN: 2184-3236. ISBN: 978-989-758-721-4. doi:10.5220/0012888500003837 (cit. on pp. 508, 509, 511, 514).

Glossary I



$(1+1)$ EA The $(1+1)$ EA is a local search algorithm that retains the best solution x_c discovered so far during the search [45,78]. In each step, it applies a unary search operator to this best-so-far solution x_c and derives a new solution x_n. If the new solution x_n is *better or equally good* when compared with x_c, i.e., not worse, then it replaces it, i.e., is stored as the new x_c. If the search space are bit strings of length n, then the $(1+1)$ EA uses a unary search operator that flips each bit independently with probability m/n, where usually $m = 1$. This operator is the main difference to randomized local search (RLS). The $(1+1)$ EA is a special case of the $(\mu+\lambda)$ evolutionary algorithm ($(\mu+\lambda)$ EA) where $\mu = \lambda = 1$.

$(1+1)$ FEA The $(1+1)$ EA with FFA plugged in [258,259,271]. On the discrete bit-string domains, this algorithm has a very specific structure: Its unary search operator flips each bit with probability $1/n$, where n is the dimension of the search space, distinguishing it from the unary operator in FRLS which flips exactly one bit. On other domains, such as permutations, the term is used synonymously with FRLS.

$(\mu+\lambda)$ EA The $(\mu+\lambda)$ EA is an evolutionary algorithm (EA) where, in each generation, λ offspring solutions are generated from the current population of μ parent solutions. The offspring and parent populations are merged, yielding $\mu+\lambda$ solutions, from which then the best μ solutions are ratained to form the parent population of the next generation. If the search space is the bit strings of length n, then this algorithm usually applies a mutation operator flipping each bit independently with probability $1/n$.

BinInt The *Binary Integer Problem* is a benchmark problem for bit-string based black-box search [235,271]. It is very similar to OneMax: It tries to maximize the number of 1 bits in the bit string of a chosen dimension n. Different from OneMax, where each bit has the same weight in the objective function, the bits of BinInt have a exponentially increasing weight: The bit string is treated as a binary-encoded integer whose value should be maximized. The objective function $f(x)$ of the minimization variant of this problem can be defined as $f(x) = 2^n - 1 - \sum_{i=1}^{n} 2^{i-1} \cdot x_i$. The problem is unimodal, i.e., has only a single global optimum and no local optimum. It is fully separable, i.e., we can optimize each variable separately.

LinHarm The Linear Harmonic Function is a benchmark problem for bit-string based black-box search [72,78,271]. It is very similar to OneMax: It tries to maximize the number of 1 bits in the bit string of a chosen dimension n. Different from OneMax, where each bit has the same weight in the objective function, the bits of LinHarm have a linear increasing weight. The objective function $f(x)$ of the minimization variant of this problem can be defined as $f(x) = \frac{1}{2}n(n+1) - \sum_{i=1}^{n} i \cdot x_i$. The problem is unimodal, i.e., has only a single global optimum and no local optimum. It is fully separable, i.e., we can optimize each variable separately. The $(1+1)$ EA can solve this problem with an expected runtime in $\Theta(n \ln n)$ [72,78].

Glossary II

2BP In the two-dimensional Bin Packing Problem (BP), the goal is to pack rectangles into rectangular bins without overlap and such that their edges are parallel to the edges of the bins [129,162,164]. The 2BP is strongly $\mathcal{NP}$-hard [164]. The variants of this problem include the 2BP|O|F, 2BP|R|F, 2BP|O|G, and 2BP|R|G [163,164]. Different objective function for the 2BP|O|F and 2BP|R|F are presented in [273]. 2BPs have many practical applications, including cutting (wood, glas) [164], packing (transportation, warehouse) [164], and newspaper page layout [227].

2BP|O|F In the oriented 2BP with free cuts, the rectangles to pack have fixed orientations, i.e., cannot be rotated, but can otherwise be arranged freely [163,164,175,274]. Permutation-based search spaces can be used for this problem by using the Improved Bottom Left (IBL) encoding. The SOTA for this problem comprises [27,33,54,106,198,274].

2BP|O|G In the oriented 2BP with guillotine cuts, the rectangles to pack have fixed orientations, i.e., cannot be rotated, and must be arranged such that they can be extracted by edge-to-edge cuts [163,164].

2BP|R|F In this variant of the 2BP with free cuts, the rectangles to pack can be rotated by 90° if need be and can freely be arranged [24,163,164,273,274]. Besides packing and cutting, this problem also occurs in scheduling and build formation in additive manufacturing [156,204]. The almost-squares-in-almost-squares (Asqas) problem [241] is also a 2BP|R|F. Search spaces based on signed permutations can be used for this problem by using the IBL encoding. The SOTA for this problem comprises [27,54,106,273,274].

2BP|R|G In this variant of the 2BP with guillotine cuts, the rectangles to pack can be rotated by 90° if need be, but must be arranged such that they can be extracted by edge-to-edge cuts [163,164].

ACO *Ant Colony Optimization* is a nature-inspired optimization method for combinatorial problems where solutions are generated by "ants" that move from node to node in a graph choosing edges based on (1) the simulated pheromone on the edges and (2) a per-edge heuristic value [74–76]. If an ant produced a good solution, "pheromone" is distributed over the edges it visited, making it more likely to be re-visited by other ants.

Asqas An "almost-square" be a rectangle with size $i \times (i+1)$ with $i \in \mathbb{N}_{\mathbf{1}}$. The *almost-squares-in-almost-squares* problem [241] is a special case of the 2BP|R|F. Here, almost-square rectangles of sizes 1×2, 2×3, …, $n \times (n+1)$ must be placed into an almost-square bin without waste. This requires that there exists a natural number $p \in \mathbb{N}_{\mathbf{1}}$ such that $p(p+1) = \sum_{i=\mathbf{1}}^{n} i(i+1)$ [241]. There are exacty five Asqas instances [241], namely with bin sizes 1×2 for $n = 1$, $p = 1$, 5×4 for $n = 3$, $p = 4$, 15×16 for $n = 8$, $p = 15$, 55×56 for $n = 20$, $p = 55$, and 119×120 for $n = 34$, $p = 119$.

Glossary III



ATSP The general case for TSPs are *asymmetric Traveling Salesperson Problems*, where the $n \times n$ distance matrix A does not need to be symmetric. In other words, it is possible that $\exists a, b \in 1..n$ with $A[a, b] \neq A[b, a]$. Notice that while such index tuple *may* exist, this is not always or necessarily the case. In this case, one may consider the STSP as a special case of the ATSP, while ATSPs may be considered as synonymous to TSPs.

BKS The *Best Known Solution* for an instance of an optimization problem is the best solution (measured based on the objective values) that has ever been reported in literature. BKSes are not necessarily globally optimal, as in many instances of $\mathcal{NP}$-hard problems, the true optima are unknown.

BP The *Bin Packing Problem* [81,216] is one of the classical problems from Operations Research (OR). The goal is to pack a set of objects into containers, using as few containers as possible. Different flavors of BPs can be distinguished, e.g., by the number of dimensions [65,162] that the objects and containers have (see, e.g., the 2BP), and/or whether the objects can be rotated [274], as well as other constraints and limitations. Both the one- and two-dimensional BP are strongly $\mathcal{NP}$-hard [164].

BRKGA The *Biased Random Key Genetic Algorithm* is a EA with a continuous search space $\mathbb{X} \subset [0, 1]^n$ and a decoding function γ translating this space to something else and biased, uniform, and elitist mating strategies [108,167]. The *bias* comes from the fact that one of the parents used for the binary operation is biased to be of higher fitness than the other parent [108]. Often, γ is the Random Keys encoding [17,18] procedure translating $\mathbb{X}$ to permutations of the first n natural numbers [107]. Sometimes, it is actually a MA further refining the translation result with a local search (LS) [107]. Sometimes the permutations are then further translated to more complex data structures, such as Gantt charts for the JSSP [107], or packing plans for 2BP 3D packing [106].

C is a programming language, which is very successful in system programming situations [69,208,249].

Concorde is an exact solver for the TSP[6,56,57], which could even solve the largest instance of TSPLib (with 85 900 cities) to optimality.

CSV *Comma-Separated Values* is a very common and simple text format for exchanging tabular or matrix data [218]. Each row in the text file represents one row in the table or matrix. The elements in the row are separated by a fixed delimiter, usually a comma (","), sometimes a semicolon (";"). Python offers some out-of-the-box CSV support in the `csv` module [60].

Glossary IV



DB A *database* is an organized collection of structured information or data, typically stored electronically in a computer system. Databases are discussed in our book *Databases* [246].

DBMS A *database management system* is the software layer located between the user or application and the database (DB). The DBMS allows the user/application to create, read, write, update, delete, and otherwise manipulate the data in the DB [269].

EA An *evolutionary algorithm* is a metaheuristic optimization method that maintains a population of candidate solutions, which undergo selection (where better solutions are chosen with higher probability) and reproduction (where mutation and recombination create a new candidate solution from one or two existing ones, repectively) [10,247].

FE *Objective function evaluations* are an implementation-independent measure of runtime for optimization algorithms [11,251,254,257]. 1 FE equals to one evaluated candidate solution during the optimization process.

FFA *Frequency Fitness Assignment* (中文: 頻率适应度分配) is a algorithm plugin for optimization methods applied to discrete or combinatorial problems with not-too-many different possible objective values [255,258,259]. It replaces the objective values in all comparisons with their absolute encounter frequency so far during the search. FFA has successfully been applied to the MaxSAT [258,259], the TSP [157,158], and the QAP [51,237], amongst others. Its properties have been intensely researched on discrete benchmark functions such as OneMax, LeadingOnes, Trap, TwoMax, Jump, and Plateau [258,259,271]. The most common realizations of FFA so far are the $(1+1)$ FEA and FRLS.

FSSP The *Flow Shop Scheduling Problem* is a variant of a JSSP where all products go through the same sequence of machines [110,138].

Gantt chart Gantt charts are two-dimensional diagrams illustrating the steps of a process [142,265]. Their horizontal axis represents the time, the vertical axis represents work packages, machines, or workers. In the diagram area, bars located at certain vertical locations indicate that the work package, machine, or worker is processed or works at the time range covered by the bar as projected to the horizontal axis.

Git is a distributed Version Control System (VCS) which allows multiple users to work on the same code while preserving the history of the code changes [223,240]. Learn more at https://git-scm.com.

GitHub is a website where software projects can be hosted and managed via the Git VCS [200,240]. Learn more at https://github.com.

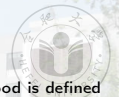
Glossary V

GRASP A *Greedy Randomized Adaptive Search Procedure* is a metaheuristic where each cycle consists of two main components [86,88,89]: First, a starting point is generated using a heuristic method (as opposed to being randomly sampled). Then, a LS is applied to refine the solution. Then, another cycle starts and proceeds in the same way. The overall best solution is retained as result. Many examples of applications of GRASPs are listed in [89]. GRASP in combination with VND has been applied to the 2BP|O|F in [198].

IBL The *improved bottom left* heuristic is a one-shot algorithm constructing a solution for 2BP [161]. For the 2BP|O|F problem variant, the input of the IBL algorithm is a permutation of the first n natural numbers, each number identifying one rectangle to be packed. For the 2BP|R|F problem flavor, the permutation elements are signed, where a negative sign indicates a rotation by 90° [161]. In [273,274] it is adapted to a multi-bin situation as follows: The algorithm iterates over the (potentially signed) permutation x and places the rectangles into the packing plan in the prescribed order. It first places the rectangle outside on top of the current bin b, with its bottom-right corner onto the top-right corner of the current bin b. It then moves the rectangle downwards and leftwards as far as possible, prioritizing the downward movement whenever possible. If the rectangle cannot be moved any further, it checks if it is completely contained in the bin b. If yes, it can remain there. Otherwise, a new bin is opened ($b \leftarrow b + 1$) and the rectangle is placed at its bottom-left corner. Once a new bin is opened, this bin is used for all further insertions. IBL is suitable as a decoding function γ inside an EA [161]. Historically, it improves upon the BL algorithm [133]: That algorithm cannot represent all possible optimal packings using a permutation, but the IBL can [161].

ILS An *iterated local search* is a variant of the LS which applies cycles of local search and perturbation [228]. First, an initial solution is generated and refined by the local search. Then the cycle begins: A perturbation, i.e., a large search move that likely destroys a good part of the solution, is applied. The resulting new solution is refined by local search. Then, an acceptance criterion decides whether we should use this or the previous solution to continue in the next cycle. ILS has successfully been applied to the QAP.

Glossary VI



Ising Model The Ising models are benchmark problems for bit-string based black-box search [72,90,91,271]. Here, a neighborhood is defined around each bit. The bit needs to have the same value as all bits in the neighborhood structure T. Thus, we try to achieve that all bits have the same value. The objective function $f(x)$ of the minimization variant of this problem family can be defined as $f(x,T) = \sum_{(i,j)\in T} |x_i - x_j|$, where T contains the index pairs of neighboring bits and is defined differently for the Ising model instances Ising 1D and Ising 2D. The problems are highly epistatic, as all variables interact with some other variables.

Ising 1D The 1D is a benchmark problem for bit-string based black-box search [72,91,271]. The bits are arranged in a one-dimensional ring and we try to minimize the number of bits that have different values than their neighbors. Hence, the neighborhood structure T is defined as $T_{\mathbf{1}D} = \{(i+1, (i+1 \bmod n)+1) \mid i \in [0..n-1]\}$. The problems are highly epistatic, as all variables interact with some other variables. The $(1+1)$ EA has an expected runtime in $\mathcal{O}(n^{\mathbf{3}})$ on 1-Dimensional Ising Model (Ising 1D) [91].

Ising 2D The 2D is a benchmark problem for bit-string based black-box search [72,90,271]. The bits are arranged in a two-dimensional torus and we try to minimize the number of bits that have different values than their neighbors. Hence, the neighborhood structure $T = T_{\mathbf{2}D}$ is defined as the set of all $\{(\alpha+\beta N, \gamma+\delta N)\}$ pairs with $\alpha,\beta,\gamma,\delta \in 0..N-1$ and either ()$\gamma = (\alpha+1) \bmod N$ and $\delta = \beta$) or ($\gamma = \alpha$ and $\delta = (\beta+1) \bmod N$) with $n = N^{\mathbf{2}}$. The problems are highly epistatic, as all variables interact with some other variables. On a similar class of problems, the $(1+1)$ EA has an exponential expected runtime [90].

Java is another very successful programming language, with roots in the C family of languages [32,168].

JSSP The *Job Shop Scheduling Problem* [31,53,110,152] is one of the most prominent and well-studied scheduling tasks. In a JSSP instance, there are m machines and k jobs [229]. Each job must be processed once by each machine in a job-specific sequence and has a job-specific processing time on each machine. The goal is to find an assignment of jobs to machines that results in an overall shortest makespan, i.e., the schedule (Gantt chart) which can complete all the jobs in the shortest time. The JSSP is $\mathcal{NP}$-complete [49,151,152]. The 8 most common sets of JSSP benchmark instances are *abz* [2], *dmu* [66], *ft* [92], *la* [154], *orb* [7], *swv* [226], *ta* [229], and *yn* [268].Their data can be found (sometimes partially) in several repositories, such as the OR-Library [19,20], [220], and [248], the latter of which also contains many results from existing works. Encodings based on permutations with repetition are proposed in [28,29,100,219] and used in [258]. The SOTA of the JSSP includes [107,120,197,199,221,245,272].

Glossary VII

Jump The Jump problem is a benchmark problem for bit-string based black-box search [78,97,271]. Like the OneMax, it tries to maximize the number of 1 bits in a bit string of chosen length n. However, the solutions representing the last ω bits around the optimum to flip are deceptive, meaning that they are actually worse and getting worse than surrounding solutions. The objective function $f(x)$ of the minimization variant is $n - \|x\|_1$ if $\|x\|_1 = n$ or $\|x\|_1 \leq n - \omega$ hold, and $\omega + \|x\|_1$ otherwise, where $\|x\|_1 = \sum_{i=1}^{n} x_i$. This means that, the larger ω, the larger the deceptive basin around the optimum that an algorithm needs to cross. An $(1+1)$ EA will converge to the resulting local optimum and thus have an expected runtime of $\Theta(n^\omega + n \ln n)$ [78].In experiments, the $(1+1)$ FEA achieves polynomial runtime on Jump, seemingly in $\mathcal{O}(n^2 \ln n)$ [258,259,271].

LABS *Low-Autocorrelation Binary Sequences* are bit strings of length n whose autocorrelation is as low as possible [37–39,67,68,98,176,179,187,194,206]. This means that sub-sequences of any length $1 \leq k < n$ should ideally never repeat. The objective function $f : \{0,1\}^n \mapsto \mathbb{N}_0$ represents the energy of a bit string x and is subject to minimization. We can define it as $f(x) = \sum_{k=1}^{n-1} \left(\sum_{i=0}^{n-1-k} s_i \cdot s_{i+k} \right)^2$ with $s_j = 2x_j - 1$. Such sequences are needed for communication engineering [38,39,187], cryptography[67,187], as well as modulation pulses in radar and sonar ranging[194]. The LABS is $\mathcal{NP}$-hard [239]. The SOTA on LABS contains [37,39,67,68,187].

LeadingOnes The LeadingOnes problem is a benchmark problem for bit-string based black-box search [213,263,271]. It tries to maximize the number of *leading* 1 bits in the bit string of a chosen dimension n. The objective function $f(x)$ of the minimization variant of this problem can be defined as $f(x) = n - \sum_{i=1}^{n} \prod_{j=1}^{i} x_j$. Different from OneMax, the contribution to the objective value of the bit at index i thus depends on the state of all the bits *before* this index. The problem is unimodal, i.e., has only a single global optimum and no local optimum. Different from OneMax, it is not separable, i.e., we can optimize each variable separately. The expected runtime of $(1+1)$ EA on LeadingOnes is in $\Theta(n^2)$ [78]. The average runtime $(1+1)$ FEA on LeadingOnes seems to be slower by factor proportional to n, i.e., seems to be in $\mathcal{O}(n^3)$ [258].

LK The *Lin-Kernighan* (LK) algorithm [160] is a SLS method for the TSP. The LK algorithm performs iterative k-opt moves, i.e., in each step, it constructs the two sets X and Y of k edges such that removing the edges in X from the current tour T and then adding the edges from Y to T results in a better tour.The LK algorithm is one of the most classical approaches to the TSP.

LKH The *Lin-Kernighan-Helsgaun* algorithm [57,118,119] is a SLS method for the TSP based on the Lin-Kernighan (LK) algorithm [160]. LKH-1 was proposed in [119] and LKH-2 in [118]. It holds the record in the *World TSP* [58] as of 2025.

Glossary VIII



LS A *local search* is an algorithm where a single "current" candidate solution is maintained and new solutions are selected from its neighborhood in each step. Most LSes are randomized, i.e., stochastic local search (SLS).

MA A *Memetic Algorithm* is an EA where each new solution is refined by a LS before it enters the population [247]. As LS, we can use, e.g., RLS, but TS [52,95] is maybe a better choise. Some SOTA algorithms in some domains may be classified as MAs, e.g., Lin-Kernighan Helsgaun (LKH) basically uses a population that is refined via a local search. MAs also worked well on the JSSP [52,120] and the TSP [95].

MaxSAT The goal of satisfiability problems is to find an assignment for n Boolean variables that make a given Boolean formula $F : \{0,1\}^n \mapsto \{0,1\}$ become true. In the *Maximum Satisfiability* problem (MaxSAT) [123], F is given in conjunctive normal form, i.e., the variables appear either directly or negated in m "or" clauses, which are all combined into one "and." The objective function $f(x)$, subject to minimization, computes the number of clauses which are false under the variable setting x. If $f(x) = 0$, then all clauses of F are true, which solves the problem. The MaxSAT problem is $\mathcal{NP}$-complete [55] and exhibits low epistasis but deceptiveness [210], making it hard for local searches. Typical benchmark instances can be found in SATLIB, which contains smaller-scale instances, and the up-to-date MaxSAT Evaluation (MSE), providing SOTA benchmarks, algorithms, and results.

metaheuristic A *metaheuristic* is an optimization algorithm which performs a cycle of trial and error during which candidate solutions are sampled from the solution space $\mathbb{Y}$ and evaluated using an objective function $f : \mathbb{Y} \mapsto \mathbb{R}$ [103,247]. The results of the objective function are used to decide which solution is promising. The neighborhoods around promising solutions *or* similarly-structured solutions are then sampled in the next cycle. Prominent metaheuristics are SLSes, SA, TS, and EAs.

moptipy is the *Metaheuristic Optimization in Python* library [257]. It has been used in several different research works, including [51,157–159,237,267,273,274]. Algorithms implemented by moptipy include EAs, $(1+1)$ EA, RLS, SA, and the $(1+1)$ FEA, amongst others. Learn more at https://thomasweise.github.io/moptipy and https://thomasweise.github.io/moptipyapps.

MSE The *MaxSAT Evaluation* is a regularly updated set of benchmark instances for the MaxSAT [130]. It also includes solver software and benchmark results [243].

Glossary IX

N-Queens The N-Queens Problem is a benchmark problem for bit-string based black-box search [72,271]. The goal is to arrange N queens on an $N \times N$ chessboard so that none can attack the other. The search space has thus dimension $n = N^2$, with each bit standing for a field and encoding whether or not a queen is placed there. The objective function $f(x)$ of the minimization variant is $N - \left(\sum_{i=1}^{n} x_i\right) - N \sum_{\xi} \max\left\{0, Q_\xi(x) - 1\right\}$ where $Q_\xi(x)$ is the number of queens in the horizontal, vertical, or diagonale ξ. The problem is quite epistatic.

OneMax OneMax is the simplest possible benchmark problem for bit-string based black-box search [79,188,271]. The goal is to maximize the number of 1 bits in the bit string of a chosen dimension n. The objective function $f(x)$ of the minimization variant of this problem can be defined as $f(x) = n - \sum_{i=1}^{n} x_i$. The problem is unimodal, i.e., has only a single global optimum and no local optimum. It is fully separable, i.e., we can optimize each variable separately. The expected runtime of $(1+1)$ EA on OneMax is in $\Theta(n \ln n)$ [70,188]. The average runtime $(1+1)$ FEA on OneMax seems to be slower by factor proportional to n, i.e., seems to be in $\mathcal{O}(n^2 \ln n)$ [258].

OR Operations Research (or Operational Research) is the application of sciences such as mathematics and computer science to the management and organization of systems, organizations, enterprises, factories, or projects. It encompasses the development and application of problem-solving methods and techniques (such as mathematical optimization, simulation, queueing theory and other stochastic models) with the goal to improve decision-making and efficiency [63].

OR-Library is a library of benchmark instances for various OR problems [19,20] located at https://people.brunel.ac.uk/~mastjjb/jeb/info.html. It contains instances of the JSSP, 1D- and 2D-Packing, Knapsack, periodic TSPs, amongst others.

Plateau The Plateau problem is a benchmark problem for bit-string based black-box search [5,271]. It is structured exactly like Jump, but instead of being deceptive, the last ω bits to flip around the optimum are neutral. The objective function $f(x)$ of the minimization variant is $n - \|x\|_1$ if $\|x\|_1 = n$ or $\|x\|_1 \leq n - \omega$ hold, and ω otherwise, where $\|x\|_1 = \sum_{i=1}^{n} x_i$. This means that, the larger ω, the larger the neutral plane around the optimum that an algorithm needs to cross. An $(1+1)$ EA has an expected runtime of $\Theta(n^\omega)$ on plateaus [5]. The average runtime of the $(1+1)$ FEA on Plateaus is slower by a factor seemingly proportional in n.

Glossary X



PostgreSQL An open source object-relational DBMS [87,191,205,234]. See https://postgresql.org for more information.

Python The Python programming language [126,155,170,250], i.e., what you will learn about in our book [250]. Learn more at https://python.org.

QAP The *Quadratic Assignment Problem* is an optimization problem where the goal is to assign a set of n facilities to a set of n locations [41,51,147,165]. Such an assignment can be represented as a permutation x of the first n natural numbers, where x_i specifies the location where facility i should be placed. For each QAP instance, a distance matrix A is given, where $A[a, b]$ specifies the distance from location a to location b, as well as a flow matrix B, where $B[i, j]$ is the amount of material flowing (e.g., daily) from facility i to facility j. The objective function $f(x)$ then rates a permutation x as $f(x) = \sum_{i=\mathbf{1}}^{n} \sum_{j=\mathbf{1}}^{n} A[x_i, x_j] * B[i, j]$. The QAP is $\mathcal{NP}$-complete [77,215] and even among this problem class, is considered as particularly hard for metaheuristics [22,41,165,228]. The QAP has applications such as building layout [61,84,148], keyboard layout [43], circuit design [85], wiring [225], and scheduling [224]. Typical benchmark instances can be found in QAPLIB. SOTA algorithms for the QAP include [22,95,177,184,231,232].

QAPLIB is a library of 136 benchmark instances for the Quadratic Assignment Problem (QAP) [40,42], 28 of which have not yet been solved to optimality [4] (at least by 2023).

Random Keys encoding is a decoding procedure $\gamma : \mathbb{X} \mapsto \mathbb{Y}$ that maps a continuous search space $\mathbb{X} \subset [0, 1]^n$ which is a subset of the n-dimensional real numbers to the space $\mathbb{Y}$ of permutations of the first n natural numbers [17,18]. For example, on $n = 5$, the vector $x = (0.49, 0.91, 0.33, 0.75, 0.51)$ is translated to the permutation $y = (3, 1, 5, 4, 2)$, because $x_{\mathbf{3}} = 0.33$ has the smallest value, putting 3 at the beginning, $x_{\mathbf{1}} = 0.49$ the second-smallest, placing 1 second, ..., and $x_{\mathbf{2}} = 0.91$ being the largest value meaning that the last number in y will be 2 [18]. The advantage of the encoding is that we can search in $\mathbb{X}$ applying arbitrary mathematical operators, crossover, and mutation procedures, but will always get correct permutations. The disadvantage is that $|\mathbb{X}| \gg |\mathbb{Y}|$ and that there actually is no meaning underpinning moves in $\mathbb{X}$ in relation to their projected result in $\mathbb{Y}$.

Glossary XI



RLS Randomized local search retains the best solution x_c discovered so far during the search and, in each step, it applies a unary search operator to this best-so-far solution x_c and derives a new solution x_n. If the new solution x_n is *better or equally good* when compared with x_c, i.e., not worse, then it replaces it, i.e., is stored as the new x_c. If the search space are bit strings of length n, then RLS uses a unary search operator that flips exactly one bit. This operator is the main difference to $(1 + 1)$ EA.

FRLS The RLS with FFA plugged in [271]. On the discrete bit-string domains, this algorithm has a very specific structure: Its unary search operator flips exactly one bit, distinguishing it from the unary operator in the $(1 + 1)$ FEA, which flips each bit with probability $1/n$. On other domains, such as permutations, we use the term synonymously with $(1 + 1)$ FEA.

RTS *Reactive Tabu Search* is a variant of TS where the length of the tabu list is adapted to the problem and to the current evolution of the search, and an escape strategy for diversifying the search is used when that is not sufficient [12–15]. It has successfully been applied to a wide range of problems, from the QAP [13,14], to Knapsack problems [12,14], the N-K landscape [12] to the training of neural networks [15].

SA *Simulated Annealing* is a SLS that sometimes accepts a worse solution [40,132,141,203,247]. The probability to do so decreases over time and with the difference in objective values, i.e.,is the lower the worse the new solution is. SA performed well on a variety of problems, including the QAP [182,238] and the JSSP [197].

SATLIB is a library of benchmark instances for the Maximum Satisfiability (MaxSAT) available at https://www.cs.ubc.ca/~hoos/SATLIB/index-ubc.html [124,125].

SGA The Standard Genetic Algorithm [10,62,105,178,185,247] was the first EA. It maintains a population of solutions and applies mutation and crossover to generate offspring solutions. It uses fitness proportionate selection to choose which solutions should "survive" into the next generation, which today is considered a very bad design choice, see, e.g., [263].

SLS *Stochastic local search* is a local search (LS), one of the simplest metaheuristic search paradigms [123]: A single "current" candidate solution is maintained and new solutions are *stochastically* sampled from its neighborhood, i.e., in a way that involves random numbers. If these solutions are deemed interesting based on their objective value, they replace the "current". The cycle continues. Basic examples are RLS and $(1 + 1)$ EA. More complex examples that add mechanisms to escape local optima on top of the SLS paradigm are TS and SA.

SOTA State-of-the-Art, i.e., the best-performing methods for a field or problem type

Glossary XII

STSP A *symmetric Traveling Salesperson Problem* is a special case of the TSP, where the $n \times n$ distance matrix A is symmetric, i.e., $A[a, b] = A[b, a] \forall a, b \in 1..n$.

Trap The Trap problem is a benchmark problem for bit-string based black-box search [78,189,271]. It is structured exactly like OneMax, however the worst and the best solution are swapped. This makes the problem highly deceptive, as following the "gradient" in the search space leads the search away from the global optimum. For bit strings of length n, the objective function $f(x)$ of the minimization variant is 0 if $\|x\|_1 = 0$ and $n - \|x\|_1 + 1$ otherwise, where $\|x\|_1 = \sum_{i=1}^{n} x_i$. An $(1 + 1)$ EA will converge to the resulting local optimum and thus have an expected runtime of $\Theta(n^n)$ [78]. In experiments, the $(1 + 1)$ FEA achieves polynomial runtime on Trap, seemingly in $\mathcal{O}(n^2 \ln n)$ [258,259,271].

TS *Tabu Search* is one of the most successful SLSes [101,102,104,134]. In order to avoid returning to previous solutions, it declares certain solutions or search moves as "tabu." For example, if a bit in a discrete problem has been flipped by the search operator, it may be forbidden to flip the bit again for some time, thus preventing the algorithm from getting trapped in a cycle. This rule may be overridden if a new best-so-far solution is discovered that outperforms all previous solutions, in which case it would be accepted (this is the so-called aspiration criterion). TS has performed well on the QAP [13,14,181,184,230,231] and on the JSSP [190,199,202,221,272].

TSP In an instance of the *Traveling Salesperson Problem*, also known as *Traveling Salesman Problem*, a set of n cities or locations as well as the distances between them are given [6,113,153,157,251,261]. The goal is to find the shortest round-trip tour that starts at one city, visits all the other cities one time each, and returns to the origin. The TSP is one of the most well-known $\mathcal{NP}$-hard combinatorial optimization problems [99,113,149]. The TSP has many applications[5], including in transportation, logistics, (circuit) plotting and drilling problems[111], telescope aiming, and genome sequencing. Solutions are often encoded as permutations x of the first n natural numbers, where each city gets a number and the order of numbers in the permutation corresponds to the order of cities in the tour. The distances between the n cities are usually given in a $n \times n$ distance matrix A, where $A[a, b]$ is the distance from city a to city b with $a, b \in 1..n$. Then, the objective function subject to minimization is $f(x) = A[x_n, x_1] + \sum_{i=1}^{n-1} A[x_i, x_{i+1}]$. Typical benchmark instances can be found in TSPLib [211,212], the 8th DIMACS challenge[135–137], and the *World TSP* repository[58].

Glossary XIII



TSPLib is a library of benchmark instances for the Traveling Salesperson Problem (TSP) [211,212]. Its original URL was available until 2025 and then became defunct. The repository https://github.com/shredderzwj/TSPLIB contains the data. Additionally, you may also check https://github.com/thomasWeise/tspSuite, which contains all the TSPLib instances as well as Java code to load them. The TSPLib contains 110 instances of the symmetric Traveling Salesperson Problems (STSPs) with scales n ranging from 14 to 85 900, but 93 instances have less than 2000 nodes. There also are 19 asymmetric Traveling Salesperson Problems (ATSPs) with scales $n \in 17..443$.

TTP The *Traveling Tournament Problem* is the combinatorial optimization problem of both efficiently and fairly organizing a tournament of n teams that play against each other in a pairwise fashion [82,267]. The efficient part boils down to arranging the games such that the total travel length is short, which is somewhat similar to the classical TSP. Initially, each team is at its home location. On each day, a team needs to travel if its scheduled game is not at its present location. On the last day, each team may need to travel back home unless their last game is a home game. The total travel length sums up the lengths of all travels over all teams. The fair part is represented in several constraints, such as doubleRoundRobin, compactness, maxStreak, and noRepeat. The TTP is $\mathcal{NP}$-hard [244].

TwoMax The TwoMax problem is a benchmark problem for bit-string based black-box search [97,242,271]. It has a global and a local optimum, both of about the same size, at opposite ends of the search space. For bit strings of length n, the objective function $f(x)$ of the minimization variant is 0 if $\|x\|_1 = n$ and $1 + n - \max\left\{\|x\|_1, n - \|x\|_1\right\}$ otherwise, where $\|x\|_1 = \sum_{i=1}^{n} x_i$. An $(1+1)$ EA has a 50/50 chance to converge to the local or global optimum and thus has an expected runtime in $\Theta(n^n)$ [96,97]. In experiments, the $(1+1)$ FEA achieves polynomial runtime seemingly in $\mathcal{O}(n^2 \ln n)$ on TwoMax [258,259,271].

Glossary XIV



unit test Software development is centered around creating the program code of an application, library, or otherwise useful system. A *unit test* is an *additional* code fragment that is not part of that productive code. It exists to execute (a part of) the productive code in a certain scenario (e.g., with specific parameters), to observe the behavior of that code, and to compare whether this behavior meets the specification [21,192,193,195,214,236]. If not, the unit test fails. The use of unit tests is at least threefold: First, they help us to detect errors in the code. Second, program code is usually not developed only once and, from then on, used without change indefinitely. Instead, programs are often updated, improved, extended, and maintained over a long time. Unit tests can help us to detect whether such changes in the program code, maybe after years, violate the specification or, maybe, cause another, depending, module of the program to violate its specification. Third, they are part of the documentation or even specification of a program.

URI A *Uniform Resource Identifier* is an identifier for an abstract or physical resource in the internet [270]. It can be a URL, a name, or both. URIs are supersets of URLs. The connection strings of the PostgreSQL DBMS are examples of Uniform Resource Identifiers (URIs).

URL A *Uniform Resource Locator* identifies a resource in the World Wide Web (WWW) and a way to obtain it by describing a network access mechanism. The most notable example of URLs is the text you write into web browsers to visit websites [26]. URLs are subsets of URIs.

VCS A *Version Control System* is a software which allows you to manage and preserve the historical development of your program code [240]. A distributed VCS allows multiple users to work on the same code and upload their changes to the server, which then preserves the change history. The most popular distributed VCS is Git.

VND *Variable Neighborhood Descend*; see VNS. VND in combination with GRASP has been applied to the 2BP|O|F in [198].

Glossary XV



VNS *Variable Neighborhood Search* is a metaheuristic framework combining a systematic change of neighborhood with LS [117,186]. A *neighborhood* structure in LS defines which solutions can be reached from a given solution within one search step. Usually, one uses a single unary operator and, hence, has a single neighborhood structure. However, for many problems, we can easily design many operators. For permutations, for example, one operator may swap two elements, another one may reverse a section of the permutation, and yet another one, while yet another one may rotate a subsequence one step to the left or right [251]. In the VNS framework, a multiple neighborhoods are investigated in a systematic fashion. Assume that we have a list of neighborhood structures. Variable Neighborhood Descend (VND) works in a deterministic fashion. It finds the best solution that can be reached in one step from the current solution under the current neighborhood. If this is better than the current solution, we accept it and continue with the first neighborhood. Otherwise, we continue with the next neighborhood in the list. In Basic VNS, instead of picking the best move from the current neighborhood, the first improving move is used. Also, at the beginning of each cycle, a random point from the current neighborhood around the current solution is chosen as starting point.

W-Model The W-Model is a tunable benchmark function for bit-string based black-box search [253,260].It allows you to adjust the amount of ruggedness, deceptiveness, epistasis, and uniform neutrality in the search space.

WWW World Wide Web [25,64]

$i!$ The factorial $a!$ of a natural number $a \in \mathbb{N}_1$ is the product of all positive natural numbers less than or equal to a, i.e., $a! = 1 * 2 * 3 * 4 * \cdots * (a-1) * a$ [47,80,169].

$i..j$ with $i, j \in \mathbb{Z}$ and $i \leq j$ is the set that contains all integer numbers in the inclusive range from i to j. It always holds that $i..j \subset \mathbb{Z}$. For example, $5..9$ is equivalent to $\{5, 6, 7, 8, 9\}$

A the $n \times n$ distance matrix. $A[a, b]$ indicates the distance between locations a and b with $a, b \in 1..n$. A distance matrix can be symmetric, in which case $A[a, b] = A[b, a]$ $\forall a, b \in 1..n$. Distance matrices represent the problem instance data in TSPs and part of the instance data of QAPs.

Glossary XVI

B the $n \times n$ flow matrix. $B[a, b]$ is the flow of goods between locations a and b with $a, b \in 1..n$. Flow matrices are one half of the problem instance data of QAPs.

$f(y)$ The *objective function* $f : \mathbb{Y} \mapsto \mathbb{R}$ can compute a cost or rating for each candidate solution y in the solution space $\mathbb{Y}$. The smaller this value, the better the solution, i.e., in the context of this work, objective functions are subject to minimization. Objective functions are not necessarily continuous and it is also quite common that they have integer or natural numbers as results, e.g., we often have $f : \mathbb{Y} \mapsto \mathbb{N}_0$ instead of $f : \mathbb{Y} \mapsto \mathbb{R}$.

$\gamma(x)$ The decoding function $\gamma : \mathbb{X} \mapsto \mathbb{Y}$ translates point from the search space $\mathbb{X}$ to one point y in the solution space $\mathbb{Y}$. It is a very common case that $\mathbb{X} = \mathbb{Y}$ and that the decoding function γ is the identity mapping. Examples for encodings that translate from a simple search space $\mathbb{X}$ to a complex solution data structure $\mathbb{Y}$ are the IBL algorithm that translates (potentially signed) permutations to two-dimensional packings on the 2BP, the permutation-based encoding of Gantt charts for the JSSP from [28,29,100,219,252], or the permutation-based encoding of game plans for the Traveling Tournament Problem (TTP) from [267]. Encodings can also translate between types of search spaces. The Random Keys encoding, for example, translates from $[0, 1]^n$ to permutations of the first n natural numbers..

$\mathrm{geom}(A)$ The *geometric mean* $\mathrm{geom}(A)$ is the nth root of the product of n *positive* values in a dataset $A = (a_0, a_1, \ldots, a_{n-1})$ with $a_i > 0$ for all $i \in 0..n$, i.e., $\mathrm{geom}(A) = \sqrt[n]{\prod_{i=0}^{n-1} a_i} = \exp\left(\frac{1}{n} \sum_{i=0}^{n-1} \ln a_i\right)$. The formula using $\exp$ and $\ln$ is equivalent to the product formula. However, due to the limited range of floating point numbers on computers [127], the product formula may lead to an overflow or underflow and the $\exp$-$\ln$ method is more reliable. In [94], it is shown that the geometric mean is the only reasonable measure of average when dealing with normalized data, where the arithmetic mean or median become unreliable. In Python, you can use function `geometric_mean` from the module `statistics` to compute the geometric mean.

k the number of jobs in an instance of the JSSP.

m the number of machines in an instance of the JSSP.

Glossary XVII

$\text{mean}(A)$ The *arithmetic mean* $\text{mean}(A)$ is an estimate of the expected value of a distribution from which a data sample A was, well, sampled. Its is computed on data sample $A = (a_0, a_1, \ldots, a_{n-1})$ as the sum of all n elements a_i in the sample data A divided by the total number n of values, i.e., $\text{mean}(A) = \frac{1}{n}\sum_{i=0}^{n-1} a_i$. In Python, you can use function `mean` from the module `statistics` to compute the arithmetic mean.

$\text{median}(A)$ The *median* $\text{median}(A)$ is the value separating the bigger-valued half from the smaller-valued half of a data sample or distribution. Its estimate is the value right in the middle of a *sorted* data sample $A = (a_0, a_1, \ldots, a_{n-1})$ where $a_{i-1} \leq a_i \; \forall i \in 1 \ldots (n-1)$ with an odd number of elements and the mean of the two values in the middle if n is even. In other words, $\text{median}(A) = a_{\frac{n-1}{2}}$ if n is odd and $\frac{1}{2}\left(a_{\frac{n}{2}-1} + a_{\frac{n}{2}}\right)$ otherwise, i.e., if n is even. In Python, you can use function `median` from the module `statistics` to compute the median.

n We usually use the value $n \in \mathbb{N}_0$ to denote the *scale*, i.e., the size of the problem to solve. This notion is very general in computer science and not limited to optimization. For example, if we sort a list or search in a list, the scale n is the length of the list, i.e., the number of elements in it. In a TSP, this may be the number of cities. In a discrete benchmark problem such as OneMax or Trap, this may be the number of bits over which the problem is defined. In a QAP, this may be the number of facilities and locations. In a MaxSAT problem, this may be the number of variables that are input to the Boolean formula that we want to become true. There are also problems where the scale is not defined straightforwardly like this. For example, in a JSSP, we have a number of machines and a number of jobs, which together define the size of the problem. But even then, the scale n relates to the size of the solution space $\mathbb{Y}$, and, hence, also of the search space $\mathbb{X}$, which usually grow exponentially with n. And this is already the main issue: The larger n, the (exponentially) larger the number of possible solutions and thus, the larger the runtime of algorithms. This is known as the "curse of dimensionality" [23,50]. Scales n are also often used in conjunction with the $\mathcal{O}$-notation to denote algorithm complexity. For example, comparison-based sorting a list of n items takes $\Theta(n \log n)$ steps [146] (because they need $\lceil \log_2 n! \rceil$ comparisons in the worst case).

$\mathbb{N}_0$ the set of the natural numbers *including* 0, i.e., 0, 1, 2, 3, and so on. It holds that $\mathbb{N}_0 \subset \mathbb{Z}$ and that $\mathbb{N}_0 = \mathbb{N}_1 \cup \{0\}$.

$\mathbb{N}_1$ the set of the natural numbers *excluding* 0, i.e., 1, 2, 3, 4, and so on. It holds that $\mathbb{N}_1 \subset \mathbb{Z}$ and that $\mathbb{N}_1 = \mathbb{N}_0 \setminus \{0\}$.

Glossary XVIII



$\mathcal{NP}$ is the class of computational problems that can be solved in polynomial time by a non-deterministic machine and can be verified in polynomial time by a deterministic machine (such as a normal computer) [109].

$\mathcal{NP}$-complete A decision problem is $\mathcal{NP}$-complete if it is in $\mathcal{NP}$ and all problems in $\mathcal{NP}$ are reducible to it in polynomial time [109,209]. A problem is $\mathcal{NP}$-complete if it is $\mathcal{NP}$-hard and if it is in $\mathcal{NP}$.

$\mathcal{NP}$-hard A problem is $\mathcal{NP}$-hard if all problems in $\mathcal{NP}$ are reducible to it in polynomial time [109]. Algorithms that guarantee to find the correct solutions of $\mathcal{NP}$-hard problems [49,55,152] have a worst-case runtime that grows exponentially with the problem scale.

$\mathbf{\Omega}(g(x))$ If $f(x) = \mathbf{\Omega}(g(x))$, then there exist positive numbers $x_\mathbf{0} \in \mathbb{R}^+$ and $c \in \mathbb{R}^+$ such that $f(x) \geq c * g(x) \geq 0$ for all $x \geq x_\mathbf{0}$ [59,143,144]. In other words, $\mathbf{\Omega}(g(x))$ describes a lower bound for function growth.

$\mathcal{O}(g(x))$ If $f(x) = \mathcal{O}(g(x))$, then there exist positive numbers $x_\mathbf{0} \in \mathbb{R}^+$ and $c \in \mathbb{R}^+$ such that $0 \leq f(x) \leq c * g(x)$ for all $x \geq x_\mathbf{0}$ [9,59,143,144,150]. In other words, $\mathcal{O}(g(x))$ describes an upper bound for function growth.

$\mathbf{\Theta}(g(x))$ If $f(x) = \mathbf{\Theta}(g(x))$, then $f(x) = \mathcal{O}(g(x))$ and $f(x) = \mathbf{\Omega}(g(x))$ [143,144], meaning that there exist positive constants $c_\mathbf{1}, c_\mathbf{2}, x_\mathbf{0} \in \mathbb{R}^+$ such that $0 \leq c_\mathbf{1} g(x) \leq f(x) \leq c_\mathbf{2} g(x)$ for all $x \geq x_\mathbf{0}$ [59]. In other words, $\mathbf{\Theta}(g(x))$ describes an exact order of function growth.

$\mathbf{o}(g(x))$ If $f(x) = \mathbf{o}(g(x))$, then there exist positive numbers $x_\mathbf{0} \in \mathbb{R}^+$ such that $0 \leq f(x) \leq c * g(x)$ for all $x \geq x_\mathbf{0}$ ***all*** $c \in \mathbb{R}^+$ [59]. Different from $\mathcal{O}(g(x))$, $\mathbf{o}(g(x))$ holds for ***all*** constants $c > 0$, whereas $\mathcal{O}(g(x))$ implies the existence of ***some*** constant $c > 0$. This basically means that $f(x)$ becomes insignificant compared to $g(x)$ as $x_\mathbf{0}$ grows toward infinity. It is often written that $\lim_{x_\mathbf{0} \to \infty} \frac{f(x)}{g(x)} = 0$. In other words, $\mathbf{o}(g(x))$ describes an upper bound for function growth that is not asymptotically tight, i.e., an upper bound that grows faster than $f(x)$.

Glossary XIX

$\text{quantile}_q^k(A)$ The q-*quantiles* are the cut points that divide a sorted data sample $A = (a_0, a_1, \ldots, a_{n-1})$ where $a_{i-1} \leq a_i \ \forall i \in 1 \ldots (n-1)$ into q equally-sized parts. $\text{quantile}_q^k(A)$ be the kth q-quantile, with $k \in 1 \ldots (q-1)$, i.e., there are $q-1$ of the q-quantiles. In the context of this book, we define $h = (n-1)\frac{k}{q}$. $\text{quantile}_q^k(A)$ then can be computed as a_h if h is integer, i.e., $h \in \mathbb{Z}$, and as $a_{\lfloor h \rfloor} + (h - \lfloor h \rfloor) * \left(a_{\lfloor h \rfloor + 1} - a_{\lfloor h \rfloor}\right)$ otherwise. It holds that $\text{quantile}_1^2(A) = \text{median}(A)$. In Python, you can use function `quantiles` from the module `statistics` to compute quantiles

$\mathbb{R}$ the set of the real numbers.

$\mathbb{R}^+$ the set of the positive real numbers, i.e., $\mathbb{R}^+ = \{x \in \mathbb{R} : x > 0\}$.

$\text{sd}(A)$ The statistical estimate $\text{sd}(A)$ of the *standard deviation* of a data sample $A = (a_0, a_1, \ldots, a_{n-1})$ with n observations is the square root of the estimated variance $\text{var}(A)$, i.e., $\text{sd}\, A = \sqrt{\text{var}(A)}$. In Python, you can use function `stdev` from the module `statistics` to compute the standard deviation.

$\text{var}(A)$ The *variance* of a distribution is the expectation of the squared deviation of the underlying random variable from its mean. The variance $\text{var}(A)$ of a data sample $A = (a_0, a_1, \ldots, a_{n-1})$ with n observations can be estimated as $\text{var}(A) = \frac{1}{n-1}\sum_{i=0}^{n-1}(a_i - \text{mean}(A))^2$. It can also be computed using $\text{var}(A) = \left(\sum_{i=0}^{n-1} a_i^2\right) - \frac{1}{n}\left(\sum_{i=0}^{n-1} a_i\right)^2$, which affords computing it in a single pass, but this method may lead to disastrous rounding errors [48]. A better choice is Welford's algorithm [145,262]: It begins by setting its running mean $m \leftarrow 0$, running variance $v \leftarrow 0$, and its running counter $i \leftarrow 0$. It then iterates over A with the current value being a_i. In each step, it sets $delta \leftarrow a_i - m$, then updates $m \leftarrow m + \delta/(i+1)$, $v \leftarrow v + \delta * (a_i - m)$, and finally $i \leftarrow i + 1$. Once all the elements of A have been processed, the sample variance is $\frac{1}{n-1}v$.

Glossary XX

$\omega(g(x))$ If $f(x) = \omega(g(x))$, then for every positive constant $c > 0$, there exists a number $x_0 \in \mathbb{R}^+$ such that $0 \leq c * g(x) < f(x)$ for all $x \geq x_0$ [59]. In other words, $f(x) = \omega(g(x))$ if and only if $g(x) = o(f(x))$. In other words, ω describes a lower bound that is not asymptotically tight.

$\mathbb{X}$ The search space $\mathbb{X}$ is the space through which an optimization algorithm navigates in order to find good solutions. The search operators of metaheuristics are used to sample points $x \in \mathbb{X}$, for example. On many benchmark problems, the objective functions f directly evaluate the elements $x \in \mathbb{X}$, i.e., are defined as $f : \mathbb{X} \mapsto \mathbb{R}$. However, very often these elements x are not the same data structures that the user is interested in and/or that the objective functions evaluate: It is common that the search space $\mathbb{X}$ is a simple and plain data structure that can be tackled with standard operations or mathematical tools, such as real vectors $\mathbb{R}^n$, permutations, or bit strings, whereas the solution space $\mathbb{Y}$ can be something more complicated. Then, the user only cares about the solutions $y \in \mathbb{Y}$ and not about the search space $\mathbb{X}$. A decoding $\gamma : \mathbb{X} \mapsto \mathbb{Y}$ is employed to translate the $x \in \mathbb{X}$ to $y = \gamma(x)$ with $y \in \mathbb{Y}$.

x a point in the search space $\mathbb{X}$.

$\mathbb{Y}$ the set of candidate solutions of an optimization problem is called the *solution space* $\mathbb{Y}$. The solution space corresponds to the data structure holding the candidate solutions y. These are the elements that are passed to the user after the optimization process is completed. They are also the parameters of the objective function(s) $f : \mathbb{Y} \mapsto \mathbb{R}$.

y a candidate solution $y \in \mathbb{Y}$ is one possible solution to an optimization problem. After the optimization process is completed, one or multiple such solutions are passed to the user.

$\mathbb{Z}$ the set of the integers numbers including positive and negative numbers and 0, i.e., . . . , -3, -2, -1, 0, 1, 2, 3, . . . , and so on. It holds that $\mathbb{Z} \subset \mathbb{R}$.